

UNIVERSITY OF PETROLEUM AND ENERGY STUDIES

Containerization & DevOps

Lab Report

Name	Pranav R Nair
SAP ID	500121466
Roll No	R2142230197
Batch	2 (CCVT)

Index of Experiments

#	Experiment Title	Page
1	Experiment 1: Comparison of Virtual Machines (VMs) and Containers	—
2	Experiment 2: Docker Installation and Container Lifecycle	—
3	Experiment 3: Deploying NGINX Using Different Base Images and Comparing Image Layers	—
4	Experiment 4: Docker Essentials — Dockerfile, .dockerignore, Tagging & Publishing	—
5	Experiment 5: Docker Volumes, Environment Variables, Monitoring & Networks	—
6	Experiment 6: Docker Run vs Docker Compose — A Practical Comparison	—
7	Experiment 7: CI/CD Pipeline using Jenkins, GitHub & Docker Hub	—
8	Experiment 9: Ansible — Agentless Configuration Management with Docker	—
9	Experiment 10: SonarQube — Static Code Analysis with Docker	—
10	Experiment 11: Container Orchestration with Docker Compose & Docker Swarm	—
11	Experiment 12: Container Orchestration with Kubernetes (k3d)	—

Objective

The primary goal of this experiment is to practically compare **Virtual Machines (VMs)** and **Containers** by deploying a web server (Nginx) using two different technologies: **Vagrant** (for VMs) and **Docker** (for Containers). By the end of this lab, you will understand the differences in setup time, resource utilization, and operational efficiency between these two approaches.

Theory: VMs vs. Containers

Before diving into the commands, it's crucial to understand what we are building.

Virtual Machines (VMs)

A **Virtual Machine** allows you to run a full guest operating system (like Ubuntu) on top of your host operating system (Windows). It uses a piece of software called a **Hypervisor** (like VirtualBox) to manage resources.

- **Pros:** Full isolation, runs any OS.
- **Cons:** Heavy resource usage (RAM/CPU), slow startup.

Containers

A **Container** allows you to package an application and its dependencies together. Unlike a VM, it shares the host OS kernel and does not require a full guest OS.

- **Pros:** Extremely lightweight, fast startup (seconds), efficient resource usage.
 - **Cons:** Less isolation than VMs, kernel dependency.
-

Prerequisites

Ensure you have the following tools installed:

- **Vagrant:** For managing virtual machine environments.
 - **VirtualBox:** The hypervisor used by Vagrant.
 - **Docker Desktop (or Engine):** For running containers.
 - **WSL 2** (Windows Subsystem for Linux): Recommended backend for Docker on Windows.
-

Part A: Virtual Machines with Vagrant

In this section, we will spin up a full Ubuntu VM and install Nginx inside it.

Step 1: Project Setup

Create a dedicated directory for your Vagrant project.

```
mkdir vm-lab
cd vm-lab
```

Step 2: Initialize Vagrant

Initialize a new Vagrant environment with an Ubuntu image.

```
vagrant init ubuntu/jammy64
```

- **Command:** `vagrant init` creates a `Vagrantfile` in your current directory. This file describes the type of machine required.
- **Argument:** `ubuntu/jammy64` specifies the "box" (image) to use. "Jammy64" refers to Ubuntu 22.04 LTS.
- **Expected Output:** A message confirming a `Vagrantfile` has been placed in the directory.

Step 3: Start the Virtual Machine

Provision and start the VM.

```
vagrant up
```

- **Command:** `vagrant up` reads the `Vagrantfile` and tells VirtualBox to create and start the machine.
- **Deep Dive:** This process involves downloading the OS image (if not cached), allocating RAM/CPU, booting the guest OS, and setting up networking (SSH forwarding).
- **Expected Output:** Logs showing the import progress, network interface setup, and SSH key generation.

Step 4: Access the VM

Log into the running VM via SSH.

```
vagrant ssh
```

- **Command:** securely connects you effectively "inside" the VM terminal.
- **Expected Output:** Your command prompt will change to something like `vagrant@ubuntu-jammy:~$`.

Step 5: Install Nginx (Inside VM)

Now that we are inside the VM, we treat it like a regular server.

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
```

- **Flags:**
 - `-y`: Automatically answers "yes" to prompts, avoiding interruption.
- **Expected Output:** Nginx installs and the service starts successfully.

Step 6: Verify Nginx

Check if the web server is responding locally inside the VM.

```
curl localhost
```

- **Expected Output:** The raw HTML code of the "Welcome to Nginx" page.

Step 7: Cleanup

Exit the VM and shut it down to free up resources.

```
# Exit the SSH session
exit

# Stop the VM (saves state)
vagrant halt

# Destroy the VM (deletes all files)
vagrant destroy
```

Part B: Containers with Docker (via WSL)

Now, we will achieve the same result (running Nginx) using Docker containers.

Step 1: Ensure WSL is Ready

If you haven't set up WSL 2 yet:

```
wsl --install
```

- **Command:** Installs the Windows Subsystem for Linux and the default Ubuntu distribution. **REQUIRES RESTART.**

Step 2: Pull the Docker Image

Download the official Ubuntu image (optional, `docker run` does this automatically if missing).

```
docker pull ubuntu
```

Step 3: Run Nginx Container

We will run an Nginx container directly. Note that we don't need to "install" Nginx manually; we just use the pre-built `nginx` image.

```
docker run -d -p 8080:80 --name nginx-container nginx
```

- **Command:** `docker run` creates and starts a container.
- **Flags:**
 - `-d` (**Detached**): Runs the container in the background. Without this, your terminal would be stuck showing server logs.
 - `-p 8080:80` (**Port Mapping**): Maps port `8080` on your host (Windows) to port `80` inside the container.
 - `--name nginx-container`: Assigns a custom name for easy management (instead of a random string).
 - `nginx`: The name of the image to use.
- **Expected Output:** A long alphanumeric string (Container ID) is returned.

Step 4: Verify Nginx

Since we mapped the port, we can access Nginx from our host machine (Windows/WSL).

```
curl localhost:8080
```

- **Expected Output:** The "Welcome to Nginx" HTML page.

Step 5: Monitor Resources

Compare the footprint of this container against the VM from Part A.

```
docker stats
```

- **Command:** Displays a live stream of container resource usage (CPU, Memory, Network).
- **Observation:** You will likely see the container using **< 20MB RAM**, whereas the Vagrant VM likely reserved **512MB - 1GB**.

Deep Dive

Why Port Mapping (-p)?

Containers run in their own isolated network namespace. Port 80 inside the container is **not** open on your computer by default. The flag `-p 8080:80` creates a "bridge," forwarding traffic from your computer's port 8080 to the container's port 80.

Image vs. Container

- **Image:** The blueprint or template (read-only). Like a Class in programming.
 - **Container:** The running instance of that image. Like an Object in programming.
-

Troubleshooting & Common Pitfalls

1. "Vagrant up" fails with Virtualization Error

- **Cause:** Virtualization technology (VT-x or AMD-V) is disabled in your computer's BIOS.
- **Fix:** Restart your PC, enter BIOS, and enable Virtualization.

2. Docker: "bind: address already in use"

- **Simulated Error:** `Error response from daemon: driver failed programming external connectivity ...`
- **Cause:** Port 8080 is already being used by another application.
- **Fix:** Change the host port mapping, e.g., `-p 8081:80`.

3. "Docker daemon is not running"

- **Cause:** Docker Desktop is closed.
 - **Fix:** Launch Docker Desktop from the Start menu and wait for the whale icon to stabilize.
-

Student: Pranav R Nair | **Batch:** 2(CCVT) | **SAP ID:** 500121466

Visuals & Outputs

```

Administrator: Windows PowerShell

System load:          0.88
Usage of /:           4.1% of 38.70GB
Memory usage:        21%
Swap usage:          0%
Processes:           126
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:28:41ff:fee4:55a4

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

nagrant@ubuntu-jammy:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:5 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2904 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [14.1 MB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [419 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1011 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [222 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy/universe Translation-en [5652 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f Metadata [286 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [217 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/multiverse Translation-en [112 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f Metadata [8372 B]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3165 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [485 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.1 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5101 kB]
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.5 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [51.1 kB]
Get:22 http://security.ubuntu.com/ubuntu jammy-security/multiverse Translation-en [10.5 kB]
Get:23 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 c-n-f Metadata [376 B]
Get:24 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [957 kB]
Get:25 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [632 B]
Get:26 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1248 kB]
Get:27 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [311 kB]
Get:28 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.1 kB]
Get:29 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [70.1 kB]
Get:30 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse Translation-en [15.5 kB]

```

```

Administrator: Windows PowerShell

Fetched 36.9 MB in 17s (2218 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
9 packages can be upgraded. Run 'apt list --upgradable' to see them.

nagrant@ubuntu-jammy:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0 libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8
  libnginx-mod-http-geoip2 libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail
  libnginx-mod-stream libnginx-mod-stream-geoip2 libtiff5 libwebp7 libxpm4 nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0 libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8
  libnginx-mod-http-geoip2 libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail
  libnginx-mod-stream libnginx-mod-stream-geoip2 libtiff5 libwebp7 libxpm4 nginx-common nginx-core
0 upgraded, 20 newly installed, 0 to remove and 9 not upgraded.
Need to get 2694 kB of archives.
After this operation, 8346 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-dejavu-core all 2.37-2build1 [1041 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fontconfig-config all 2.13.1-4.2ubuntu5 [29.1 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 libdeflate0 amd64 1.10-2 [70.0 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/main amd64 libfontconfig1 amd64 2.13.1-4.2ubuntu5 [131 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjpeg-turbo8 amd64 2.1.2-0ubuntu1 [134 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjpeg8 amd64 8c-2ubuntu10 [2264 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtiff5 amd64 4.3.0-6ubuntu0.12 [29.2 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libwebp7 amd64 1.2.2-0ubuntu0.22.04.2 [206 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtiff5 amd64 4.3.0-6ubuntu0.12 [185 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libxpm4 amd64 1:3.5.12-1ubuntu0.22.04.2 [36.7 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgd3 amd64 2.3.0-2ubuntu2.3 [129 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx-common all 1.18.0-6ubuntu14.7 [40.1 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-geoip2 amd64 1.18.0-6ubuntu14.7 [12.0 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-image-filter amd64 1.19.0-6ubuntu14.7 [15.5 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-xslt-filter amd64 1.18.0-6ubuntu14.7 [13.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-mail amd64 1.18.0-6ubuntu14.7 [45.8 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-stream amd64 1.18.0-6ubuntu14.7 [73.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-stream-geoip2 amd64 1.18.0-6ubuntu14.7 [10.1 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx-core amd64 1.18.0-6ubuntu14.7 [483 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx amd64 1.18.0-6ubuntu14.7 [3878 B]
Fetched 2694 kB in 4s (758 kB/s)
Preconfiguring packages ...
Selecting previously unselected package fonts-dejavu-core.
(Reading database ... 64286 files and directories currently installed.)
Preparing to unpack .../80-fonts-dejavu-core_2.37-2build1_all.deb ...
Unpacking fonts-dejavu-core (2.37-2build1) ...
Selecting previously unselected package fontconfig-config.
Preparing to unpack .../01-fontconfig-config_2.13.1-4.2ubuntu5_all.deb ...
Unpacking fontconfig-config (2.13.1-4.2ubuntu5) ...
Selecting previously unselected package libdeflate0:amd64.
Preparing to unpack .../02-libdeflate0_1.10-2_amd64.deb ...
Unpacking libdeflate0:amd64 (1.10-2) ...
Selecting previously unselected package libfontconfig1:amd64.

```

```

pranav@Pranav: ~$ sudo systemctl start docker
[sudo] password for pranav:
pranav@Pranav:~$ sudo systemctl start docker
pranav@Pranav:~$ sudo usermod -aG docker $USER
pranav@Pranav:~$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
pranav@Pranav:~$ docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
eaf8753feae0: Pull complete
700146c8ad64: Pull complete
119d43eec815: Pull complete
57f0dd1befe2: Pull complete
10b68cfefee1: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
e2dd2dbe6277: Download complete
785250c9bf9e: Download complete
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
5c61af1494ec   nginx-container  0.00%    13.3MiB / 7.594GiB   0.17%    2.33kB / 1.4kB  98.3kB / 12.3kB  17

```

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> vagrant --version
vagrant 2.4.9
PS C:\WINDOWS\system32> cd vm-lab
PS C:\WINDOWS\system32\vm-lab> vagrant init ubuntu/jammy64
Vagrantfile already exists in this directory. Remove it before running 'vagrant init'.
PS C:\WINDOWS\system32\vm-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Clearing any previously set forwarded ports...
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
==> default: Adapter 1: nat
==> default: Forwarding ports...
==> default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
==> default: SSH address: 127.0.0.1:2222
==> default: SSH username: vagrant
==> default: SSH auth method: private key
==> default: Warning: Connection reset. Retrying...
==> default: Warning: Connection aborted. Retrying...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
==> default: The guest additions on this VM do not match the installed version of
==> default: VirtualBox! In most cases this is fine, but in rare cases it can
==> default: prevent things such as shared folders from working properly. If you see
==> default: shared folder errors, please make sure the guest additions within the
==> default: virtual machine match the version of VirtualBox you have installed on
==> default: your host and reload your VM.
==> default: Guest Additions Version: 6.0.0 r127566
==> default: VirtualBox Version: 7.2
==> default: Mounting shared folders...
==> default: C:\Windows\System32\vm-lab => /vagrant
==> default: Machine already provisioned. Run 'vagrant provision' or use the '--provision'
==> default: flag to force provisioning. Provisioners marked to run always will still run.
PS C:\WINDOWS\system32\vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-164-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jan 24 04:03:16 UTC 2026

```

Objective

The goal of this experiment is to master the fundamental **Docker Container Lifecycle**. You will learn how to:

1. **Pull** images from a registry.
2. **Run** containers with specific configurations (port mapping, detached mode).
3. **Manage** containers (stop, remove) and clean up resources (images).

Prerequisites

- Docker Desktop installed and running.
- Basic understanding of CLI commands.

Procedure

Step 1: Pull a Docker Image

Before we can run a container, we need the "blueprint" or image locally.

```
docker pull nginx
```

- **Command:** `docker pull` downloads an image from a Docker Registry (default: Docker Hub).
- **Argument:** `nginx` is the name of the official image for the Nginx web server. Since we didn't specify a tag (like `nginx:1.21`), Docker defaults to `nginx:latest`.
- **Expected Output:**

```
Using default tag: latest
...
Status: Downloaded newer image for nginx:latest
```

Step 2: Run the Container

Now we instantiate the image into a running container.

```
docker run -d -p 8080:80 --name my-web-server nginx
```

- **Command:** `docker run` creates and starts a container.
- **Flags:**

- `-d` (**Detached**): Runs the container in the background. If you omit this, the container logs will hijack your terminal.
- `-p 8080:80` (**Port Mapping**): Maps **Host Port 8080** -> **Container Port 80**. This allows you to access the web server at `localhost:8080`.
- `--name`: Assigns a readable name (`my-web-server`) to the container.
- **Expected Output**: A long unique container ID (e.g., `a1b2c3d4 ...`).

Step 3: Verify Running Containers

Check the status of your active containers.

```
docker ps
```

- **Command**: Lists only **running** containers.
- **Flags (Optional)**: Use `-a` (`docker ps -a`) to see **all** containers, including stopped ones.
- **Expected Output**: A table showing Container ID, Image, Command, Created Time, Status (Up X seconds), Ports, and Names.

Step 4: Stop the Container

Gracefully shut down the application.

```
docker stop my-web-server
```

- **Command**: Sends a `SIGTERM` signal to the main process inside the container, asking it to stop safely.
- **Argument**: You can use either the **Container ID** or the **Name** (`my-web-server`).
- **Expected Output**: The command echoes the container name.

Step 5: Remove the Container

Stopping a container does not delete it; it just puts it in an "Exited" state. To free up disk space, we must remove it.

```
docker rm my-web-server
```

- **Command**: Deletes the container's mutable layer and metadata.
- **Note**: You cannot remove a *running* container unless you use the `-f` (force) flag. Always stop first!
- **Expected Output**: The command echoes the container name.

Step 6: Remove the Image (Cleanup)

If you no longer need the Nginx image, remove it to save disk space.

```
docker rmi nginx
```

- **Command:** `rmi` stands for **Remove Image**.
- **Expected Output:** Logs showing "Untagged: ..." and "Deleted: ...".

Deep Dive

The Container Lifecycle

A container moves through specific states:

1. **Created:** The container layer is initialized but not started.
2. **Running:** The main process (PID 1) is active.
3. **Paused:** Processes are frozen (SIGSTOP).
4. **Exited:** The main process has stopped (either finished or crashed).
5. **Dead:** The container is non-functional and difficult to remove.

Docker Hub

When you run `docker pull`, where does the file come from? It comes from **Docker Hub** (hub.docker.com), the world's largest library and community for container images. It is similar to GitHub, but for binaries/images instead of source code.

Troubleshooting & Common Pitfalls

1. "Conflict: The container name is already in use"

- **Error:** `Error response from daemon: Conflict. The container name "/my-web-server" is already in use ...`
- **Cause:** You tried to run a new container with the same name as an existing one (even if the old one is stopped).
- **Fix:**
 - Remove the old one: `docker rm my-web-server`
 - **OR** Use a different name: `--name my-web-server-2`

2. "Cannot destroy container ... (it is running)"

- **Error:** `Error response from daemon: You cannot remove a running container ...`
- **Cause:** You ran `docker rm` on a live container.
- **Fix:** Stop it first (`docker stop <name>`), then remove it. Or force remove it (`docker rm -f <name>`).

Student: Pranav R Nair | **Batch:** 2(CCVT) | **SAP ID:** 500121466

Visuals & Outputs

```

pranav@Pranav: ~
This message is shown once a day. To disable it please create the
/home/pranav/.hushlogin file.
pranav@Pranav:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
pranav@Pranav:~$ docker run -d -p 8080:80 nginx
d5b0597671f310d6dca20c44815bdef3851cb35ad39b89e47d2e9bf319a45562
pranav@Pranav:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
d5b0597671f3   nginx         "/docker-entrypoint..." 11 seconds ago Up 10 seconds 0.0.0.0:8080->80/
tcp, [::]:8080->80/tcp
39f35339b42f   elasticsearch:8.8.0 "/bin/tini -- /usr/l..." 10 hours ago   Up 10 hours (healthy) 0.0.0.0:9200->920
0/tcp, [::]:9200->9200/tcp, 0.0.0.0:9300->9300/tcp, [::]:9300->9300/tcp
fa9720943526   redis:7-alpine "docker-entrypoint.s..." 33 hours ago   Up 33 hours (healthy) 0.0.0.0:6379->637
9/tcp, [::]:6379->6379/tcp
83d70f09e2d7   postgres:15-alpine "docker-entrypoint.s..." 33 hours ago   Up 33 hours (healthy) 0.0.0.0:5432->543
2/tcp, [::]:5432->5432/tcp
pranav@Pranav:~$ docker stop d5b0597671f3
d5b0597671f3
pranav@Pranav:~$ docker rm d5b0597671f3
d5b0597671f3
pranav@Pranav:~$ docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
pranav@Pranav:~$

```

Objective

Deploy NGINX using three different Docker image strategies — the **Official image**, a **Ubuntu-based** custom image, and an **Alpine-based** custom image — then compare their image sizes, build times, layer structures, and production suitability.

Theory: Why Base Images Matter

Before running commands, it's essential to understand the concept of a **base image** and why it has a massive impact on your containerized application.

What is a Base Image?

Every Docker image is built in **layers**. The very first layer is the **base image** — the foundational operating system or runtime that your application is built upon (specified by the `FROM` instruction in a Dockerfile). Everything you install or configure adds layers on top.

The Three Strategies

Strategy	Base	Philosophy
Official <code>nginx</code>	Debian (Slim)	Pre-optimized by the NGINX team; ready out-of-the-box.
Ubuntu + NGINX	<code>ubuntu:22.04</code>	Full-featured OS; install NGINX manually via <code>apt</code> .
Alpine + NGINX	<code>alpine:latest</code>	Ultra-minimal Linux distro (~5 MB); install NGINX via <code>apk</code> .

Why Alpine Images Are Smaller

Alpine Linux uses **musl libc** and **BusyBox** instead of the standard GNU C Library (glibc) and GNU coreutils. This means:

- **No unnecessary utilities:** Alpine ships with only the bare essentials.
- **musl libc** is a lightweight alternative to glibc, producing much smaller binaries.
- **Package manager (`apk`)** is designed for speed and minimal disk footprint; the `--no-cache` flag avoids storing the package index locally.
- **Result:** An Alpine base image is typically **~5–8 MB**, compared to Ubuntu's **~28–30 MB** (compressed).

Why Ubuntu Images Are Avoided in Production

While Ubuntu is excellent for learning and debugging, it is **rarely used as a production base image** because:

- **Bloated size:** Ubuntu ships with hundreds of packages you don't need in a container (e.g., `apt`, `bash`, `systemd` stubs, locales).
- **Larger attack surface:** More packages = more potential CVEs (Common Vulnerabilities and Exposures).
- **Slower CI/CD:** Larger images take longer to pull, push, and deploy — multiplied across hundreds of containers in a cluster.
- **Wasted resources:** In Kubernetes, every node caches images. A 200 MB image vs. a 16 MB image adds up quickly at scale.

Prerequisites

Ensure you have the following tools installed and running:

- **Docker Desktop** (or Docker Engine on Linux)
- **WSL 2** (for Windows users — recommended Docker backend)
- Basic familiarity with `docker run`, `Dockerfile`, and port mapping

Part 1: Deploy NGINX Using the Official Image (Recommended Approach)

This is the simplest and most production-ready method.

Step 1: Pull the Image

```
docker pull nginx:latest
```

- **Command:** `docker pull` downloads an image from Docker Hub.
- **Argument:** `nginx:latest` pulls the most recent stable build of the official NGINX image, maintained by the NGINX team.
- **Expected Output:**

```
latest: Pulling from library/nginx
4f4efe02d542: Pull complete
47cd406a84ef: Pull complete
...
Status: Downloaded newer image for nginx:latest
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

- **Command:** `docker run` creates and starts a new container.
- **Flags:**
 - `-d` (**Detached mode**): Runs the container in the background. Without this, your terminal would be hijacked by NGINX's access logs.
 - `--name nginx-official`: Assigns a human-readable name so you can reference it easily (instead of a random hex ID).
 - `-p 8080:80` (**Port mapping**): Maps **Host port 8080** → **Container port 80**. NGINX inside the container listens on port 80, but your host machine accesses it via port 8080.
 - `nginx`: The image to instantiate.
- **Expected Output:** A long container ID string, e.g.,
`07e57d4f16258c9fb20ca60942074b1417eaae627ba5dd2d4478ea686d00fe8f`.

Step 3: Verify the Deployment

```
curl http://localhost:8080
```

- **Command:** Sends an HTTP GET request to the NGINX server.
- **Expected Output:** The raw HTML of the "**Welcome to nginx!**" page, confirming the server is live.

Step 4: Inspect the Image

```
docker images nginx
```

- **Expected Output:**

IMAGE	ID	DISK USAGE	CONTENT SIZE
nginx:latest	341bf0f3ce6c	240MB	65.8MB

- **Key Observation:** The official image is **~240 MB** on disk. It is based on Debian (slim variant) and comes pre-optimized with NGINX configured, entrypoint scripts, and environment variable templating.

Part 2: Custom NGINX Using Ubuntu Base Image

Here we build NGINX from scratch on a full Ubuntu OS to understand the overhead of a general-purpose base image.

Step 1: Create the Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Line-by-line breakdown:

Line	Purpose
<code>FROM ubuntu:22.04</code>	Uses Ubuntu 22.04 LTS as the base. This pulls the full OS (~28 MB compressed).
<code>RUN apt-get update</code>	Refreshes the package index so <code>apt</code> knows what's available.
<code>apt-get install -y nginx</code>	Installs NGINX. The <code>-y</code> flag auto-confirms prompts.
<code>apt-get clean</code>	Removes cached <code>.deb</code> files to reduce image size.
<code>rm -rf /var/lib/apt/lists/*</code>	Deletes the package index files (no longer needed after install).
<code>EXPOSE 80</code>	Documents that the container listens on port 80 (informational only).
<code>CMD ["nginx", "-g", "daemon off;"]</code>	Starts NGINX in the foreground . By default, NGINX daemonizes itself, but Docker requires the main process to stay in the foreground (PID 1) or the container exits immediately.

Step 2: Build the Image

```
docker build -t nginx-ubuntu .
```

- **Command:** `docker build` reads the Dockerfile and constructs an image layer by layer.
- **Flags:**
 - `-t nginx-ubuntu` (**Tag**): Names the resulting image `nginx-ubuntu` for easy reference.
 - `.` (**Build context**): Tells Docker to use the current directory as the context (where to find the Dockerfile and any files to `COPY`).
- **Expected Output:** Build completes in approximately **92.5 seconds** (network dependent):

```
[+] Building 92.5s (7/7) FINISHED
```

Step 3: Run the Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

- **Flags:** Same as Part 1, but using port **8081** on the host (to avoid conflicts with the official container on 8080).
- **Expected Output:** A container ID string.

Step 4: Inspect the Image

```
docker images nginx-ubuntu
```

- **Expected Output:**

IMAGE	ID	DISK USAGE	CONTENT SIZE
nginx-ubuntu:latest	36bec77fa9b0	199MB	52.7MB

- **Key Observation:** The Ubuntu-based image is **199 MB**, and took **92.5 seconds** to build. The bulk of the time and size comes from `apt-get update` downloading package metadata and installing NGINX plus its dependencies.

Part 3: Custom NGINX Using Alpine Base Image

Now we achieve the same result with a fraction of the resources.

Step 1: Create the Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Line-by-line breakdown:

Line	Purpose
<code>FROM alpine:latest</code>	Uses Alpine Linux — a security-oriented, lightweight distro (~5 MB).
<code>RUN apk add --no-cache nginx</code>	Installs NGINX using Alpine's <code>apk</code> package manager. <code>--no-cache</code> avoids saving the package index locally, keeping the image small.

Line	Purpose
<code>EXPOSE 80</code>	Documents the listening port.
<code>CMD ["nginx", "-g", "daemon off;"]</code>	Runs NGINX in the foreground (same reason as Ubuntu).

Step 2: Build the Image

```
docker build -t nginx-alpine .
```

- **Expected Output:** Build completes in approximately **12.4 seconds** — over **7x faster** than the Ubuntu build:

```
[+] Building 12.4s (7/7) FINISHED
```

Step 3: Run the Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

- **Flags:** Uses port **8082** on the host.

Step 4: Inspect the Image

```
docker images nginx-alpine
```

- **Expected Output:**

IMAGE	ID	DISK USAGE	CONTENT SIZE
nginx-alpine:latest	aba85e8e3dbd	16MB	4.74MB

- **Key Observation:** The Alpine-based image is only **16 MB** — roughly **15x smaller** than the official image and **12x smaller** than the Ubuntu image.

Part 4: Image Pull Time and Size Comparison

I. Measured Image Pull & Build Times

These are the actual times measured during the experiment:

Image	Pull/Build Time	Disk Usage	Content Size
<code>nginx:latest</code> (Official)	~8 layers pulled	240 MB	65.8 MB
<code>nginx-ubuntu</code> (Custom)	92.5 seconds build	199 MB	52.7 MB
<code>nginx-alpine</code> (Custom)	12.4 seconds build	16 MB	4.74 MB

Takeaway: Alpine images are **7.5x faster to build** and **15x smaller** than the official image. In a CI/CD pipeline that builds and deploys hundreds of times a day, this difference is enormous.

II. Why This Matters in Production

- **Network bandwidth:** Pulling a 16 MB image vs. a 240 MB image across 50 Kubernetes nodes saves **~11 GB** of transfer per deployment.
- **Cold start latency:** Smaller images mean faster container startup, critical for auto-scaling.
- **Registry storage:** Docker registries (ECR, GCR, Docker Hub) charge by storage. Smaller images = lower costs.

Part 5: Layer Inspection (Deep Dive)

Compare Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

Layer Comparison Summary

Image	Total Layers	Largest Layer	Key Observation
<code>nginx</code> (Official)	~17 layers	87.4 MB (Debian base) + 86.6 MB (NGINX install)	Many helper scripts, env vars, and entrypoint configs
<code>nginx-ubuntu</code>	~8 layers	87.5 MB (Ubuntu base) + 58.6 MB (apt install)	Full Ubuntu filesystem included
<code>nginx-alpine</code>	~5 layers	9.11 MB (Alpine base) + 2.2 MB (apk install)	Minimal — only what's needed

Deep Dive: How Docker Layers Work

Each instruction in a Dockerfile (`FROM` , `RUN` , `COPY` , etc.) creates a new **read-only layer**. When a container runs, Docker adds a thin **writable layer** on top.

Writable Container	← Runtime changes (logs, temp files)
CMD / ENTRYPOINT	← Layer 3 (0 bytes – just metadata)
RUN apk add nginx	← Layer 2 (2.2 MB for Alpine)
FROM alpine:latest	← Layer 1 (9.11 MB base)

Why fewer layers matter:

- Fewer layers = smaller image = faster pulls
- Docker caches layers, so unchanged layers aren't rebuilt
- Combining `RUN` commands with `&&` reduces the total number of layers

Part 6: Functional Task — Serve a Custom HTML Page

Step 1: Create the HTML File

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

Step 2: Run with a Volume Mount

```
docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

- **Flags:**
 - `-v $(pwd)/html:/usr/share/nginx/html` (**Volume mount**): Maps your local `html/` directory into the container's web root. This means any changes you make to `html/index.html` on your host are **instantly reflected** inside the container — no rebuild needed.
- **Expected Output:** A container ID. Visit `http://localhost:8083` to see your custom page.

Why Volume Mounts?

- **Development speed:** Edit files locally, see changes instantly.
- **Data persistence:** Container data survives restarts.

- **Separation of concerns:** Config/content lives outside the image, making the image reusable.



Part 7: Comprehensive Comparison Summary

Image Comparison Table

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Disk Usage	240 MB	199 MB	16 MB
Content Size	65.8 MB	52.7 MB	4.74 MB
Build Time	Pre-built	~92.5 seconds	~12.4 seconds
Layer Count	~17	~8	~5
Ease of Use	★ ★ ★ (Plug & Play)	★ ★ (Manual setup)	★ ★ (Manual setup)
Debugging Tools	Limited	Excellent (<code>apt</code> , <code>bash</code>)	Minimal (<code>ash</code> shell)
Security Surface	Medium	Large	Small
Production Ready	✓ Yes	⚠ Rarely	✓ Yes

When to Use What

Use Case	Recommended Image
Production web server / reverse proxy	Official <code>nginx</code> or Alpine
Learning Linux internals + NGINX config	Ubuntu-based
Microservices / Kubernetes pods	Alpine-based
CI/CD build artifacts	Alpine-based
Heavy debugging / troubleshooting	Ubuntu-based



Troubleshooting & Common Pitfalls

1. Container Exits Immediately

- **Symptom:** `docker ps` shows nothing after `docker run`.
- **Cause:** NGINX defaults to running as a **daemon** (background process). When it daemonizes, PID 1 exits, and Docker thinks the container is done.

- **Fix:** Always include `CMD ["nginx", "-g", "daemon off;"]` in your Dockerfile to keep NGINX in the foreground.

2. "Bind: address already in use"

- **Error:** `Error response from daemon: driver failed programming external connectivity ...`
- **Cause:** Another process (or container) is already using that host port.
- **Fix:** Change the host port, e.g., `-p 8084:80`, or stop the conflicting container with `docker stop <name>`.

3. "Conflict: container name is already in use"

- **Error:** `The container name "/nginx-official" is already in use ...`
- **Cause:** A container with that name already exists (even if stopped).
- **Fix:** Remove the old container with `docker rm nginx-official`, or use a different name.

4. Alpine: "ash" shell instead of "bash"

- **Symptom:** Running `docker exec -it <container> bash` fails with `OCI runtime exec failed`.
- **Cause:** Alpine uses `ash` (from BusyBox), not `bash`.
- **Fix:** Use `docker exec -it <container> sh` instead.

5. Build takes too long / hangs at `apt-get update`

- **Cause:** Slow network or DNS issues inside the Docker build environment.
- **Fix:** Ensure Docker has internet access. On Windows, restart Docker Desktop.

Visuals & Outputs

```

pranav@Pranav: ~
pranav@Pranav:~$ docker pull nginx:latest
latest: Pulling from library/nginx
4f4efe02d542: Pull complete
47cd406a84ef: Pull complete
7b6cb8ccac7b: Pull complete
bae5a1799a80: Pull complete
f73400a233fd: Pull complete
46bf3a120c8e: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
pranav@Pranav:~$ docker run -d --name nginx-official -p 8080:80 nginx
07e57d4f16258c9fb20ca60942074b1417eaae627ba5dd2d4478ea686d00fe8f
pranav@Pranav:~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and

```

```

pranav@Pranav: ~
07e57d4f16258c9fb20ca60942074b1417eaae627ba5dd2d4478ea686d00fe8f
pranav@Pranav:~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
pranav@Pranav:~$ docker images nginx

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	341bf0f3ce6c	240MB	65.8MB	U

```

pranav@Pranav:~$

```

```

pranav@Pranav: ~
nginx:latest 341bf0f3ce6c 240MB 65.8MB U
pranav@Pranav:~$ nano Dockerfile
pranav@Pranav:~$ docker build -t nginx-ubuntu .
[+] Building 92.5s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 218B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a
=> => sha256:b1cba2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 29.54MB / 29.54MB
=> => extracting sha256:b1cba2e842ca52b95817f958faf99734080c78e92e43ce609cde9244867b49ed 1.4s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lis
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:c18a47ead832aab91beld7174080c0b291ec48d8d5622faf1a5606b337f02b94
=> => exporting config sha256:caf7b797e67e7ef62059a7d4dccecb5b9fd140028b23851b6949dd2e021e08b7
=> => exporting attestation manifest sha256:56c1c847c7282a604f7b69a8fabb792b0c6f18d12492d4f76bf62b051f4159ed
=> => exporting manifest list sha256:36bec77fa9b0d0454cddc710ded873ccb12e17ac437ed83e1c8fa63114e27074
=> => naming to docker.io/library/nginx-ubuntu:latest
=> => unpacking to docker.io/library/nginx-ubuntu:latest
pranav@Pranav:~$ docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
f5ee08e7c23a68adf7c1e5f00cb9b8eedb2db56f063abbc9dd690842dba386fc
pranav@Pranav:~$ docker images nginx-ubuntu

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
nginx-ubuntu:latest 36bec77fa9b0 199MB 52.7MB U
pranav@Pranav:~$

```

```

pranav@Pranav: ~
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
nginx-ubuntu:latest 36bec77fa9b0 199MB 52.7MB U
pranav@Pranav:~$ nano Dockerfile
pranav@Pranav:~$ docker build -t nginx-alpine .
[+] Building 12.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 134B
=> [internal] load metadata for docker.io/library/alpine:latest
=> [auth] library/alpine:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c
=> [2/2] RUN apk add --no-cache nginx
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:2bbf020505d15410ba0f45b8079f4edf847fb9a4f6af9f9b95d0d22e863ecec8
=> => exporting config sha256:8d16857f272a35851cf20f0a718ad6ffff7d6aa448a98a9c4ef237aac07c0d02c
=> => exporting attestation manifest sha256:5ab6ef4e5262641cf797ff89de81abe805d5769eaacc1712d7dbd063cea6a890
=> => exporting manifest list sha256:aba85e8e3dbdd0768c4eaec9c611d581c25cceb650409ff7678da2d793b3c3c
=> => naming to docker.io/library/nginx-alpine:latest
=> => unpacking to docker.io/library/nginx-alpine:latest
pranav@Pranav:~$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
d76ebb8f9726e0d762d75802f21f8f014b3283d9ac19baa4a2ee4173c82a94fb
pranav@Pranav:~$ docker images nginx-alpine

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
nginx-alpine:latest aba85e8e3dbd 16MB 4.74MB U
pranav@Pranav:~$

```

```

pranav@Pranav: ~
nginx-alpine:latest    aba85e8e3dbd    16MB    4.74MB    U
pranav@Pranav:~$ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx-alpine:latest    aba85e8e3dbd    16MB    4.74MB    U
nginx-ubuntu:latest    36bec77fa9b0    199MB    52.7MB    U
nginx:latest           341bf0f3ce6c    240MB    65.8MB    U
pranav@Pranav:~$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE                CREATED              CREATED BY          SIZE      COMMENT
341bf0f3ce6c         2 weeks ago         CMD ["nginx" "-g" "daemon off;"] 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         STOPSIGNAL SIGQUIT 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         EXPOSE map[80/tcp:{}] 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENTRYPOINT ["/docker-entrypoint.sh"] 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>            2 weeks ago         COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0
<missing>            2 weeks ago         COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB     buildkit.dockerfile.v0
<missing>            2 weeks ago         COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB     buildkit.dockerfile.v0
<missing>            2 weeks ago         COPY docker-entrypoint.sh / # buildkit 8.19kB     buildkit.dockerfile.v0
<missing>            2 weeks ago         RUN /bin/sh -c set -x && groupadd --syst... 86.6MB     buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV DYNPKG_RELEASE=1-trixie 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV PKG_RELEASE=1-trixie 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV ACME_VERSION=0.3.1 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV NJS_RELEASE=1-trixie 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV NJS_VERSION=0.9.5 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         ENV NGINX_VERSION=1.29.5 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         LABEL maintainer=NGINX Docker Maintainers <d... 0B         buildkit.dockerfile.v0
<missing>            2 weeks ago         # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB     debuerreotype 0.17
IMAGE                CREATED              CREATED BY          SIZE      COMMENT
36bec77fa9b0         3 minutes ago       CMD ["nginx" "-g" "daemon off;"] 0B         buildkit.dockerfile.v0
<missing>            3 minutes ago       EXPOSE [80/tcp] 0B         buildkit.dockerfile.v0
<missing>            3 minutes ago       RUN /bin/sh -c apt-get update && apt-get... 58.6MB     buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) CMD ["/bin/bash"] 0B         buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) ADD file:52c0e467fa2e92f10... 87.5MB     buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) LABEL org.opencontainers... 0B         buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) LABEL org.opencontainers... 0B         buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH 0B         buildkit.dockerfile.v0
<missing>            9 days ago          /bin/sh -c #(nop) ARG RELEASE 0B         buildkit.dockerfile.v0
IMAGE                CREATED              CREATED BY          SIZE      COMMENT
aba85e8e3dbd         52 seconds ago      CMD ["nginx" "-g" "daemon off;"] 0B         buildkit.dockerfile.v0
<missing>            52 seconds ago      EXPOSE [80/tcp] 0B         buildkit.dockerfile.v0
<missing>            52 seconds ago      RUN /bin/sh -c apk add --no-cache nginx # bu... 2.2MB     buildkit.dockerfile.v0
<missing>            3 weeks ago         CMD ["/bin/sh"] 0B         buildkit.dockerfile.v0
<missing>            3 weeks ago         ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /... 9.11MB     buildkit.dockerfile.v0
pranav@Pranav:~$

```

```

pranav@Pranav:~$ mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html

pranav@Pranav:~$ docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
890ec462377bde37cd34dbda921b3258e2f5ee28cd0c5ddc35a6d6b1993a452e
pranav@Pranav:~$ |

```

Optional Read: NGINX Web Server Deep Dive

1. What is NGINX?

NGINX is a:

- High-performance **web server**
- **Reverse proxy**
- **Load balancer**
- **API gateway**

It is event-driven and asynchronous, which makes it far more efficient than traditional thread-based servers like Apache under high load.

2. Hosting Static Files (Most Common Use)

Install NGINX on a Linux Server

Ubuntu / Debian:

```
sudo apt update
sudo apt install nginx
```

RHEL / CentOS:

```
sudo dnf install nginx
```

Start and enable the service:

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

Default Web Root

NGINX serves files from `/var/www/html` by default. Place your content there:

```
sudo nano /var/www/html/index.html
```

Access via `http://<server-ip>` in your browser.

3. Understanding NGINX Configuration

Path	Purpose
<code>/etc/nginx/nginx.conf</code>	Main configuration file
<code>/etc/nginx/sites-available/</code>	Individual site configurations
<code>/etc/nginx/sites-enabled/</code>	Symlinks to active site configs

NGINX uses **server blocks** (similar to Apache virtual hosts) to define how requests are handled.

4. Hosting a Custom Website (Server Block)

Create the site directory and config

```
sudo mkdir -p /var/www/myapp
sudo chown -R www-data:www-data /var/www/myapp
```

Create the NGINX configuration

```
server {
    listen 80;
    server_name myapp.local;

    root /var/www/myapp;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Enable and reload

```
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/
sudo nginx -t          # Test configuration syntax
sudo systemctl reload nginx
```

5. NGINX as a Reverse Proxy

Use case: Frontend → NGINX → Backend app (Node.js, Python, Go)

```
server {
    listen 80;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Why this matters:

- Hide backend ports from the public internet

- Central SSL/TLS termination
- Easy horizontal scaling

6. Load Balancing

```
upstream backend_pool {
    server 10.0.0.1:8080;
    server 10.0.0.2:8080;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend_pool;
    }
}
```

Algorithms: Round-robin (default), Least connections, IP hash.

7. HTTPS / SSL Termination

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/ssl/cert.pem;
    ssl_certificate_key /etc/ssl/key.pem;

    location / {
        root /var/www/html;
    }
}
```

8. NGINX vs Apache

Feature	NGINX	Apache
Architecture	Event-driven	Process/thread

Feature	NGINX	Apache
Static files	Faster	Slower
Reverse proxy	Excellent	Average
<code>.htaccess</code> support	No	Yes
Memory usage	Low	Higher

Student: Pranav R Nair | **Batch:** 2(CCVT) | **SAP ID:** 500121466

Objective

Containerize a Python Flask application using a `Dockerfile`, optimize the build context with `.dockerignore`, master image tagging and versioning strategies, implement multi-stage builds for production-ready images, and publish the final image to Docker Hub.

Theory: The Docker Build Pipeline

Before running commands, it's essential to understand the workflow that takes your source code and transforms it into a deployable container image.

The Build Pipeline

```
Source Code → Dockerfile → docker build → Image → docker run → Container
                ↑                               ↓
            .dockerignore                docker push → Registry
            (filters context)                (Docker Hub)
```

Key Concepts

Concept	What It Is	Why It Matters
Dockerfile	A text file with instructions to build an image layer by layer	Defines the reproducible "recipe" for your container
.dockerignore	A file that excludes paths from the build context	Reduces image size, speeds up builds, prevents secrets from leaking
Tagging	Naming and versioning images (e.g., <code>myapp:1.0</code>)	Enables version control, rollbacks, and organized registries
Multi-stage builds	Using multiple <code>FROM</code> statements to separate build and runtime	Produces smaller, more secure production images
Docker Hub	A cloud-hosted registry for storing and sharing images	Enables team collaboration and CI/CD pipelines

Prerequisites

- Docker Desktop installed and running
- A Docker Hub account (free at hub.docker.com)

- Basic knowledge of Python (for understanding the Flask app)
- Familiarity with `docker run` and port mapping (covered in Experiments 1–3)

Part 1: Create the Application

Step 1: Set Up the Project Directory

```
mkdir my-flask-app
cd my-flask-app
```

- **Command:** Creates a new folder and navigates into it. This folder will become the **build context** — everything Docker can "see" during the build.

Step 2: Create the Flask Application

Create `app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Line-by-line breakdown:

Line	Purpose
<code>from flask import Flask</code>	Imports the Flask web framework
<code>app = Flask(__name__)</code>	Creates a Flask application instance
<code>@app.route('/')</code>	Defines the root URL endpoint
<code>app.run(host='0.0.0.0', port=5000)</code>	Starts the server on all interfaces (<code>0.0.0.0</code>) at port 5000. Using <code>0.0.0.0</code> is critical — <code>127.0.0.1</code> would only be accessible inside the container

Step 3: Create the Requirements File

Create `requirements.txt`:

```
Flask==2.3.3
```

- This pins the Flask version to ensure reproducible builds across different environments.

Part 2: Write the Dockerfile

Step 1: Create the Dockerfile

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

Instruction-by-instruction breakdown:

Instruction	What It Does	Why This Way
<code>FROM python:3.9-slim</code>	Sets the base image to Python 3.9 on a slim Debian variant (~120 MB vs ~900 MB for full)	<code>slim</code> removes docs, man pages, and unused packages to save space
<code>WORKDIR /app</code>	Sets <code>/app</code> as the working directory for all subsequent commands	Keeps things organized; avoids dumping files in <code>/</code>

Instruction	What It Does	Why This Way
<code>COPY requirements.txt .</code>	Copies only the requirements file first	Enables Docker layer caching — dependencies are only reinstalled when <code>requirements.txt</code> changes
<code>RUN pip install --no-cache-dir -r requirements.txt</code>	Installs Python dependencies	<code>--no-cache-dir</code> saves ~50 MB by not caching pip's download cache
<code>COPY app.py .</code>	Copies the application code	Done <i>after</i> pip install so code changes don't invalidate the dependency cache
<code>EXPOSE 5000</code>	Documents that the container listens on port 5000	Informational — does not actually open the port
<code>CMD ["python", "app.py"]</code>	Defines the default command to run when the container starts	Uses exec form (JSON array) which runs directly as PID 1

🧐 Deep Dive: Why Copy Requirements Before Code?

Docker caches each layer. If you change `app.py` but not `requirements.txt`, Docker reuses the cached dependency layer — saving minutes on each build. This is called the **dependency caching pattern**:

```

Layer 1: FROM python:3.9-slim      ← Cached (base image unchanged)
Layer 2: WORKDIR /app              ← Cached
Layer 3: COPY requirements.txt .    ← Cached (file unchanged)
Layer 4: RUN pip install ...        ← Cached (requirements unchanged)
Layer 5: COPY app.py .             ← REBUILT (code changed)
Layer 6: CMD ["python", "app.py"]  ← REBUILT

```

🔧 Part 3: Configure .dockerignore

Step 1: Create the .dockerignore File

```

# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py

```

Why .dockerignore Is Critical

Without .dockerignore	With .dockerignore
<code>.git/</code> folder (often 50+ MB) gets copied into the build	Excluded — saves space and build time
<code>.env</code> with secrets gets baked into the image	Excluded — prevents credential leaks
<code>__pycache__/</code> adds stale bytecode	Excluded — Python generates fresh bytecode inside the container
<code>venv/</code> with host-OS libraries gets included	Excluded — container installs its own dependencies
Build context transfer: 50+ MB	Build context transfer: < 1 KB

Part 4: Build and Tag the Image

Step 1: Basic Build

```
docker build -t my-flask-app .
```

- **Command:** `docker build` reads the Dockerfile and constructs an image.
- **Flags:**
 - `-t my-flask-app` (**Tag**): Names the image `my-flask-app` with an implicit `:latest` tag.
 - `.` (**Build context**): Tells Docker to use the current directory. Docker sends all files (minus `.dockerignore` exclusions) to the daemon.
- **Expected Output** (first build, ~69 seconds):

```
[+] Building 69.0s (11/11) FINISHED
⇒ [internal] load build definition from Dockerfile
⇒ [1/5] FROM docker.io/library/python:3.9-slim
⇒ [2/5] WORKDIR /app
⇒ [3/5] COPY requirements.txt .
⇒ [4/5] RUN pip install --no-cache-dir -r requirements.txt
⇒ [5/5] COPY app.py .
⇒ naming to docker.io/library/my-flask-app:latest
```

Step 2: Version Tagging

```
# Tag with a specific version number
docker build -t my-flask-app:1.0 .
```

- **What changes:** Instead of `:latest`, the image gets the explicit tag `:1.0`.
- **Expected Output** (~6.6 seconds — cached):

```
[+] Building 6.6s (10/10) FINISHED
⇒ CACHED [2/5] WORKDIR /app
⇒ CACHED [3/5] COPY requirements.txt .
⇒ CACHED [4/5] RUN pip install ...
⇒ CACHED [5/5] COPY app.py .
```

Note: The rebuild is nearly instant because Docker reuses all cached layers.

Step 3: Multi-Tag Build

```
# Apply multiple tags in a single build
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

- **Flags:** Multiple `-t` flags apply multiple tags to the **same** image (same SHA). This is efficient — no duplication.
- **Expected Output:** Build completes in ~2.3 seconds (fully cached), with both tags applied.

Step 4: Tag for a Custom Registry

```
# Prefix with your Docker Hub username for publishing
docker build -t username/my-flask-app:1.0 .
```

- **Convention:** The format `username/image:tag` is required for pushing to Docker Hub. The `username/` prefix maps to your Docker Hub repository.

Step 5: Re-tag an Existing Image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

- **Command:** `docker tag` creates a **new alias** for an existing image. It does NOT copy the image — both tags point to the same underlying layers.
- **Use case:** Promoting a tested image to a release version without rebuilding.

Part 5: Inspect the Image

View Image Details

```
# List all images
docker images

# Show layer history
docker history my-flask-app

# Full JSON inspection
docker inspect my-flask-app
```

`docker history` **output** reveals each Dockerfile instruction as a layer:

IMAGE	CREATED	CREATED BY	SIZE
82d629e35149	2 minutes ago	CMD ["python" "app.py"]	0B
<missing>	2 minutes ago	EXPOSE [5000/tcp]	0B
<missing>	2 minutes ago	COPY app.py . # buildkit	12.3kB
<missing>	2 minutes ago	RUN pip install --no-cache-dir -r ...	13.4MB
<missing>	2 minutes ago	COPY requirements.txt . # buildkit	12.3kB
<missing>	2 minutes ago	WORKDIR /app	0B

docker inspect output shows comprehensive metadata including:

- **RepoTags:** All tags pointing to this image (`my-flask-app:1.0` , `my-flask-app:latest` , `my-flask-app:v1.0`)
- **ExposedPorts:** `5000/tcp`
- **Cmd:** `["python", "app.py"]`
- **WorkingDir:** `/app`
- **Architecture:** `amd64`
- **Size:** `48796436` bytes (~48.8 MB content size)

Part 6: Run and Manage Containers

Step 1: Run the Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

- **Flags:**
 - `-d` (**Detached**): Runs in the background.
 - `-p 5000:5000` (**Port mapping**): Maps host port 5000 → container port 5000.
 - `--name flask-container`: Assigns a readable name.

Step 2: Verify the Application

```
# Test the endpoint
curl http://localhost:5000
```

- **Expected Output:** `Hello from Docker!`

```
# View running containers
docker ps
```

- **Expected Output:**

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
9cc9edec60e0	my-flask-app	"python app.py"	Up 15 seconds	0.0.0.0:5000→5000/tcp

```
# View container logs
docker logs flask-container
```

- **Expected Output:**

```
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
172.17.0.1 - - [26/Feb/2026 05:01:20] "GET / HTTP/1.1" 200 -
```

Step 3: Container Lifecycle Management

```
# Stop the container (sends SIGTERM)
docker stop flask-container

# Restart a stopped container
docker start flask-container

# Remove a stopped container
docker rm flask-container

# Force remove a running container (sends SIGKILL)
docker rm -f flask-container
```

- **Flags:**
 - **-f (Force):** Skips the stop step and immediately kills the container. Use with caution — this doesn't give the application time to clean up.

Part 7: Multi-Stage Builds

Why Multi-Stage Builds?

In a standard build, build tools (compilers, package managers) remain in the final image even though they're not needed at runtime. Multi-stage builds solve this by using **separate stages** — one for building, one for running.

Step 1: Create the Multi-Stage Dockerfile

Create `Dockerfile.multistage`:

```

# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in a virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user (security best practice)
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]

```

Key instructions explained:

Instruction	Purpose
<code>FROM ... AS builder</code>	Names this stage "builder" so we can reference it later
<code>RUN python -m venv /opt/venv</code>	Creates an isolated virtual environment for dependencies
<code>COPY --from=builder /opt/venv /opt/venv</code>	Copies only the virtual environment from the builder stage into the fresh runtime image — build tools and caches are left behind
<code>RUN useradd -m -u 1000 appuser</code>	Creates a non-root user for security — if the app is compromised, the attacker doesn't have root access

Instruction	Purpose
<code>USER appuser</code>	Switches to the non-root user for all subsequent commands

Step 2: Build and Compare

```
# Build the regular image
docker build -t flask-regular .

# Build the multi-stage image
docker build -f Dockerfile.multistage -t flask-multistage .

# Compare sizes
docker images | grep flask-
```

- **Flag:** `-f Dockerfile.multistage` tells Docker to use a specific Dockerfile instead of the default `Dockerfile`.
- **Expected Output:**

<code>flask-multistage:latest</code>	<code>189MB</code>	<code>46.2MB</code>
<code>flask-regular:latest</code>	<code>200MB</code>	<code>48.8MB</code>

Observation: The multi-stage image is **~5% smaller** in this case. The savings are more dramatic with compiled languages (Go, Java, C++) where the build toolchain can add hundreds of MB.

Part 8: Publishing to Docker Hub

Step 1: Log In to Docker Hub

```
docker login
```

- **Expected Output:** `Authenticating with existing credentials... Login Succeeded`
- **Note:** If no stored credentials, Docker prompts for username and password.

Step 2: Tag for Docker Hub

```
# Tag with your Docker Hub username
docker tag my-flask-app:latest nairp126/my-flask-app:1.0
docker tag my-flask-app:latest nairp126/my-flask-app:latest
```

- **Convention:** The format `<username>/<repository>:<tag>` is mandatory for Docker Hub pushes.

Step 3: Push the Image

```
# Push the versioned tag
docker push nairp126/my-flask-app:1.0

# Push the latest tag
docker push nairp126/my-flask-app:latest
```

- **Expected Output** (first push):

```
The push refers to repository [docker.io/nairp126/my-flask-app]
833f8696a6de: Pushed
38513bd72563: Pushed
...
1.0: digest: sha256:82d629e35149 ... size: 856
```

- **Expected Output** (second push — layers already exist):

```
833f8696a6de: Already exists
3d8e44f4b704: Layer already exists
...
latest: digest: sha256:82d629e35149 ... size: 856
```

Key insight: Docker pushes layers individually. If a layer already exists on the registry, it's skipped — making subsequent pushes near-instant.

Step 4: Pull and Run from Docker Hub

```
# Pull from Docker Hub (simulating another machine)
docker pull nairp126/my-flask-app:latest

# Run the pulled image
docker run -d -p 5000:5000 nairp126/my-flask-app:latest
```

Part 9: Cleanup

```
# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Nuclear option: remove everything unused
docker system prune -a
```

- **Flags:**

- **-a (All):** Removes ALL unused images, not just dangling ones. Use with caution — this will delete cached base images too.



Essential Docker Commands Cheatsheet

Command	Purpose	Example
<code>docker build</code>	Build image from Dockerfile	<code>docker build -t myapp .</code>
<code>docker run</code>	Create and start a container	<code>docker run -d -p 3000:3000 myapp</code>
<code>docker ps</code>	List running containers	<code>docker ps -a</code> (include stopped)
<code>docker images</code>	List local images	<code>docker images</code>
<code>docker tag</code>	Create a new tag for an image	<code>docker tag myapp:latest myapp:v1</code>
<code>docker login</code>	Authenticate with Docker Hub	<code>docker login</code>
<code>docker push</code>	Upload image to registry	<code>docker push username/myapp:1.0</code>
<code>docker pull</code>	Download image from registry	<code>docker pull username/myapp</code>
<code>docker rm</code>	Remove a container	<code>docker rm -f container-name</code>
<code>docker rmi</code>	Remove an image	<code>docker rmi image-name</code>
<code>docker logs</code>	View container output	<code>docker logs -f container-name</code>
<code>docker exec</code>	Run a command inside a container	<code>docker exec -it container sh</code>
<code>docker history</code>	Show image layer history	<code>docker history myapp</code>
<code>docker inspect</code>	Show full image/container metadata	<code>docker inspect myapp</code>



Troubleshooting & Common Pitfalls

1. Build Fails with "failed to resolve source metadata"

- **Error:** `ERROR: failed to solve: python:3.9-slim: failed to resolve source metadata ... lookup registry-1.docker.io: no such host`
- **Cause:** Docker cannot reach Docker Hub (DNS resolution failure or Docker Desktop not running).
- **Fix:** Restart Docker Desktop and ensure you have internet connectivity. Wait for the Docker whale icon to stabilize.

2. "Container name is already in use"

- **Error:** `Conflict. The container name "/flask-container" is already in use`
- **Cause:** A container with that name exists (even if stopped).
- **Fix:** Remove it first: `docker rm flask-container`, or use a different name.

3. "Cannot remove a running container"

- **Error:** `Error response from daemon: cannot remove container "flask-container": container is running`
- **Cause:** You tried `docker rm` on a live container.
- **Fix:** Stop it first (`docker stop flask-container`), then remove. Or force: `docker rm -f flask-container`.

4. "Tag does not exist" when pushing

- **Error:** `tag does not exist: nairp126/my-flask-app:latest`
- **Cause:** You tried to push an image that hasn't been tagged with your Docker Hub username prefix.
- **Fix:** Tag it first: `docker tag my-flask-app:latest nairp126/my-flask-app:latest`

5. Application returns "Connection refused" after `docker run`

- **Cause:** Flask is bound to `127.0.0.1` (localhost only), which is unreachable from outside the container.
- **Fix:** Ensure `app.run(host='0.0.0.0', port=5000)` uses `0.0.0.0` to listen on all interfaces.

Visuals & Outputs

```

pranav@Pranav: ~/my-flask-a
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Thu Feb 26 04:49:01 UTC 2026

System load:  2.38               Processes:    40
Usage of /:   1.0% of 1006.85GB   Users logged in: 2
Memory usage: 33%              IPv4 address for eth0: 192.168.206.154
Swap usage:   0%

This message is shown once a day. To disable it please create the
/home/pranav/.hushlogin file.
pranav@Pranav:~$ mkdir my-flask-app
cd my-flask-app
pranav@Pranav:~/my-flask-app$ nano app.py
pranav@Pranav:~/my-flask-app$ nano requirements.txt
pranav@Pranav:~/my-flask-app$ nano Dockerfile
pranav@Pranav:~/my-flask-app$ nano .dockerignore
pranav@Pranav:~/my-flask-app$ docker build -t my-flask-app .
[+] Building 0.7s (2/2) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 361B                             0.0s
=> ERROR [internal] load metadata for docker.io/library/python:3.9-slim 0.2s

> [internal] load metadata for docker.io/library/python:3.9-slim:
Dockerfile:2
-----
1 | # Use Python base image
2 | >>> FROM python:3.9-slim
3 |
4 | # Set working directory

ERROR: failed to build: failed to solve: python:3.9-slim: failed to resolve source metadata for docker.io/library/python:3.9-slim: failed to do request: Head "https://registry-1.docker.io/v2/library/python/manifests/3.9-slim": dialing registry-1.docker.io:443 container via direct connection because Docker Desktop has no HTTPS proxy: connecting to registry-1.docker.io:443: dial tcp: lookup registry-1.docker.io: no such host
pranav@Pranav:~/my-flask-app$ docker build -t my-flask-app .
[+] Building 69.0s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 361B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 6.4s
=> [auth] library/python:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.1s

```

```

pranav@Pranav: ~/my-flask-a
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 274B                                  0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13c 44.6s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13c 0.1s
=> sha256:fc74430849022d13b0dd4b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB 13.9s
=> sha256:ea56f685404ad8f1680d322f152d2cfe62115b30dda481c2c450078315beb508 251B / 251B 1.6s
=> sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB 5.9s
=> sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB 40.4s
=> extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 2.0s
=> extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 0.3s
=> extracting sha256:fc74430849022d13b0dd4b8969a953f842f59c6e9d1a0c2c83d710affa286c08 1.4s
=> extracting sha256:ea56f685404ad8f1680d322f152d2cfe62115b30dda481c2c450078315beb508 0.2s
=> [internal] load build context                                0.1s
=> => transferring context: 321B                                  0.0s
=> [2/5] WORKDIR /app                                         0.6s
=> [3/5] COPY requirements.txt .                               0.4s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 14.4s
=> [5/5] COPY app.py .                                        0.1s
=> exporting to image                                         1.9s
=> => exporting layers                                           1.3s
=> => exporting manifest sha256:5bf4e44f225ffdb4a342ad33e9863019248b09af03448971713aa4b46c356cbc 0.0s
=> => exporting config sha256:74e8dbe520b591d7243c597212155201c4c0f81cd9177cf688bfabf6d2245fe 0.0s
=> => exporting attestation manifest sha256:bb2ce0940a7a133a7ee35a267afa07b3ed71d58d8b086367fb33278b2c07612d 0.0s
=> => exporting manifest list sha256:072bcc2f4f2a75f7e02f0f5729c2ed211f673978e6f69afb2fbddb770d8eb64a 0.0s
=> => naming to docker.io/library/my-flask-app:latest          0.0s
=> => unpacking to docker.io/library/my-flask-app:latest       0.5s
pranav@Pranav:~/my-flask-app$ docker images

IMAGE                                ID                                DISK USAGE  Info →  U  In Use
docker/desktop-kubernetes:kubernetes-v1.34.1-cni-v1.7.1-critools-v1.33.0-cri-dockerd-v0.3.20-1-debian 12d6673564e0 592MB      185MB
docker/desktop-storage-provisioner:v2.0 115d77efe6e2 59.2MB     17.3MB
docker/desktop-storage-provisioner:v3.0 57d2b6ad1c6f 80.3MB     23.9MB  U
docker/desktop-vpnkit-controller:dc331cb22850be0cdd97c84a9cfecaf44a1afb6e 7ecf567ea070 47MB       10.8MB
docker/desktop-vpnkit-controller:v4.0 bdaff3408b1c 52.6MB     11.9MB  U
dpape/pgadmin4:latest 50700ac17936 823MB      185MB
elasticsearch:8.8.0 5c28849be5e9 1.98GB     641MB
grafana/grafana:latest ba93c9d192e5 995MB      211MB  U
java-app:1.0 aeab387b556e 1.3GB      363MB
my-flask-app:latest 072bcc2f4f2a 200MB      48.8MB
neo4j:5.16-community 4c11171dd0db 1.29GB     360MB
nginx-alpine:latest d661e47411af 802MB      294MB  U
nginx-ubuntu:latest aba85e8e3d4b 16MB       4.74MB
nginx:latest 36bec77fa9b0 199MB      52.7MB
postgres:15 341bf0f3cee6c 240MB      65.8MB
postgres:15 a7711af921c3 633MB      164MB

```

```

pranav@Pranav: ~/my-flask-a
postgres:15-alpine                d326a0ce8bce      392MB      110MB      U
postgres:16-alpine                97ff59a4e30e      395MB      111MB      U
prom/alertmanager:latest          cc54cc450174      119MB      39.1MB     U
prom/prometheus:latest           1f0f50f06aca      503MB      134MB      U
redis:7-alpine                    8b01dd377f02      61.2MB     17.7MB     U
registry.k8s.io/coredns/coredns:v1.12.1  e8c262566636     101MB      22.4MB     U
registry.k8s.io/etcd:3.6.4-0      e36c00168342     273MB      74.3MB     U
registry.k8s.io/kube-apiserver:v1.34.1  b9d7c117f8ac     118MB      27.1MB     U
registry.k8s.io/kube-controller-manager:v1.34.1  2b47c1b01f5      101MB      22.8MB     U
registry.k8s.io/kube-proxy:v1.34.1     913cc83ca0b5     102MB      26MB       U
registry.k8s.io/kube-scheduler:v1.34.1  6e9fbc4e25a5     73.5MB     17.4MB     U
registry.k8s.io/pause:3.10          ee6521f290b2     1.06MB     318kB      U
registry.k8s.io/pause:3.10.1         278fb9dbcca9     1.06MB     318kB      U
secure_wipe-api-gateway:latest       c0067f9ff201     936MB      240MB      U
secure_wipe-audit-service:latest     8a9adf70579e     905MB      234MB      U
secure_wipe-certificate-service:latest 60cc74252179     947MB      242MB      U
secure_wipe-integration-service:latest ae2028f9e150     957MB      245MB      U
secure_wipe-pdf-service:latest       18f788d0dc59     983MB      252MB      U
secure_wipe-pdf-worker:latest        68879411d065     983MB      252MB      U
secure_wipe-signature-service:latest 9152abdb6616     925MB      239MB      U
secure_wipe-tenant-management:latest 94ecbdf4d4fe     971MB      247MB      U
secure_wipe-verification-service:latest 7eb454f537be     906MB      234MB      U
secure_wipe-workflow-service:latest 523047e0f881     918MB      235MB      U
ubuntu:latest                      cd1dba651b30     119MB      31.7MB     U

pranav@Pranav:~/my-flask-app$ docker build -t my-flask-app:1.0 .
[*] Building 6.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 274B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5bf4e44f225ffdb4a342ad33e9863019248b09af03448971713aa4b46c356cbc
=> => exporting config sha256:74e8dbe520b591d7243c597212155201c4c0f81cd91777cf688bfabf6d2245fe
=> => exporting attestation manifest sha256:bd0c8d64ee11ec4b0edb5cd5ec6f209545db2cf29b262b284966e4fc62a61bb
=> => exporting manifest list sha256:56ca81266d677d3c177c2e76ac36ae7f09b39fdec2c1329a7e181fab4e3349
=> => naming to docker.io/library/my-flask-app:1.0
=> => unpacking to docker.io/library/my-flask-app:1.0

```

```

pranav@Pranav: ~/my-flask-a
pranav@Pranav:~/my-flask-app$ docker build -t my-flask-app:latest -t my-flask-app:1.0 .
[*] Building 2.3s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 274B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5bf4e44f225ffdb4a342ad33e9863019248b09af03448971713aa4b46c356cbc
=> => exporting config sha256:74e8dbe520b591d7243c597212155201c4c0f81cd91777cf688bfabf6d2245fe
=> => exporting attestation manifest sha256:752aa307a8c34dd0e39d2f10c6d4df1bb06af7418864b9623ee53a7ec195434a
=> => exporting manifest list sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac855eaddfd8
=> => naming to docker.io/library/my-flask-app:latest
=> => unpacking to docker.io/library/my-flask-app:latest
=> => naming to docker.io/library/my-flask-app:1.0
=> => unpacking to docker.io/library/my-flask-app:1.0

pranav@Pranav:~/my-flask-app$ docker build -t username/my-flask-app:1.0 .
[*] Building 2.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 274B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5bf4e44f225ffdb4a342ad33e9863019248b09af03448971713aa4b46c356cbc
=> => exporting config sha256:74e8dbe520b591d7243c597212155201c4c0f81cd91777cf688bfabf6d2245fe
=> => exporting attestation manifest sha256:d0d0749d45a32bf094331dd770897fb09fee15b1ec186800ef6a351ca8178547
=> => exporting manifest list sha256:1df4554c81f21676f84f18a3cfb68b65435c5dd1d4ac31b058f2f358e4987bb3
=> => naming to docker.io/username/my-flask-app:1.0

```

```

pranav@Pranav: ~/my-flask-a  x  +  -
=> => unpacking to docker.io/username/my-flask-app:1.0 0.0s
pranav@Pranav:~/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
pranav@Pranav:~/my-flask-app$ docker images

IMAGE                                         ID                               DISK USAGE  Info + U In Use
docker/desktop-kubernetes:kubernetes-v1.34.1-cni-v1.7.1-critools-v1.33.0-cni-dockerd-v0.3.20-1-debian 12d6673564e0 592MB 185MB
docker/desktop-storage-provisioner:v2.0      115d77ef6e2 59.2MB 17.3MB
docker/desktop-storage-provisioner:v3.0      57d2b6ad1c6f 80.3MB 23.9MB U
docker/desktop-vpnkit-controller:dc331cb22850be0cdd97c84a9cfecaf44a1afb66 7ecf567ea070 47MB 10.8MB
docker/desktop-vpnkit-controller:v4.0       bdaff3408b1c 52.6MB 11.9MB U
dpag/pgadmin4:latest                         50700ac17936 823MB 185MB
elasticsearch:8.8.0                         5c28849be5e9 1.98GB 641MB
grafana/grafana:latest                       ba93c9d192e5 995MB 211MB U
java-app:1.0                                aeab387b556e 1.3GB 363MB
my-flask-app:1.0                             82d629e35149 200MB 48.8MB
my-flask-app:latest                          82d629e35149 200MB 48.8MB
my-flask-app:v1.0                           82d629e35149 200MB 48.8MB
myrepo/java:1.0                             41c1171dd0db 1.29GB 360MB
neo4j:5.16-community                         d661e47411af 802MB 294MB U
nginx-alpine:latest                         aba85e8e3dbd 16MB 4.74MB
nginx-ubuntu:latest                         36bec77fa9b0 199MB 52.7MB
nginx:latest                                341bf0f3ce6c 240MB 65.8MB
postgres:15                                 a7711af921c3 633MB 164MB
postgres:15-alpine                          d326a0ce8bce 392MB 110MB U
postgres:16-alpine                          97ff59a4e30e 395MB 111MB U
prom/alertmanager:latest                    cc54cc450174 119MB 39.1MB U
prom/prometheus:latest                      1f0f50f06aca 503MB 134MB U
redis:7-alpine                              8b81dd37ff02 61.2MB 17.7MB U
registry.k8s.io/coredns/coredns:v1.12.1    e8c262566636 101MB 22.4MB U
registry.k8s.io/etcd:3.6.4-0                e36e081683d2 273MB 74.3MB U
registry.k8s.io/kube-apiserver:v1.34.1      b9d7c117f8ac 118MB 27.1MB U
registry.k8s.io/kube-controller-manager:v1.34.1 2bf47c1b01f5 101MB 22.8MB U
registry.k8s.io/kube-proxy:v1.34.1          913cc83ca0b5 102MB 26MB U
registry.k8s.io/kube-scheduler:v1.34.1      6e9fbc4e25a5 73.5MB 17.4MB U
registry.k8s.io/pause:3.10                  ee6521f290b2 1.06MB 318kB U
registry.k8s.io/pause:3.10.1                278fb9dbcca9 1.06MB 318kB U
secure_wipe-api-gateway:latest              c0067f9ff201 936MB 240MB U
secure_wipe-audit-service:latest            8a9adf70579e 905MB 234MB U
secure_wipe-certificate-service:latest       60cc74252179 947MB 242MB U
secure_wipe-integration-service:latest       ae2028f9e150 957MB 245MB U
secure_wipe-pdf-service:latest              18f788d0dc59 983MB 252MB U
secure_wipe-pdf-worker:latest                68879411d065 983MB 252MB U
secure_wipe-signature-service:latest         9152abdb6616 925MB 239MB U
secure_wipe-tenant-management:latest         94ecbdf44fe1 971MB 247MB U
secure_wipe-verification-service:latest      7eb454f537be 906MB 234MB U

```

```

pranav@Pranav: ~/my-flask-a  x  +  -
pranav@Pranav:~/my-flask-app$ docker history my-flask-app
IMAGE      CREATED          CREATED BY          SIZE      COMMENT
82d629e35149 2 minutes ago   CMD ["python" "app.py"] 0B        buildkit.dockerfile.v0
<missing>    2 minutes ago   EXPOSE [5000/tcp]      0B        buildkit.dockerfile.v0
<missing>    2 minutes ago   COPY app.py . # buildkit 12.3kB    buildkit.dockerfile.v0
<missing>    2 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r... 13.4MB    buildkit.dockerfile.v0
<missing>    2 minutes ago   COPY requirements.txt . # buildkit 12.3kB    buildkit.dockerfile.v0
<missing>    2 minutes ago   WORKDIR /app           8.19kB    buildkit.dockerfile.v0
<missing>    3 months ago   CMD ["python3"]        0B        buildkit.dockerfile.v0
<missing>    3 months ago   RUN /bin/sh -c set -eux; for src in idle3 p... 16.4kB    buildkit.dockerfile.v0
<missing>    3 months ago   RUN /bin/sh -c set -eux; savedAptMark="$({a... 45.7MB    buildkit.dockerfile.v0
<missing>    3 months ago   ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8... 0B        buildkit.dockerfile.v0
<missing>    3 months ago   ENV PYTHON_VERSION=3.9.25 0B        buildkit.dockerfile.v0
<missing>    3 months ago   RUN GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3... 0B        buildkit.dockerfile.v0
<missing>    3 months ago   RUN /bin/sh -c set -eux; apt-get update; a... 4.94MB    buildkit.dockerfile.v0
<missing>    3 months ago   ENV LANG=C.UTF-8       0B        buildkit.dockerfile.v0
<missing>    3 months ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... 0B        buildkit.dockerfile.v0
<missing>    4 months ago   # debian.sh --arch 'amd64' out/ 'trixie' '01... 87.4MB    debuerreotype 0.16

pranav@Pranav:~/my-flask-app$ docker inspect my-flask-app
[
  {
    "Id": "sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddff8",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0"
    ],
    "RepoDigests": [
      "my-flask-app@sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddff8"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-02-26T04:58:29.774318157Z",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "Cmd": [
        "python",
        "app.py"
      ]
    }
  ]
}

```

```
pranav@Pranav: ~/my-flask-x$ curl http://localhost:5000
{
  "Cmd": [
    "python",
    "app.py"
  ],
  "WorkingDir": "/app",
  "ArgsEscaped": true
},
{
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 48796436,
  "RootFS": {
    "Type": "layers",
    "Layers": [
      "sha256:d7c97cb6f1fe7cae982649e9f55efe201212e8acaa64bd668c083b204e4efd4c",
      "sha256:4f237755fbae411472cb9e0f874fa97abb8d684f8fdcc6bd485edc8b6267a9c93",
      "sha256:298992e09a0315a8f26375a553f216f5ad343decf392956fd2de5ef7aa1f8dfea",
      "sha256:cbf6b54339a8b4u971408bd33dd69adc7db2ed9c9dbf7c533ba09efecda5f33f",
      "sha256:1ff70ac892d22558bd69c6d1b1791d27c15c6782a88fa42b6fa8863a96821de",
      "sha256:adf656bb398b21c52b39ae9f59e0f860f7804f391f5c85e2883c21uu",
      "sha256:258ec41f3394d4241c465b-930a4700e1b3c39a56063578586a575f3d2c1279",
      "sha256:80b4ae03af191a41c4dce46fb72339535db1520217d094ef42d6dae8340bfa"
    ]
  },
  "Metadata": {
    "LastTagTime": "2026-02-26T05:00:09.983498877Z"
  },
  "Descriptor": {
    "mediaType": "application/vnd.oci.image.index.v1+json",
    "digest": "sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddff8",
    "size": 856
  },
  "Identity": {
    "Build": [
      {
        "Ref": "g6bnox0z4lnwqbofctz2grhvj",
        "CreatedAt": "2026-02-26T04:59:55.964965184Z"
      }
    ]
  }
}
}

pranav@Pranav:~/my-flask-app$ docker run -d -p 5000:5000 --name flask-container my-flask-app
9cc9edec60e0e05f568c99fa181ba9d804a61ed439a4103dad906c11438b116
pranav@Pranav:~/my-flask-app$ curl http://localhost:5000
```

```

pranav@Pranav:~/my-flask-app$ docker ps
Hello from Docker!
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
9cc9edec60e0   my-flask-app                        "python app.py"         15 seconds ago   Up 15 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
bb087e9d3fe7   redis:7-alpine                     "docker-entrypoint.s..." 34 hours ago     Up 34 hours (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
bd2cf2c080a3   postgres:16-alpine                 "docker-entrypoint.s..." 34 hours ago     Up 34 hours (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
952ef83308e1   neo4j:5.16-community               "tiny -g --/startu..." 34 hours ago     Up 34 hours (healthy)   0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0
:7687->7687/tcp, [::]:7687->7687/tcp
pranav@Pranav:~/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [26/Feb/2026 05:01:20] "GET / HTTP/1.1" 200 -
pranav@Pranav:~/my-flask-app$ docker stop flask-container
flask-container
pranav@Pranav:~/my-flask-app$ docker start flask-container
flask-container
pranav@Pranav:~/my-flask-app$ docker rm flask-container
Error response from daemon: cannot remove container "flask-container": container is running: stop the container before removing or force remove
pranav@Pranav:~/my-flask-app$ docker rm -f flask-container
flask-container
pranav@Pranav:~/my-flask-app$ nano Dockerfile
Command 'nano' not found, did you mean:
  command 'nano' from snap nano (7.2-pkg-4057)
  command 'nano' from deb nano (7.2-ubuntu0.1)
See 'snap info <snapname>' for additional versions.
pranav@Pranav:~/my-flask-app$ nano Dockerfile
pranav@Pranav:~/my-flask-app$ nano Dockerfile
pranav@Pranav:~/my-flask-app$ mv Dockerfile Dockerfile.multistage
pranav@Pranav:~/my-flask-app$ nano Dockerfile.multistage
pranav@Pranav:~/my-flask-app$ docker build -t flask-regular .
[+] Building 3.6s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 3.0s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 274B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slimsha256:2d97f6910b16bd338d3060f261f53f144965f755599aabbacdale1e3cf 0.0s
=> => resolve docker.io/library/python:3.9-slimsha256:2d97f6910b16bd338d3060f261f53f144965f755599aabbacdale1e3cf 0.0s

```

```

pranav@Pranav: ~/my-flask-a
=> [internal] load build context                                0.0s
=> => transferring context: 638                                0.0s
=> CACHED [2/5] WORKDIR /app                                  0.0s
=> CACHED [3/5] COPY requirements.txt .                       0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                 0.0s
=> exporting to image                                         0.3s
=> => exporting layers                                         0.0s
=> => exporting manifest sha256:5b74e44f225ffdb4a342ad33e9863019248b09af03448971713aa4b46c356cbc 0.0s
=> => exporting config sha256:74e8dbe520b591d7243c597212155201c4c0f81cd91777cf688bfabf6d2245fe 0.0s
=> => exporting attestation manifest sha256:266b96b107488c192505f65a5281a7a3b31a3e9735de986f508b83f0527a29e5 0.1s
=> => exporting manifest list sha256:dbcb76541145fc6a8d03debb77b3e4963f09cc159d6af84ee0897ec8f6c7d6a3 0.0s
=> => naming to docker.io/library/flask-regular:latest        0.0s
=> => unpacking to docker.io/library/flask-regular:latest      0.0s
pranav@Pranav:~/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 39.1s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile.multistage 0.0s
=> => transferring dockerfile: 675B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 274B                                   0.0s
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablac 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 638                                    0.0s
=> CACHED [builder 2/5] WORKDIR /app                          0.0s
=> CACHED [builder 3/5] COPY requirements.txt .                0.0s
=> [builder 4/5] RUN python -m venv /opt/venv                  9.0s
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt 21.9s
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv       0.8s
=> [stage-1 4/5] COPY app.py .                                 0.1s
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser                 0.7s
=> exporting to image                                         3.6s
=> => exporting layers                                         2.3s
=> => exporting manifest sha256:a65a738c681943d434d01dfc5549338ce140eac0338d3fc048ceca4a3b34e83 0.0s
=> => exporting config sha256:0278a422e2bd7255cfc54a846cf7da93635324b404e7482277a5a539f585477c3 0.0s
=> => exporting attestation manifest sha256:799a841aa9a386334a378a2b53240d1f92d1a1a71a13287578d04e0ab1697407 0.1s
=> => exporting manifest list sha256:bff49156c9b2ea3c64b99d1ee9bda4b05a47661a3aa9ede757c852f377d7013d 0.0s
=> => naming to docker.io/library/flask-multistage:latest      0.0s
=> => unpacking to docker.io/library/flask-multistage:latest    1.0s
pranav@Pranav:~/my-flask-app$ docker images | grep flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      bff49156c9b2      219MB      52.1MB
flask-regular:latest        dcb76541145       200MB      48.8MB
my-flask-app:1.0            82d629e35149      200MB      48.8MB
my-flask-app:latest         82d629e35149      200MB      48.8MB
my-flask-app:v1.0           82d629e35149      200MB      48.8MB

```

```

pranav@Pranav: ~/my-flask-a
pranav@Pranav:~/my-flask-app$ docker images | grep flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      bff49156c9b2      219MB      52.1MB
flask-regular:latest        dcb76541145       200MB      48.8MB
my-flask-app:1.0            82d629e35149      200MB      48.8MB
my-flask-app:latest         82d629e35149      200MB      48.8MB
my-flask-app:v1.0           82d629e35149      200MB      48.8MB
username/my-flask-app:1.0    1df4554c81f2      200MB      48.8MB
pranav@Pranav:~/my-flask-app$ nano Dockerfile.multistage
pranav@Pranav:~/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 17.2s (12/12) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile.multistage 0.0s
=> => transferring dockerfile: 382B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.6s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 274B                                   0.0s
=> [builder 1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablac 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf 0.1s
=> [internal] load build context                                0.0s
=> => transferring context: 638                                    0.0s
=> CACHED [builder 2/4] WORKDIR /app                          0.0s
=> CACHED [builder 3/4] COPY requirements.txt .                0.0s
=> [builder 4/4] RUN pip install --no-cache-dir --prefix=/install -r requirements.txt 12.5s
=> [stage-1 3/5] COPY --from=builder /install /usr/local       0.3s
=> [stage-1 4/5] COPY app.py .                                 0.1s
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser                 0.7s
=> exporting to image                                         0.9s
=> => exporting layers                                         0.6s
=> => exporting manifest sha256:d7bc72e0d6c1514c106722afcb89f2b8424d520a6575965ed72c1a02b63f4596 0.0s
=> => exporting config sha256:a5a86312847077cb121a4086f476b8236d383cc8c482c9878ab48094fae47e1b 0.0s
=> => exporting attestation manifest sha256:a40bfff27d5f90a2cbfe0ad79e35b00d2440d8e7f41f737ef5feca2988b27e1b1 0.0s
=> => exporting manifest list sha256:90abb1ec155fa1cd01b6cb26d6aff3a08635ab9d206862d9ba6d7b8d8f52a203 0.0s
=> => naming to docker.io/library/flask-multistage:latest      0.0s
=> => unpacking to docker.io/library/flask-multistage:latest    0.2s
pranav@Pranav:~/my-flask-app$ docker images | grep flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      90abb1ec155f      189MB      46.2MB
flask-regular:latest        dcb76541145       200MB      48.8MB
my-flask-app:1.0            82d629e35149      200MB      48.8MB
my-flask-app:latest         82d629e35149      200MB      48.8MB
my-flask-app:v1.0           82d629e35149      200MB      48.8MB
username/my-flask-app:1.0    1df4554c81f2      200MB      48.8MB
pranav@Pranav:~/my-flask-app$ docker login
Authenticating with existing credentials... [Username: nairp126]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

```

```

pranav@Pranav: ~/my-flask-a x + v
pranav@Pranav:~/my-flask-app$ docker login
Authenticating with existing credentials... [Username: nairp126]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
pranav@Pranav:~/my-flask-app$ docker push nairp126/my-flask-app:latest
The push refers to repository [docker.io/nairp126/my-flask-app]
tag does not exist: nairp126/my-flask-app:latest
pranav@Pranav:~/my-flask-app$ docker tag my-flask-app:latest nairp126/my-flask-app:1.0
pranav@Pranav:~/my-flask-app$ docker tag my-flask-app:latest nairp126/my-flask-app:latest
pranav@Pranav:~/my-flask-app$ docker push nairp126/my-flask-app:1.0
The push refers to repository [docker.io/nairp126/my-flask-app]
833f8696a6de: Pushed
38513bd72563: Pushed
fc7443084902: Pushed
f125f346e945: Pushed
ea56f685404a: Pushed
bfebe46ca287: Pushed
d11fd03ad816: Pushed
b3ec39b36ae8: Pushed
3d8e44f4b704: Pushed
1.0: digest: sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddf8 size: 856
pranav@Pranav:~/my-flask-app$ docker push nairp126/my-flask-app:latest
The push refers to repository [docker.io/nairp126/my-flask-app]
833f8696a6de: Already exists
3d8e44f4b704: Layer already exists
f125f346e945: Layer already exists
38513bd72563: Layer already exists
ea56f685404a: Layer already exists
bfebe46ca287: Layer already exists
d11fd03ad816: Layer already exists
fc7443084902: Layer already exists
b3ec39b36ae8: Layer already exists
latest: digest: sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddf8 size: 856
pranav@Pranav:~/my-flask-app$ docker pull nairp126/my-flask-app:latest
latest: Pulling from nairp126/my-flask-app
Digest: sha256:82d629e35149717d9b033db6499c4fb220e697d717b029334faac8556eaddf8
Status: Image is up to date for nairp126/my-flask-app:latest
docker.io/nairp126/my-flask-app:latest
pranav@Pranav:~/my-flask-app$ docker run -d -p 5000:5000 nairp126/my-flask-app:latest
e27bbb25a5bf43cde3b69edb4f820a8211f5fa11f4bf9acab6e35a1f78cf83c
pranav@Pranav:~/my-flask-app$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] n
pranav@Pranav:~/my-flask-app$ docker image prune

```

Student: Pranav R Nair | **Batch:** 2(CCVT) | **SAP ID:** 500121466

Objective

To understand and demonstrate Docker's four operational pillars — **persistent data storage** (volumes & bind mounts), **runtime configuration** (environment variables), **observability** (stats, logs, events), and **container-to-container communication** (custom bridge networks) — by building and monitoring a multi-service application stack.

Student Information

Attribute	Details
Name	Pranav R Nair
SAP ID	500121466
Roll No	R2142230197
Batch	2(CCVT)

Part 1: Docker Volumes — Persistent Data Storage

Lab 1: Understanding Container Data Lifecycle

The Problem: Container Data is Ephemeral

By default, all files created inside a container live in its **writable layer**. This layer is tied to the container — if you `docker rm` it, the data is gone forever.

Important Nuance: Data survives `docker stop` and `docker start` — it's only destroyed on `docker rm`. This is a common misconception.

Step 1: Create a container and write data

```
docker run -d --name test-container ubuntu sleep 3600
```

Flag	Purpose
<code>-d</code>	Detached mode — runs in the background so we can exec into it

Flag	Purpose
<code>--name test-container</code>	Assigns a human-readable name instead of a random hash
<code>sleep 3600</code>	Keeps the container alive for 1 hour (ubuntu would exit immediately without a long-running process)

```
# Create directory and write a file inside the running container
docker exec test-container bash -c "mkdir -p /data && echo 'Hello World' > /data/message.txt

# Verify the file exists
docker exec test-container cat /data/message.txt
```

Expected Output:

```
Hello World
```

Flag	Purpose
<code>docker exec</code>	Runs a command inside a running container (does not start a new one)
<code>bash -c "..."</code>	Runs a compound shell command (mkdir + echo) in one exec call
<code>-p</code> (in <code>mkdir -p</code>)	Creates parent directories as needed — won't error if <code>/data</code> doesn't exist

Step 2: Stop, remove, and re-create — data is gone

```
# Stop the container (data SURVIVES this step)
docker stop test-container

# Remove the container (data is DESTROYED here)
docker rm test-container

# Start a new container with the same name
docker run -d --name test-container ubuntu sleep 3600

# Try to read the file
docker exec test-container cat /data/message.txt
```

Expected Output:

```
cat: /data/message.txt: No such file or directory
```

Deep Dive — Why? Every container has a thin **writable layer** on top of the image's read-only layers (Union Filesystem). When you `docker rm`, Docker deletes that writable layer and all its contents. The new container starts from a fresh writable layer — it has no memory of the old container's files.

Solution: Docker Volumes — Store data outside the container's writable layer so it persists independently.

Screenshot — Demonstrating ephemeral data loss after `docker rm`:

```
pranav@Pranav: ~$ docker run -d --name test-container ubuntu sleep 3600
5548b627523e022f34d38d8913a0139b6001ff94c5a6f29a00a996bfa2fe5d69
pranav@Pranav: ~$ docker exec test-container bash -c "mkdir -p /data && echo 'Hello World' > /data/message.txt"
pranav@Pranav: ~$ docker exec test-container cat /data/message.txt
Hello World
pranav@Pranav: ~$ docker stop test-container
docker rm test-container
test-container
test-container
pranav@Pranav: ~$ docker run -d --name test-container ubuntu sleep 3600
f8ab127e6a52ade06724af7c2496f5a3b6dd9482516fbb6d021456bd7decc015
pranav@Pranav: ~$ docker exec test-container cat /data/message.txt
cat: /data/message.txt: No such file or directory
pranav@Pranav: ~$ mkdir -p ~/nginx-config
pranav@Pranav: ~$ cat << 'EOF' > ~/nginx-config/nginx.conf
server {
    listen 80;
    server_name localhost;
    location / {
        default_type text/plain;
        return 200 "Hello from mounted config!";
    }
}
EOF
pranav@Pranav: ~$ docker run -d --name nginx-custom -p 8080:80 -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf nginx
c6bc09872afb91f5eccc6336d085013aaec46bc5085f7bc3df6bf5d70604ded8d8
pranav@Pranav: ~$ curl http://localhost:8080
Hello from mounted config!
pranav@Pranav: ~$ docker rm -f nginx-custom
nginx-custom
pranav@Pranav: ~$ docker run -d --name flask-env-lab -p 5000:5000 -v ~/flask-env-lab:/app flask-env-lab
flask-env-lab
pranav@Pranav: ~$ cd ~/flask-env-lab && cat << 'EOF' > app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return jsonify({
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    })
```

⚡ Lab 2: Volume Types

Docker provides three types of storage mounts:

Type	Managed By	Location	Persists After <code>docker rm</code> ?	Best For
Anonymous Volume	Docker	<code>/var/lib/docker/volumes/<hash></code>	Yes (but hard to find)	Throwaway data
Named Volume	Docker	<code>/var/lib/docker/volumes/<name></code>	Yes ☒	Databases, production data
Bind Mount	You (Host OS)	Any host directory	Yes ☒	Development, config files

1. Anonymous Volumes

```
docker run -d -v /app/data --name web1 nginx
```

Flag	Purpose
<code>-v</code> <code>/app/data</code>	Creates an anonymous volume mounted at <code>/app/data</code> inside the container. Since no name is given, Docker generates a random hash as the volume name.

```
# Check the volume – you'll see a long hex hash as the name
docker volume ls
```

```
# See exactly where it's mounted
docker inspect web1 | grep -A 5 Mounts
```

Problem with anonymous volumes: They're hard to identify and reuse. You can't easily attach a new container to the same anonymous volume because you'd need to know the random hash.

2. Named Volumes (Recommended for Production)

```
# Create a named volume explicitly
docker volume create mydata

# Attach it to a container
docker run -d -v mydata:/app/data --name web2 nginx
```

Flag	Purpose
<code>-v</code> <code>mydata:/app/data</code>	Mounts the named volume <code>mydata</code> at <code>/app/data</code> inside the container. If <code>mydata</code> doesn't exist, Docker auto-creates it.

```
# List volumes – you'll see "mydata" clearly
docker volume ls

# See metadata: mountpoint, driver, creation date
docker volume inspect mydata
```

3. Bind Mounts (Map Host Directory → Container)

```
# Create a directory on your host machine
mkdir ~/myapp-data

# Mount it directly into the container
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
```

Flag	Purpose
------	---------

<code>-v ~/myapp-data:/app/data</code>	Maps the host directory <code>~/myapp-data</code> into the container at <code>/app/data</code> . Changes in either location are reflected instantly in the other.
--	--

```
# Write from the HOST
echo "From Host" > ~/myapp-data/host-file.txt

# Read from INSIDE the container
docker exec web3 cat /app/data/host-file.txt
```

Expected Output:

```
From Host
```

Deep Dive — Bind Mounts for Development: Bind mounts are the backbone of Docker development workflows. You edit code on your host with your IDE, and the container serves it instantly — no rebuild needed. This is why `docker run -v $(pwd):/app` is so common in development.

⚡ Lab 3: Practical Volume Examples**Example 1: Database with Persistent Storage**

```
# Start MySQL with a named volume for data persistence
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

Flag	Purpose
------	---------

<code>-v mysql-data:/var/lib/mysql</code>	MySQL stores all database files in <code>/var/lib/mysql</code> . By mounting a named volume here, the database survives container removal.
---	--

Flag	Purpose
<code>-e</code> <code>MYSQL_ROOT_PASSWORD=secret</code>	Required environment variable — MySQL refuses to start without a root password.

```
# Simulate a crash: stop and delete the container
docker stop mysql-db
docker rm mysql-db

# Start a brand new container with the SAME volume
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
# All databases, tables, and rows from the old container are intact!
```

Why this works: The volume `mysql-data` is independent of any container. Removing `mysql-db` deletes the container but NOT the volume. The new container simply reconnects to the same storage.

Example 2: Web Server with Custom Configuration (Bind Mount)

```
# Create the config directory
mkdir -p ~/nginx-config

# Write a custom NGINX configuration
cat << 'EOF' > ~/nginx-config/nginx.conf
server {
    listen 80;
    server_name localhost;
    location / {
        default_type text/plain;
        return 200 "Hello from mounted config!";
    }
}
EOF
```

Fix Applied: The `default_type text/plain;` directive was added. Without it, NGINX returns the response with a default `application/octet-stream` content type, causing browsers and `curl` to attempt a file download instead of displaying the text.

```
# Run NGINX with the config file bind-mounted
docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
```

Flag	Purpose
<code>-p 8080:80</code>	Maps host port <code>8080</code> to container port <code>80</code> (NGINX's default listening port)
<code>-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf</code>	Replaces NGINX's default site config with our custom file. Changes to the host file take effect on <code>nginx -s reload</code> .

```
# Test – should display the text, not download a file
curl http://localhost:8080
```

Expected Output:

```
Hello from mounted config!
```

🔥 Lab 4: Volume Management Commands

```
# List all volumes
docker volume ls

# Create a named volume explicitly
docker volume create app-volume

# View detailed volume metadata (mountpoint, driver, labels)
docker volume inspect app-volume

# Remove ALL volumes not attached to any container (DANGEROUS in production)
docker volume prune

# Remove a specific volume by name
docker volume rm app-volume

# Copy a file from host into a running container's volume
docker cp local-file.txt container-name:/path/in/volume/
```

Common Pitfall: `docker volume rm` will fail if any container (even a stopped one) is using the volume. Remove the container first with `docker rm`, then the volume.

Part 2: Environment Variables

⚡ Lab 1: Setting Environment Variables at Runtime

Environment variables are the standard mechanism for configuring containers without rebuilding images. They follow the **12-Factor App** methodology.

Method 1: Using `-e` Flag (Inline)

```
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app
```

Flag	Purpose
<code>-e KEY=VALUE</code>	Sets a single environment variable inside the container. Can be repeated for multiple variables.

Method 2: Using `--env-file` (File-Based)

```
# Create a .env file
cat << 'EOF' > .env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
EOF

# Pass everything in the file at once
docker run -d \
  --env-file .env \
  --name app2 \
  my-app
```

Flag	Purpose
<code>--env-file .env</code>	Loads ALL key-value pairs from the file. Keeps secrets out of your command history.

Security Tip: Never commit `.env` files to git. Add them to `.gitignore`. For production, use Docker Secrets or a vault.

Method 3: In the Dockerfile (Build-Time Defaults)

```
# Set default values – overridable at runtime with -e
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0
```

These defaults are baked into the image. Runtime `-e` flags always take priority over Dockerfile `ENV` values.

⚡ Lab 2: Building and Testing a Flask App with Environment Variables

Step 1: Create the application

Fix Applied: The original notes referenced testing against a Flask app and `my-node-app`, but neither image existed. The commands below create the app from scratch.

```
# Create project directory
mkdir -p ~/flask-env-lab && cd ~/flask-env-lab

# Create requirements.txt
echo "Flask==2.3.2" > requirements.txt

# Create the Flask application
cat << 'EOF' > app.py
import os
from flask import Flask, jsonify

app = Flask(__name__)

# Read environment variables with safe defaults
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return jsonify({
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    })

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
EOF
```

Step 2: Create the Dockerfile

```
cat << 'EOF' > Dockerfile
FROM python:3.9-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

ENV PORT=5000 \
    DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
EOF
```

ENV Variable	Purpose
<code>PYTHONUNBUFFERED=1</code>	Forces Python to print output immediately instead of buffering — critical for <code>docker logs</code> to work in real-time
<code>PYTHONDONTWRITEBYTECODE=1</code>	Prevents <code>.pyc</code> cache files — keeps the container filesystem clean

Step 3: Build the image

```
docker build -t flask-app .
```

Expected Output: Build completes with `[+] Building X.Xs (11/11) FINISHED`

Screenshot — Flask app build and monitoring:

```

pranav@Pranav: ~
})

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
EOF
pranav@Pranav:~/flask-env-lab$ cat << 'EOF' > Dockerfile
FROM python:3.9-slim
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV PORT=5000 \
    DEBUG=false
EXPOSE 5000
CMD ["python", "app.py"]
EOF
pranav@Pranav:~/flask-env-lab$ docker build -t flask-app .
[+] Building 4.8s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 286B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.8s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf 0.0s
=> [internal] load build context                                0.0s
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O      BLOCK I/O   PIDS
4b534c56828e   my-flask-app  0.18%    46.44MiB / 7.594GiB  0.60%    1.36kB / 792B  537kB / 0B   3
^Cpranav@Pranav:~/flask-env-lab$ docker logs -f my-flask-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.4:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 813-371-836
172.17.0.1 - - [12/Mar/2026 17:35:03] "GET /config HTTP/1.1" 200 -
^Cpranav@Pranav:~/flask-env-lab$ docker network create app-network
3bc8e83859a32123b26b88c91eb0ad10a5b366fe2c5d5f6a0daca948a0f6c2b3

```

⚡ Lab 3: Test Environment Variables in Action

```

# Run with custom environment variables
docker run -d \
    --name my-flask-app \
    -p 5000:5000 \
    -e DATABASE_HOST="prod-db.example.com" \
    -e DEBUG="true" \
    -e API_KEY="sk-123456" \
    flask-app

```

```

# Verify environment is set correctly inside the container
docker exec my-flask-app env
docker exec my-flask-app printenv DATABASE_HOST

```

Expected Output:

```
prod-db.example.com
```

```

# Test the /config endpoint
curl http://localhost:5000/config

```

Expected Output:

```
{"db_host": "prod-db.example.com", "debug": true, "has_api_key": true}
```

Deep Dive — ENV Precedence: Runtime `-e` flags override Dockerfile `ENV` values. The Dockerfile set `DEBUG=false`, but we passed `-e DEBUG="true"` at runtime, so the app runs in debug mode. This is how the same image works in dev (`debug=true`) and prod (`debug=false`).

Part 3: Docker Monitoring

⚡ Lab 1: `docker stats` — Real-Time Resource Metrics

```
# Live dashboard for ALL running containers (Ctrl+C to exit)
docker stats

# Stats for specific containers only
docker stats my-flask-app nginx-custom

# Custom format - cleaner output
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"

# One-time snapshot (no live update) - useful for scripts
docker stats --no-stream
```

Column	What It Shows
CPU %	Percentage of host CPU used by this container
MEM USAGE / LIMIT	Current memory consumption vs. allowed maximum
MEM %	Percentage of allowed memory currently in use
NET I/O	Total bytes sent/received over the network
BLOCK I/O	Total bytes read/written to disk
PIDS	Number of processes running inside the container

Example Output from Lab:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
4b534c56828e	my-flask-app	0.18%	46.44MiB / 7.594GiB	0.60%	1.36kB / 792B	537kB / 0B

⚡ Lab 2: `docker top` — Process Monitoring

```
# View all processes running inside a container
docker top my-flask-app

# Full process listing (like ps -ef inside the container)
docker top my-flask-app -ef
```

Deep Dive — PID Namespaces: Inside the container, the Flask process is PID 1. On the host, it has a completely different PID. `docker top` shows the **host-side PIDs**, which is useful for debugging from outside the container.

⚡ Lab 3: `docker logs` — Application Logs

```
# View all historical logs
docker logs my-flask-app

# Follow logs in real-time (like tail -f)
docker logs -f my-flask-app

# Last 50 lines only
docker logs --tail 50 my-flask-app

# Logs with timestamps
docker logs -t my-flask-app

# Logs since a specific time
docker logs --since 2026-03-12 my-flask-app

# Combine: follow + last 50 lines + timestamps
docker logs -f --tail 50 -t my-flask-app
```

Flag	Purpose
<code>-f</code> (follow)	Streams new logs as they arrive — press <code>Ctrl+C</code> to stop
<code>--tail N</code>	Shows only the last N lines instead of the entire history
<code>-t</code>	Prepends RFC 3339 timestamps to every log line
<code>--since</code>	Filters to logs after a given timestamp or relative duration (e.g., <code>10m</code>)

Flask App Log Output:

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
172.17.0.1 - - [12/Mar/2026 17:35:03] "GET /config HTTP/1.1" 200 -
```

⚡ Lab 4: docker inspect — Container Details

```
# Full JSON dump of ALL container metadata
docker inspect my-flask-app

# Get specific values using Go templates
docker inspect --format='{{.State.Status}}' my-flask-app      # "running"
docker inspect --format='{{.NetworkSettings.IPAddress}}' my-flask-app # "172.17.0.4"
docker inspect --format='{{.Config.Env}}' my-flask-app        # all env vars

# Resource limits
docker inspect --format='{{.HostConfig.Memory}}' my-flask-app # memory limit (0 = unlimited)
docker inspect --format='{{.HostConfig.NanoCpus}}' my-flask-app # CPU limit
```

⚡ Lab 5: docker events — System-Wide Event Stream

```
# Watch all Docker events live (container starts, stops, dies, etc.)
docker events

# Only container events
docker events --filter 'type=container'

# Only "start" events
docker events --filter 'event=start'

# Custom format
docker events --format '{{.Type}} {{.Action}} {{.Actor.Attributes.name}}'
```

⚡ Lab 6: Practical Monitoring Script

```
#!/bin/bash
# monitor.sh - Simple Docker monitoring dashboard

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo

echo "1. Running Containers:"
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
echo

echo "2. Resource Usage:"
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}\t{{.Pids}}"
echo

echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format '{{.Time}} {{.Type}} {{.Action}}' | tail -5
echo

echo "4. System Disk Usage:"
docker system df
```

How to use: Save as `monitor.sh`, run `chmod +x monitor.sh`, then `./monitor.sh`. This gives you a single-command health check for all running containers.

Part 4: Docker Networks

⚡ Lab 1: Understanding Docker Network Types

```
# List the three default networks Docker creates at installation
docker network ls
```

Expected Output:

NETWORK ID	NAME	DRIVER	SCOPE
abc123	bridge	bridge	local
def456	host	host	local
ghi789	none	null	local

Network	Driver	Purpose
<code>bridge</code>	bridge	Default for standalone containers. Each gets its own IP in a private subnet.

Network	Driver	Purpose
host	host	Container shares the host's network stack directly — no isolation.
none	null	Complete network isolation — only loopback interface.

⚡ Lab 2: Network Types in Practice

1. Custom Bridge Network (Recommended for multi-container apps)

```
# Create a user-defined bridge network
docker network create app-network

# Inspect it
docker network inspect app-network

# Run two containers on the same network
docker run -d --name web1 --network app-network nginx
docker run -d --name web2 --network app-network nginx

# Containers can communicate using names (Docker DNS)
docker exec web1 curl http://web2
```

Deep Dive — Why Custom Bridge? The default `bridge` network does NOT provide DNS resolution — containers must use IP addresses. A user-defined bridge automatically runs Docker's embedded DNS at `127.0.0.11`, enabling container-name-based discovery. Always create a custom bridge for multi-container setups.

2. Host Network

```
# Container shares host's network - NGINX runs directly on host port 80
docker run -d --name host-app --network host nginx
curl http://localhost
```

Caution: No port mapping needed (no `-p` flag), but multiple containers can't share the same port. Use on Linux servers for performance-critical applications only.

3. None Network

```
# Complete isolation – no network access at all
docker run -d --name isolated-app --network none alpine sleep 3600

# Only the loopback interface exists
docker exec isolated-app ifconfig
```

⚡ Lab 3: Network Management Commands

```
# Create a network with custom subnet and gateway
docker network create \
  --driver bridge \
  --subnet 172.20.0.0/16 \
  --gateway 172.20.0.1 \
  my-subnet

# Connect a running container to a second network
docker network connect app-network existing-container

# Disconnect a container from a network
docker network disconnect app-network container-name

# Remove a specific network
docker network rm app-network

# Remove ALL unused networks
docker network prune
```

⚡ Lab 4: Multi-Container Communication (DNS Proof)

Goal: Prove that Docker DNS resolves container names across a custom network

Fix Applied: The original notes attempted to run a nonexistent `node-app` image. We replaced it with an `alpine` container using `ping` to prove DNS resolution — simpler and guaranteed to work.

```
# Create the network
docker network create app-network

# Start PostgreSQL database
docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

# Prove DNS works: ping the database by NAME from an alpine container
docker run --rm --network app-network alpine ping -c 4 postgres-db
```

Expected Output:

```
PING postgres-db (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.104 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.165 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.092 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.062 ms

--- postgres-db ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
```

What just happened: The alpine container resolved the name `postgres-db` → `172.18.0.2` using Docker's embedded DNS. No `/etc/hosts` editing, no IP hardcoding. This is exactly how microservices find each other in Docker.

Screenshot — PostgreSQL pull and DNS ping proof:

```

pranav@Pranav: ~
3bc8e83859a32123b26b88c91eb0ad10a5b366fe2c5d5f6a0daca948a0f6c2b3
pranav@Pranav:~/flask-env-lab$ docker run -d --name postgres-db --network app-network \
-e POSTGRES_PASSWORD=secret \
postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
206356c42440: Already exists
0c725cebedaa: Pull complete
c2358c04556d: Pull complete
bae50de223a4: Pull complete
b6de334a9157: Pull complete
67266521af9d: Pull complete
fd349cbbb730: Pull complete
b488546e671b: Pull complete
8d9526eade8e: Pull complete
2f8ddeccdf2d: Pull complete
f0c010f1448f: Pull complete
f080c6a4c94c: Pull complete
e01de8c32e6c: Pull complete
2e180f475c84: Pull complete
Digest: sha256:f30e3de0ac9cc938dac627ef2231099867c694b5f949fad924c8c977428c399
Status: Downloaded newer image for postgres:15
7fce3194b2b411a65571c83f1eb6cad57017b505046cace43006709e2a4f3ee
pranav@Pranav:~/flask-env-lab$ docker run --rm --network app-network alpine ping -c 4 postgres-db
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
PING postgres-db (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.104 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.165 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.092 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.062 ms

--- postgres-db ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.062/0.105/0.165 ms
pranav@Pranav:~/flask-env-lab$ cd ~
mkdir -p app app-logs
cat << 'EOF' > .env.production
API_SECRET=my_production_secret
EOF
pranav@Pranav:~$ docker network create myapp-network
628c626065e7b6f2b3fbd64421540288fdcc712302baff296a174bf60fa4424

```

⚡ Lab 5: Network Inspection & Debugging

```

# View all containers on a network and their IPs
docker network inspect app-network

# Get a container's IP address
docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' postgres-

# DNS resolution test from inside a container
docker exec web1 nslookup web2

# Internet connectivity test
docker exec web1 ping -c 4 google.com

# Check published port mappings
docker port nginx-custom

```

⚡ Lab 6: Port Publishing vs Exposing

```
# PORT PUBLISHING - makes container accessible from outside Docker
docker run -d -p 8080:80 --name app1 nginx
# Host port 8080 → Container port 80

# DYNAMIC PORT - Docker picks a random available host port
docker run -d -P --name app2 nginx
docker port app2
# Shows: 80/tcp → 0.0.0.0:32768

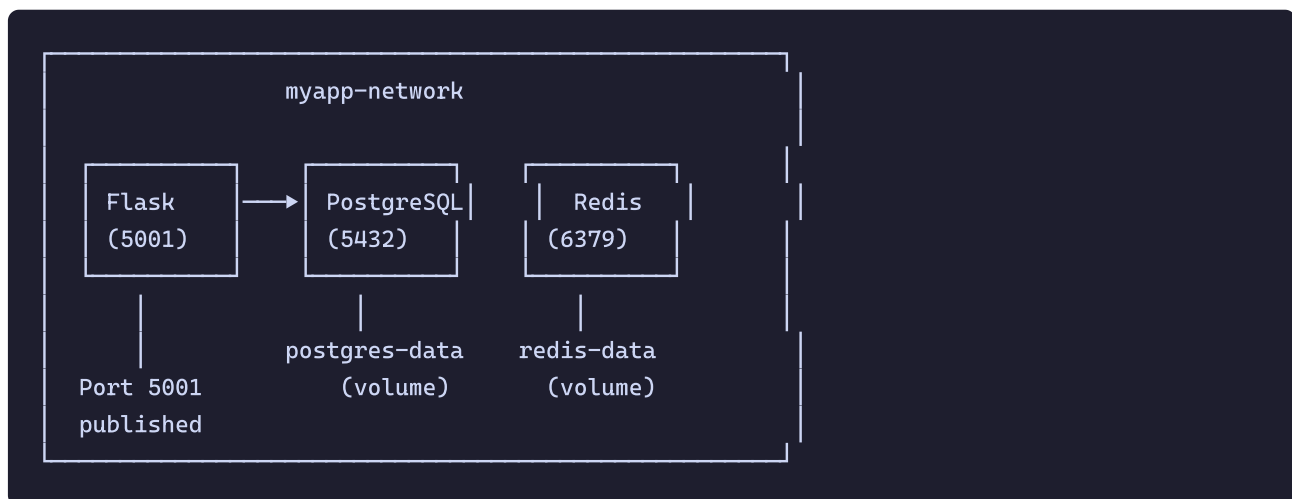
# LOCALHOST ONLY - accessible only from the host machine itself
docker run -d -p 127.0.0.1:8080:80 --name app3 nginx

# MULTIPLE PORTS - for HTTPS + HTTP
docker run -d -p 80:80 -p 443:443 --name app4 nginx
```

Concept	Dockerfile EXPOSE	Runtime -p
What it does	Documentation only — declares intent	Actually creates the port mapping
Accessible?	❌ Not from outside	✅ Yes, from host and network
Required?	No (but recommended)	Yes, for external access

Part 5: Complete Real-World Example — Multi-Service Stack

Architecture



Prerequisites Setup

Fix Applied: The original notes did not include setup commands for required host directories and the `.env.production` file. Without these, Docker's bind mounts and `--env-file` would crash the container on startup.

```
# Create required host directories
cd ~
mkdir -p app app-logs

# Create the production environment file
cat << 'EOF' > .env.production
API_SECRET=my_production_secret
EOF
```

Step-by-Step Deployment

Step 1: Create the application network

```
docker network create myapp-network
```

Step 2: Start PostgreSQL with persistent storage

```
docker run -d \
  --name postgres \
  --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=mydatabase \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:15
```

Flag	Purpose
<code>--network myapp-network</code>	Places the database on the custom network for DNS discovery
<code>-e POSTGRES_DB=mydatabase</code>	Auto-creates a database named <code>mydatabase</code> on first startup
<code>-v postgres-data:/var/lib/postgresql/data</code>	Named volume ensures database files survive container removal

Step 3: Start Redis cache

```
docker run -d \
  --name redis \
  --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine
```

No port publishing for Redis — it's only accessible within `myapp-network`. This is a security best practice.

Step 4: Deploy the Flask application

```
docker run -d \
  --name prod-flask-app \
  --network myapp-network \
  -p 5001:5000 \
  -v ~/app-logs:/var/log/app \
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  -e DATABASE_HOST="postgres" \
  --env-file .env.production \
  flask-app
```

Flag	Purpose
<code>-p 5001:5000</code>	Maps host port <code>5001</code> to the Flask app's port <code>5000</code> (5000 may already be in use by the earlier lab)
<code>-v ~/app-logs:/var/log/app</code>	Bind mount — application logs are written to the host for external log aggregators
<code>-e DATABASE_URL="...@postgres:5432/..."</code>	The hostname <code>postgres</code> resolves via Docker DNS to the PostgreSQL container's IP
<code>--env-file .env.production</code>	Loads secrets from a file — keeps them out of your bash history

Screenshot — Full stack deployment and `docker ps` output:

```

pranav@Pranav: ~
pranav@Pranav:~$ docker run -d --name postgres --network myapp-network \
-e POSTGRES_PASSWORD=mysecretpassword \
-e POSTGRES_DB=mydatabase \
-v postgres-data:/var/lib/postgresql/data \
postgres:15
b186a4c96fc26092539bb061455d3de4662e281dd6d94fde8b526d47f6709a03
pranav@Pranav:~$ docker run -d --name redis --network myapp-network \
-v redis-data:/data \
redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
bc1da058f299: Pull complete
09a5a0c32a23: Pull complete
bd53938b1271: Pull complete
9fe3744a2eac: Pull complete
7ad54b3c4cef: Pull complete
2a97533d89ca: Pull complete
4f4fb700ef54: Pull complete
1c2c8d1ee428: Pull complete
Digest: sha256:8b81dd37ff027bec4e516d41acfbe9fe2460070dc6d4a4570a2ac5b9d59df065
Status: Downloaded newer image for redis:7-alpine
ac556a94f8b8a3799d283efef8348929d240f4c7feeb5646b8feaf1c4cd9c6c2
pranav@Pranav:~$ docker run -d --name prod-flask-app --network myapp-network \
-p 5001:5000 \
-v ~/app-logs:/var/log/app \
-e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
-e REDIS_URL="redis://redis:6379" \
-e DATABASE_HOST="postgres" \
--env-file .env.production \
flask-app
215ef5167e03b89b7b1c39f2a6d510838016aafcd86566d31e6a11e793b08eb7
pranav@Pranav:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
215ef5167e03   flask-app     "python app.py"         6 seconds ago Up 6 seconds  0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp   prod-flask-app
ac556a94f8b8   redis:7-alpine "docker-entrypoint.s..." 15 seconds ago Up 15 seconds  6379/tcp                                     redis
b186a4c96fc2   postgres:15   "docker-entrypoint.s..." 38 seconds ago Up 38 seconds  5432/tcp                                     postgres
7fece3194b2b   postgres:15   "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  5432/tcp                                     postgres-db
4b534c56828e   flask-app     "python app.py"         4 minutes ago  Up 4 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   my-flask-app
ccbc09872afb   nginx         "/docker-entrypoint. ..." 5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp       nginx-custom
f8ab127e6a52   ubuntu       "sleep 3600"            12 minutes ago Up 12 minutes                                     test-container
pranav@Pranav:~$ curl http://localhost:5001/config
{"db_host":"postgres","debug":false,"has_api_key":false}

```

Step 5: Verify the stack

```

# All three containers should be running
docker ps

# Monitor resource usage across all services
docker stats postgres redis prod-flask-app

# Check application logs
docker logs -f prod-flask-app

# Test network connectivity between services
docker exec prod-flask-app ping -c 2 postgres
docker exec prod-flask-app ping -c 2 redis

# View the entire network topology
docker network inspect myapp-network

# Test the API endpoint
curl http://localhost:5001/config

```

Expected Output:

```

{"db_host":"postgres","debug":false,"has_api_key":false}

```

`db_host` correctly shows `postgres` (the container name), proving Docker DNS resolved the connection string.

Screenshot — Final verification with curl and config endpoint:

```

pranav@Pranav: ~
ccbc09872afb  nginx          "/docker-entrypoint..." 5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  nginx-custom
f8ab127e6a52  ubuntu         "sleep 3600"             12 minutes ago Up 12 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  test-container

pranav@Pranav:~$ curl http://localhost:5001/config
{"db_host":"postgres", "debug":false, "has_api_key":false}
pranav@Pranav:~$ docker rm -f $(docker ps -aq)
215ef5167e03
ac556a94f8b8
b186a4c96fc2
7fce3194b2b
4b534c56828e
ccbc09872afb
f8ab127e6a52
d04aa5df4094
f3a2e7f3c5fc
90c4f61dc230
bdfcd86fc51f
4a0970748b52
pranav@Pranav:~$ docker network prune -f
Deleted Networks:
app-network
myapp-network

pranav@Pranav:~$ docker volume prune -f
Deleted Volumes:
26aab778eca930acf6c9f2afe5872fb0641ad3f1a7e08a93c3ef08e8b5090714

Total reclaimed space: 40.3MB
pranav@Pranav:~$

```

Cleanup

```

# Stop and remove ALL containers
docker rm -f $(docker ps -aq)

# Remove all unused networks
docker network prune -f

# Remove all unused volumes (CAUTION: destroys all persistent data)
docker volume prune -f

# Remove unused images
docker image prune -f

```

Screenshot — Full cleanup:

```

pranav@Pranav: ~
ccbc09872afb  nginx      "/docker-entrypoint..."  5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  nginx-custom
f8ab127e6a52  ubuntu    "sleep 3600"              12 minutes ago  Up 12 minutes  test-container

pranav@Pranav:~$ curl http://localhost:5001/config
{"db_host":"postgres","debug":false,"has_api_key":false}

pranav@Pranav:~$ docker rm -f $(docker ps -aq)
215ef5167e03
ae556a94f8b8
b186a4c96fc2
7feca3194b2b
4b534c56828e
ccbc09872afb
f8ab127e6a52
d04aa5df4094
f3a2e7f3c5fc
90c4f61dc230
bdfcd86fc51f
4a0970748b52

pranav@Pranav:~$ docker network prune -f
Deleted Networks:
app-network
myapp-network

pranav@Pranav:~$ docker volume prune -f
Deleted Volumes:
26aab778eca930acf6c9f2afe5872fb0641ad3f1a7e08a93c3ef08e8b5090714

Total reclaimed space: 40.3MB
pranav@Pranav:~$

```

🔧 Common Pitfalls & Troubleshooting

Problem	Symptom	Solution
Container exits immediately	<code>STATUS: Exited (0)</code>	Ensure the main process runs in the foreground. For ubuntu, use <code>sleep 3600</code> or <code>bash</code> with <code>-it</code> .
Data lost after restart	"No such file" after <code>docker stop/start</code>	Data survives <code>stop/start</code> . It's only lost after <code>docker rm</code> . Use volumes to persist beyond removal.
<code>curl</code> downloads file instead of showing text	Response prompts download	Add <code>default_type text/plain;</code> to your NGINX config.
Flask app image not found	<code>Unable to find image 'flask-app:latest'</code>	You must <code>docker build -t flask-app .</code> first.
Containers can't find each other by name	<code>Could not resolve host</code>	Use a custom bridge network, not the default bridge.
Port already in use	<code>Bind for 0.0.0.0:5000 failed: port is already allocated</code>	Use a different host port: <code>-p 5001:5000</code>
<code>--env-file</code> crashes container	<code>open .env.production: no such file or directory</code>	Create the file before running the container.

Problem	Symptom	Solution
Bind mount directory creates as root	Permissions denied inside container	Create the host directory before running: <code>mkdir -p ~/app-logs</code>



Quick Reference Cheatsheet

Volumes

```

docker volume create <name>           # Create named volume
docker run -v <volume>:/path           # Mount named volume
docker run -v /host/path:/container/path # Bind mount
docker volume ls                       # List all volumes
docker volume inspect <name>          # View details
docker volume rm <name>               # Delete volume
docker volume prune                    # Remove ALL unused

```

Environment Variables

```

docker run -e KEY=value                # Inline variable
docker run --env-file .env             # From file
docker exec <container> env            # List all env vars
docker exec <container> printenv KEY   # Get one value
# Dockerfile: ENV KEY=value           # Build-time default

```

Monitoring

```

docker stats                          # Live resource dashboard
docker stats --no-stream              # One-time snapshot
docker logs -f <container>           # Stream logs
docker logs --tail 50 -t <container> # Last 50 + timestamps
docker top <container>                # Process list
docker inspect <container>           # Full JSON metadata
docker events                         # System event stream
docker system df                      # Disk usage summary

```

Networks

```

docker network create <name>                # Create bridge
docker run --network <name> ...              # Join network
docker network connect <network> <container> # Add to network
docker network disconnect <network> <container> # Remove from network
docker network inspect <name>                # View topology
docker network rm <name>                     # Delete network
docker network prune                         # Remove unused

```

Key Takeaways

1. **Volumes** persist data beyond container lifecycle — always use **named volumes** for databases and production data.
2. **Bind mounts** are ideal for development — edit on your host, see changes instantly in the container.
3. **Environment variables** configure containers dynamically — use `-e` for quick tests, `--env-file` for production.
4. `docker stats`, `docker logs`, and `docker events` form a complete observability toolkit for debugging.
5. **Custom bridge networks** provide DNS-based service discovery — containers find each other by name, not IP.
6. **Never publish database ports** unless absolutely required — keep them on internal networks only.
7. Container data survives `stop/start` but is **destroyed on** `docker rm` — this is the #1 misconception.
8. Always create required host directories and `.env` files **before** running containers with bind mounts.

Experiment 5 covers the four pillars of operational Docker: storage, configuration, monitoring, and networking. Together, these form the foundation for building production-ready containerized applications.

Student: Pranav R Nair | **SAP ID:** 500121466 | **Batch:** 2(CCVT)

Objective

Systematically compare the imperative `docker run` approach against the declarative `docker-compose` approach across single-container, multi-container, resource-limited, and custom-built application scenarios — and understand exactly why each tool exists.

Parts Overview

Part	Focus
Part A	Theory — core concepts, flags, and common pitfalls
Part B — Task 1	Single Nginx container: <code>docker run</code> vs Compose
Part B — Task 2	Multi-container WordPress + MySQL
Part C — Task 3	Convert <code>docker run</code> commands to Compose
Part C — Task 4	Resource limits: <code>docker run</code> vs Compose
Part D — Task 5	Node.js app with custom Dockerfile + Compose <code>build:</code>
Part D — Task 6	Multi-Stage Python/FastAPI Dockerfile with Compose
Cleanup	System prune — remove all experiment artefacts

PART A — Theory

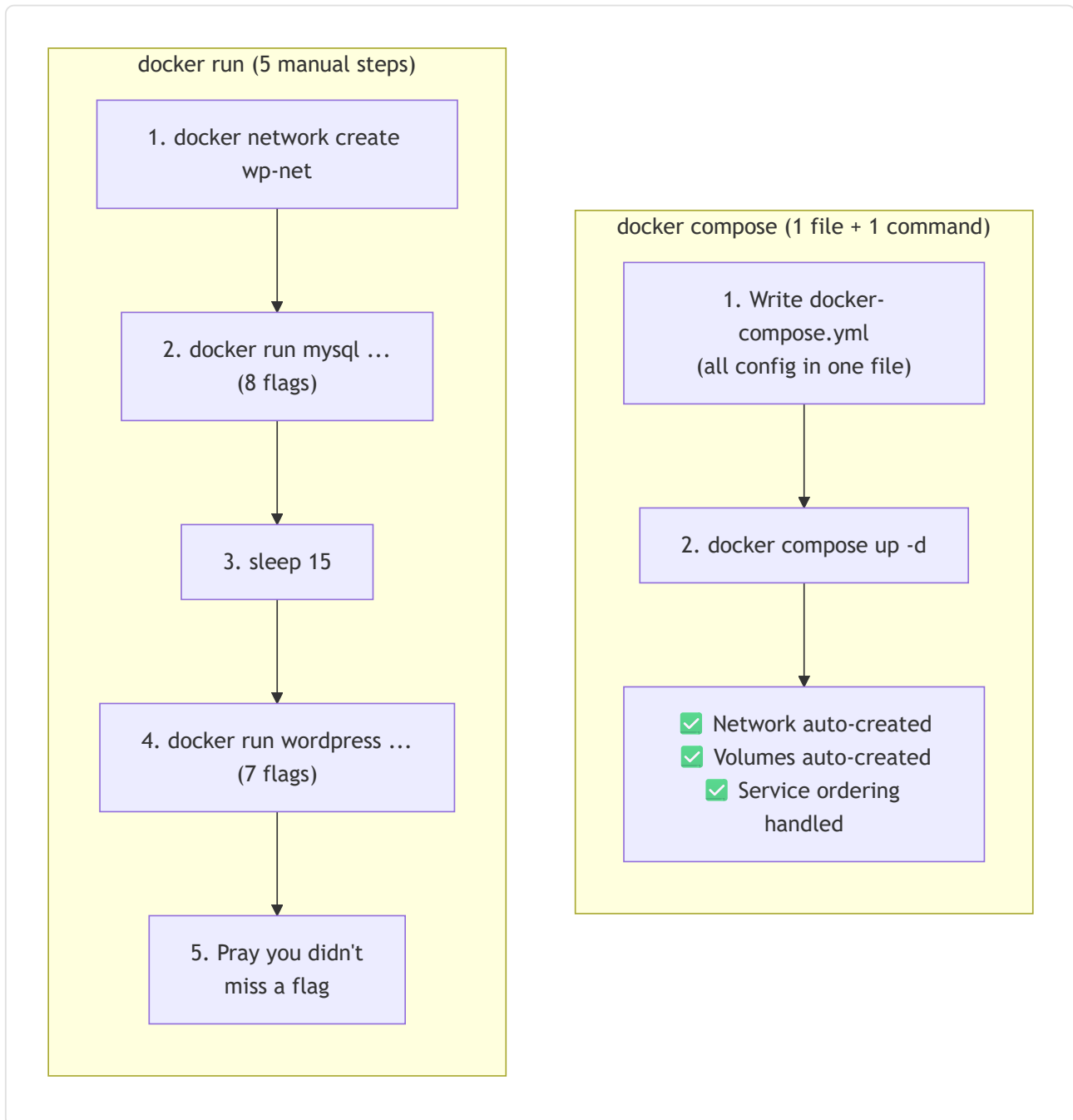
1. `docker run` vs Docker Compose — The Core Difference

`docker run` is **imperative** — you specify every step explicitly via CLI flags. `docker compose` is **declarative** — you describe the desired end state in a YAML file.

Dimension	<code>docker run</code> (Imperative)	<code>docker compose</code> (Declarative)
Configuration	CLI flags typed each time	<code>docker-compose.yml</code> — committed to Git
Multi-container	Multiple separate commands	Single file, single command
Readability	Hard to read long commands	Structured YAML

Dimension	<code>docker run</code> (Imperative)	<code>docker compose</code> (Declarative)
Reproducibility	Easy to miss a flag	Consistent every run
Networking	Manual <code>--network</code> flags	Auto-created shared network
Best for	Quick one-off tests, debugging	Dev environments, CI/CD, production

Workflow Comparison: Deploying WordPress + MySQL



2. Flag-to-YAML Mapping (Complete Reference)

docker run Flag	docker-compose.yml Key	What it Does
<code>-p 8080:80</code>	<code>ports: ["8080:80"]</code>	Maps host port 8080 → container port 80
<code>-v ./html:/app</code>	<code>volumes: [".:/html:/app"]</code>	Bind mount: host folder into container
<code>-v pgdata:/data</code>	<code>volumes: ["pgdata:/data"]</code> + top-level <code>volumes:</code>	Named Docker-managed volume
<code>-e KEY=value</code>	<code>environment: [KEY=value]</code>	Sets env var inside the container
<code>--name myapp</code>	<code>container_name: myapp</code>	Human-readable container name
<code>--network mynet</code>	<code>networks: [mynet]</code> + top-level <code>networks:</code>	Custom network membership
<code>--restart unless-stopped</code>	<code>restart: unless-stopped</code>	Auto-restart policy
<code>--memory="256m"</code>	<code>deploy.resources.limits.memory: 256m</code>	RAM cap
<code>--cpus="0.5"</code>	<code>deploy.resources.limits.cpus: "0.5"</code>	CPU cap (0.5 = half a core)
<code>-d</code>	<code>docker compose up -d</code>	Detached (background) mode
<code>--hostname myhost</code>	<code>hostname: myhost</code>	Container hostname

3. Restart Policies — Explained

Policy	Behaviour	When to Use
<code>no</code> (default)	Never restart	One-off tasks, jobs
<code>always</code>	Always restart — even after <code>docker stop</code>	Critical services that must always be up
<code>unless-stopped</code>	Restart automatically, but NOT if you manually stopped it	Web servers, APIs
<code>on-failure</code>	Only restart on non-zero exit code	Batch jobs that may fail

Key Difference: `always` vs `unless-stopped`: If you run `docker stop myapp` and then reboot the machine, a container with `restart: always` will start again. A container with `restart: unless-stopped` will NOT — it remembers you stopped it intentionally.

4. Volume Types — Deep Dive

Type	Syntax (docker run)	Syntax (Compose)	Where Data Lives	Survives <code>docker rm</code> ?
Bind Mount	<code>-v \$(pwd)/html:/app</code>	<code>- ./html:/app</code>	Your host filesystem	✓ Yes (it's your file)
Named Volume	<code>-v pgdata:/var/lib/data</code>	<code>- pgdata:/var/lib/data</code> <code>+ volumes: pgdata:</code>	<code>/var/lib/docker/volumes/</code>	✓ Yes (until <code>docker volume rm</code>)
Anonymous Volume	<code>-v /var/lib/data</code>	not recommended	<code>/var/lib/docker/volumes/</code>	✗ No — removed with container

Rule of Thumb: Use **bind mounts** for development (live code editing). Use **named volumes** for databases and persistent state. Avoid anonymous volumes.

5. `depends_on` — What It Does and Doesn't Do

```
wordpress:
  depends_on:
    - mysql
```

✓ **What it does:** Starts `mysql` container *before* `wordpress`.

✗ **What it does NOT do:** It does NOT wait for MySQL to be *ready to accept connections*. MySQL takes 10–15 seconds to fully initialize after the container starts. If WordPress connects too early, it gets `Connection`

refused .

The correct production pattern:

```
mysql:
  image: mysql:5.7
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    retries: 5
    timeout: 5s

wordpress:
  depends_on:
    mysql:
      condition: service_healthy # Waits for healthcheck to pass
```

6. image: vs build: in Compose

	image:	build:
Source	Pulls from Docker Hub or registry	Builds from local <code>Dockerfile</code>
When to use	Standard software (nginx, mysql, postgres)	Your own application code
Speed	Fast (already built, cached)	Slower first time, cached after
Example	<code>image: nginx:alpine</code>	<code>build: .</code> or <code>build: context: ./app</code>

You can also combine them (build locally, tag it):

```
services:
  app:
    build: .
    image: myapp:1.0 # The built image gets this tag
```

7. deploy: Resource Limits — When Do They Work?

```
deploy:
  resources:
    limits:
      memory: 256m
      cpus: "0.5"
```

Mode	Behaviour
Docker Compose v2 (Docker Engine 20.10+)	✓ <code>resources.limits</code> works in normal <code>docker compose up</code>
Docker Swarm (<code>docker stack deploy</code>)	✓ Full <code>deploy:</code> block works — replicas, placement, update config
Old Compose v1 (<code>docker-compose</code> binary)	✗ <code>deploy:</code> block is silently ignored

To check your version:

```
docker compose version # Must show Compose v2.x for full support
```

8. Custom Networks — When You Must Declare Them

All services in a single `docker-compose.yml` are automatically placed on a **shared default network** and can reach each other by service name — no configuration needed.

You must declare a **custom network** explicitly when:

- You want to isolate groups of services
- You reference a named network inside a service (you **MUST** also declare it at the top level)
- You want to connect containers from separate Compose files

Bug from this lab: If you write `networks: [app-net]` inside a service but forget the top-level `networks: app-net:` block, Compose throws: `network app-net declared as external, but could not be found`.

PART B — Practical Tasks

Task 1: Single Container — `docker run` vs Docker Compose

Prerequisite Setup

🔧 **Fix Added:** Without the `html/` directory and an `index.html` file, the volume mount either fails or serves an empty directory, causing the NGINX welcome page not to appear.

```
# Create the HTML directory and a test page
mkdir -p html
echo "<h1>Hello from Docker Run</h1>" > html/index.html
```

Step 1A: Run Nginx Using `docker run`

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

Flag	Purpose
<code>-d</code>	Detached mode — container runs in the background, terminal stays free
<code>--name lab-nginx</code>	Gives the container a stable, memorable name for use in later commands
<code>-p 8081:80</code>	Maps host port 8081 → container port 80 (NGINX listens on 80 internally)
<code>-v \$(pwd)/html:/usr/share/nginx/html</code>	Bind mount: serves your local <code>html/</code> directory instead of the NGINX default page
<code>nginx:alpine</code>	Minimal Alpine-based NGINX image — about 40MB vs 150MB for the default

Verify the container is running:

```
docker ps
```

Expected Output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a3f9b1c2d4e5	nginx:alpine	"/docker-entrypoint..."	5 seconds ago	Up 4 seconds	0.0.0.

Test in browser: <http://localhost:8081> should show **"Hello from Docker Run"**.

```

pranav@Pranav: ~
pranav@Pranav:~$ # Create the html directory and a test page first
mkdir -p html
echo "<h1>Hello from Docker Run</h1>" > html/index.html
pranav@Pranav:~$ # Run the container
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
6d2fb8e26cd6864e07f209eede99b8a1e127ad85f081d198a4db4e1143e13522
pranav@Pranav:~$ # Verify
docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
lab-nginx      6d2fb8e26cd6   nginx:alpine            "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
fastapi_container 0668ffa0cf83   project1-backend        "uvicorn main:app --..." 2 days ago    Up 26 seconds (health: starting) 0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp
postgres_container 4f7b2000238d   project1-db             "docker-entrypoint.s..." 2 days ago    Up 26 seconds (healthy) 5432/tcp
pranav@Pranav:~$ # Stop and remove
docker stop lab-nginx
docker rm lab-nginx
lab-nginx
lab-nginx
pranav@Pranav:~$ # Create docker-compose.yml
cat > docker-compose.yml <<'EOF'
version: '3.8'
> ^C
pranav@Pranav:~$ # Create docker-compose.yml
cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
EOF
pranav@Pranav:~$ # Run
docker compose up -d
WARN[0000] /home/pranav/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✔ Network pranav_default Created
0.0s

```

Deep Dive — Why Port Mapping? NGINX listens on port 80 *inside* the container's isolated network namespace. The host machine has no direct access to that namespace. The `-p 8081:80` flag instructs Docker's userland proxy to forward any traffic arriving on host port 8081 into the container's port 80. The host port (8081) can be any free port; the container port (80) must match what the process inside listens on.

Cleanup:

```

docker stop lab-nginx
docker rm lab-nginx

```

Step 1B: Same Setup Using Docker Compose

Create `docker-compose.yml`:

```

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html

```

```
docker compose up -d
```

Verify:

```
docker compose ps
```

Expected Output:

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
lab-nginx	nginx:alpine	"/docker-entrypoint..."	nginx	4 seconds ago	Up 3 seconds

```
pranav@Pranav: ~
pranav@Pranav:~$ # Run
docker compose up -d
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
  ✓ Network pranav_default Created 0.0s
  ✓ Container lab-nginx Created 0.1s
pranav@Pranav:~$ # Verify
docker compose ps
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME      IMAGE      COMMAND                  SERVICE    CREATED      STATUS      PORTS
lab-nginx nginx:alpine "/docker-entrypoint..." nginx       6 seconds ago Up 5 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
pranav@Pranav:~$ # Stop and clean up
docker compose down
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
  ✓ Container lab-nginx Removed 0.3s
  ✓ Network pranav_default Removed 0.2s
pranav@Pranav:~$ # Step 1: Create network
docker network create wp-net
Error response from daemon: network with name wp-net already exists
pranav@Pranav:~$
# Step 2: Run MySQL
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
docker: Error response from daemon: Conflict. The container name "/mysql" is already in use by container "f4d4991d7fb3cce8fbce088d12ff12baa0a3ffe18537eed39991a5c1fa0a94a1". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
pranav@Pranav:~$ # Wait for MySQL to initialize (important - skip this and WordPress will fail)
sleep 15
pranav@Pranav:~$ # Step 3: Run WordPress
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  -e WORDPRESS_DB_USER=root \
  wordpress:latest
docker: Error response from daemon: Conflict. The container name "/wordpress" is already in use by container "ea3bc820186c035f76c1e4784e1c62a74ed9abfcc7b91650b9da0100f63a78f9". You have to remove (or rename) that container to be able to reuse that name.

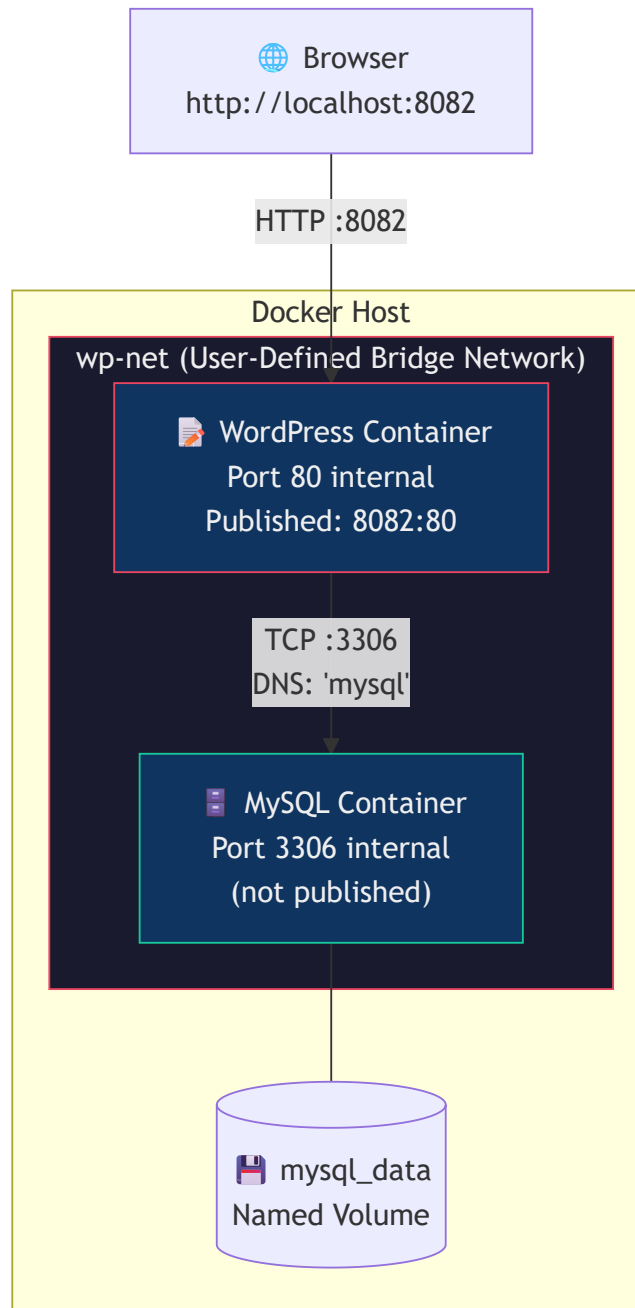
Run 'docker run --help' for more information
```

Cleanup:

```
docker compose down
```

Task 2: Multi-Container Application — WordPress + MySQL

Architecture Diagram



WordPress resolves the hostname `mysql` to the MySQL container's IP automatically via Docker's embedded DNS server. The named volume `mysql_data` persists database files across container restarts.

2A: Using `docker run` (Manual Way)

🔧 **Fix 1 Added:** `WORDPRESS_DB_USER=root` was missing. Without it, WordPress cannot authenticate with MySQL, causing the setup wizard to fail with a database connection error.

🔧 **Fix 2 Added:** `sleep 15` was missing between MySQL and WordPress startup. MySQL takes 10–15 seconds to initialize. Without the delay, WordPress attempts to connect to MySQL before it's ready, causing connection errors.

Step 1: Create an isolated network

```
docker network create wp-net
```

Command part	Purpose
<code>docker network create</code>	Creates a user-defined bridge network
<code>wp-net</code>	Name for the network — used as a DNS domain by Docker

Step 2: Start MySQL

```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  -e MYSQL_USER=root \
  mysql:5.7
```

Flag	Purpose
<code>--network wp-net</code>	Connects MySQL to the named network so WordPress can reach it by hostname
<code>-e MYSQL_ROOT_PASSWORD=secret</code>	Sets the root password — required by MySQL image
<code>-e MYSQL_DATABASE=wordpress</code>	Auto-creates a database named <code>wordpress</code> on first run
<code>-e MYSQL_USER=root</code>	✅ Fix: Explicitly sets the user WordPress will connect as

Step 3: Wait for MySQL to initialize

```
sleep 15
```

Why `sleep 15` ? The MySQL container starts in seconds, but the actual database engine inside takes much longer to initialize, create system tables, and set up the configured database. WordPress doesn't retry connections — it fails on the first attempt if MySQL isn't ready.

Step 4: Start WordPress

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  -e WORDPRESS_DB_USER=root \
  wordpress:latest
```

Flag	Purpose
<code>--network wp-net</code>	Connects WordPress to the same network as MySQL
<code>-e WORDPRESS_DB_HOST=mysql</code>	<code>mysql</code> resolves to the MySQL container's IP via Docker DNS
<code>-e WORDPRESS_DB_USER=root</code>	✅ Fix: Must match the MySQL user configured above
<code>-e WORDPRESS_DB_PASSWORD=secret</code>	Must match <code>MYSQL_ROOT_PASSWORD</code>

Test: `http://localhost:8082` — you should see the WordPress setup wizard.

```

pranav@Pranav: ~
Run 'docker run --help' for more information
pranav@Pranav:~$ # Verify both containers are running
docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS                               NAMES
0668ffa0cf83   project1-backend  "uvicorn main:app --..."  2 days ago    Up 2 minutes (unhealthy)  0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp  fastapi_con
4f7b2000238d   project1-db      "docker-entrypoint.s..."  2 days ago    Up 2 minutes (healthy)    5432/tcp                                postgres_co
ntainer
pranav@Pranav:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS                               NAMES
ea3bc820186c   wordpress:latest  "docker-entrypoint.s..."  20 hours ago   Exited (0) 14 hours ago                                     wordpress
f4d4991d7fb3   mysql:5.7       "docker-entrypoint.s..."  20 hours ago   Exited (0) 14 hours ago                                     mysql
0668ffa0cf83   project1-backend  "uvicorn main:app --..."  2 days ago    Up 3 minutes (unhealthy)  0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp  fastapi_c
ontainer
4f7b2000238d   project1-db      "docker-entrypoint.s..."  2 days ago    Up 3 minutes (healthy)    5432/tcp                                postgres_
container
pranav@Pranav:~$ docker restart ea3bc820186c
ea3bc820186c
pranav@Pranav:~$ docker restart f4d4991d7fb3
f4d4991d7fb3
pranav@Pranav:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS                               NAMES
ea3bc820186c   wordpress:latest  "docker-entrypoint.s..."  20 hours ago   Up 19 seconds        0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  wordpress
f4d4991d7fb3   mysql:5.7       "docker-entrypoint.s..."  20 hours ago   Up 4 seconds         3306/tcp, 33060/tcp                 mysql
0668ffa0cf83   project1-backend  "uvicorn main:app --..."  2 days ago    Up 4 minutes (unhealthy)  0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp  fastapi_c
ontainer
4f7b2000238d   project1-db      "docker-entrypoint.s..."  2 days ago    Up 4 minutes (healthy)    5432/tcp                                postgres_
container
pranav@Pranav:~$ # Cleanup
pranav@Pranav:~$ docker stop wordpress mysql
pranav@Pranav:~$ docker rm wordpress mysql
pranav@Pranav:~$ docker network rm wp-net
pranav@Pranav:~$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress

```

Cleanup:

```

docker stop wordpress mysql
docker rm wordpress mysql
docker network rm wp-net

```

2B: Using Docker Compose

 **Fix noted:** `WORDPRESS_DB_USER` Added here. For production, also add a proper MySQL healthcheck.

```

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      # Production-safe: wait for MySQL to be ready
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5
      timeout: 5s

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_USER: root
      WORDPRESS_DB_PASSWORD: secret
      WORDPRESS_DB_NAME: wordpress
      # Fix: was missing in original
    depends_on:
      mysql:
        condition: service_healthy
        # Waits for healthcheck, not just container start

volumes:
  mysql_data:

```

```

docker compose up -d
docker compose ps

```

Expected Output:

NAME	IMAGE	SERVICE	STATUS
mysql	mysql:5.7	mysql	running (healthy)
wordpress	wordpress:latest	wordpress	running

```

mysql
wp-net
pranav@Pranav:~$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
      WORDPRESS_DB_USER: root
    depends_on:
      - mysql

volumes:
  mysql_data:
EOF
pranav@Pranav:~$ docker compose up -d
WARN[0000] /home/pranav/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 4/4
✔Network pranav_default      Created           0.0s
✔Volume pranav_mysql_data    Created           0.0s
✔Container pranav-mysql-1     Created           0.0s
✔Container pranav-wordpress-1 Created           0.0s
pranav@Pranav:~$ # Verify
docker compose ps
docker compose logs wordpress
WARN[0000] /home/pranav/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
pranav-mysql-1      mysql:5.7            "docker-entrypoint.s..." mysql      9 seconds ago  Up 8 seconds  3306/tcp, 33060/tcp
pranav-wordpress-1  wordpress:latest     "docker-entrypoint.s..." wordpress  9 seconds ago  Up 8 seconds  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
WARN[0000] /home/pranav/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
wordpress-1 | WordPress not found in /var/www/html - copying now...
wordpress-1 | Complete! WordPress has been successfully copied to /var/www/html
wordpress-1 | No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; copying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRES
S_DB_PASSWORD WORDPRESS_DB_USER)

```

Deep Dive — Docker DNS: Within the `wp-net` user-defined network (or the default Compose network), Docker runs an embedded DNS server. When WordPress resolves `mysql`, the DNS server returns the internal IP of the MySQL container — automatically. This is why `WORDPRESS_DB_HOST: mysql` works without you knowing the actual IP address.

Cleanup:

```
docker compose down -v # -v also removes the mysql_data volume
```

PART C — Conversion Tasks

Task 3: Convert `docker run` to Docker Compose

Problem 1: Basic Web Application

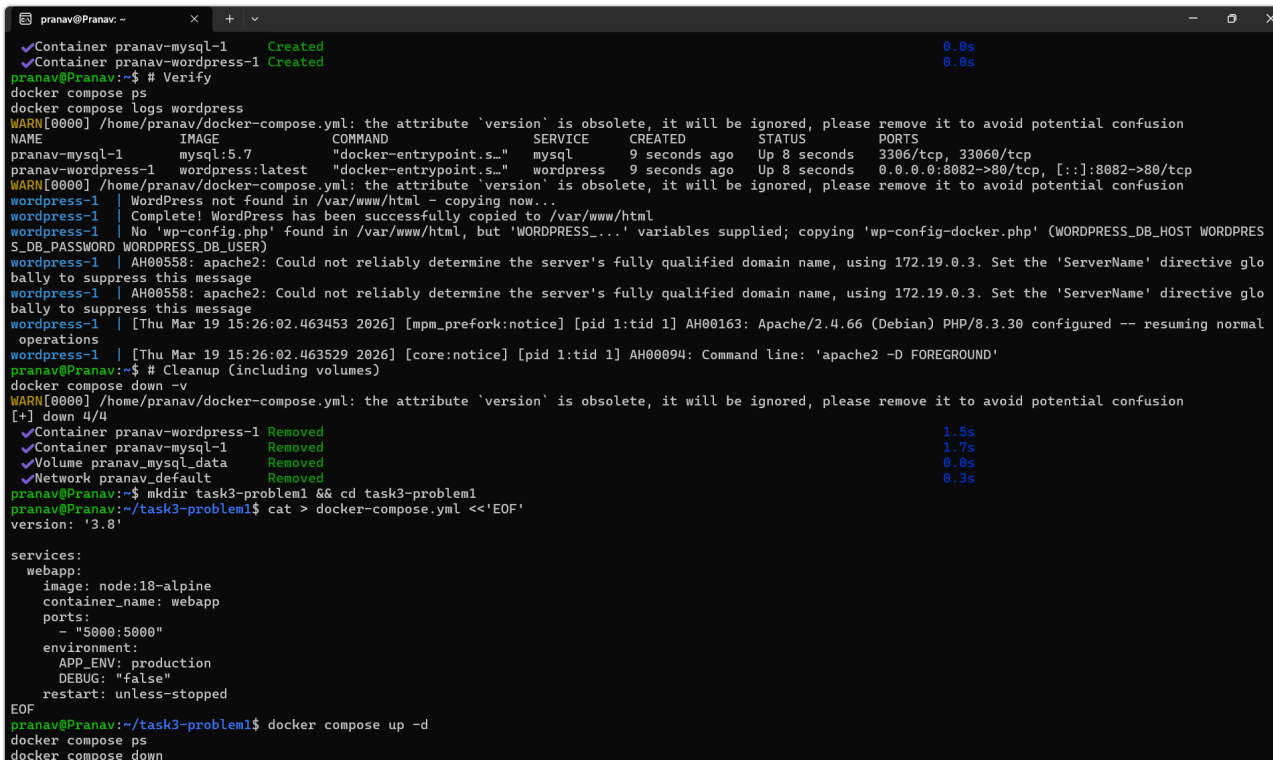
Given `docker run` command:

```
docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
```

Equivalent `docker-compose.yml` :

```
services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

```
docker compose up -d
docker compose ps
```



```
pranav@Pranav: ~
✓Container pranav-mysql-1 Created 0.0s
✓Container pranav-wordpress-1 Created 0.0s
pranav@Pranav:~$ # Verify
docker compose ps
docker compose logs wordpress
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
pranav-mysql-1 mysql:5.7 "docker-entrypoint.s..." mysql 9 seconds ago Up 8 seconds 3306/tcp, 33060/tcp
pranav-wordpress-1 wordpress:latest "docker-entrypoint.s..." wordpress 9 seconds ago Up 8 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
wordpress-1 | WordPress not found in /var/www/html - copying now...
wordpress-1 | Complete! WordPress has been successfully copied to /var/www/html
wordpress-1 | No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; copying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRES
$ DB_PASSWORD WORDPRESS_DB_USER)
wordpress-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.19.0.3. Set the 'ServerName' directive glo
bally to suppress this message
wordpress-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.19.0.3. Set the 'ServerName' directive glo
bally to suppress this message
wordpress-1 | [Thu Mar 19 15:26:02.463453 2026] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.66 (Debian) PHP/8.3.30 configured -- resuming normal
operations
wordpress-1 | [Thu Mar 19 15:26:02.463529 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
pranav@Pranav:~$ # Cleanup (including volumes)
docker compose down -v
WARN[0000] /home/pranav/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 4/4
✓Container pranav-wordpress-1 Removed 1.5s
✓Container pranav-mysql-1 Removed 1.7s
✓Volume pranav_mysql_data Removed 0.0s
✓Network pranav_default Removed 0.3s
pranav@Pranav:~$ mkdir task3-problem1 && cd task3-problem1
pranav@Pranav:~/task3-problem1$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
EOF
pranav@Pranav:~/task3-problem1$ docker compose up -d
docker compose ps
docker compose down
```

Problem 2: Volume + Custom Network Configuration

🔧 **Fix Added:** The original Compose solution referenced `app-net` inside each service but was missing the top-level `networks:` block. Docker Compose throws an error if you reference a custom network without declaring it at the root level.

Given `docker run` commands:

```
docker network create app-net

docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

docker run -d \
  --name backend \
  --network app-net \
  -p 8000:8000 \
  -e DB_HOST=postgres-db \
  -e DB_USER=admin \
  -e DB_PASS=secret \
  python:3.11-slim
```

Correct `docker-compose.yml` (with the network fix):

```

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net          # Attach to custom network

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db  # Docker DNS resolves this to the postgres container
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net          # Attach to custom network

volumes:
  pgdata:                # Named volume – persists after docker compose down

networks:
  app-net:               # ✅ Fix: must be declared here or Compose throws an error

```

```

docker compose up -d
docker compose ps

```

```

pranav@Pranav: ~
restart: unless-stopped
EOF
pranav@Pranav:~/task3-problem1$ docker compose up -d
docker compose ps
docker compose down
cd -
WARN[0000] /home/pranav/task3-problem1/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] up 7/7
✓Image node:18-alpine          Pulled          16.9s
✓Network task3-problem1_default Created        0.0s
✓Container webapp              Created        0.2s
WARN[0000] /home/pranav/task3-problem1/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME      IMAGE          COMMAND                  SERVICE  CREATED      STATUS          PORTS
webapp    node:18-alpine "docker-entrypoint.s..." webapp    1 second ago Up Less than a second 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
WARN[0000] /home/pranav/task3-problem1/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] down 2/2
✓Container webapp              Removed        0.4s
✓Network task3-problem1_default Removed        0.3s
pranav@Pranav:~$ mkdir task3-problem2 && cd task3-problem2
pranav@Pranav:~/task3-problem2$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:

```

Deep Dive — Why Must You Declare Custom Networks? Compose's default network is created implicitly. But any network you name explicitly in `networks:` at the service level is treated as a reference to a named resource. Compose needs to know whether to create it (`app-net:`) or assume it already exists externally (`app-net: external: true`). Without the declaration, Compose can't determine its intent and fails.

Cleanup:

```
docker compose down -v
```

Task 4: Resource Limits Conversion

Given docker run command:

```

docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine

```

Equivalent docker-compose.yml:

```
services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
          memory: 256m
          cpus: "0.5"
```

```
docker compose up -d
```

 **Verification Added:** The original task had no way to confirm limits were applied. Use `docker stats` to verify:

```
docker stats limited-app --no-stream
```

Expected Output:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
a4f2b1c0d3e6	limited-app	0.00%	3.664MiB / 256MiB	1.43%	...	...	2

The `MEM LIMIT` column shows `256MiB` — confirming the resource cap is enforced.

```

pranav@Pranav: ~
DB_PASS: secret
depends_on:
  - postgres-db
networks:
  - app-net

volumes:
EOFpp-net:
pranav@Pranav:~/task3-problem2$ cat > docker-compose.yml <<'EOF'
version: '3.8'
mkdir task3-problem2 && cd task3-problem2
cat > docker-compose.yml <<'EOF'

version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net

volumes:
EOFpp-net:
pranav@Pranav:~/task3-problem2$ # Bring up everything with one command
docker compose up -d
yaml: unmarshal errors:
  line 36: mapping key "networks" already defined at line 32
pranav@Pranav:~/task3-problem2$ cat > docker-compose.yml <<'EOF'
version: '3.8'

```

Deep Dive — **deploy:** Behaviour Across Modes:

Mode	Result
<code>docker compose up</code> with Compose v2 + Engine 20.10+	✅ <code>resources.limits</code> is applied
<code>docker stack deploy</code> (Swarm)	✅ Full <code>deploy:</code> block honoured
Old <code>docker-compose</code> v1 binary	❌ Entire <code>deploy:</code> block silently ignored

Cleanup:

```
docker compose down
```

PART D — Custom Dockerfile Tasks

Task 5: Replace Standard Image with Dockerfile (Node.js App)

Instead of using `node:18-alpine` directly from Docker Hub, we build our own application image.

Step 1: Create `app.js`

```
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Step 2: Create Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Dockerfile Instruction	Purpose
<code>FROM node:18-alpine</code>	Base image — Node.js on Alpine (minimal ~120MB)
<code>WORKDIR /app</code>	Sets working directory inside container; all subsequent paths are relative to this
<code>COPY app.js .</code>	Copies <code>app.js</code> from build context into <code>/app/</code> inside the image
<code>EXPOSE 3000</code>	Documents the port; does not actually publish it
<code>CMD ["node", "app.js"]</code>	Default process to run when the container starts

Step 3: Create docker-compose.yml

```
services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

Step 4: Build and Run

```
docker compose up --build -d
```

Flag	Purpose
<code>--build</code>	Forces a rebuild of the image before starting — even if it already exists
<code>-d</code>	Detached mode

 **Verification Added:** The original said "verify in browser" but gave no terminal command. Use `curl` for scriptable verification:

```
curl http://localhost:3000
```

Expected Output:

Docker Compose Build Lab

```
pranav@Pranav: ~
pranav@Pranav:~/task3-problem2$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net

volumes:
  pgdata:

EOF
pranav@Pranav:~/task3-problem2$ cat > docker-compose.yml <<'EOF'
version: '3.8'
pranav@Pranav:~/task3-problem2$ docker compose up -d
# Bring up everything with one command
pranav@Pranav:~/task3-problem2$ # Bring up everything with one command
pranav@Pranav:~/task3-problem2$ docker compose up -d
WARN[0000] /home/pranav/task3-problem2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] up 9/9
✔Image python:3.11-slim Pulled 8.9s
✔Network task3-problem2_app-net Created 0.0s
✔Volume task3-problem2_pgdata Created 0.0s
✔Container postgres-db Created 0.3s
✔Container backend Created 0.0s
pranav@Pranav:~/task3-problem2$ # Verify
pranav@Pranav:~/task3-problem2$ docker compose ps
WARN[0000] /home/pranav/task3-problem2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
```

Step 5: Modify and Rebuild

 **Commands Added:** The original described this in words only. Here are the exact commands:

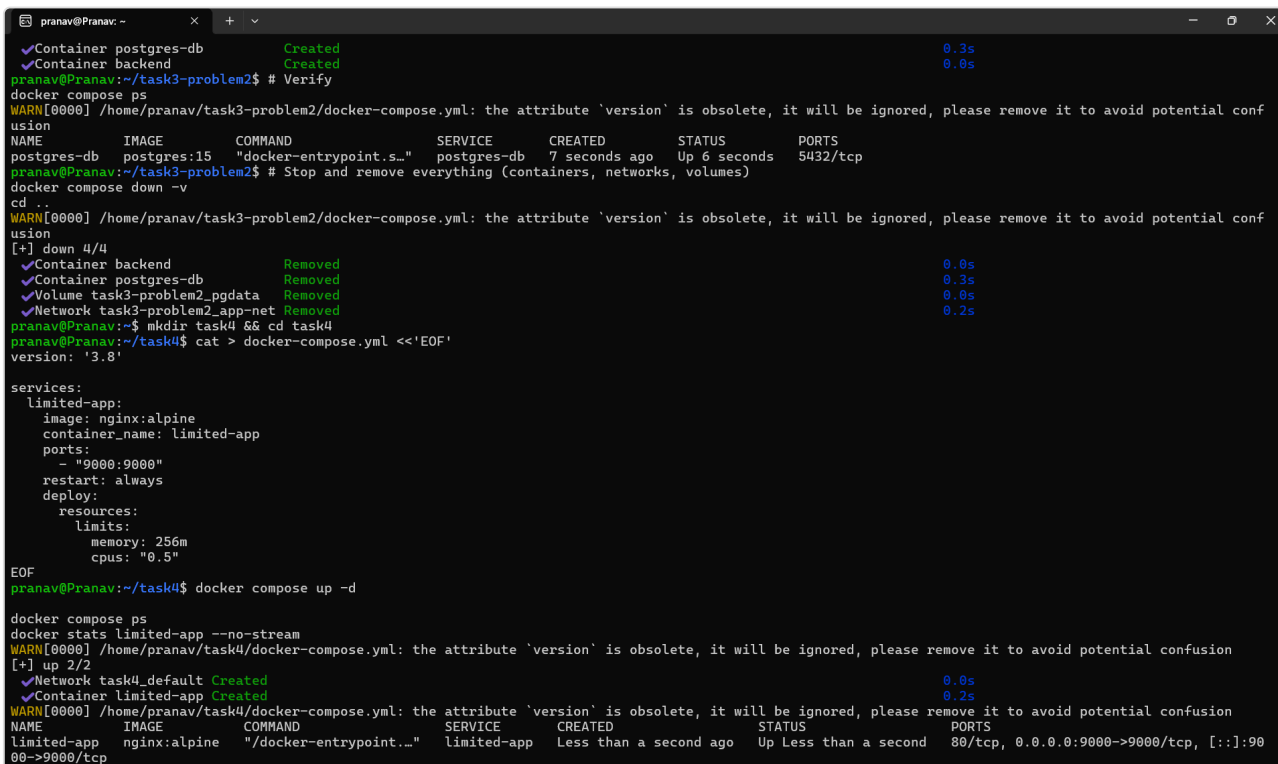
```
# Modify the response message using sed (no editor needed)
sed -i 's/Docker Compose Build Lab/Updated: Docker Compose Build Lab v2/' app.js

# Rebuild and restart
docker compose up --build -d

# Verify the change
curl http://localhost:3000
```

Expected Output:

```
Updated: Docker Compose Build Lab v2
```



```
pranav@Pranav: ~
✓Container postgres-db Created 0.3s
✓Container backend Created 0.0s
pranav@Pranav:~/task3-problem2$ # Verify
docker compose ps
WARN[0000] /home/pranav/task3-problem2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED        STATUS        PORTS
postgres-db   postgres:15    "docker-entrypoint.s..." postgres-db 7 seconds ago  Up 6 seconds  5432/tcp
pranav@Pranav:~/task3-problem2$ # Stop and remove everything (containers, networks, volumes)
docker compose down -v
cd ..
WARN[0000] /home/pranav/task3-problem2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 4/4
✓Container backend Removed 0.0s
✓Container postgres-db Removed 0.3s
✓Volume task3-problem2_pgdata Removed 0.0s
✓Network task3-problem2_app-net Removed 0.2s
pranav@Pranav:~$ mkdir task4 && cd task4
pranav@Pranav:~/task4$ cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
          memory: 256m
          cpus: "0.5"
EOF
pranav@Pranav:~/task4$ docker compose up -d

docker compose ps
docker stats limited-app --no-stream
WARN[0000] /home/pranav/task4/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✓Network task4_default Created 0.0s
✓Container limited-app Created 0.2s
WARN[0000] /home/pranav/task4/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED        STATUS        PORTS
limited-app     nginx:alpine    "/docker-entrypoint..." limited-app Less than a second ago Up Less than a second 80/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
```

Cleanup

```
docker compose down
docker rmi task5-nodeapp # Remove the locally built image
```

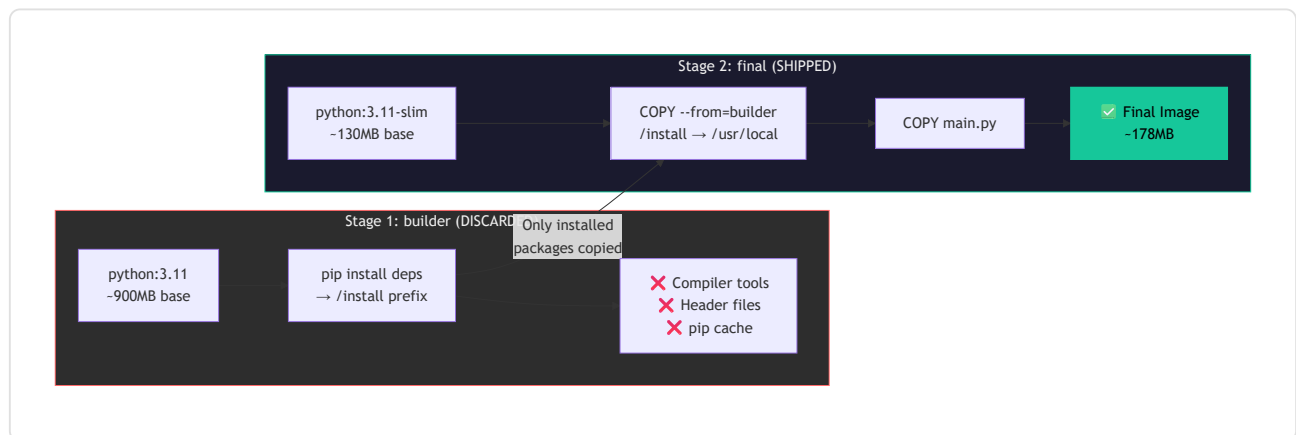
Task 6: Multi-Stage Dockerfile — Python FastAPI App

🔧 **Entire Task Rewritten:** The original document described this task in words but provided no actual code. All four files below were written from scratch.

Why Multi-Stage Builds?

A single-stage Python build image includes the compiler toolchain, header files, and build dependencies — all of which are unnecessary at runtime. A multi-stage build discards the "builder" layer and only ships the final runtime artifacts, dramatically reducing image size and attack surface.

Multi-Stage Build Pipeline



Result: ~900MB builder image is discarded entirely. Only the ~48MB of installed Python packages survives into the final ~178MB image — an **80% size reduction**.

File 1: `main.py`

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from Multi-Stage FastAPI", "env": "docker"}
```

File 2: `requirements.txt`

```
fastapi==0.110.0
uvicorn==0.27.1
```

File 3: Multi-Stage Dockerfile

```

# — Stage 1: builder —————
FROM python:3.11 AS builder

WORKDIR /install

COPY requirements.txt .

# Install dependencies into a separate prefix (easy to copy selectively)
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

# — Stage 2: runtime (final image) —————
FROM python:3.11-slim AS final

WORKDIR /app

# Copy only the installed packages – build tools stay in Stage 1
COPY --from=builder /install /usr/local

# Copy application code
COPY main.py .

EXPOSE 8080

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]

```

Stage	Image used	What it contains
<code>builder</code>	<code>python:3.11</code> (~900MB)	Full Python + pip + compiler tools + installed packages
<code>final</code>	<code>python:3.11-slim</code> (~130MB)	Only: slim Python + the installed packages from builder

Key techniques:

- `--prefix=/install` — installs packages to a known path so `COPY --from=` can target them precisely
- `--no-cache-dir` — prevents pip's HTTP cache from bloating the layer
- `COPY --from=builder` — the only connection between stages; everything else in `builder` is discarded

File 4: docker-compose.yml

```
services:
  fastapi:
    build:
      context: .
      dockerfile: Dockerfile
      target: final          # Build only up to the 'final' stage
    container_name: fastapi-app
    ports:
      - "8080:8080"
    environment:
      APP_ENV: development
    volumes:
      - ./main.py:/app/main.py  # Live code reload in development
```

Build and Run

```
docker compose up --build -d
```

 **Verification Added:**

```
curl http://localhost:8080
```

Expected Output:

```
{"message": "Hello from Multi-Stage FastAPI", "env": "docker"}
```

```

pranav@Pranav: ~
b79a89b4ee17 limited-app 0.00% 12.88MiB / 256MiB 5.03% 626B / 84B 0B / 8.19kB 17
pranav@Pranav:~/task4$ docker compose down
cd ..
WARN[0000] /home/pranav/task4/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
  ✓ Container limited-app Removed 0.4s
  ✓ Network task4_default Removed 0.3s
pranav@Pranav:~$ mkdir task5 && cd task5
pranav@Pranav:~/task5$ # Step 1: Create app.js
cat > app.js <<'EOF'
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
EOF
pranav@Pranav:~/task5$ # Step 2: Create Dockerfile
cat > Dockerfile <<'EOF'
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
EOF
pranav@Pranav:~/task5$ # Step 3: Create docker-compose.yml
cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
EOF
pranav@Pranav:~/task5$ # Build and run
docker compose up --build -d
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.5s (10/10) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 488B 0.0s

```

Compare Image Sizes

```
docker images | grep -E "python|fastapi"
```

Expected Output (approximate):

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE	
fastapi-app	latest	a1b2c3d4e5f6	2 minutes ago	178MB	← final stage only
python	3.11	f6e5d4c3b2a1	2 weeks ago	912MB	← builder stage base
python	3.11-slim	1a2b3c4d5e6f	2 weeks ago	130MB	← runtime base

The final application image is 178MB vs ~912MB if we had used the full `python:3.11` as the single stage — a **80% size reduction**.

```

pranav@Pranav: ~
- "3000:3000"
EOF
pranav@Pranav:~/task5$ # Build and run
docker compose up --build -d
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.5s (10/10) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 488B                                           0.0s
=> [internal] load build definition from Dockerfile                      0.0s
=> => transferring dockerfile: 123B                                     0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine        0.0s
=> [internal] load .dockerignore                                         0.0s
=> => transferring context: 2B                                           0.0s
=> [1/3] FROM docker.io/library/node:18-alpine                         0.1s
=> [internal] load build context                                         0.1s
=> => transferring context: 154B                                         0.0s
=> [2/3] WORKDIR /app                                                  0.0s
=> [3/3] COPY app.js .                                                 0.0s
=> exporting to image                                                  0.1s
=> => exporting layers                                                  0.0s
=> => writing image sha256:3da27ab2c8ae872810e5ae7e9bbcd8cd6a5f932bc47bef814e6ab3e873397a0 0.0s
=> => naming to docker.io/library/task5-nodeapp                        0.0s
=> => resolving provenance for metadata file                            0.0s
[+] up 3/3
✔Image task5-nodeapp          Built                                0.6s
✔Network task5_default        Created                             0.0s
✔Container custom-node-app     Created                             0.0s
pranav@Pranav:~/task5$ # Verify
docker compose ps
curl http://localhost:3000
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                  SERVICE    CREATED          STATUS          PORTS
custom-node-app     task5-nodeapp       "docker-entrypoint.s..." nodeapp     7 seconds ago    Up 7 seconds    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
Docker Compose Build La# Step 4: Modify message and rebuild
sed -i 's/Docker Compose Build Lab/Updated Message v2/' app.js

docker compose up --build -d # rebuilds image automatically

curl http://localhost:3000 # now prints: Updated Message v2
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.3s (10/10) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 488B                                           0.0s
=> [internal] load build definition from Dockerfile                      0.0s
=> => transferring dockerfile: 123B                                     0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine        0.0s
=> [internal] load .dockerignore                                         0.0s
=> => transferring context: 2B                                           0.0s
=> [1/3] FROM docker.io/library/node:18-alpine                         0.0s
=> [internal] load build context                                         0.0s
=> => transferring context: 148B                                         0.0s
=> CACHED [2/3] WORKDIR /app                                            0.0s
=> [3/3] COPY app.js .                                                 0.0s
=> exporting to image                                                  0.1s
=> => exporting layers                                                  0.0s
=> => writing image sha256:4d68212f9c662222c406d88d0c63ce2e2b7d520680ac5639245fc08e043fb3ab 0.0s
=> => naming to docker.io/library/task5-nodeapp                        0.0s
=> => resolving provenance for metadata file                            0.0s
[+] up 2/2
✔Image task5-nodeapp          Built                                0.45s
✔Container custom-node-app     Recreated                             10.3s
curl: (56) Recv failure: Connection reset by peer
pranav@Pranav:~/task5$ curl http://localhost:3000 # now prints: Updated Message v2
pranav@Pranav:~/task5$ # Cleanup~/task5$ # Cleanup
docker compose down
docker rmi task5-nodeapp # remove built image
cd ..
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container custom-node-app     Removed                                10.3s
✔Network task5_default        Removed                                0.2s
Untagged: task5-nodeapp:latest
Deleted: sha256:4d68212f9c662222c406d88d0c63ce2e2b7d520680ac5639245fc08e043fb3ab
pranav@Pranav:~$ mkdir task6 && cd task6
pranav@Pranav:~/task6$ # Step 1: Create a simple FastAPI app
cat > main.py <<'EOF'
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from Multi-Stage Docker Build!"}

```

```

pranav@Pranav: ~
x + v
docker compose up --build -d # rebuilds image automatically

curl http://localhost:3000 # now prints: Updated Message v2
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.3s (10/10) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 488B                                           0.0s
=> [internal] load build definition from Dockerfile                      0.0s
=> => transferring dockerfile: 123B                                     0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine        0.0s
=> [internal] load .dockerignore                                         0.0s
=> => transferring context: 2B                                           0.0s
=> [1/3] FROM docker.io/library/node:18-alpine                         0.0s
=> [internal] load build context                                         0.0s
=> => transferring context: 148B                                         0.0s
=> CACHED [2/3] WORKDIR /app                                            0.0s
=> [3/3] COPY app.js .                                                 0.0s
=> exporting to image                                                  0.1s
=> => exporting layers                                                  0.0s
=> => writing image sha256:4d68212f9c662222c406d88d0c63ce2e2b7d520680ac5639245fc08e043fb3ab 0.0s
=> => naming to docker.io/library/task5-nodeapp                        0.0s
=> => resolving provenance for metadata file                            0.0s
[+] up 2/2
✔Image task5-nodeapp          Built                                0.45s
✔Container custom-node-app     Recreated                             10.3s
curl: (56) Recv failure: Connection reset by peer
pranav@Pranav:~/task5$ curl http://localhost:3000 # now prints: Updated Message v2
pranav@Pranav:~/task5$ # Cleanup~/task5$ # Cleanup
docker compose down
docker rmi task5-nodeapp # remove built image
cd ..
WARN[0000] /home/pranav/task5/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container custom-node-app     Removed                                10.3s
✔Network task5_default        Removed                                0.2s
Untagged: task5-nodeapp:latest
Deleted: sha256:4d68212f9c662222c406d88d0c63ce2e2b7d520680ac5639245fc08e043fb3ab
pranav@Pranav:~$ mkdir task6 && cd task6
pranav@Pranav:~/task6$ # Step 1: Create a simple FastAPI app
cat > main.py <<'EOF'
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from Multi-Stage Docker Build!"}

```

Cleanup

```

docker compose down
docker rmi fastapi-app

```

Additional Screenshots — Lab Progress

```

pranav@Pranav: ~
pranav@Pranav:~/task6$ # Step 2: Create requirements.txt
cat > requirements.txt <<'EOF'
fastapi==0.110.0
uvicorn==0.29.0
EOF
pranav@Pranav:~/task6$ # Step 3: Multi-Stage Dockerfile
cat > Dockerfile <<'EOF'
# === Stage 1: Builder ===
FROM python:3.11 AS builder

WORKDIR /build

COPY requirements.txt .

# Install dependencies into a local folder (not system)
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

# === Stage 2: Final (slim image) ===
FROM python:3.11-slim AS final

WORKDIR /app

# Copy only installed packages from builder
COPY --from=builder /install /usr/local

COPY main.py .

EXPOSE 8080

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
EOF
pranav@Pranav:~/task6$ # Step 4: docker-compose.yml
cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  fastapi-app:
    build:
      context: .
      dockerfile: Dockerfile
    target: final
    container_name: fastapi-multistage
    ports:
      - "8080:8080"
    environment:
      APP_ENV: production

```

```

pranav@Pranav: ~
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
EOF
pranav@Pranav:~/task6$ # Step 4: docker-compose.yml
cat > docker-compose.yml <<'EOF'
version: '3.8'

services:
  fastapi-app:
    build:
      context: .
      dockerfile: Dockerfile
      target: final
    container_name: fastapi-multistage
    ports:
      - "8080:8080"
    environment:
      APP_ENV: production
    volumes:
      - ./main.py:/app/main.py # hot-reload during development
EOF
pranav@Pranav:~/task6$ # Build and run
docker compose up --build -d
WARN[0000] /home/pranav/task6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Building 104.3s (17/17) FINISHED
=> [internal] load local bake definitions           0.0s
=> => reading from stdin 5298                      0.0s
=> [internal] load build definition from Dockerfile  0.0s
=> => transferring dockerfile: 5288                 0.0s
=> [internal] load metadata for docker.io/library/python:3.11 3.1s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.0s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                    0.0s
=> => transferring context: 2B                       0.0s
=> [builder 1/4] FROM docker.io/library/python:3.11@sha256:ff461875d046c85ecc529e93cf2a0004f29df705661949362141 95.2s
=> => resolve docker.io/library/python:3.11@sha256:ff461875d046c85ecc529e93cf2a0004f29df70566194936214115b36703d 0.0s
=> => sha256:ff461875d046c85ecc529e93cf2a0004f29df70566194936214115b36703d866 10.32kB / 10.32kB 0.0s
=> => sha256:e1388095fc3d7fd4f5611bd6b70464d8dc602a6189c9d5689def96add4b74a3a 6.35kB / 6.35kB 0.0s
=> => sha256:b012eb15df0bce418c03ec940325aee6aa4300d771c325728855697e620c63a 25.62MB / 25.62MB 20.2s
=> => sha256:ee3a0e7d7ff0c84203cab438fcf345647c8121bbd80506a3c692f8608a14c4f4 67.78MB / 67.78MB 36.7s
=> => sha256:990d2ceca3883d62ee15b9e7c06da32a9f9a6bb95d5c0b47548581e6e0a38d50 2.32kB / 2.32kB 0.0s
=> => sha256:8f6ad8580ba46fa8ee628532c70b8dc82406179d5493b0b09ec19fc03d4c5b373 49.30MB / 49.30MB 29.7s
=> => sha256:8688d9f2f670804eb217cf80cf8063bd13a1951e92e65caca8e5b736f5e1b 236.00MB / 236.00MB 87.8s
=> => extracting sha256:8f6ad8580ba46fa8ee628532c70b8dc82d06179d5493b0b09ec19fc03d4c5b373 1.6s
=> => sha256:66063df90a44c03620e1790b680bad5509bc860518ee257a157d6262916b680a 6.09MB / 6.09MB 33.9s
=> => extracting sha256:b012eb15df0bce418c03ec940325aee6aa4300d771c325728855697e620c63a 0.5s
=> => sha256:1589b6e505d3cd8ceb2b87cebc53c22b3bd9b858def90d5c108605b5d58db28 23.98MB / 23.98MB 44.5s
=> => sha256:8cd65a420aac1587da1c19e7cc7bd6f61b226f69fa9ec6f9d3f6215b9bf33cf2 249B / 249B 37.7s

```

```

pranav@Pranav: ~
docker compose up --build -d
WARN[0000] /home/pranav/task6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 104.3s (17/17) FINISHED
=> [internal] load local bake definitions
=> reading from stdin 528B
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 528B
=> [internal] load metadata for docker.io/library/python:3.11
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> transferring context: 2B
=> [builder 1/4] FROM docker.io/library/python:3.11@sha256:ff461875d046c85ecc529e93cf2a0004f29df705661949362141 95.2s
=> resolve docker.io/library/python:3.11@sha256:ff461875d046c85ecc529e93cf2a0004f29df70566194936214115b36703d 0.0s
=> sha256:ff461875d046c85ecc529e93cf2a0004f29df70566194936214115b36703d866 10.32kB / 10.32kB
=> sha256:e1388005fc3d7fd4f5611bd6b70464d8dc602a6189c9d5689def96add4b74a3a 6.35kB / 6.35kB
=> sha256:b012eb15dff0bce418c03ec940325aee6aa4300d771c325728855697e620c63a 25.62MB / 25.62MB
=> sha256:ee3a0e7d77f0c84203cab438fc345647c8121bbd80506a3c692f8608a14c4f4 67.78MB / 67.78MB
=> sha256:990d2ceca3883d62ee15b9e7c06da32a9f9a6bb95d5c0b47548581e6e0a38d50 2.32kB / 2.32kB
=> sha256:8f6ad858d0a46fa8ee628532c70b8dc82d06179d543b0b09ec19fc03d4c5b373 49.30MB / 49.30MB
=> sha256:8688d0f2f567884eb217c6f80efa063bdb13a1951e92e6c5cac1ae5b736f5e1b 236.08MB / 236.08MB
=> extracting sha256:8f6ad858d0a46fa8ee628532c70b8dc82d06179d543b0b09ec19fc03d4c5b373 1.6s
=> sha256:66063df90a44c93620e1790b680bad5509bc860518ee257a157d6262916b680a 6.09MB / 6.09MB
=> extracting sha256:b012eb15dff0bce418c03ec940325aee6aa4300d771c325728855697e620c63a 0.5s
=> sha256:1589b6e505d3cd8ceb2b87cebc53c22b3bd9b858def90d5c108605bbd58d8b28 23.98MB / 23.98MB
=> sha256:8cd65a420aac1587dalc19e7cc7bd6f61b226f69f9ec6f9d3f6215b9bf33cf2 249B / 249B
=> extracting sha256:ee3a0e7d77f0c84203cab438fc345647c8121bbd80506a3c692f8608a14c4f4 4.8s
=> extracting sha256:8688d0f2f567884eb217c6f80efa063bdb13a1951e92e6c5cac1ae5b736f5e1b 5.7s
=> extracting sha256:66063df90a44c93620e1790b680bad5509bc860518ee257a157d6262916b680a 0.3s
=> extracting sha256:1589b6e505d3cd8ceb2b87cebc53c22b3bd9b858def90d5c108605bbd58d8b28 0.7s
=> extracting sha256:8cd65a420aac1587dalc19e7cc7bd6f61b226f69f9ec6f9d3f6215b9bf33cf2 0.0s
=> [final 1/4] FROM docker.io/library/python:3.11-slim
=> [internal] load build context
=> transferring context: 254B
=> [final 2/4] WORKDIR /app
=> [builder 2/4] WORKDIR /build
=> [builder 3/4] COPY requirements.txt .
=> [builder 4/4] RUN pip install --prefix=/install --no-cache-dir -r requirements.txt 4.3s
=> [final 3/4] COPY --from=builder /install /usr/local
=> [final 4/4] COPY main.py .
=> exporting to image
=> exporting layers
=> writing image sha256:4f7cc05ea33b8428ea91f0415dca860cd3d7707cbe714fac2322c5bc1c3f21e0 0.0s
=> naming to docker.io/library/task6-fastapi-app
=> resolving provenance for metadata file
[+] up 3/3
Image task6-fastapi-app Built 96.3s

```

```

=> sha256:990d2ceca3883d62ee15b9e7c06da32a9f9a6bb95d5c0b47548581e6e0a38d50 2.32kB / 2.32kB
=> sha256:8f6ad858d0a46fa8ee628532c70b8dc82d06179d543b0b09ec19fc03d4c5b373 49.30MB / 49.30MB
=> sha256:8688d0f2f567884eb217c6f80efa063bdb13a1951e92e6c5cac1ae5b736f5e1b 236.08MB / 236.08MB
=> extracting sha256:8f6ad858d0a46fa8ee628532c70b8dc82d06179d543b0b09ec19fc03d4c5b373 1.6s
=> sha256:66063df90a44c93620e1790b680bad5509bc860518ee257a157d6262916b680a 6.09MB / 6.09MB
=> extracting sha256:b012eb15dff0bce418c03ec940325aee6aa4300d771c325728855697e620c63a 0.5s
=> sha256:1589b6e505d3cd8ceb2b87cebc53c22b3bd9b858def90d5c108605bbd58d8b28 23.98MB / 23.98MB
=> sha256:8cd65a420aac1587dalc19e7cc7bd6f61b226f69f9ec6f9d3f6215b9bf33cf2 249B / 249B
=> extracting sha256:ee3a0e7d77f0c84203cab438fc345647c8121bbd80506a3c692f8608a14c4f4 4.8s
=> extracting sha256:8688d0f2f567884eb217c6f80efa063bdb13a1951e92e6c5cac1ae5b736f5e1b 5.7s
=> extracting sha256:66063df90a44c93620e1790b680bad5509bc860518ee257a157d6262916b680a 0.3s
=> extracting sha256:1589b6e505d3cd8ceb2b87cebc53c22b3bd9b858def90d5c108605bbd58d8b28 0.7s
=> extracting sha256:8cd65a420aac1587dalc19e7cc7bd6f61b226f69f9ec6f9d3f6215b9bf33cf2 0.0s
=> [final 1/4] FROM docker.io/library/python:3.11-slim
=> [internal] load build context
=> transferring context: 254B
=> [final 2/4] WORKDIR /app
=> [builder 2/4] WORKDIR /build
=> [builder 3/4] COPY requirements.txt .
=> [builder 4/4] RUN pip install --prefix=/install --no-cache-dir -r requirements.txt 4.3s
=> [final 3/4] COPY --from=builder /install /usr/local
=> [final 4/4] COPY main.py .
=> exporting to image
=> exporting layers
=> writing image sha256:4f7cc05ea33b8428ea91f0415dca860cd3d7707cbe714fac2322c5bc1c3f21e0 0.0s
=> naming to docker.io/library/task6-fastapi-app
=> resolving provenance for metadata file
[+] up 3/3
Image task6-fastapi-app Built 96.3s
Network task6_default Created 0.0s
Container fastapi-multistage Created 0.1s
pranav@Pranav:~/task6$ # Verify
curl http://localhost:8080/
{"message":"Hello from # Compare image sizes!"}pranav@Pranav:~/task6$ # Compare image sizes
docker images | grep -E "python|fastapi"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
python:3.11-slim e67db9b14d09 124MB 0B
task6-fastapi-app:latest 4f7cc05ea33b 139MB 0B U
pranav@Pranav:~/task6$ # Cleanup
docker compose down
docker rmi task6-fastapi-app
cd .
WARN[0000] /home/pranav/task6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
Container fastapi-multistage Removed 0.6s
Network task6_default Removed 0.3s
Untagged: task6-fastapi-app:latest

```

Common Pitfalls

Pitfall	Symptom	Root Cause	Fix
Missing <code>html/</code> directory	NGINX serves 403 Forbidden	Volume mount path doesn't exist on host	<code>mkdir -p html && echo ... > html/index.html</code>
Missing <code>WORDPRESS_DB_USER</code>	WordPress setup fails with DB error	WordPress doesn't know which MySQL user to connect as	Add <code>-e WORDPRESS_DB_USER=root</code>
No <code>sleep</code> between MySQL and WordPress (<code>docker run</code>)	WordPress: "Error establishing a database connection"	MySQL container starts in ~1s, MySQL engine needs 10–15s	Add <code>sleep 15</code> between the two <code>docker run</code> commands
Missing top-level <code>networks:</code> in Compose	<code>network app-net declared as external, but could not be found</code>	Referenced network not declared at Compose root level	Add <code>networks: app-net:</code> at the bottom of the file
<code>deploy:</code> not applying limits	Container has no memory cap despite the config	Running old <code>docker-compose</code> v1 (silently ignores <code>deploy:</code>)	Use <code>docker compose</code> (v2) — check with <code>docker compose version</code>
<code>docker compose up --build</code> not rebuilding	Stale image served despite code changes	Compose uses layer cache; host file wasn't COPYed	Use <code>docker compose build --no-cache</code> if cache is stale
<code>depends_on</code> not enough	App crashes on start because DB isn't ready	<code>depends_on</code> only waits for container START, not DB READY	Add a <code>healthcheck:</code> to the DB service + <code>condition: service_healthy</code>

Final Cleanup

 **Cleanup Section Added:** Run these commands at the end of the lab to remove all experiment artefacts.

```
# Stop and remove all stopped containers
docker container prune -f

# Remove all dangling images (untagged build intermediates)
docker image prune -f

# Remove all unused volumes (CAUTION: removes persistent data)
docker volume prune -f

# Remove all unused networks
docker network prune -f

# Or: remove everything at once (containers, images, networks, volumes)
docker system prune -a --volumes -f
```

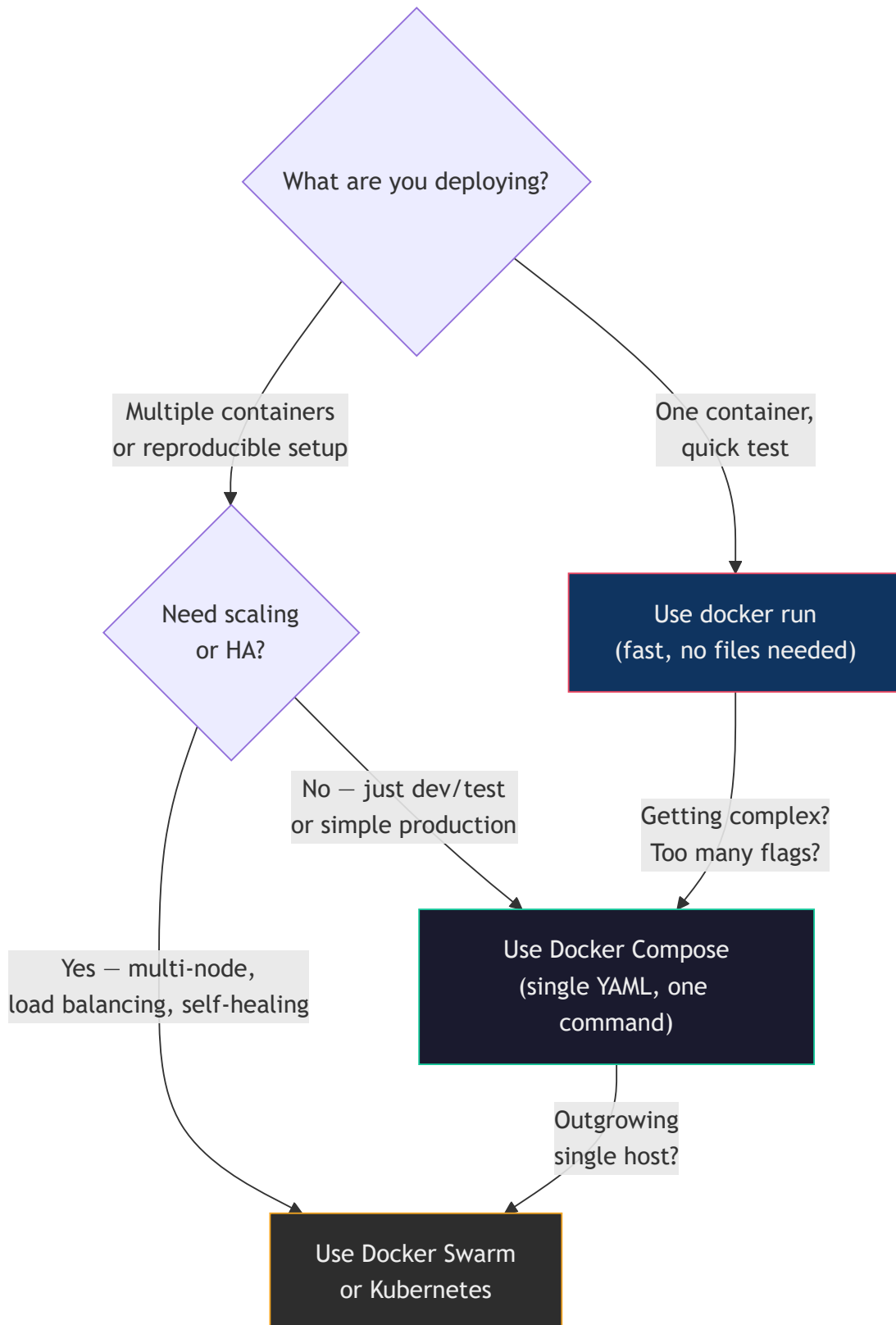
⚠ **Warning:** `docker system prune -a --volumes` removes ALL unused resources — not just from this experiment. Only run it if you want a completely clean Docker environment.

Quick Reference Cheatsheet

Action	docker run	docker compose
Start single container	<code>docker run -d -p 8081:80 nginx</code>	<code>docker compose up -d</code>
View running containers	<code>docker ps</code>	<code>docker compose ps</code>
View logs	<code>docker logs <name></code>	<code>docker compose logs -f</code>
Stop container	<code>docker stop <name></code>	<code>docker compose down</code>
Remove container	<code>docker rm <name></code>	included in <code>docker compose down</code>
Scale service	✗ Manual	<code>docker compose up --scale web=3</code>
Build from Dockerfile	<code>docker build -t myapp . && docker run myapp</code>	<code>docker compose up --build</code>
Check resource usage	<code>docker stats <name> --no-stream</code>	<code>docker stats --no-stream</code>



Decision Flowchart — When to Use Which Tool



Key Takeaways

1. `docker run` is best for quick, one-off containers and debugging. Every option must be typed manually — easy to miss flags.
2. **Docker Compose** is best for reproducible, multi-container setups. The YAML file is the single source of truth.
3. `depends_on` **is not enough** — always combine it with a `healthcheck` and `condition: service_healthy` for databases.
4. **Every named network must be declared at the top level** of the Compose file or Compose will throw an error.
5. **Multi-stage builds** dramatically reduce final image size — ship only what the runtime needs.
6. `docker stats` is the fastest way to verify resource limits are actually enforced.

Experiment Summary: This lab demonstrated that `docker run` and `docker compose` are not competing tools — they are different levels of abstraction. Master the flag-to-YAML mapping, understand the common pitfalls (especially MySQL readiness and network declarations), and you will be equipped to confidently deploy and debug any containerized application.

Student: Pranav R Nair | **SAP ID:** 500121466 | **Batch:** 2(CCVT)

Objective

Design and implement a complete **CI/CD pipeline** where every `git push` to GitHub automatically triggers Jenkins to build a Docker image and push it to Docker Hub — achieving fully automated continuous integration and delivery.

Theory — What Is CI/CD?

The Assembly Line Analogy

Think of a **car manufacturing plant**:

Factory Stage	CI/CD Equivalent
Designer draws blueprints	Developer writes code and pushes to GitHub
Motion sensor detects new blueprint arrived	GitHub webhook fires an HTTP POST to Jenkins
Assembly line starts automatically	Jenkins pipeline begins executing stages
Parts are assembled into a car	Docker builds the application into an image
Quality inspection	Tests run (if configured)
Car is shipped to the dealership	Docker image is pushed to Docker Hub

Without CI/CD, a human has to walk each blueprint to the factory, start the machines, inspect the car, and drive it to the dealership manually. CI/CD **automates the entire chain**.

CI (Continuous Integration)

Every commit pushed to the repository is **automatically built and tested**. The goal is to catch bugs early — before they reach production.

CD (Continuous Delivery / Deployment)

Once the build passes, the **artifact** (in our case, a Docker image) is automatically **delivered** to a registry (Docker Hub) or **deployed** to a server.

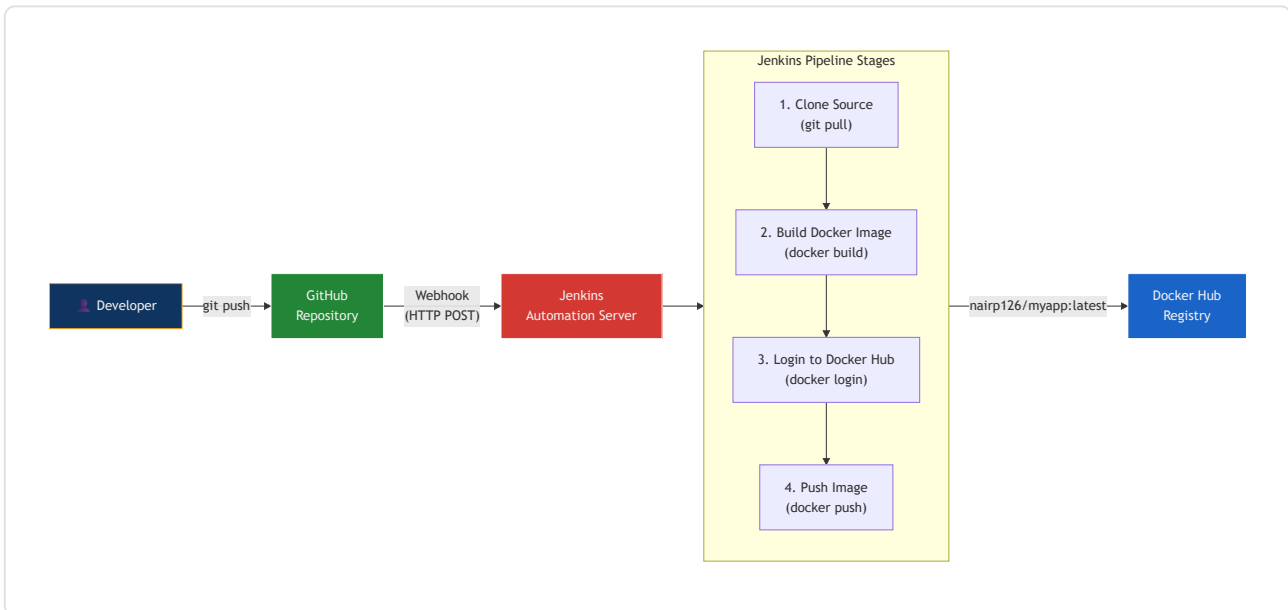
Jenkins

Jenkins is a **web-based automation server** that:

- Provides a browser-based dashboard for managing pipelines
- Uses **Pipeline as Code** — the build definition lives in a `Jenkinsfile` in your repo

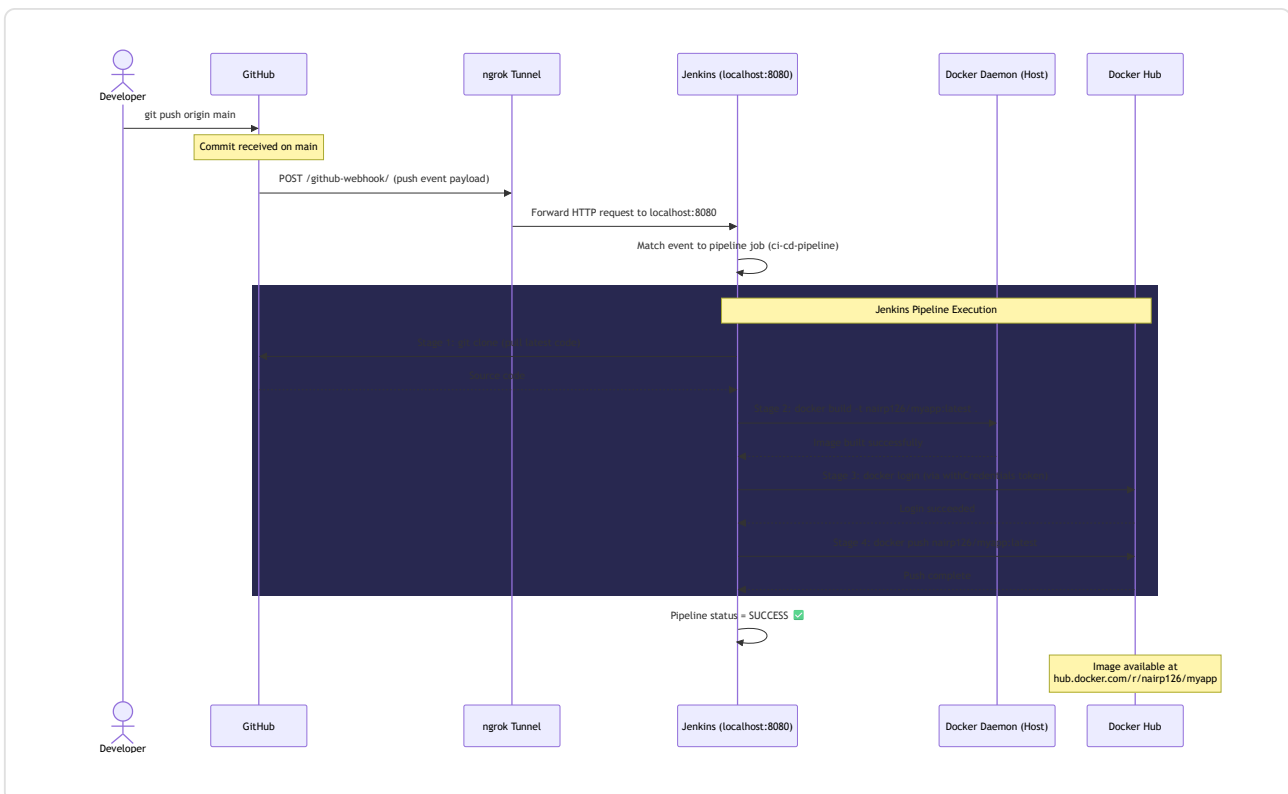
- Has a plugin ecosystem (GitHub, Docker, Slack, Kubernetes, etc.)
- Runs on `localhost:8080` by default

The Pipeline in This Lab



Complete Event Sequence — From `git push` to Docker Hub

The flowchart above shows *what* happens. This sequence diagram shows *who talks to whom, and when*:



Key observations from the sequence:

1. **GitHub initiates** — The developer pushes code, but it's GitHub that calls Jenkins (not the other way around)
2. **ngrok bridges the gap** — Without it, GitHub can't reach `localhost:8080`
3. **Jenkins orchestrates** — It doesn't build images itself; it tells the Docker daemon what to do via the mounted socket
4. **Docker Hub is the final destination** — The built image is pushed to a public registry, available for deployment anywhere

Prerequisites

- Docker & Docker Compose installed (Docker Desktop on Windows with WSL integration)
- A GitHub account with a personal access token
- A Docker Hub account with an access token
- Basic Linux command-line knowledge

Part A — GitHub Repository Setup

Step 1 — Create Project Directory

```
mkdir ~/my-app && cd ~/my-app
```

Step 2 — Create the Flask Application

`app.py`

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

Code	Purpose
<code>Flask(__name__)</code>	Creates a Flask web application instance
<code>@app.route("/")</code>	Maps the root URL <code>/</code> to the <code>home()</code> function
<code>host="0.0.0.0"</code>	Listens on all network interfaces (not just localhost) — required inside a container

Code	Purpose
<code>port=80</code>	Standard HTTP port

`requirements.txt`

```
flask
```

Pip will install Flask and its dependencies (Werkzeug, Jinja2, etc.) during the Docker build.

Step 3 — Create the Dockerfile

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

Line-by-Line Breakdown

Instruction	What It Does	Why
<code>FROM python:3.10-slim</code>	Uses a minimal Python 3.10 base image (~45 MB)	Slim variant excludes compilers and docs — smaller image, faster pulls
<code>WORKDIR /app</code>	Sets <code>/app</code> as the working directory inside the container	All subsequent commands run from here
<code>COPY . .</code>	Copies all project files into the container	Brings <code>app.py</code> , <code>requirements.txt</code> into the image
<code>RUN pip install -r requirements.txt</code>	Installs Python dependencies at build time	Baked into the image layer — no network needed at runtime
<code>EXPOSE 80</code>	Documents that the container listens on port 80	Informational metadata — does not actually publish the port
<code>CMD ["python", "app.py"]</code>	Default command when the container starts	Exec form (no shell wrapping) — proper signal handling

Step 4 — Create the Jenkinsfile

The `Jenkinsfile` is the **Pipeline as Code** definition — it tells Jenkins exactly what to do:

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = "nairp126/myapp"
  }

  stages {

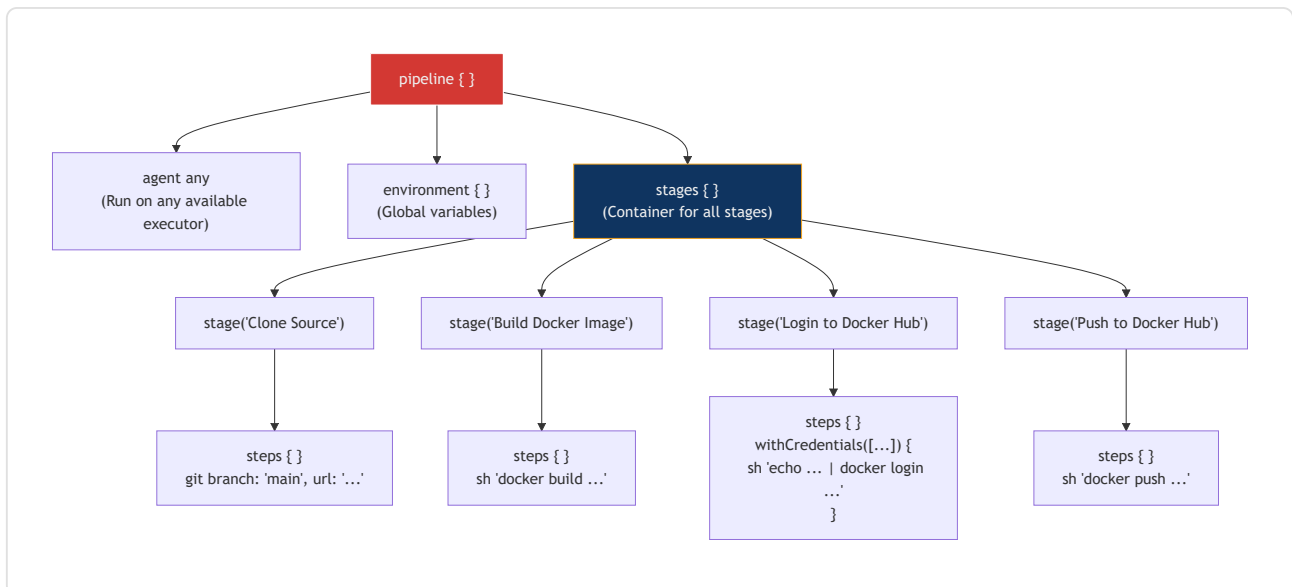
    stage('Clone Source') {
      steps {
        git branch: 'main', url: 'https://github.com/nairp126/my-app.git'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME:latest .'
      }
    }

    stage('Login to Docker Hub') {
      steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
          sh 'echo $DOCKER_TOKEN | docker login -u nairp126 --password-stdin'
        }
      }
    }

    stage('Push to Docker Hub') {
      steps {
        sh 'docker push $IMAGE_NAME:latest'
      }
    }
  }
}
```

Jenkinsfile Structure Explained



Keyword	Purpose
<code>pipeline { }</code>	Root block — everything goes inside this
<code>agent any</code>	Run the pipeline on any available Jenkins executor (in our case, the same Docker host)
<code>environment { }</code>	Define variables available to all stages — <code>IMAGE_NAME</code> avoids repeating <code>nairp126/myapp</code>
<code>stages { }</code>	Groups all pipeline phases
<code>stage('Name') { }</code>	A single logical phase — shown as a block in the Jenkins UI
<code>steps { }</code>	Contains the actual commands to execute
<code>git</code>	Built-in Jenkins step to clone a Git repository
<code>sh</code>	Execute a shell command
<code>withCredentials</code>	Temporarily injects a stored secret into an environment variable

Deep Dive — withCredentials (The Tricky Part)

This is the most confusing block for beginners. Here's what's happening:

```

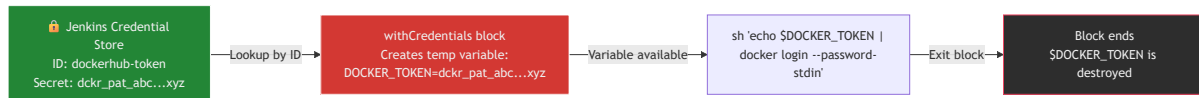
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
  sh 'echo $DOCKER_TOKEN | docker login -u nairp126 --password-stdin'
}

```

Step-by-step:

1. Jenkins looks up the credential stored with ID `dockerhub-token` in its credential store

2. It creates a **temporary** environment variable called `DOCKER_TOKEN` containing the secret value
3. Inside the `{ }` block, `$DOCKER_TOKEN` resolves to the actual token
4. `echo $DOCKER_TOKEN | docker login --password-stdin` pipes the token into Docker login securely
5. Once the block ends, `$DOCKER_TOKEN` is destroyed — it's never written to logs or disk



Why not just write the password in the Jenkinsfile? Because the Jenkinsfile is committed to GitHub — anyone with repo access could read it. `withCredentials` keeps secrets in Jenkins's encrypted store, never in version control.

Step 5 — Initialize Git and Push to GitHub

```
git init
git add .
git commit -m "Initial CI/CD setup"
git branch -M main
git remote add origin https://github.com/nairp126/my-app.git
git push -u origin main
```

Command	Purpose
<code>git init</code>	Initialize a new Git repository
<code>git add .</code>	Stage all files for commit
<code>git commit -m "..."</code>	Create the initial commit
<code>git branch -M main</code>	Rename the default branch from <code>master</code> to <code>main</code>
<code>git remote add origin</code>	Link local repo to the GitHub remote
<code>git push -u origin main</code>	Push to GitHub and set <code>main</code> as the upstream tracking branch

Screenshots — Project creation and git push:

```

pranav@Pranav: ~/my-app
pranav@Pranav:~$ mkdir ~/my-app && cd ~/my-app
pranav@Pranav:~/my-app$ cat > app.py << 'EOF'
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
EOF
pranav@Pranav:~/my-app$ cat > requirements.txt << 'EOF'
flask
EOF
pranav@Pranav:~/my-app$ cat > Dockerfile << 'EOF'
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
EOF
pranav@Pranav:~/my-app$ cat > Jenkinsfile << 'EOF'
pipeline {
    Docker
    FROM python:3.10-slim
    Jenkinsfile
    FROM python:3.10-slim
    FROM python:3.10-slim
    FROM python:3.10-slim
    FROM python:3.10-slim
    WORKDIR /app
    COPY . .
    RUN pip install -r requirements.txt
    EXPOSE 80
    CMD ["python", "app.py"]
    EOF
pranav@Pranav:~/my-app$ cat > Jenkinsfile << 'EOF'
pipeline {
    agent any

    environment {
        IMAGE_NAME = "nairp126/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {

```

```

CMD ["python", "app.py"]
EOF
pranav@Pranav:~/my-app$ cat > Jenkinsfile << 'EOF'
pipeline {
    agent any

    environment {
        IMAGE_NAME = "nairp126/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/nairp126/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u YOUR_DOCKERHUB_USERNAME --password-stdin'
                }
            }
        }
    }
}
EOF } } } sh 'docker push $IMAGE_NAME:latest'
pranav@Pranav:~/my-app$ git init
git add .
git commit -m "Initial CI/CD setup"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/my-app.git
git push -u origin main
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:     git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:

```

```

pranav@Pranav: ~/my-app$ git push -u origin main
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
remote: Permission to nairp126/my-app.git denied to nairp126.
fatal: unable to access 'https://github.com/nairp126/my-app.git/': The requested URL returned error: 403
pranav@Pranav:~/my-app$ git push -u origin main
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 927 bytes | 927.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/nairp126/my-app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
pranav@Pranav:~/my-app$ cd ~
mkdir jenkins-setup && cd jenkins-setup
pranav@Pranav:~/jenkins-setup$ cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
EOF
pranav@Pranav:~/jenkins-setup$ docker compose up -d
WARN[0000] /home/pranav/jenkins-setup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 16/16
✔Image jenkins/jenkins:lts          Pulled           6.4s
✔Network jenkins-setup_default      Created         0.0s
✔Volume jenkins-setup_jenkins_home  Created         0.0s
✔Container jenkins                  Created         0.1s
pranav@Pranav:~/jenkins-setup$ docker ps

```

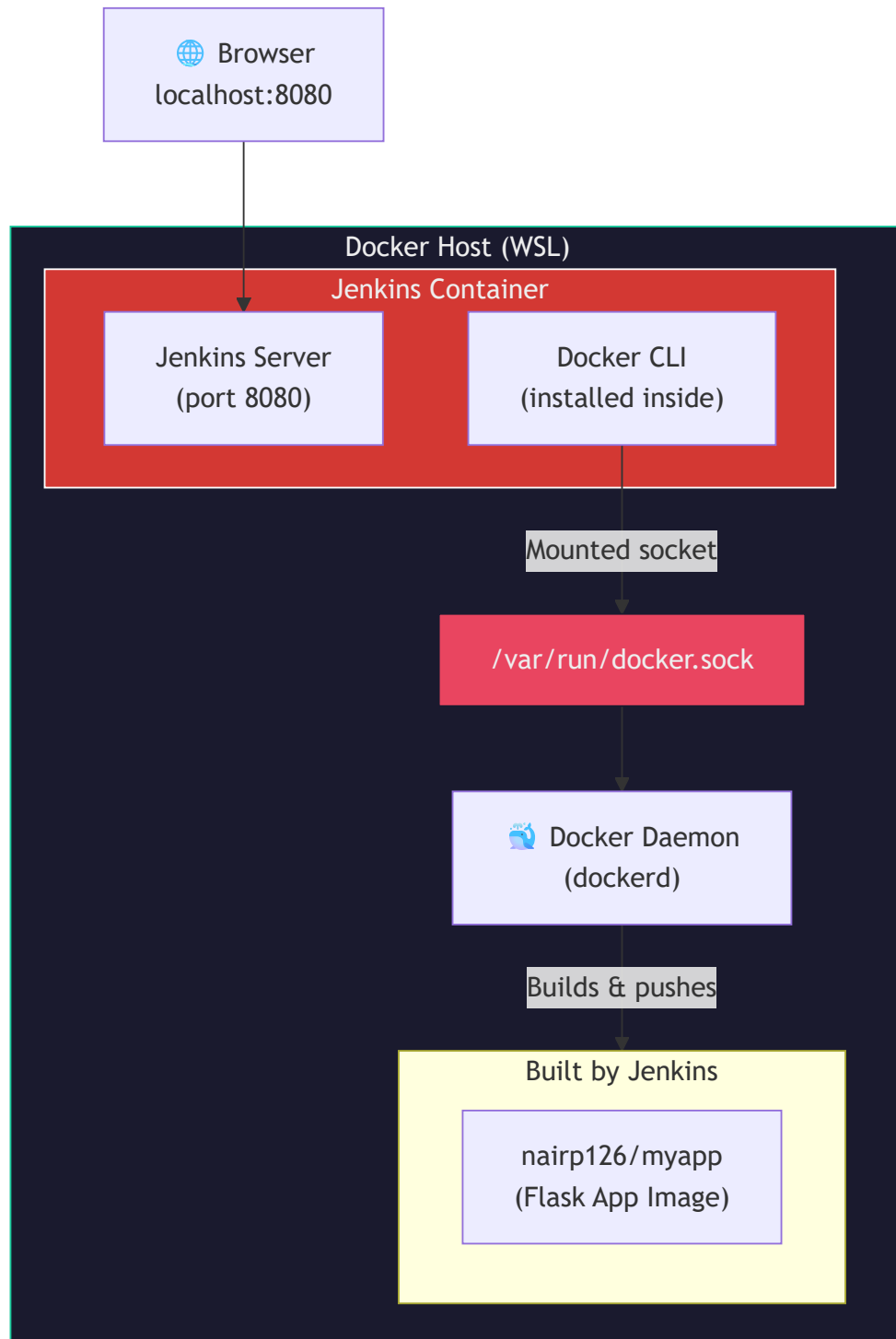
Part B — Jenkins Setup Using Docker

Why Run Jenkins in Docker?

Running Jenkins as a container provides:

- **Isolation** — Jenkins and its dependencies don't clutter your host system
- **Reproducibility** — Same `docker-compose.yml` = same Jenkins on any machine
- **Persistence** — Named volumes preserve Jenkins data across container restarts

Architecture — Jenkins + Docker Socket



Key concept: Jenkins runs *inside* a container, but uses the **host's Docker daemon** via the mounted socket. When Jenkins runs `docker build`, it's the host's Docker that does the actual building. This is called **Docker-outside-of-Docker (DooD)**.

Step 1 — Create Docker Compose File

```
mkdir ~/jenkins-setup && cd ~/jenkins-setup
```

`docker-compose.yml`

```
version: '3.8'

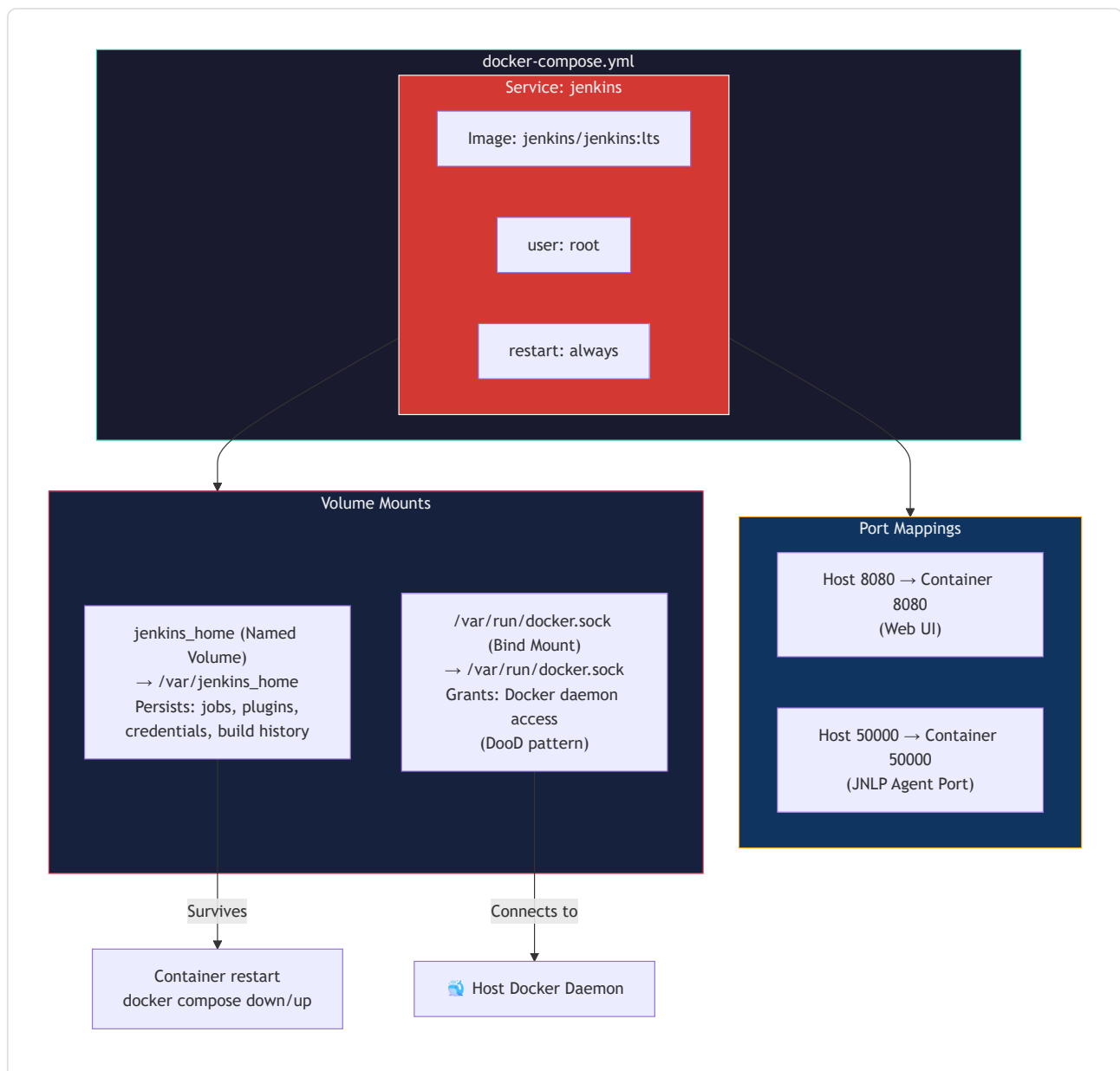
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Configuration Breakdown

Key	Value	Purpose
<code>image</code>	<code>jenkins/jenkins:lts</code>	Official Jenkins Long-Term Support image — stable, patched regularly
<code>container_name</code>	<code>jenkins</code>	Fixed name for easy <code>docker exec</code> access
<code>restart</code>	<code>always</code>	Auto-restart on crash or Docker daemon restart
<code>ports: 8080</code>	<code>8080:8080</code>	Jenkins web UI — access via browser
<code>ports: 50000</code>	<code>50000:50000</code>	JNLP agent port — used by remote build agents (not needed in single-node setup, but included for completeness)
<code>volumes: jenkins_home</code>	Named volume	Persists Jenkins configuration, jobs, plugins, and build history across container restarts
<code>volumes: docker.sock</code>	Bind mount	Gives Jenkins access to the host Docker daemon (Docker-outside-of-Docker pattern)
<code>user</code>	<code>root</code>	Required for writing to the Docker socket — the socket is owned by <code>root:docker</code>

Docker Compose Service Architecture



Why two ports?

Port	Protocol	Used For	Needed in This Lab?
8080	HTTP	Jenkins web dashboard and webhook endpoint	✓ Yes — core functionality
50000	TCP/JNLP	Communication with remote Jenkins agents (distributed builds)	✗ No — we use a single-node setup, but included for future expansion

Why `user: root` ?

The Docker socket (`/var/run/docker.sock`) is owned by `root:docker` on the host. The Jenkins container runs as the `jenkins` user by default, which doesn't have permission to read/write the socket. Setting `user: root` is the simplest workaround for a lab environment.

Production alternative: Instead of running as root, add the `jenkins` user to the `docker` group inside the container, or use a Docker socket proxy like [Tecnativa/docker-socket-proxy](https://github.com/Tecnativa/docker-socket-proxy) to limit API access.

Step 2 — Start Jenkins

```
docker compose up -d
```

Flag	Purpose
<code>up</code>	Creates and starts all services defined in <code>docker-compose.yml</code>
<code>-d</code>	Detached mode — runs in background

Note: Use `docker compose` (with a space), not `docker-compose` (with a hyphen). See the deep dive below.

Deep Dive — `docker-compose` VS `docker compose`

	<code>docker-compose</code> (V1)	<code>docker compose</code> (V2)
Type	Standalone Python binary	Built-in Docker CLI plugin (Go)
Installation	Separate install (<code>pip install docker-compose</code> or download binary)	Ships with Docker Desktop and <code>docker-ce-cli</code> ≥ 20.10
Command	<code>docker-compose up -d</code>	<code>docker compose up -d</code>
Status	⚠️ Deprecated (July 2023) — no longer receives updates	✅ Actively maintained
Performance	Slower (Python startup overhead)	Faster (native Go binary)
Container naming	Uses <code>_</code> separator: <code>project_service_1</code>	Uses <code>-</code> separator: <code>project-service-1</code>

What changed? Docker Inc. rewrote Compose from Python (V1) to Go (V2) and integrated it directly into the Docker CLI as a plugin. The old `docker-compose` binary still works if installed, but new installations won't have it. Always use the V2 syntax (`docker compose` with a space) in modern environments.

Expected Output:

```
[+] Running 16/16
✓ Image jenkins/jenkins:lts Pulled 6.4s
✓ Network jenkins-setup_default Created 0.0s
✓ Volume jenkins-setup_jenkins_home Created 0.0s
✓ Container jenkins Created 0.1s
```

Step 3 — Get the Initial Admin Password

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Part	Meaning
<code>docker exec -it jenkins</code>	Run a command inside the running <code>jenkins</code> container with an interactive terminal
<code>cat</code> <code>/var/jenkins_home/secrets/initialAdminPassword</code>	Read the auto-generated admin password

Expected Output: A 32-character hex string like `5200d1dec42b44d98e0010ab7d81dcf1`

Copy this password and paste it at `http://localhost:8080` to unlock Jenkins.

Step 4 — Install Docker Inside the Jenkins Container

⚠ **This step is NOT in the lab PDF** but is **required** for the pipeline to work.

The `jenkins/jenkins:lts` image does not include the Docker CLI. Without it, the `sh 'docker build ...'` stage fails with `docker: command not found`.

```
docker exec -it jenkins bash
apt-get update && apt-get install -y docker.io
docker --version
exit
```

Command	Purpose
<code>docker exec -it jenkins bash</code>	Open an interactive shell inside the Jenkins container
<code>apt-get update</code>	Refresh the package index
<code>apt-get install -y docker.io</code>	Install the Docker CLI package
<code>docker --version</code>	Verify installation (should show <code>Docker version 26.x.x</code>)

Command	Purpose
<code>exit</code>	Return to the host shell

Why this works: After installation, the Docker CLI inside Jenkins communicates with the Docker daemon via the mounted socket (`/var/run/docker.sock`). It doesn't run its own daemon — it uses the host's.

Screenshots — Jenkins startup, admin password, and Docker installation:

```

pranav@Pranav:~/my-app$ git push -u origin main
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
remote: Permission to nairp126/my-app.git denied to nairp126.
fatal: unable to access 'https://github.com/nairp126/my-app.git/': The requested URL returned error: 403
pranav@Pranav:~/my-app$ git push -u origin main
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 927 bytes | 927.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/nairp126/my-app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
pranav@Pranav:~/my-app$ cd ~
mkdir jenkins-setup && cd jenkins-setup
pranav@Pranav:~/jenkins-setup$ cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
EOF
pranav@Pranav:~/jenkins-setup$ docker compose up -d
WARN[0000] /home/pranav/jenkins-setup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 16/16
✔Image jenkins/jenkins:lts      Pulled          6.4s
✔Network jenkins-setup_default  Created          0.0s
✔Volume jenkins-setup_jenkins_home Created          0.0s
✔Container jenkins              Created          0.1s
pranav@Pranav:~/jenkins-setup$ docker ps

```

```

jenkins_home:
EOF
pranav@Pranav: ~/jenkins-setup$ docker compose up -d
WARN[0000] /home/pranav/jenkins-setup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confu
sion
[*] up 16/16
✓Image jenkins/jenkins:lts Pulled 6.4s
✓Network jenkins-setup_default Created 0.0s
✓Volume jenkins-setup_jenkins_home Created 0.0s
✓Container jenkins Created 0.1s
pranav@Pranav: ~/jenkins-setup$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7dbb2f1f9b47 jenkins/jenkins:lts "/usr/bin/tini -- /u" 26 seconds ago Up 25 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->5
0000/tcp, [::]:50000->50000/tcp jenkins
pranav@Pranav: ~/jenkins-setup$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
5200d1dec42b44d98e0010ab7d81dcf1
pranav@Pranav: ~/jenkins-setup$ docker exec -it jenkins bash
root@7dbb2f1f9b47:/# apt-get update && apt-get install -y docker.io
docker --version
exit
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [114 kB]
Fetched 10.0 MB in 2s (4420 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
apparmor containerd criu dbus dbus-bin dbus-daemon dbus-session-bus-common dbus-system-bus-common dbus-user-session
dmsup docker-buildx docker-cli gettext-base iproute2 iptables libapparmor1 libbpf1 libcap2 libcap2-bin libcompell
libcryptsetup12 libdbus-1-3 libdevmapper1.02.1 libelf1t64 libintl-perl libintl-xs-perl libip4tc2 libip6tc2
libjansson4 libjson-c5 libkmod2 liblmdb0 libmodule-find-perl libnet1 libnetfilter-contrack3 libnfnetlink0
libnftables1 libnftnl1 libnl-3-200 libnss-systemd libpam-cap libpam-systemd libproc-processtable-perl
libprotobuf-c1 libprotobuf32t64 libpython3-stdlib libpython3.13-minimal libpython3.13-stdlib libreadline8t64
libsort-naturally-perl libsystemd-shared libterm-readkey-perl libtirpc-common libtirpc3t64 libxtables12
linux-sysctl-defaults media-types needrestart netbase nftables python3 python3-minimal python3-protobuf
python3-pycrui python3.13 python3.13-minimal readline-common runc systemd systemd-cryptsetup systemd-sysv
systemd-timesyncd xz-utils
Suggested packages:
apparmor-profiles-extra apparmor-utils containernetworking-plugins docker-doc aufs-tools btrfs-progs cgroupfs-mount
debootstrap e2fsprogs rinse rootlesskit xfsprogs zfs-fuse | zfsutils-linux firewallld kmod libtss2-rc0t64
libarchive13t64 libdw1t64 libgcrypt20 libgqrencode4 needrestart-session | libnotify-bin iucode-tool

```

```

sysctl: permission denied on key "net.ipv4.conf.lo.rp_filter"
sysctl: permission denied on key "net.ipv4.conf.default.accept_source_route"
sysctl: permission denied on key "net.ipv4.conf.eth0.accept_source_route"
sysctl: permission denied on key "net.ipv4.conf.lo.accept_source_route"
sysctl: permission denied on key "net.ipv4.conf.default.promote_secondaries"
sysctl: permission denied on key "net.ipv4.conf.eth0.promote_secondaries"
sysctl: permission denied on key "net.ipv4.ping_group_range"
sysctl: permission denied on key "fs.protected_hardlinks"
sysctl: permission denied on key "fs.protected_symlinks"
sysctl: permission denied on key "fs.protected_regular"
sysctl: permission denied on key "fs.protected_fifos"
sysctl: permission denied on key "vm.max_map_count"
sysctl: permission denied on key "kernel.pid_max"
Processing triggers for libc-bin (2.41-12) ...
Processing triggers for systemd (257.9-1-deb13u1) ...
Docker version 26.1.5+dfsg1, build a72d7cd
exit
pranav@Pranav: ~/jenkins-setup$ cd ~/my-app
pranav@Pranav: ~/my-app$ cat > Jenkinsfile << 'EOF'
pipeline {
  agent any

  environment {
    IMAGE_NAME = "nairp126/myapp"
  }

  stages {
    stage('Clone Source') {
      steps {
        git branch: 'main', url: 'https://github.com/nairp126/my-app.git'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME:latest .'
      }
    }

    stage('Login to Docker Hub') {
      steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
          sh 'echo $DOCKER_TOKEN | docker login -u nairp126 --password-stdin'
        }
      }
    }
  }
}

```

Step 5 — Complete Initial Setup in Browser

1. Open <http://localhost:8080>

2. Paste the initial admin password
3. Click **"Install suggested plugins"** (wait for installation to complete)
4. Create your admin user (username, password, email)
5. Accept the default Jenkins URL

Part C — Jenkins Configuration

Step 1 — Add Docker Hub Credentials

Path: Manage Jenkins → Credentials → (global) → Add Credentials

Field	Value
Kind	Secret text
Secret	Your Docker Hub access token (generate from hub.docker.com → Account Settings → Security → New Access Token)
ID	<code>dockerhub-token</code>
Description	Docker Hub access token

The `ID` must exactly match the `credentialsId` in the Jenkinsfile. If they don't match, the pipeline fails with `CredentialNotFoundException`.

Step 2 — Create the Pipeline Job

1. Dashboard → New Item
2. Name: `ci-cd-pipeline`
3. Type: Pipeline
4. Click **OK**

Configure the pipeline:

Setting	Value
Build Triggers	☒ GitHub hook trigger for GITScm polling
Pipeline Definition	Pipeline script from SCM
SCM	Git
Repository URL	<code>https://github.com/nairp126/my-app.git</code>

Setting	Value
Branch Specifier	<code>*/main</code>
Script Path	<code>Jenkinsfile</code>

Click **Save**.

Part D — GitHub Webhook Integration

Why Webhooks?

Without webhooks, you would have to manually click "Build Now" in Jenkins after every code push. Webhooks make it **automatic** — GitHub tells Jenkins "new code arrived" via an HTTP POST request.

The Problem: Jenkins on Localhost

The lab PDF assumes Jenkins runs on a public server with a fixed IP. But we're running Jenkins inside WSL on `localhost` — GitHub's servers can't reach it over the internet.

Solution: Use **ngrok** to create a temporary public tunnel.

Step 1 — Install ngrok

```
curl -Lo ngrok.zip https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-amd64.zip
sudo apt install unzip -y
unzip ngrok.zip
sudo mv ngrok /usr/local/bin/
```

Step 2 — Authenticate ngrok

```
ngrok config add-authtoken YOUR_NGROK_AUTH_TOKEN
```

Get your token from dashboard.ngrok.com.

Step 3 — Start the Tunnel

```
ngrok http 8080
```

Expected Output:

```

Session Status    online
Forwarding        https://polygenistic-reina-uncasked.ngrok-free.dev → http://localhost:8080

```

The HTTPS URL printed in the **Forwarding** line is your **public Jenkins URL**.

How ngrok Works



Step 4 — Configure GitHub Webhook

Path: GitHub Repo → Settings → Webhooks → Add webhook

Field	Value
Payload URL	<code>https://<your-ngrok-url>/github-webhook/</code> (trailing slash is required)
Content type	<code>application/json</code>
Which events?	Just the push event
Active	☒ Checked

Important: Copy the **exact** URL from the ngrok terminal output. Every ngrok restart generates a new random URL — you must update the webhook if ngrok restarts.

Screenshot — ngrok showing successful webhook delivery (200 OK):

```

ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             Pranav Nair (Plan: Free)
Version             3.37.3
Region              India (in)
Latency              61ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://polygenistic-reina-uncasked.ngrok-free.dev -> http://localhost:8080

Connections
  ttl   opn   rt1   rt5   p50   p90
    1    0    0.00  0.00  30.27 30.27

HTTP Requests
-----
20:35:53.833 UTC POST /github-webhook/      200 OK

```

Part E — Testing the Pipeline

Test 1 — Manual Build

Click "**Build Now**" in the Jenkins dashboard to verify the pipeline works before testing webhooks.

Test 2 — Webhook-Triggered Build

Make a code change, commit, and push:

```

cd ~/my-app
echo "# test" >> README.md
git add .
git commit -m "Test webhook"
git push

```

What should happen automatically:

1. GitHub fires a webhook POST to the ngrok URL
2. ngrok tunnels the request to Jenkins on localhost:8080
3. Jenkins receives the event and starts the pipeline
4. Clone → Build → Login → Push stages execute

5. Docker image `nairp126/myapp:latest` appears on Docker Hub

Screenshots — Pushing changes and verifying webhook:

```

pranav@Pranav: ~/my-app  x  Ubuntu  x  +  v
    steps {
    }
    sh 'docker build -t $IMAGE_NAME:latest .'
  }
}

stage('Login to Docker Hub') {
  steps {
    withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
      sh 'echo $DOCKER_TOKEN | docker login -u nairp126 --password-stdin'
    }
  }
}

EOF } } } sh 'docker push $IMAGE_NAME:latest'
pranav@Pranav:~/my-app$ git add Jenkinsfile
git commit -m "Fix: use main branch in Jenkinsfile"
git push
[main fe056c7] Fix: use main branch in Jenkinsfile
1 file changed, 2 insertions(+), 2 deletions(-)
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 332 bytes | 332.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/nairp126/my-app.git
   e942221..fe056c7  main -> main
pranav@Pranav:~/my-app$ cd ~
pranav@Pranav:~$ curl -Lo ngrok.zip https://bin.equinox.io/c/BJyJlMqVY4c/ngrok-v3-stable-linux-amd64.zip
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 11.1M  100 11.1M    0     0 1355k      0  0:00:08  0:00:08 --:--:-- 1361k
pranav@Pranav:~$ sudo apt install unzip -y
unzip ngrok.zip
sudo mv ngrok /usr/local/bin/
[sudo] password for pranav:
Sorry, try again.
[sudo] password for pranav:
Sorry, try again.
[sudo] password for pranav:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:

```

```

pranav@Pranav:~/my-app  x  Ubuntu  x  +  v
pranav@Pranav:~/my-app$ cd ~
pranav@Pranav:~$ curl -Lo ngrok.zip https://bin.equinox.io/c/BJyJlMqVY4c/ngrok-v3-stable-linux-amd64.zip
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 11.1M  100 11.1M    0     0 1355k      0  0:00:08  0:00:08 --:--:-- 1361k
pranav@Pranav:~$ sudo apt install unzip -y
unzip ngrok.zip
sudo mv ngrok /usr/local/bin/
[sudo] password for pranav:
Sorry, try again.
[sudo] password for pranav:
Sorry, try again.
[sudo] password for pranav:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  zip
The following NEW packages will be installed:
  unzip
0 upgraded, 1 newly installed, 0 to remove and 29 not upgraded.
Need to get 174 kB of archives.
After this operation, 384 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 unzip amd64 6.0-28ubuntu4.1 [174 kB]
Fetched 174 kB in 1s (144 kB/s)
Selecting previously unselected package unzip.
(Reading database ... 58297 files and directories currently installed.)
Preparing to unpack .../unzip_6.0-28ubuntu4.1_amd64.deb ...
Unpacking unzip (6.0-28ubuntu4.1) ...
Setting up unzip (6.0-28ubuntu4.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Archive: ngrok.zip
  inflating: ngrok
pranav@Pranav:~$ ngrok config add-authtoken 3BaZXWm9FTLfnN36MJZil6L9Dgq_3vyHNEu8kVAbcFLXE62ZQ
Auth token saved to configuration file: /home/pranav/.config/ngrok/ngrok.yml
pranav@Pranav:~$ ngrok http 8080
pranav@Pranav:~$ cd ~/my-app
pranav@Pranav:~/my-app$ sed -i 's/Hello from CI/CD Pipeline!/Hello from CI/CD Pipeline! v2/' app.py
pranav@Pranav:~/my-app$ git add .
git commit -m "Test webhook trigger"
git push
[main 6244795] Test webhook trigger
1 file changed, 1 insertion(+), 1 deletion(-)
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/nairp126/my-app.git/'

```

```

pranav@Pranav: ~/my-app  x  Ubuntu
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 291 bytes | 291.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com:nairp126/my-app.git
   fe056c7..6244795  main -> main
pranav@Pranav:~/my-app$ git add .
git commit -m "Test webhook trigger"
git push
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Everything up-to-date
pranav@Pranav:~/my-app$ cd ~/my-app
echo "# test" >> README.md
git add .
git commit -m "Test webhook"
git push
[main 01e0756] Test webhook
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
Username for 'https://github.com': nairp126
Password for 'https://nairp126@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 271 bytes | 271.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com:nairp126/my-app.git
   6244795..01e0756  main -> main
pranav@Pranav:~/my-app$ |

```

Changes Made from Lab PDF

The lab PDF contained several placeholders and assumptions that required changes for a real implementation:

#	What Changed	Lab PDF Had	Changed To	Why
1	Git branch	<code>git 'https:// ... '</code> (no branch)	<code>git branch: 'main', url: '... '</code>	Jenkins defaults to <code>master</code> ; GitHub uses <code>main</code>
2	IMAGE_NAME	<code>your-dockerhub- username/myapp</code>	<code>nairp126/myapp</code>	Replaced placeholder with actual username
3	Docker login	<code>docker login -u your- dockerhub-username</code>	<code>docker login -u nairp126</code>	Same — replaced placeholder
4	Docker CLI install	Not mentioned at all	<code>apt-get install -y docker.io</code> inside Jenkins	Jenkins image has no Docker CLI — pipeline fails without it
5	Compose command	<code>docker-compose up -d</code>	<code>docker compose up -d</code>	Standalone <code>docker- compose</code> binary not available; plugin version works

#	What Changed	Lab PDF Had	Changed To	Why
6	Webhook URL	<code>http://<your-server-ip>:8080/github-webhook/</code>	<code>https://<ngrok-url>/github-webhook/</code>	Jenkins on localhost is not reachable by GitHub — ngrok creates a public tunnel

Errors Encountered & Resolutions

Error 1 — Couldn't find any revision to build

```
ERROR: Couldn't find any revision to build. Verify the repository and branch configuration
```

Root Cause: The Jenkinsfile used `git 'https:// ...'` without specifying a branch. Jenkins defaults to `master`, but the GitHub repo uses `main`.

Fix: Changed the Clone Source stage to explicitly pass `branch: 'main'`:

```
git branch: 'main', url: 'https://github.com/nairp126/my-app.git'
```

Then committed and pushed the fix:

```
git add Jenkinsfile
git commit -m "Fix: use main branch in Jenkinsfile"
git push
```

Error 2 — Webhook Not Triggering Builds

Symptom: After configuring the webhook and pushing code, Jenkins didn't start a new build.

Root Cause: The webhook payload URL still contained the example placeholder from the guide, not the actual ngrok URL. Since ngrok generates a unique random URL every time it starts, the correct URL had to be read from the ngrok terminal output.

Fix:

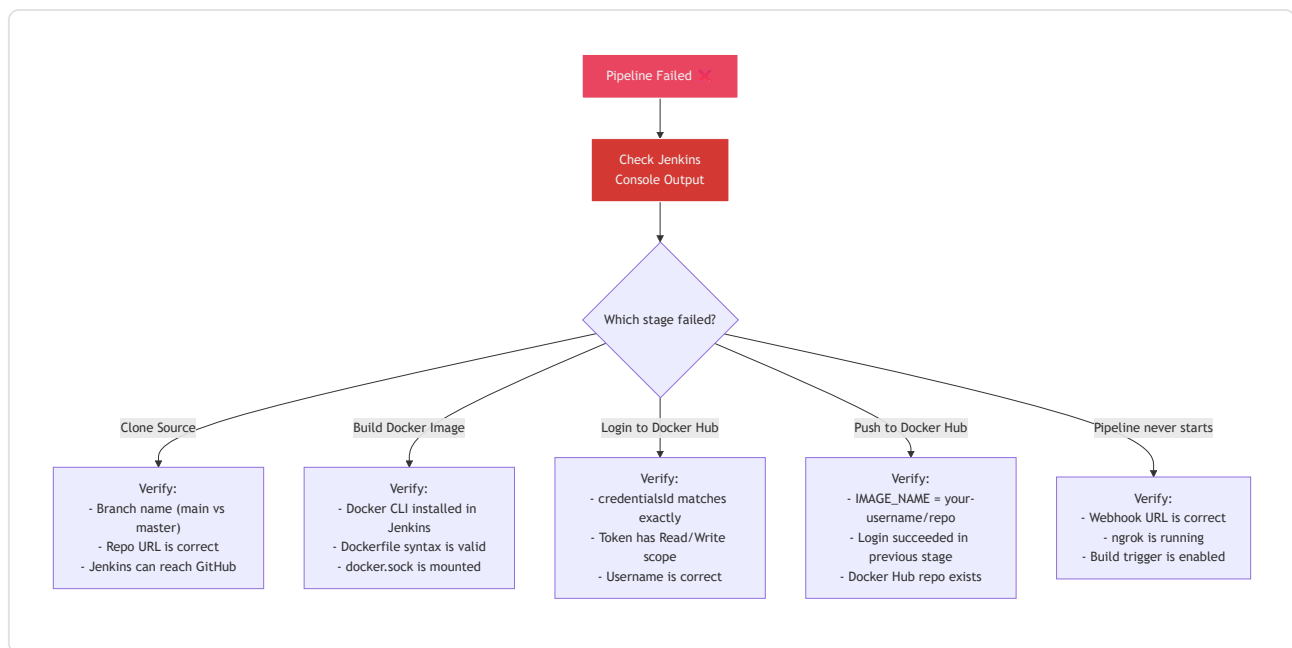
1. Checked the ngrok terminal for the actual forwarding URL
2. Went to GitHub → Repository → Settings → Webhooks → Edit
3. Updated the Payload URL with the correct ngrok URL
4. After saving, GitHub's "Recent Deliveries" showed a green ✓ confirming successful delivery
5. Subsequent pushes automatically triggered Jenkins builds

Troubleshooting — Common Pitfalls

#	Symptom	Likely Cause	Fix
1	<code>docker: command not found</code> in Jenkins console	Docker CLI not installed inside the Jenkins container	Run <code>docker exec -it jenkins bash</code> then <code>apt-get update && apt-get install -y docker.io</code>
2	<code>permission denied when accessing /var/run/docker.sock</code>	Jenkins container running as non-root user	Set <code>user: root</code> in <code>docker-compose.yml</code> , or add the jenkins user to the <code>docker</code> group inside the container
3	<code>Couldn't find any revision to build</code>	Jenkinsfile cloning <code>master</code> but repo uses <code>main</code>	Change <code>git 'url'</code> to <code>git branch: 'main', url: 'url'</code> in the Jenkinsfile
4	Webhook shows ✗ in GitHub (delivery failed)	ngrok URL expired, wrong URL, or Jenkins not running	Restart ngrok, copy the new Forwarding URL, update the webhook in GitHub Settings
5	<code>denied: requested access to the resource is denied</code> on docker push	Wrong Docker Hub username, or token lacks push permission	Verify <code>IMAGE_NAME</code> matches your Docker Hub username; regenerate the access token with Read/Write scope
6	<code>CredentialNotFoundException</code>	<code>credentialsId</code> in Jenkinsfile doesn't match the ID in Jenkins credential store	Go to Manage Jenkins → Credentials and verify the exact ID string
7	Jenkins UI loads but pipeline never starts after push	<code>GitHub hook trigger for GITScm polling</code> not enabled in the job	Edit the pipeline job → Build Triggers → check the GitHub hook trigger checkbox

Debugging Workflow

When a pipeline fails, follow this systematic approach:



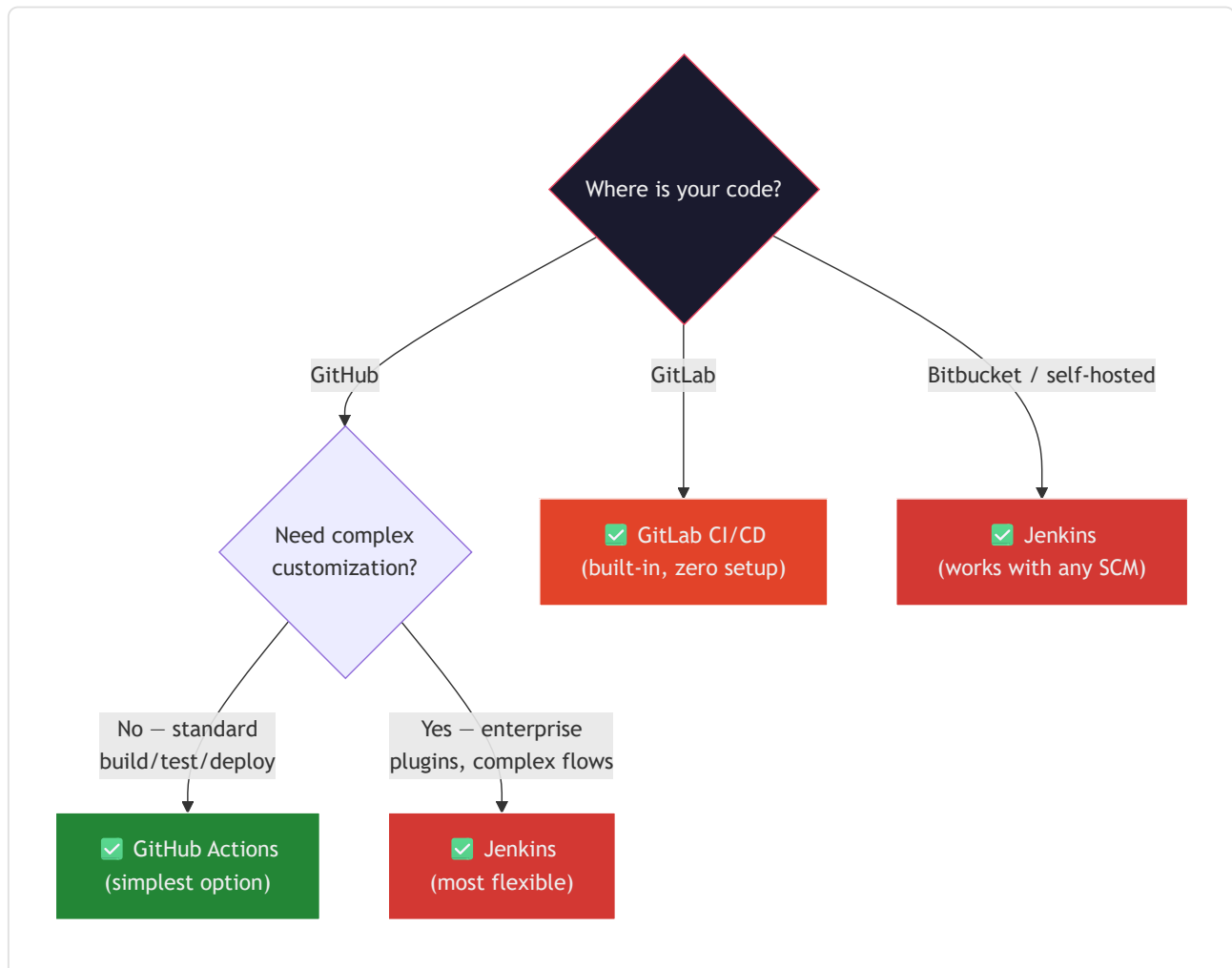
CI/CD Tools Comparison — Where Does Jenkins Fit?

Jenkins is one of many CI/CD tools. Understanding the landscape helps in interviews and real-world tool selection:

Feature	Jenkins	GitHub Actions	GitLab CI/CD
Type	Self-hosted server	Cloud-hosted (SaaS)	Self-hosted or SaaS
Setup	Install + configure yourself	Zero setup — built into GitHub	Built into GitLab
Pipeline definition	<code>Jenkinsfile</code> (Groovy)	<code>.github/workflows/*.yaml</code> (YAML)	<code>.gitlab-ci.yaml</code> (YAML)
Language	Groovy DSL	YAML	YAML
Plugin ecosystem	1,800+ plugins	GitHub Marketplace (Actions)	Built-in features
Cost	Free (open source)	Free tier (2,000 min/month)	Free tier (400 min/month)
Infrastructure	You manage the server	GitHub manages runners	GitLab manages runners (or self-host)
Docker integration	Via plugins + socket mounting	Native (runs in containers)	Native (Docker executor)
Learning curve	Steep (Groovy, plugins, admin)	Moderate (YAML)	Moderate (YAML)

Feature	Jenkins	GitHub Actions	GitLab CI/CD
Best for	Complex enterprise pipelines, custom workflows	GitHub-hosted projects, open source	GitLab-hosted projects, DevSecOps

When to Choose Each



Interview insight: Jenkins is the oldest and most flexible CI/CD tool, but modern teams increasingly prefer GitHub Actions or GitLab CI for simpler setup and YAML-based configuration. Jenkins remains dominant in enterprise environments with complex, legacy, or multi-SCM pipelines.

Key Terminology — Glossary

Term	Definition
CI (Continuous Integration)	Practice of automatically building and testing code after every commit to a shared repository
CD (Continuous Delivery/Deployment)	Automatically delivering built artifacts (images, binaries) to a registry or production environment
Jenkins	Open-source automation server with a web dashboard, plugin ecosystem, and Pipeline as Code support
Jenkinsfile	A Groovy script checked into the repo that defines the pipeline stages (Pipeline as Code)
Pipeline	A sequence of automated stages (clone, build, test, deploy) executed by Jenkins
Stage	A logical phase in a pipeline (e.g., "Build Docker Image") — shown as a block in the Jenkins UI
Agent	The executor where pipeline stages run; <code>agent any</code> = run on any available node
Webhook	An HTTP callback — GitHub sends a POST request to Jenkins when code is pushed
ngrok	A tunneling tool that exposes a local port (e.g., <code>localhost:8080</code>) to the public internet via a temporary HTTPS URL
Docker Socket	<code>/var/run/docker.sock</code> — Unix socket for communicating with the Docker daemon; mounted into Jenkins for DooD
DooD (Docker-outside-of-Docker)	Pattern where a container uses the host's Docker daemon by mounting the Docker socket
DinD (Docker-in-Docker)	Pattern where a separate Docker daemon runs inside a container; requires <code>--privileged</code>
<code>withCredentials</code>	Jenkins pipeline step that temporarily injects a stored secret into an environment variable
<code>--password-stdin</code>	Docker login flag that reads the password from standard input instead of the command line (prevents it from appearing in process listings)
Artifact	The output of a CI/CD pipeline — in this lab, a Docker image pushed to Docker Hub
Personal Access Token (PAT)	A token-based authentication mechanism used as a secure alternative to passwords for GitHub and Docker Hub APIs
SCM (Source Code Management)	A system for tracking code changes — Git, SVN, etc.; Jenkins uses SCM to locate the Jenkinsfile

Term	Definition
GitHub Actions	GitHub's built-in CI/CD service — pipelines defined in YAML, runs on GitHub-hosted runners, zero setup
GitLab CI/CD	GitLab's built-in CI/CD service — pipelines defined in <code>.gitlab-ci.yml</code> , integrated with GitLab's DevSecOps features
Docker Compose V1 vs V2	V1 was a standalone Python binary (<code>docker-compose</code>); V2 is a Go plugin integrated into Docker CLI (<code>docker compose</code>). V1 is deprecated since July 2023
JNLP (Java Network Launch Protocol)	Protocol used by Jenkins agents to connect to the Jenkins controller over port 50000

Viva / Interview Preparation

Q1: What is the role of the Jenkinsfile, and why is it stored in the same repository as the application code?

Answer:

The Jenkinsfile defines the CI/CD pipeline stages — clone, build, login, push — as Groovy code. Storing it in the repository follows the **Pipeline as Code** practice, which provides:

- **Version control:** Pipeline changes are tracked in Git alongside the application code
- **Review process:** Pipeline modifications go through the same PR/code review workflow
- **Reproducibility:** Any commit in the repo's history includes the exact pipeline definition that was used at that point in time
- **Single source of truth:** The pipeline definition lives with the code it builds — no configuration drift between the Jenkins server and the actual pipeline

Without Pipeline as Code, pipeline configuration lives only in the Jenkins UI (as a job configuration), which is fragile, not version-controlled, and easily lost when Jenkins is recreated.

Q2: Explain the security model of `withCredentials`. Why is it better than hardcoding secrets in the Jenkinsfile?

Answer:

How `withCredentials` works:

1. Secrets are stored in Jenkins's encrypted credential store (backed by AES-128/256 encryption)
2. The `withCredentials` block retrieves the secret by its ID and creates a **temporary** environment variable
3. The variable is available **only** inside the `{ }` block — it's destroyed when the block exits

4. Jenkins actively **masks** the secret value in console logs — if the secret appears in output, it's replaced with

Why hardcoding is dangerous:

```
// ❌ NEVER DO THIS – the password is in version control
sh 'docker login -u user -p MyS3cretP@ss'
```

- The Jenkinsfile is committed to GitHub — anyone with repo access can read the password
- The password appears in Jenkins console logs in plaintext
- Password rotation requires changing code, committing, and pushing

withCredentials eliminates all three risks: secrets stay encrypted in Jenkins, are masked in logs, and can be rotated in the Jenkins UI without touching code.

Q3: What is Docker-outside-of-Docker (DooD), and why is it used in this lab instead of Docker-in-Docker (DinD)?

Answer:

DooD (Docker-outside-of-Docker) — the approach used in this lab:

```
volumes:
  - /var/run/docker.sock:/var/run/docker.sock
```

Jenkins container mounts the host's Docker socket, so when Jenkins runs `docker build`, it's the **host's Docker daemon** that builds the image. The images are built on the host, visible to the host, and can be pushed directly from the host's daemon.

DinD (Docker-in-Docker) — an alternative:

A separate Docker daemon runs **inside** the Jenkins container. Images built this way are isolated from the host.

Why DooD was chosen:

Factor	DooD	DinD
Setup complexity	Simple — just mount the socket	Complex — requires <code>--privileged</code> flag
Layer cache	Shared with host — faster rebuilds	Separate cache — slower
Security	Socket access = root (known trade-off)	<code>--privileged</code> = even more dangerous
Image visibility	Images available on host immediately	Images isolated inside inner Docker

For a learning lab, DooD is simpler, faster, and achieves the same result. In production CI/CD systems (like Kubernetes-based Jenkins), DinD or Kaniko (daemonless builds) are preferred for better isolation.

Key Takeaways

1. **Jenkinsfile = Pipeline as Code** — always store it in the repo, never configure pipelines only through the UI
2. **Never hardcode secrets** — use `withCredentials` to inject secrets from Jenkins's encrypted store
3. **Webhooks make CI/CD automatic** — without them, you'd need to click "Build Now" manually after every push
4. **Docker socket mounting** is a deliberate security trade-off — it gives Jenkins full control over the host's Docker daemon
5. The `master` → `main` **branch rename** is a common gotcha — always explicitly specify `branch: 'main'` in Jenkins Git steps
6. **ngrok** bridges the gap between localhost and the internet — essential when GitHub needs to send webhooks to a local Jenkins instance

Objective

Set up Ansible on a control node (WSL2 Ubuntu), build SSH-enabled Docker containers as managed nodes, and use Ansible playbooks to automate package installation, file creation, and system information gathering across all four servers simultaneously.

Theory

The Problem: Manual Server Management at Scale

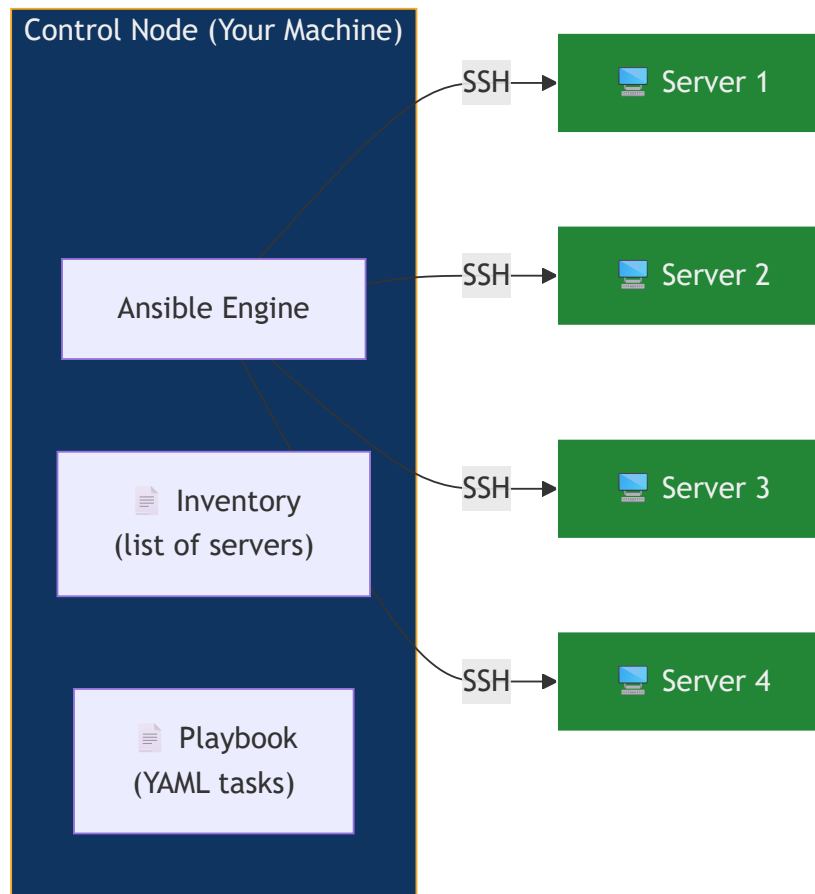
Imagine you manage 100 production servers. A critical security patch needs to be applied — **today**. Manually:

1. SSH into server 1 → run `apt update && apt upgrade` → verify → logout
2. Repeat 99 more times
3. Pray you didn't miss one or make a typo

This is **slow, error-prone, inconsistent, and doesn't scale**. What if one server gets a different package version? What if you forget server #47?

The Solution: Ansible

Ansible is an open-source automation tool that lets you define tasks in simple YAML files (playbooks) and execute them across hundreds of servers **simultaneously** over SSH — with **no agent software** required on the target machines.



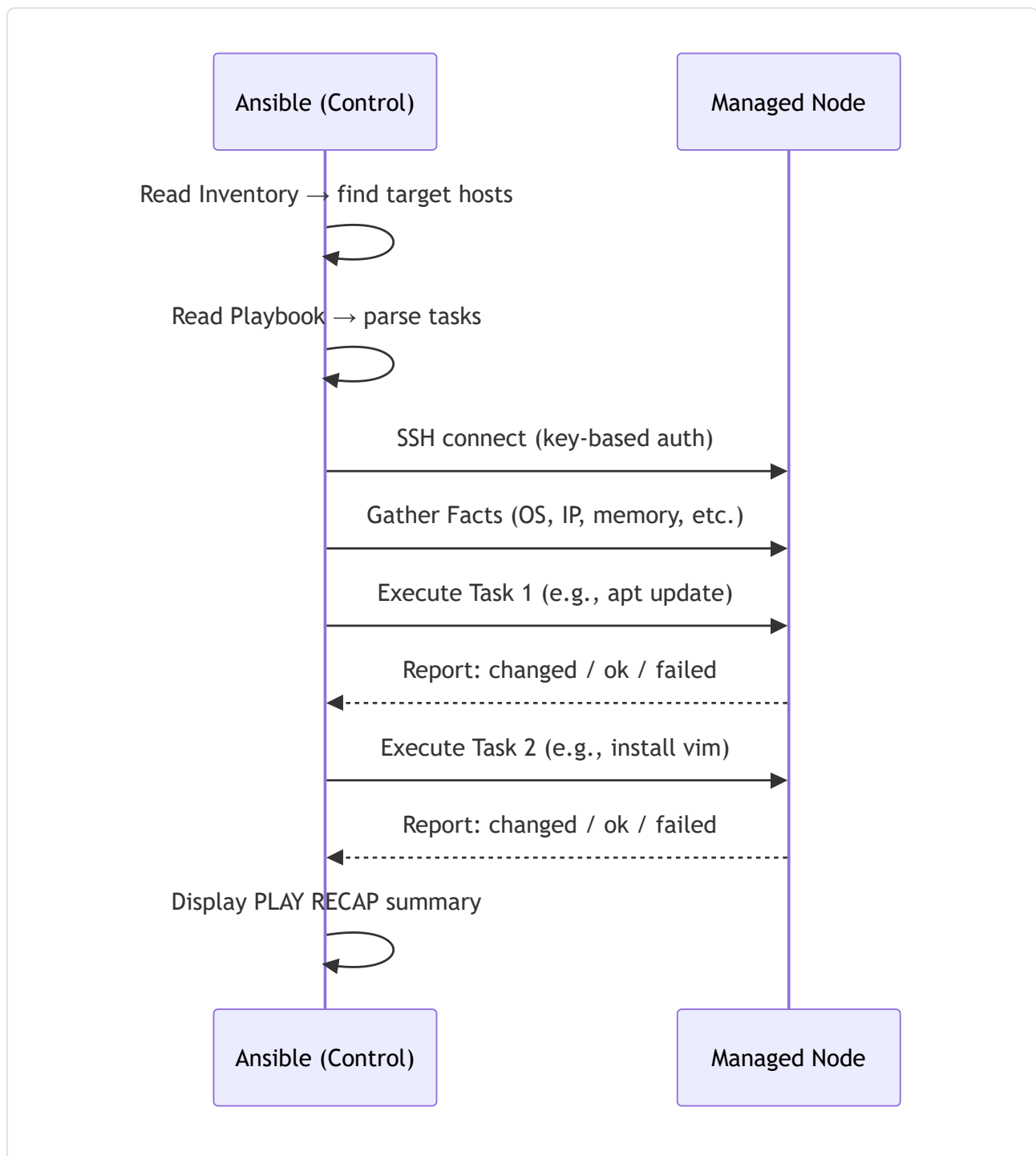
Key Ansible Concepts

Component	Description
Control Node	Machine with Ansible installed — sends commands
Managed Nodes	Target servers — no Ansible needed, only SSH + Python
Inventory	INI/YAML file listing managed nodes and connection details
Playbook	YAML file defining a sequence of automation tasks
Task	A single action (e.g., install a package, create a file)
Module	Built-in function that performs a task (e.g., <code>apt</code> , <code>copy</code> , <code>command</code>)
Role	Reusable, structured collection of playbooks and vars

Why Ansible Wins — Key Properties

Property	Meaning
Agentless	Uses SSH — no software to install/maintain on managed nodes
Idempotent	Running a playbook twice produces the same result — no duplicates
Declarative	You describe the desired state ("nginx must be installed"), not the steps
Push-based	Changes are initiated from the control node immediately
YAML syntax	Human-readable, version-controllable, easy to learn

How Ansible Executes a Playbook



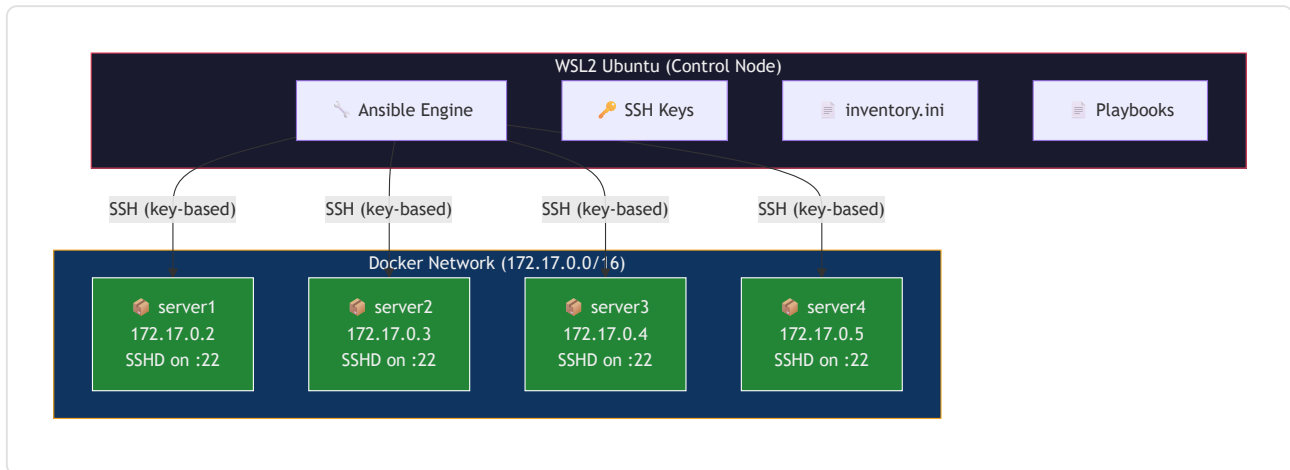
Ansible vs Other Tools

Feature	Ansible	Chef	Puppet	SaltStack
Architecture	Agentless (SSH)	Agent-based	Agent-based	Agent-based
Language	YAML	Ruby DSL	Puppet DSL	YAML/Python
Learning curve	Low	High	Medium	Medium
Push/Pull	Push	Pull	Pull	Both

Feature	Ansible	Chef	Puppet	SaltStack
Setup effort	Minimal	High	High	Medium

Our Lab Architecture

In this experiment, we simulate a real multi-server environment using Docker containers:



Each Docker container runs Ubuntu with:

- OpenSSH server (sshd) listening on port 22
- Our public key pre-installed in `/root/.ssh/authorized_keys`
- Python3 installed (Ansible requires it on managed nodes)

Prerequisites

- WSL2 with Ubuntu installed
- Docker Desktop running
- Basic terminal/command-line knowledge

Part A — Installation & Setup

Phase 1: Install Ansible

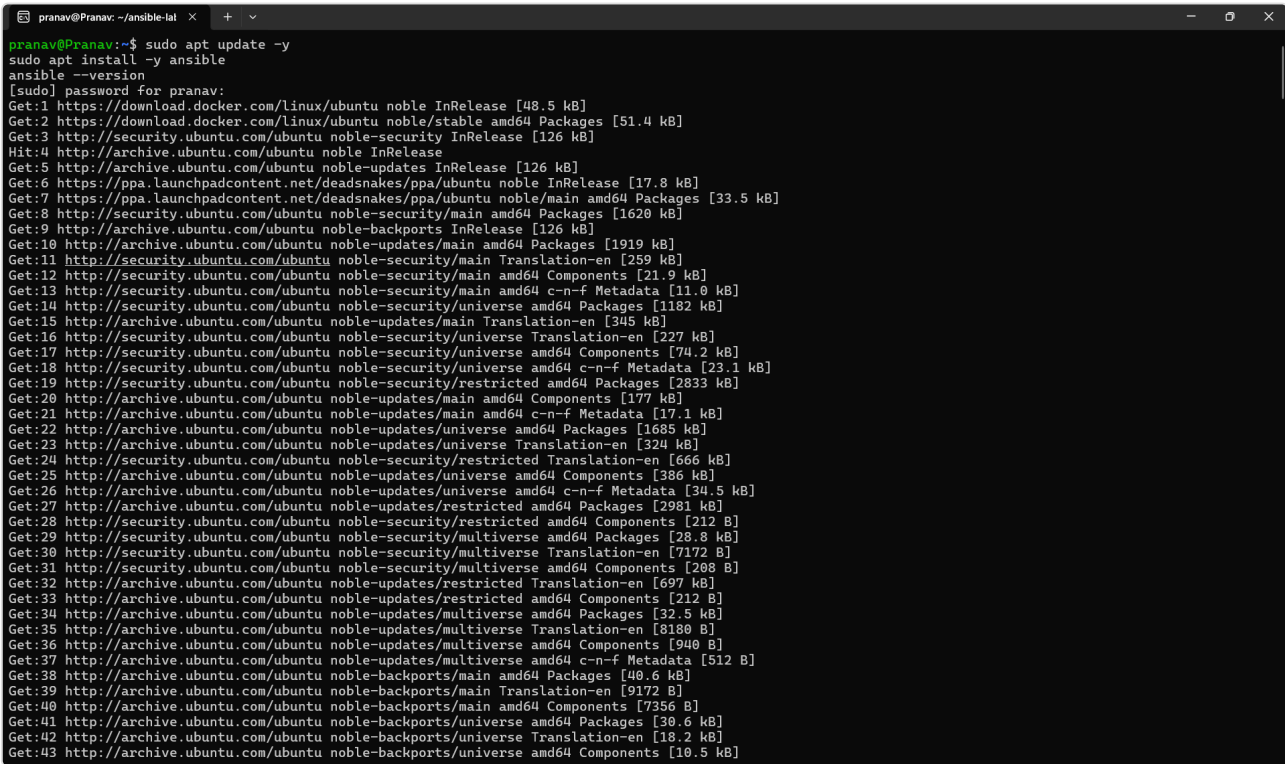
```
sudo apt update -y
sudo apt install -y ansible
ansible --version
```

Expected output: Ansible version info with Python path and config file location.

Verify with a local ping test:

```
ansible localhost -m ping
```

```
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```



```
pranav@Pranav: ~/ansible-lab
pranav@Pranav:~$ sudo apt update -y
sudo apt install -y ansible
ansible --version
[sudo] password for pranav:
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [51.4 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:6 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble InRelease [17.8 kB]
Get:7 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble/main amd64 Packages [33.5 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1620 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1919 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [259 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.9 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.0 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1182 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [345 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [227 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [23.1 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2833 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.1 kB]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1685 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [324 kB]
Get:24 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [666 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.5 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2981 kB]
Get:28 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:29 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:30 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [7172 B]
Get:31 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [697 kB]
Get:33 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:34 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [32.5 kB]
Get:35 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [8180 B]
Get:36 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:37 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [512 B]
Get:38 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [40.6 kB]
Get:39 http://archive.ubuntu.com/ubuntu noble-backports/main Translation-en [9172 B]
Get:40 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7356 B]
Get:41 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [30.6 kB]
Get:42 http://archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [18.2 kB]
Get:43 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
```

Phase 2: Generate SSH Key Pair

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""
```

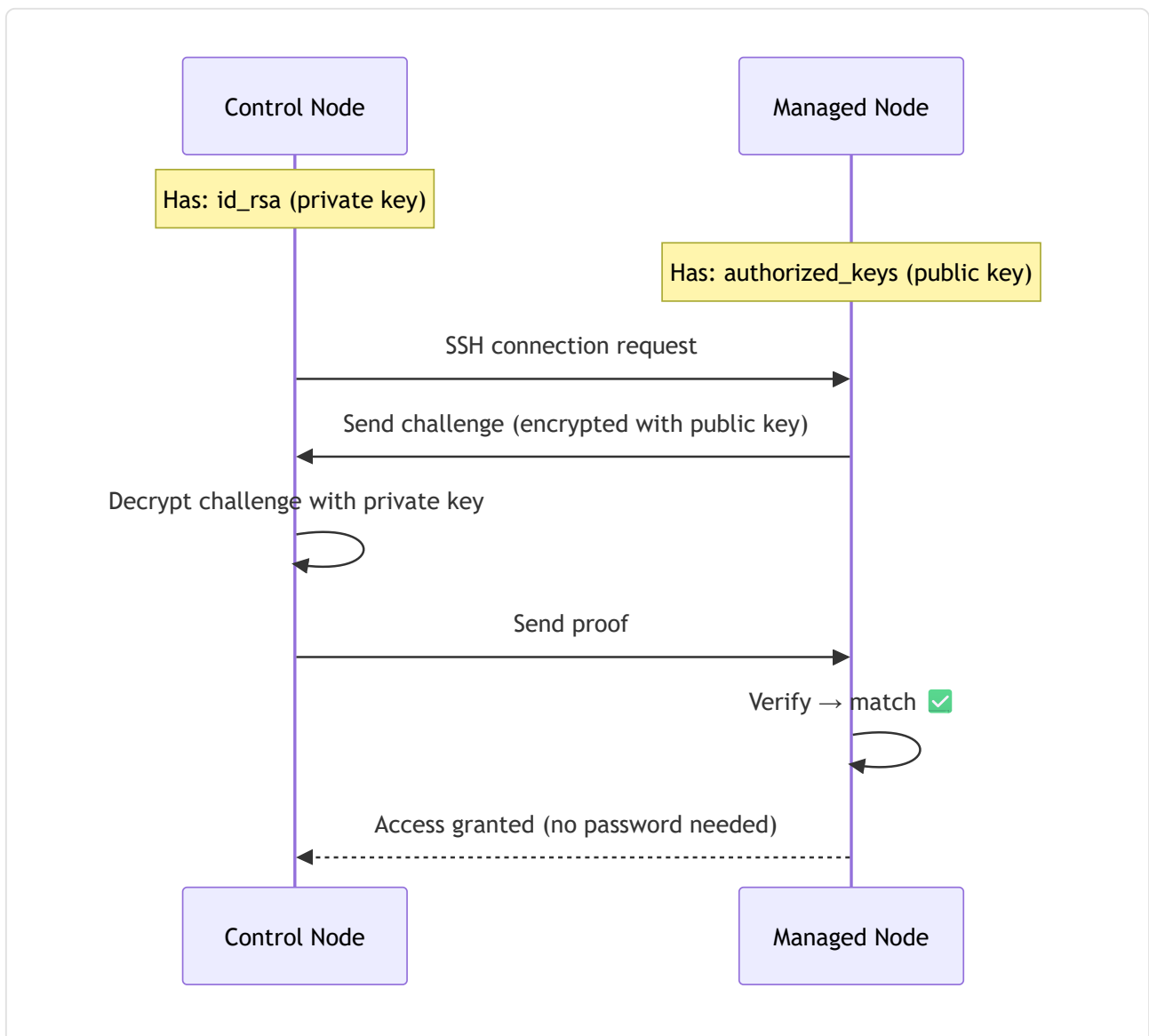
Flag	Purpose
-t rsa	Key type: RSA algorithm
-b 4096	Key size: 4096 bits (strong encryption)
-f ~/.ssh/id_rsa	Output file path — skips the interactive "enter file" prompt
-N ""	Empty passphrase — critical for Ansible (it can't type a passphrase interactively)

Why `-N ""` ? Without this flag, `ssh-keygen` prompts for a passphrase. If you set one, Ansible will **hang indefinitely** on every SSH connection because it cannot provide the passphrase during automated playbook runs.

This creates two files:

File	Location	Purpose
<code>id_rsa</code> (Private Key)	Control node (<code>~/.ssh/</code>)	Used by Ansible to authenticate — never share
<code>id_rsa.pub</code> (Public Key)	Copied into managed nodes	Placed in <code>~/.ssh/authorized_keys</code> to grant access

SSH Key Authentication Flow



```

pranav@Pranav: ~/ansible-lab
pranav@Pranav:~$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""
Generating public/private rsa key pair.
Created directory '/home/pranav/.ssh'.
Your identification has been saved in /home/pranav/.ssh/id_rsa
Your public key has been saved in /home/pranav/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:UQTFSGoixW7XguSv8UAPcB95oHzr1NAb+KQ1fIDB6HI pranav@Pranav
The key's randomart image is:
+--[RSA 4096]-----+
|  ..oo+o++          |
|  .o=,+o=..         |
|  .B+oB+O           |
|  ..E++@.*          |
|  =++.S.            |
|   ooo .            |
|    =.              |
|     .              |
+-----+
pranav@Pranav:~$ ls ~/.ssh/id_rsa ~/.ssh/id_rsa.pub
/home/pranav/.ssh/id_rsa /home/pranav/.ssh/id_rsa.pub
pranav@Pranav:~$ mkdir -p ~/ansible-lab
cd ~/ansible-lab
pranav@Pranav:~/ansible-lab$ cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
pranav@Pranav:~/ansible-lab$ cat > Dockerfile << 'EOF'
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server

RUN mkdir -p /run/sshd && \
  echo 'root:password' | chpasswd && \
  sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
  sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
  sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && \
  chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && \
  chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

```

Phase 3: Build the Docker Image

Create the working directory and copy keys:

```

mkdir -p ~/ansible-lab && cd ~/ansible-lab
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .

```

Create the Dockerfile:

```

FROM ubuntu

RUN apt update -y
RUN apt install -y python3 openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH: enable root login, disable password auth, enable key auth
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Create .ssh directory with correct permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys into the image
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper file permissions
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix PAM login issue for SSH
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

# Start SSH daemon in foreground
CMD ["/usr/sbin/sshd", "-D"]

```

Dockerfile Breakdown

Line	Purpose
<code>FROM ubuntu</code>	Base image — latest Ubuntu
<code>apt install python3 openssh-server</code>	Python3 for Ansible, sshd for SSH
<code>echo 'root:password' chpasswd</code>	Sets root password (fallback)
<code>PermitRootLogin yes</code>	Allows SSH as root
<code>PasswordAuthentication no</code>	Disables password login — forces key-based auth
<code>PubkeyAuthentication yes</code>	Enables SSH key authentication
<code>COPY id_rsa.pub ... authorized_keys</code>	Installs our public key — grants access

Line	Purpose
<code>chmod 600 / 644</code>	SSH requires strict file permissions or it refuses to work
<code>pam_loginuid.so</code> fix	Prevents PAM errors when SSHing into containers
<code>CMD ["/usr/sbin/sshd", "-D"]</code>	Runs SSH daemon in foreground — keeps container alive

Build the image:

```
docker build -t ubuntu-server .
```

Clean up keys from the build directory:

```
rm id_rsa id_rsa.pub
```

```
pranav@Pranav: ~/ansible-lab
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]
EOF
pranav@Pranav:~/ansible-lab$ docker build -t ubuntu-server .
[+] Building 106.4s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                  0.1s
=> => transferring dockerfile: 828B                                  0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                     0.1s
=> => transferring context: 2B                                         0.0s
=> [1/9] FROM docker.io/library/ubuntu:latest                      0.0s
=> [internal] load build context                                    0.1s
=> => transferring context: 4.20kB                                     0.0s
=> [2/9] RUN apt update -y                                          11.8s
=> [3/9] RUN apt install -y python3 python3-pip openssh-server     89.1s
=> [4/9] RUN mkdir -p /run/ssh && echo 'root:password' | chpasswd && sed -i 's/#PermitRootLogin prohibi 0.6s
=> [5/9] RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh            0.4s
=> [6/9] COPY id_rsa /root/.ssh/id_rsa                             0.1s
=> [7/9] COPY id_rsa.pub /root/.ssh/authorized_keys                0.1s
=> [8/9] RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/authorized_keys 0.5s
=> [9/9] RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd 0.4s
=> exporting to image                                              3.1s
=> exporting layers                                                3.0s
=> writing image sha256:df802819b42903d6d388de4334314bdb81e73462d5e30ec3360c0b117d99af96 0.0s
=> naming to docker.io/library/ubuntu-server                       0.0s
pranav@Pranav:~/ansible-lab$ docker images | grep ubuntu-server
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ubuntu-server:latest      df802819b429    667MB      0B
pranav@Pranav:~/ansible-lab$ rm id_rsa id_rsa.pub
pranav@Pranav:~/ansible-lab$ for i in {1..4}; do
  echo -e "\nCreating server${i}"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done
Creating server1
aabe9c9a1b97146298ef56e86741b82bcb6892813b8c338b89a71a8e0b562bc6
IP of server1 is 172.17.0.2
Creating server2
c3e369d7854106e59b59e17f40fc45de5484bc3ed419ec2172e5d73dfd80b6b9
IP of server2 is 172.17.0.3
Creating server3
133a103c4fd537c745d82b9015c048d06caa0248425a68b633e6b64bd7a73431
IP of server3 is 172.17.0.4
```

Phase 4: Launch 4 Server Containers

```
for i in {1..4}; do
  echo -e "\nCreating server${i}"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done
```

Flag	Purpose
<code>-d</code>	Detached mode — run in background
<code>--rm</code>	Auto-remove container when stopped
<code>-p 220\${i}:22</code>	Map host port 2201–2204 to container SSH port 22
<code>--name server\${i}</code>	Name containers server1 through server4

Verify all containers are running:

```
docker ps
```

```

pranav@Pranav: ~/ansible-lab
IP of server2 is 172.17.0.3

Creating server3
133a103c4fd537c745d82b9015c848d06caa0248425a68b633e6b64bd7a73431
IP of server3 is 172.17.0.4

Creating server4
9b044682c0999a4a3bb6dec2a02eb1e550fb93383f031d8977a0dbf3ca7f222c
IP of server4 is 172.17.0.5
pranav@Pranav:~/ansible-lab$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
9b044682c099   ubuntu-server  "/usr/sbin/sshd -D"      5 seconds ago Up 4 seconds  0.0.0.0:2204->22/tcp, [::]:2204->22/tcp
133a103c4fd5   ubuntu-server  "/usr/sbin/sshd -D"      5 seconds ago Up 4 seconds  0.0.0.0:2203->22/tcp, [::]:2203->22/tcp
c3e369d78541   ubuntu-server  "/usr/sbin/sshd -D"      6 seconds ago Up 5 seconds  0.0.0.0:2202->22/tcp, [::]:2202->22/tcp
aabe9c9a1b97   ubuntu-server  "/usr/sbin/sshd -D"      6 seconds ago Up 6 seconds  0.0.0.0:2201->22/tcp, [::]:2201->22/tcp
7dbb2f1f9b47   jenkins/jenkins:lts  "/usr/bin/tini -- /u..." 3 weeks ago   Up 7 minutes  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
a97f974ff7ae   myjenkins-blueocean:2.541.3-1  "/usr/bin/tini -- /u..." 3 weeks ago   Up 7 minutes  jenkins-blueocean

pranav@Pranav:~/ansible-lab$ cd ~/ansible-lab

echo "[servers]" > inventory.ini
for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

cat << 'EOF' >> inventory.ini

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
pranav@Pranav:~/ansible-lab$ cat inventory.ini
[servers]
172.17.0.2
172.17.0.3
172.17.0.4
172.17.0.5

[servers:vars]
ansible_user=root

```

Phase 5: Create the Ansible Inventory

```
cd ~/ansible-lab

# Auto-generate inventory from running container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
    docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >>
done

# Add connection variables
cat << 'EOF' >> inventory.ini

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

Verify:

```
cat inventory.ini
```

Expected output:

```
[servers]
172.17.0.2
172.17.0.3
172.17.0.4
172.17.0.5

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

Inventory Variable Breakdown

Variable	Purpose
<code>ansible_user=root</code>	SSH as root user
<code>ansible_ssh_private_key_file=~/.ssh/id_rsa</code>	Path to the private key for authentication
<code>ansible_python_interpreter=/usr/bin/python3</code>	Explicitly set Python path — prevents auto-discovery warnings

Why `~/.ssh/id_rsa` and not `./id_rsa`? Relative paths break if you run Ansible from a different directory. Also, keys on NTFS (Windows mounts in WSL2) get `0777` permissions, causing SSH to reject them with "UNPROTECTED PRIVATE KEY FILE!" error. Keys in `~/.ssh/` on the Linux filesystem have correct `600` permissions.

Phase 6: Test Connectivity

Critical step — disable host key checking first:

```
export ANSIBLE_HOST_KEY_CHECKING=False
```

Why? First-time SSH connections show "The authenticity of host... can't be established. Are you sure (yes/no)?" — Ansible can't type "yes" in non-interactive mode, so the connection **fails silently**.

Test SSH manually:

```
SERVER1_IP=$(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server1)
ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no root@$SERVER1_IP "echo SSH works"
```

Test Ansible connectivity to all servers:

```
ansible all -i inventory.ini -m ping
```

Flag	Purpose
<code>all</code>	Target all hosts in inventory
<code>-i inventory.ini</code>	Specify inventory file
<code>-m ping</code>	Use the <code>ping</code> module (tests SSH + Python, not ICMP)

Expected output (all 4 servers SUCCESS):

```
172.17.0.2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.3 | SUCCESS => { ... }
172.17.0.4 | SUCCESS => { ... }
172.17.0.5 | SUCCESS => { ... }
```

```

pranav@Pranav: ~/ansible-lab
172.17.0.4
172.17.0.5

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
pranav@Pranav:~/ansible-lab$ export ANSIBLE_HOST_KEY_CHECKING=False
pranav@Pranav:~/ansible-lab$ SERVER1_IP=$(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server1)
ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no root@$SERVER1_IP "echo SSH works"
Warning: Permanently added '172.17.0.2' (ED25519) to the list of known hosts.
SSH works
pranav@Pranav:~/ansible-lab$ ansible all -i inventory.ini -m ping
172.17.0.5 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
pranav@Pranav:~/ansible-lab$ cat > ~/ansible-lab/update.yml << 'EOF'
---
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file

```

Part B — Running Playbooks

Phase 7: Playbook 1 — Update & Install Packages

Create `update.yml`:

```

---
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on {{ inventory_hostname }}"

```

Playbook Breakdown

Field	Purpose
---	YAML document start marker (required)
name:	Human-readable description of this play
hosts: all	Run on all hosts in inventory
become: yes	Escalate privileges (sudo)
tasks:	List of actions to perform

Task	Module	What It Does
Update apt	apt	Refreshes package cache and performs dist-upgrade
Install packages	apt	Installs vim, htop, wget — state: present means "ensure installed"
Create test file	copy	Creates a file with dynamic content using Jinja2 template ({{ inventory_hostname }})

Run the playbook:

```
ansible-playbook -i inventory.ini update.yml
```

Expected output:

```
PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [172.17.0.2]
ok: [172.17.0.3]
ok: [172.17.0.4]
ok: [172.17.0.5]

TASK [Update apt packages] *****
changed: [172.17.0.2]
changed: [172.17.0.3]
...

TASK [Install required packages] *****
changed: [172.17.0.2]
...

TASK [Create test file] *****
changed: [172.17.0.2]
...

PLAY RECAP *****
172.17.0.2  : ok=4   changed=3   unreachable=0   failed=0
172.17.0.3  : ok=4   changed=3   unreachable=0   failed=0
172.17.0.4  : ok=4   changed=3   unreachable=0   failed=0
172.17.0.5  : ok=4   changed=3   unreachable=0   failed=0
```

Understanding PLAY RECAP

Counter	Meaning
ok	Tasks that succeeded (including "already in desired state")
changed	Tasks that made actual changes on the server
unreachable	Hosts that couldn't be contacted via SSH
failed	Tasks that errored out
skipped	Tasks skipped due to conditions

```

pranav@Pranav: ~/ansible-lab
upgrade: dist
- name: Install required packages
  apt:
    name: ["vim", "htop", "wget"]
    state: present
- name: Create test file
  copy:
    dest: /root/ansible_test.txt
    content: "Configured by Ansible on {{ inventory_hostname }}"
EOF
pranav@Pranav:~/ansible-lab$ ansible-playbook -i inventory.ini update.yml

PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [172.17.0.5]
ok: [172.17.0.2]
ok: [172.17.0.3]
ok: [172.17.0.4]

TASK [Update apt packages] *****
changed: [172.17.0.4]
changed: [172.17.0.2]
changed: [172.17.0.5]
changed: [172.17.0.3]

TASK [Install required packages] *****
changed: [172.17.0.2]
changed: [172.17.0.5]
changed: [172.17.0.3]
changed: [172.17.0.4]

TASK [Create test file] *****
changed: [172.17.0.2]
changed: [172.17.0.3]
changed: [172.17.0.4]
changed: [172.17.0.5]

PLAY RECAP *****
172.17.0.2      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.3      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.4      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.5      : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
pranav@Pranav:~/ansible-lab$ cat > ~/ansible-lab/playbook1.yml << 'EOF'

```

Phase 8: Playbook 2 — System Info & Advanced Tasks

Create `playbook1.yml`:

```

---
- name: Configure multiple servers
  hosts: servers
  become: yes

  tasks:
    - name: Update apt package index
      apt:
        update_cache: yes

    - name: Install Python 3 (latest available)
      apt:
        name: python3
        state: latest

    - name: Create test file with content
      copy:
        dest: /root/test_file.txt
        content: |
          This is a test file created by Ansible
          Server name: {{ inventory_hostname }}
          Current date: {{ ansible_date_time.date }}

    - name: Display system information
      command: uname -a
      register: uname_output

    - name: Show disk space
      command: df -h
      register: disk_space

    - name: Print results
      debug:
        msg:
          - "System info: {{ uname_output.stdout }}"
          - "Disk space: {{ disk_space.stdout_lines }}"

```

New Concepts in This Playbook

Concept	Explanation
<code>state: latest</code>	Ensures the latest version is installed (vs <code>present</code> which accepts any version)
<code>content: </code>	YAML multi-line string (literal block scalar) — preserves line breaks
<code>{{ ansible_date_time.date }}</code>	Ansible fact — auto-gathered system info available as variables

Concept	Explanation
<code>register: uname_output</code>	Captures command output into a variable for later use
<code>debug:</code> module	Prints messages to the console — used for displaying registered variables

Run:

```
ansible-playbook -i inventory.ini playbook1.yml
```

```

pranav@Pranav: ~/ansible-lab$ cat > ~/ansible-lab/playbook1.yml << 'EOF'
---
- name: Configure multiple servers
  hosts: servers
  become: yes

  tasks:
    - name: Update apt package index
      apt:
        update_cache: yes

    - name: Install Python 3 (latest available)
      apt:
        name: python3
        state: latest

    - name: Create test file with content
      copy:
        dest: /root/test_file.txt
        content: |
          This is a test file created by Ansible
          Server name: {{ inventory_hostname }}
          Current date: {{ ansible_date_time.date }}

    - name: Display system information
      command: uname -a
      register: uname_output

    - name: Show disk space
      command: df -h
      - "Disk space: {{ disk_space.stdout_lines }}"
EOF
pranav@Pranav:~/ansible-lab$ ansible-playbook -i inventory.ini playbook1.yml

PLAY [Configure multiple servers] *****

TASK [Gathering Facts] *****
ok: [172.17.0.2]
ok: [172.17.0.5]
ok: [172.17.0.3]
ok: [172.17.0.4]

TASK [Update apt package index] *****
ok: [172.17.0.4]
ok: [172.17.0.5]
ok: [172.17.0.2]
ok: [172.17.0.3]

```

```

pranav@Pranav: ~/ansible-lab
PLAY [Configure multiple servers] *****
TASK [Gathering Facts] *****
ok: [172.17.0.2]
ok: [172.17.0.5]
ok: [172.17.0.3]
ok: [172.17.0.4]

TASK [Update apt package index] *****
ok: [172.17.0.4]
ok: [172.17.0.5]
ok: [172.17.0.2]
ok: [172.17.0.3]

TASK [Install Python 3 (latest available)] *****
ok: [172.17.0.2]
ok: [172.17.0.4]
ok: [172.17.0.5]
ok: [172.17.0.3]

TASK [Create test file with content] *****
changed: [172.17.0.5]
changed: [172.17.0.2]
changed: [172.17.0.4]
changed: [172.17.0.3]

TASK [Display system information] *****
changed: [172.17.0.2]
changed: [172.17.0.3]
changed: [172.17.0.4]
changed: [172.17.0.5]

TASK [Show disk space] *****
changed: [172.17.0.2]
changed: [172.17.0.3]
changed: [172.17.0.4]
changed: [172.17.0.5]

TASK [Print results] *****
ok: [172.17.0.2] => {
  "msg": [
    "System info: Linux aabe9c9alb97 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 GNU/Linux",
    "Disk space: ['Filesystem      Size  Used Avail Use% Mounted on', 'overlay          1007G   19G  937G   2% /', 'tmpfs           3.8G     0  3.8G   0% /proc/acpi', 'tmpfs           3.8G     0  3.8G   0% /sys/firmware']"
  ]
}

```

Phase 9: Verify Changes

```

# Verify ansible_test.txt (from update.yml)
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

# Verify test_file.txt (from playbook1.yml)
ansible all -i inventory.ini -m command -a "cat /root/test_file.txt"

# Double-check via docker exec directly
for i in {1..4}; do
  echo "=== server${i} ==="
  docker exec server${i} cat /root/ansible_test.txt
done

```

Expected output:

```

172.17.0.2 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.2
172.17.0.3 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.3
...

```

```

pranav@Pranav: ~/ansible-lab
}
ok: [172.17.0.4] => {
  "msg": [
    "System info: Linux 133a103c4fd5 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 GNU/Linux"
  ],
  "Disk space: ['Filesystem      Size  Used Avail Use% Mounted on', 'overlay          1007G   19G  937G   2% /', 'tmpfs          64M    0   64M   0% /dev/shm', '1007G   19G  937G   2% /etc/hosts', 'tmpfs          3.8G    0   3.8G   0% /proc/acpi', 'tmpfs          3.8G    0   3.8G   0% /proc/scsi', 'tmpfs          3.8G    0   3.8G   0% /sys/firmware']"
}
ok: [172.17.0.5] => {
  "msg": [
    "System info: Linux 9b044682c099 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 GNU/Linux"
  ],
  "Disk space: ['Filesystem      Size  Used Avail Use% Mounted on', 'overlay          1007G   19G  937G   2% /', 'tmpfs          64M    0   64M   0% /dev/shm', '1007G   19G  937G   2% /etc/hosts', 'tmpfs          3.8G    0   3.8G   0% /proc/acpi', 'tmpfs          3.8G    0   3.8G   0% /proc/scsi', 'tmpfs          3.8G    0   3.8G   0% /sys/firmware']"
}
}

PLAY RECAP *****
172.17.0.2      : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.3      : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.4      : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.5      : ok=7    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

pranav@Pranav:~/ansible-lab$ # Check ansible_test.txt was created on all servers (from update.yml)
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

# Check test_file.txt was created on all servers (from playbook1.yml)
ansible all -i inventory.ini -m command -a "cat /root/test_file.txt"

# Double-check via docker exec directly
for i in {1..4}; do
  echo "=== server${i} ==="
  docker exec server${i} cat /root/ansible_test.txt
done
172.17.0.2 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.2
172.17.0.3 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.3
172.17.0.4 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.4
172.17.0.5 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.5
172.17.0.2 | CHANGED | rc=0 >>
This is a test file created by Ansible

```

Phase 10: Cleanup

```

for i in {1..4}; do
  docker stop server${i}
done

```

Since we used `--rm`, containers are automatically removed when stopped.

```

pranav@Pranav: ~/ansible-lab
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

# Check test_file.txt was created on all servers (from playbook1.yml)
ansible all -i inventory.ini -m command -a "cat /root/test_file.txt"

# Double-check via docker exec directly
for i in {1..4}; do
  echo "=== server${i} ==="
  docker exec server${i} cat /root/ansible_test.txt
done
172.17.0.2 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.2
172.17.0.3 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.3
172.17.0.4 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.4
172.17.0.5 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.5
172.17.0.2 | CHANGED | rc=0 >>
This is a test file created by Ansible
Server name: 172.17.0.2
Current date: 2026-04-24
172.17.0.3 | CHANGED | rc=0 >>
This is a test file created by Ansible
Server name: 172.17.0.3
Current date: 2026-04-24
172.17.0.5 | CHANGED | rc=0 >>
This is a test file created by Ansible
Server name: 172.17.0.5
Current date: 2026-04-24
172.17.0.4 | CHANGED | rc=0 >>
This is a test file created by Ansible
Server name: 172.17.0.4
Current date: 2026-04-24
=== server1 ===
Configured by Ansible on 172.17.0.2=== server2 ===
Configured by Ansible on 172.17.0.3=== server3 ===
Configured by Ansible on 172.17.0.4=== server4 ===
pranav@Pranav:~/ansible-lab$ for i in {1..4}; do
  docker stop server${i}
done
server1
server2
server3
server4
pranav@Pranav:~/ansible-lab$

```

Deep Dive: Understanding Idempotency

Run `update.yml` twice:

- **First run:** `changed=3` — packages installed, file created
- **Second run:** `changed=0` — packages already installed, file already exists with same content

This is **idempotency** — the defining property of configuration management tools. It means you can safely re-run playbooks without fear of duplicate installations or broken states.

```

First run:  Install vim → changed ✓
Second run: vim already installed → ok (no change) ✓

```

Common Pitfalls & Troubleshooting

Problem	Cause	Fix
<code>UNREACHABLE! Host key verification failed</code>	First SSH connection requires "yes" confirmation	<code>export ANSIBLE_HOST_KEY_CHECKING=False</code>

Problem	Cause	Fix
<code>UNPROTECTED PRIVATE KEY FILE!</code>	Key file permissions too open (0777 on NTFS)	Use <code>~/.ssh/id_rsa</code> on Linux filesystem, not NTFS mount
<code>Permission denied (publickey)</code>	Wrong key or key not in <code>authorized_keys</code>	Verify: <code>ssh -i ~/.ssh/id_rsa root@<ip></code> manually
<code>Missing sudo password</code>	<code>become: yes</code> without password	Using root user directly, or add <code>ansible_become_password</code>
<code>No module named 'apt'</code>	Running playbook on non-Debian host	Ensure containers are Ubuntu-based
Container exits immediately	No foreground process	Ensure <code>CMD ["/usr/sbin/sshd", "-D"]</code> — <code>-D</code> keeps sshd in foreground
<code>ansible_connection must be one of [ssh, local ...]</code>	Inventory uses <code>ansible_connection=2201</code> (port number)	<code>ansible_connection</code> is a type (ssh/local), not a port — use direct container IPs

Optional: Part B — Local & Advanced Exercises

Ad-hoc Commands

```
# List all modules
ansible-doc -l

# Search for AWS modules
ansible-doc -l | grep aws

# View module documentation
ansible-doc apt
```

Ansible Vault (Secrets Management)

```
# Create encrypted file
ansible-vault create secrets.yml

# Edit encrypted file
ansible-vault edit secrets.yml

# Run playbook with vault password
ansible-playbook -i inventory.ini playbook.yml --ask-vault-pass
```

Ansible Galaxy (Reusable Roles)

```
# Install community collections
ansible-galaxy collection install community.general
```

Lab Fixes Summary

The original lab sheet had several issues that were fixed for this experiment. These are documented for educational purposes:

Change	Original (Broken)	Fixed	Why
SSH keygen	<code>ssh-keygen -t rsa -b 4096</code> (interactive)	Added <code>-f ~/.ssh/id_rsa -N ""</code>	Ansible can't type passphrases interactively
Dockerfile	<code>apt install python3 python3-pip</code>	Removed <code>python3-pip</code>	pip via apt is deprecated on Ubuntu 22.04+; Ansible only needs python3
Host key checking	Not mentioned	<code>export ANSIBLE_HOST_KEY_CHECKING=False</code>	SSH prompts "are you sure?" which hangs non-interactive Ansible
Inventory entries	<code>ansible_connection=2201</code> (wrong)	Direct container IPs only	<code>ansible_connection</code> expects a type (ssh/local), not a port number
Docker run flags	<code>-p --rm 2222:22</code> (broken order)	<code>--rm -p 220\${i}:22</code>	<code>--rm</code> between <code>-p</code> and its argument breaks flag parsing
Key file path	<code>./id_rsa</code> (relative)	<code>~/.ssh/id_rsa</code> (absolute)	Relative paths break from different directories; NTFS permissions cause errors

Glossary

Term	Definition
Ansible	Open-source agentless automation tool for configuration management, deployment, and orchestration

Term	Definition
Control Node	The machine where Ansible is installed and playbooks are executed from
Managed Node	Target server that Ansible configures — requires only SSH + Python
Inventory	File listing managed nodes, grouped and with connection variables
Playbook	YAML file defining a sequence of automation tasks to execute
Task	A single action in a playbook (e.g., install a package)
Module	Built-in Ansible function (e.g., <code>apt</code> , <code>copy</code> , <code>command</code> , <code>debug</code>)
Role	Reusable, structured automation bundle — playbooks + vars + templates
Agentless	No software needed on managed nodes — Ansible uses SSH
Idempotent	Running the same operation multiple times yields the same result
Facts	System info Ansible auto-collects (OS, IP, memory, date, etc.)
Register	Captures task output into a variable for use in later tasks
Become	Privilege escalation (sudo) — <code>become: yes</code> in playbooks
PLAY RECAP	Summary at the end showing ok/changed/failed/unreachable per host
Ad-hoc command	One-off Ansible commands without a playbook (<code>ansible all -m ping</code>)
Ansible Vault	Encryption tool for securing sensitive data (passwords, keys)
SSH key pair	Cryptographic key pair — private key (authenticates) + public key (authorizes)
SSHD	SSH daemon — the server process that listens for SSH connections

References

- [Ansible Official Docs](#)
- [Ansible Getting Started](#)
- [Spacelift Ansible Tutorial](#)
- [Ansible Tower](#)

Objective

Deploy a SonarQube server using Docker Compose, scan a Java application with intentional code issues, and analyze the results through the web dashboard and REST API to understand automated static code analysis in a CI/CD pipeline.

Theory

The Problem: Bugs Found Too Late

Code bugs and security vulnerabilities are often discovered during testing — or worse, after deployment in production. Manual code reviews are **slow, inconsistent, and don't scale** as teams grow.

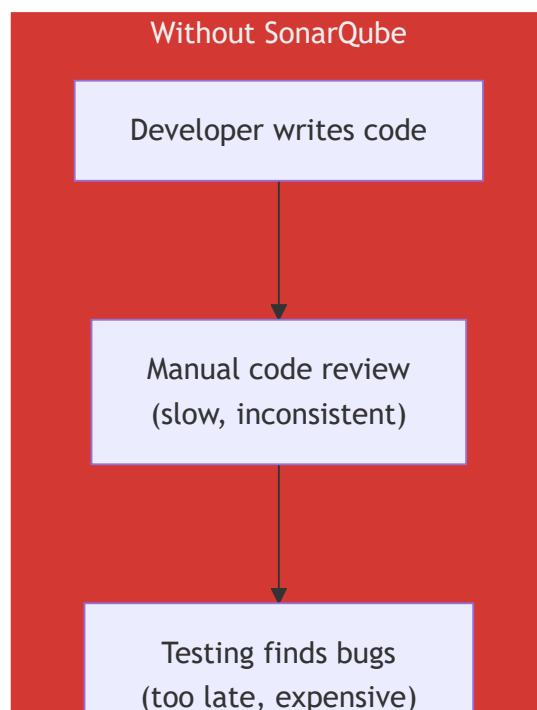
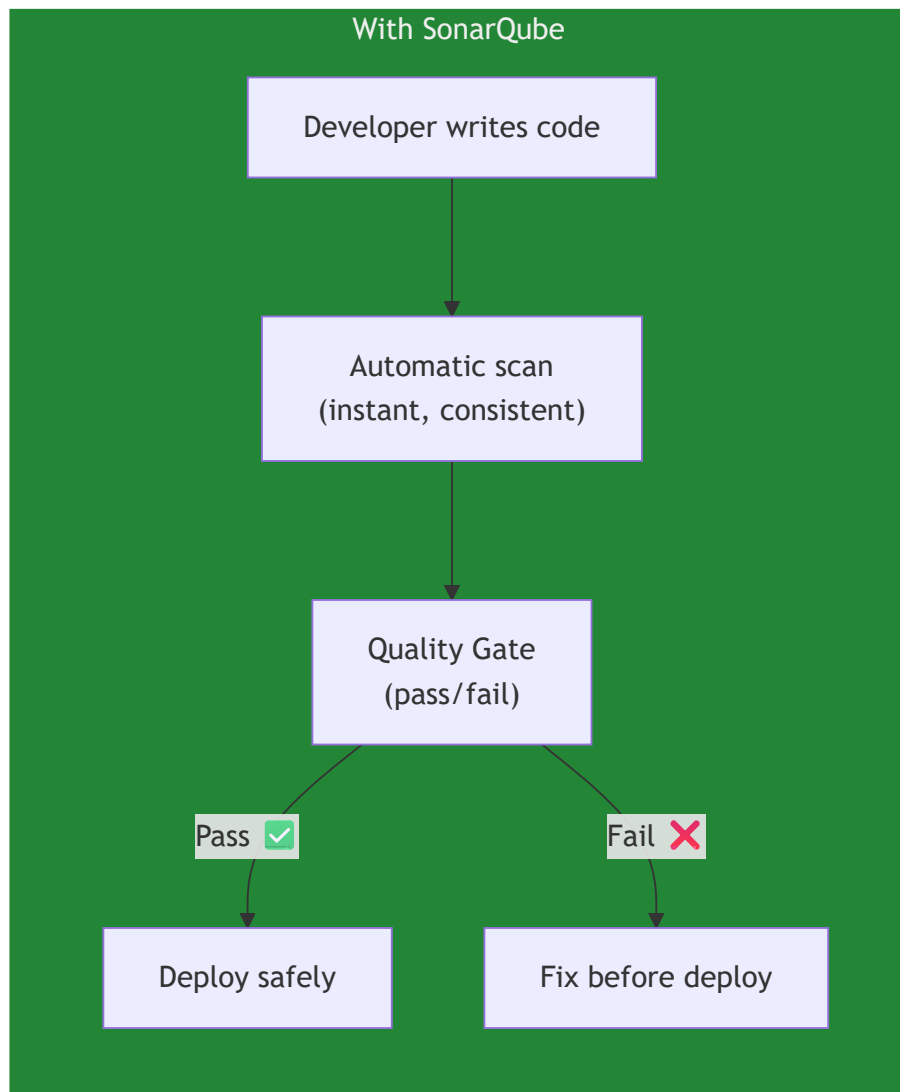
Consider this: a team of 10 developers each pushes 3 commits per day. That's 30 code changes daily. Can a senior engineer manually review every line for:

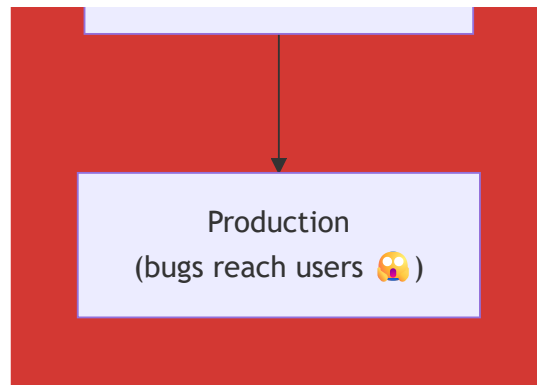
- Division by zero?
- SQL injection?
- Null pointer dereferences?
- Unused variables?
- Copy-pasted code blocks?

No. This is where **static code analysis** comes in.

What is SonarQube?

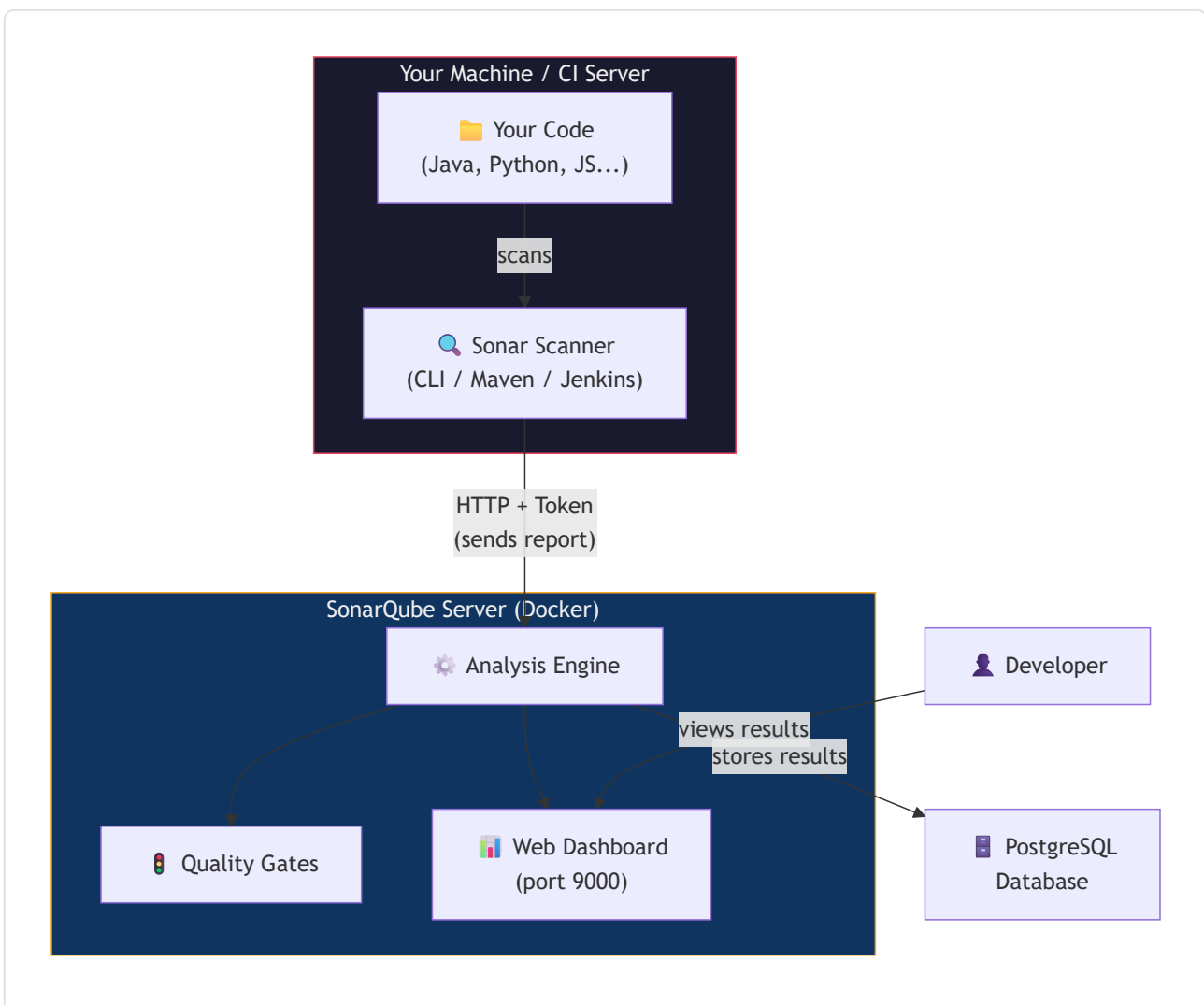
SonarQube is an open-source platform that automatically scans source code for bugs, security vulnerabilities, and maintainability issues — **without running the code**. This is called **static analysis** (analyzing code at rest, not at runtime).





SonarQube Architecture — Server + Scanner

SonarQube has **two separate components**:



Component	Role	Analogy
SonarQube Server	Stores results, applies rules, shows dashboard	The examiner — receives work, grades it, shows the report

Component	Role	Analogy
Sonar Scanner	Reads source code, detects issues, sends report	The student — does the work and submits it
PostgreSQL	Persists all analysis data	The filing cabinet — stores all grades permanently

Why Both Are Required

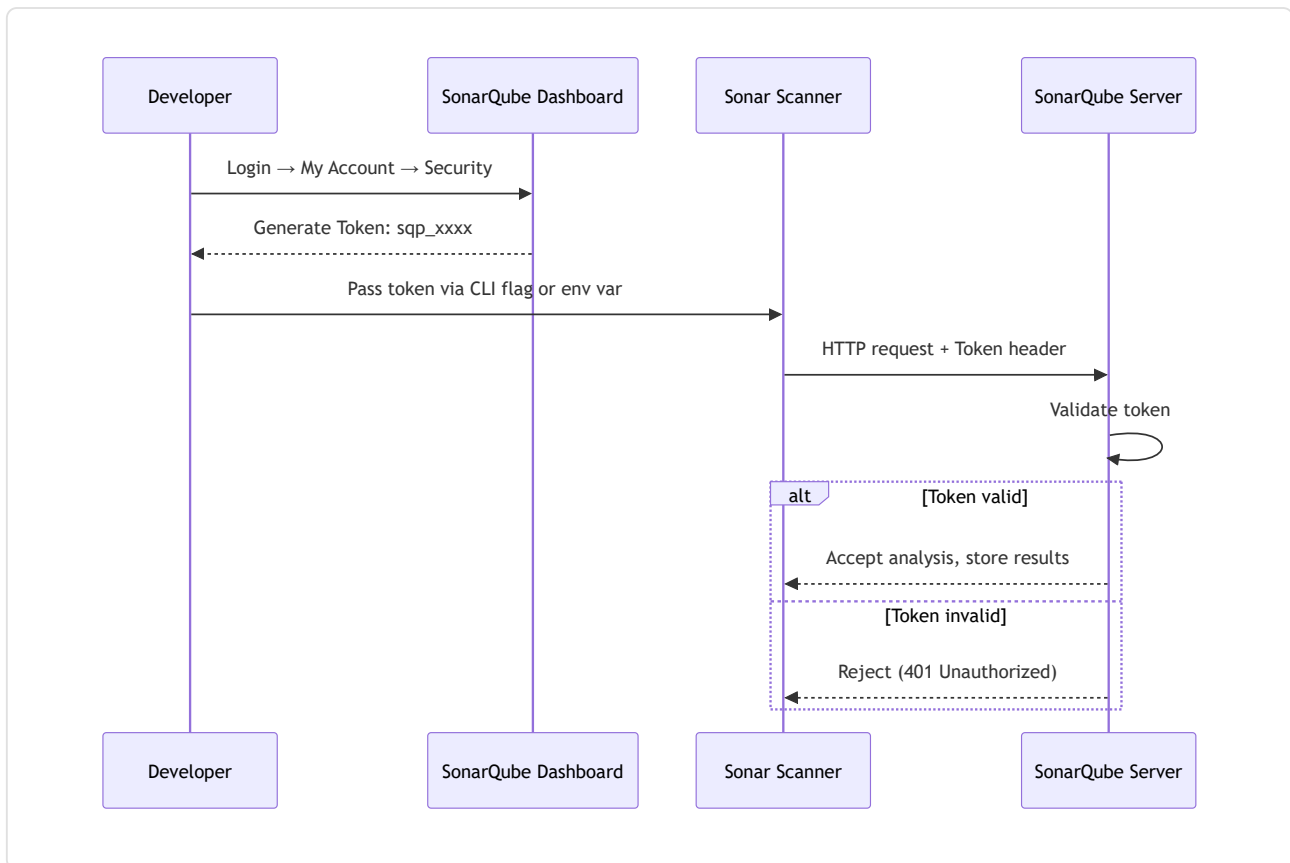
Scenario	Result
Only Server installed	Dashboard is empty — no code gets analyzed
Only Scanner installed	Nowhere to send results — analysis is wasted
Both installed together	Full pipeline works 

Issue Categories SonarQube Detects

Category	Icon	Meaning	Example
Bug		Code that will likely crash or behave incorrectly at runtime	Division by zero, null pointer dereference
Vulnerability		Security weakness an attacker could exploit	SQL injection, hardcoded passwords
Code Smell		Code that works but is poorly written	Unused variables, empty catch blocks
Technical Debt		Estimated time to fix all issues	"57 minutes to fix all smells"
Coverage		Percentage of code tested by unit tests	0% means no tests exist
Duplication		Repeated copy-pasted code blocks	Two identical methods

Token-Based Authentication

The Scanner authenticates with the Server using a **token** — not a username/password:



Hands-on Lab

Prerequisites

- WSL2 with Ubuntu
- Docker Desktop running
- Terminal access

Phase 1: Install Java JDK & Maven

```

sudo apt update -y
sudo apt install -y openjdk-17-jdk maven

# Verify both
java -version
mvn -version
  
```

Package

Purpose

openjdk-17-jdk

Java compiler — needed to compile the sample app

Package	Purpose
maven	Build tool — compiles Java, manages dependencies, runs the SonarQube scanner plugin

```
pranav@Pranav: ~/sonarqube
pranav@Pranav:~$ sudo apt update -y
sudo apt install -y openjdk-17-jdk maven

# Verify both
java -version
mvn -version

[sudo] password for pranav:
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble InRelease
Hit:7 https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64 InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
52 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-17-jdk is already the newest version (17.0.18+8-1~24.04.1).
The following additional packages will be installed:
  libaopalliance-java libapache-pom-java libatinject-jsr330-api-java libcdi-api-java libcommons-cli-java
  libcommons-io-java libcommons-lang3-java libcommons-parent-java liberror-prone-java
  libgeronimo-annotation-1.3-spec-java libgeronimo-interceptor-3.0-spec-java libguava-java libguice-java libjansi-java
  libjsr305-java libmaven-parent-java libmaven-resolver-java libmaven-shared-utils-java libmaven3-core-java
  libplexus-cipher-java libplexus-classworlds-java libplexus-component-annotations-java libplexus-interpolation-java
  libplexus-sec-dispatcher-java libplexus-utils2-java libsisu-inject-java libsisu-plexus-java libslf4j-java
  libwagon-file-java libwagon-http-shaded-java libwagon-provider-api-java
Suggested packages:
  libatinject-jsr330-api-java-doc libel-api-java libcommons-io-java-doc libasm-java libcglib-java libjsr305-java-doc
  libmaven-shared-utils-java-doc liblogback-java libplexus-utils2-java-doc junit4 testng libcommons-logging-java
  liblog4j1.2-java
The following NEW packages will be installed:
  libaopalliance-java libapache-pom-java libatinject-jsr330-api-java libcdi-api-java libcommons-cli-java
  libcommons-io-java libcommons-lang3-java libcommons-parent-java liberror-prone-java
  libgeronimo-annotation-1.3-spec-java libgeronimo-interceptor-3.0-spec-java libguava-java libguice-java libjansi-java
  libjsr305-java libmaven-parent-java libmaven-resolver-java libmaven-shared-utils-java libmaven3-core-java
  libplexus-cipher-java libplexus-classworlds-java libplexus-component-annotations-java libplexus-interpolation-java
  libplexus-sec-dispatcher-java libplexus-utils2-java libsisu-inject-java libsisu-plexus-java libslf4j-java
  libwagon-file-java libwagon-http-shaded-java libwagon-provider-api-java maven
0 upgraded, 32 newly installed, 0 to remove and 52 not upgraded.
Need to get 10.1 MB of archives.
After this operation, 13.1 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 libapache-pom-java all 29-2 [5284 B]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 libatinject-jsr330-api-java all 1.0+ds1-5 [5348 B]
```

Phase 2: Set WSL2 Kernel Parameters

This is the most critical step. Without it, SonarQube's embedded Elasticsearch crashes silently.

```
sudo sysctl -w vm.max_map_count=524288
sudo sysctl -w fs.file-max=131072
```

Parameter	WSL2 Default	Required	Purpose
vm.max_map_count	65530	524288	Max memory-mapped areas for Elasticsearch
fs.file-max	varies	131072	Max open file descriptors system-wide

These reset on WSL restart. For permanent fix, add them to `/etc/sysctl.conf`.

Phase 3: Start SonarQube with Docker Compose

Create the working directory:

```
mkdir -p ~/sonarqube-lab && cd ~/sonarqube-lab
```

Create `docker-compose.yml`:

```

services:

# Database – stores all SonarQube data persistently
sonar-db:
  image: postgres:13
  container_name: sonar-db
  environment:
    POSTGRES_USER: sonar
    POSTGRES_PASSWORD: sonar
    POSTGRES_DB: sonarqube
    POSTGRES_HOST_AUTH_METHOD: trust
  volumes:
    - sonar-db-data:/var/lib/postgresql/data
  networks:
    - sonarqube-lab

# SonarQube Server – web UI + analysis engine
sonarqube:
  image: sonarqube:lts-community
  container_name: sonarqube
  ports:
    - "9000:9000"
  environment:
    SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
    SONAR_JDBC_USERNAME: sonar
    SONAR_JDBC_PASSWORD: sonar
  volumes:
    - sonar-data:/opt/sonarqube/data
    - sonar-extensions:/opt/sonarqube/extensions
  depends_on:
    - sonar-db
  networks:
    - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge

```

Docker Compose Breakdown

Key	Purpose
<code>postgres:13</code>	PostgreSQL database image — stores analysis results

Key	Purpose
<code>sonarqube:lts-community</code>	SonarQube Long-Term Support community edition
<code>ports: "9000:9000"</code>	Expose dashboard on <code>http://localhost:9000</code>
<code>SONAR_JDBC_URL</code>	Tells SonarQube how to connect to PostgreSQL
<code>depends_on: sonar-db</code>	Start database before SonarQube server
Named <code>volumes</code>	Persist data across container restarts
<code>networks: sonarqube-lab</code>	Isolated bridge network — containers communicate by name

Start the containers:

```
docker compose up -d
docker compose logs -f sonarqube
```

Wait until you see "SonarQube is operational" in the logs, then press `Ctrl+C`.

```
pranav@Pranav: ~/sonarqube
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.10, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.6.87.2-microsoft-standard-wsl2", arch: "amd64", family: "unix"
pranav@Pranav:~$ sudo sysctl -w vm.max_map_count=524288
sudo sysctl -w fs.file-max=131072
vm.max_map_count = 524288
fs.file-max = 131072
pranav@Pranav:~$ mkdir -p ~/sonarqube-lab
cd ~/sonarqube-lab
pranav@Pranav:~/sonarqube-lab$ cat > ~/sonarqube-lab/docker-compose.yml << 'EOF'
services:

  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
      POSTGRES_HOST_AUTH_METHOD: trust
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
EOF
driver: bridge
pranav@Pranav:~/sonarqube-lab$ cd ~/sonarqube-lab
docker compose up -d
```

```

pranav@Pranav: ~/sonarqube
docker compose logs -f sonarqube
[+] up 30/30
✓Image postgres:13                               Pulled                               63.6ss
✓Image sonarqube:lts-community                    Pulled                               124.5s
✓Network sonarqube-lab_sonarqube-lab              Created                             0.1s
✓Volume sonarqube-lab_sonar-db-data               Created                             0.0s
✓Volume sonarqube-lab_sonar-data                  Created                             0.0s
✓Volume sonarqube-lab_sonar-extensions             Created                             0.0s
✓Container sonar-db                               Created                             0.3s
✓Container sonarqube                              Created                             0.0s
sonarqube | 2026.04.24 17:50:53 INFO app[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube | 2026.04.24 17:50:53 INFO app[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:36903]
sonarqube | 2026.04.24 17:50:53 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/sonarqube/el
asticssearch/bin/elasticsearch
sonarqube | 2026.04.24 17:50:53 INFO app[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube | 2026.04.24 17:50:56 INFO es[[o.e.n.Node] version[7.17.15], pid[26], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/2023-11-10T22:
03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
sonarqube | 2026.04.24 17:50:56 INFO es[[o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube | 2026.04.24 17:50:56 INFO es[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs
/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -D
file.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimi
zation=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true,
-Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:Ma
xDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distr
ibution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=false]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] loaded module [analysis-common]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] loaded module [lang-painless]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] loaded module [parent-join]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] loaded module [reindex]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] loaded module [transport-netty4]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.p.PluginsService] no plugins loaded
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.e.NodeEnvironment] using [1] data paths, mounts [[/opt/sonarqube/data (/dev/sdf)]]], net usable_space [935.9g
b], net total_space [1006.8gb], types [ext4]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.e.NodeEnvironment] heap size [512mb], compressed ordinary object pointers [true]
sonarqube | 2026.04.24 17:50:57 INFO es[[o.e.n.Node] node name [sonarqube], node ID [8zfFmvBPTtKyGdXWZZGVRA], cluster name [sonarqube], roles [data_froze
n_master, remote_cluster_client, data, data_content, data_hot, data_warm, data_cold, ingest]
sonarqube | 2026.04.24 17:50:59 INFO es[[o.e.t.NettyAllocator] creating NettyAllocator with the following configs: [name=unpooled, suggested_max_allocati
on_size=256kb, factors=[es.unsafe.use_unpooled_allocator=null, g1gc_enabled=true, g1gc_region_size=1mb, heap_size=512mb]]
sonarqube | 2026.04.24 17:50:59 INFO es[[o.e.i.r.RecoverySettings] using rate limit [40mb] with [default=40mb, read=0b, write=0b, max=0b]
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.d.DiscoveryModule] using discovery type [zen] and seed hosts providers [settings]
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.g.DanglingIndicesState] gateway.auto_import_dangling_indices is disabled, dangling indices will not be autom
atically detected or imported and must be managed manually
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.n.Node] initialized
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.n.Node] starting ...
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.t.TransportService] publish_address [127.0.0.1:36903], bound_addresses [127.0.0.1:36903]
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.b.BootstrapChecks] explicitly enforcing bootstrap checks
sonarqube | 2026.04.24 17:51:00 INFO es[[o.e.c.c.Coordinator] setting initial configuration to VotingConfiguration{8zfFmvBPTtKyGdXWZZGVRA}

```

Verify Server Health

```
curl -s http://localhost:9000/api/system/status | python3 -m json.tool
```

Expected output when healthy:

```
{
  "id": "xxxxxxx",
  "version": "9.9.x",
  "status": "UP"
}
```

Open <http://localhost:9000> in your browser. Default login: **admin / admin** (you'll be prompted to change the password on first login — use **Admin@1234**).

Phase 4: Generate Authentication Token

1. Open `http://localhost:9000`
2. Log in as admin
3. Click your user icon (top right) → "My Account"
4. Click the "Security" tab
5. Under "Generate Tokens", type: scanner-token
6. Click "Generate"
7. Copy the token immediately – it is shown only once!
Format: `sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX`

Store it as an environment variable:

```
export SONAR_TOKEN="sqp_your_actual_token_here"
echo $SONAR_TOKEN    # verify it's set
```

Security best practice: Never hardcode tokens in `pom.xml` or source files. If you commit them to Git, anyone can access your SonarQube server.

```
pranav@Pranav: ~/sonarqube
sonarqube | 2026.04.24 18:01:08 INFO web[AZ3AoD+eS6IRuydoAABe][o.s.u.c.UpdateCenter] The plugin 'vbnet' version : 8.51.0.59060 has not been found on the u
pdate center.
sonarqube | 2026.04.24 18:01:08 INFO web[AZ3AoD+eS6IRuydoAABe][o.s.u.c.UpdateCenter] The plugin 'web' version : 3.7.1.3306 has not been found on the updat
e center.
sonarqube | 2026.04.24 18:01:08 INFO web[AZ3AoD+eS6IRuydoAABe][o.s.u.c.UpdateCenter] The plugin 'xml' version : 2.7.0.3820 has not been found on the updat
e center.
^C
pranav@Pranav:~/sonarqube-lab$ # Paste your actual token here
export SONAR_TOKEN="sqa_0f41947acf700fce8ae9a4dffa034eb951c8d12a8"
# Verify it's set
echo $SONAR_TOKEN
sqa_0f41947acf700fce8ae9a4dffa034eb951c8d12a8
pranav@Pranav:~/sonarqube-lab$ mkdir -p ~/sonarqube-lab/sample-java-app/src/main/java/com/example
cd ~/sonarqube-lab/sample-java-app
pranav@Pranav:~/sonarqube-lab/sample-java-app$ cat > ~/sonarqube-lab/sample-java-app/src/main/java/com/example/Calculator.java << 'EOF'
package com.example;

public class Calculator {

    // BUG: Division by zero not handled
    public int divide(int a, int b) {
        return a / b;
    }

    // CODE SMELL: Unused variable
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    // VULNERABILITY: SQL Injection risk
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    // CODE SMELL: Duplicated code
    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
    }
}

EOF }
```

Phase 5: Create the Sample Java Project

```
mkdir -p ~/sonarqube-lab/sample-java-app/src/main/java/com/example
cd ~/sonarqube-lab/sample-java-app
```

Create `src/main/java/com/example/Calculator.java` — a class with **intentional bugs, vulnerabilities, and code smells**:

```

package com.example;

public class Calculator {

    // BUG: Division by zero not handled
    // If someone calls divide(5, 0), this crashes at runtime
    public int divide(int a, int b) {
        return a / b;
    }

    // CODE SMELL: Unused variable
    // 'unused' is declared but never read - clutters the code
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;    // ← SonarQube flags this
        return result;
    }

    // VULNERABILITY: SQL Injection risk
    // Concatenating user input into SQL is exploitable
    // Attacker could pass: "1 OR 1=1" to get all users
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    // CODE SMELL: Duplicated code
    // These two methods are identical - should be one method
    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;    // ← exact duplicate
        }
        return result;
    }

    // BUG: Null pointer risk
    // If name is null, .toUpperCase() throws NullPointerException
    public String getName(String name) {
        return name.toUpperCase();
    }
}

```

```
// CODE SMELL: Empty catch block
// Exception is caught but silently ignored - hides errors
public void riskyOperation() {
    try {
        int x = 10 / 0;
    } catch (Exception e) {
        // ← never leave catch blocks empty
    }
}
}
```

Create `pom.xml`:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.projectName>Sample Java Application</sonar.projectName>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <!-- Token is NOT stored here - passed via CLI -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>

```

```

pranav@Pranav: ~/sonarqube
int result = 0;
for (int i = 0; i < b; i++) {
    result = result + a;
}
} // empty catch - hides errors{
pranav@Pranav:~/sonarqube-lab/sample-java-app$ cat > ~/sonarqube-lab/sample-java-app/pom.xml << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>sample-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <sonar.projectKey>sample-java-app</sonar.projectKey>
        <sonar.projectName>Sample Java Application</sonar.projectName>
        <sonar.host.url>http://localhost:9000</sonar.host.url>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-resources-plugin</artifactId>
                <version>3.9.1</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.3.0</version>
            </plugin>
        </plugins>
    </build>
</project>
pranav@Pranav:~/sonarqube-lab/sample-java-app$ cd ~/sonarqube-lab/sample-java-app
mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.pom (8.1 kB at
15 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.3.0/maven-jar-plugin-3.3.0.pom

```

Phase 6: Compile and Scan

Step 1 — Compile first (creates `target/classes/`):

```
cd ~/sonarqube-lab/sample-java-app
mvn compile
```

Why compile first? The SonarQube Maven plugin needs compiled `.class` files for **bytecode analysis** — detecting null pointer risks, unused variables, etc. Without compiled classes, SonarQube can only do text-level scanning and misses most bugs.

```

pranav@Pranav: ~/sonarqube
pranav@Pranav:~/sonarqube-lab/sample-java-app$ cd ~/sonarqube-lab/sample-java-app
mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.jar (8.1 kB at 15 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/23/maven-plugins-23.pom (9.2 kB at 204 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/22/maven-parent-22.pom (30 kB at 633 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/22/maven-parent-22.pom (30 kB at 633 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/11/apache-11.pom (15 kB at 23 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.jar (30 kB at 602 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.jar (30 kB at 602 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom (10 kB at 227 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom (10 kB at 227 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/24/maven-plugins-24.pom (11 kB at 13 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/23/maven-parent-23.pom (33 kB at 693 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/23/maven-parent-23.pom (33 kB at 693 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/13/apache-13.pom (14 kB at 297 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.jar (43 kB at 39 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.jar (43 kB at 39 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom (27 kB at 563 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.pom (27 kB at 563 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom (766 B at 15 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.pom (766 B at 15 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom (2.0 kB at 40 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-parent/1.3/hamcrest-parent-1.3.pom (2.0 kB at 40 kB/s)
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ sample-app ---
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-api/2.0.6/maven-plugin-api-2.0.6.pom (1.5 kB at 29 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-api/2.0.6/maven-plugin-api-2.0.6.pom (1.5 kB at 29 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven/2.0.6/maven-2.0.6.pom (9.0 kB at 177 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven/2.0.6/maven-2.0.6.pom (9.0 kB at 177 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/5/maven-parent-5.pom (15 kB at 324 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/5/maven-parent-5.pom (15 kB at 324 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom (3.4 kB at 71 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom (3.4 kB at 71 kB/s)

```

Step 2 — Run the SonarQube scan:

```
mvn sonar:sonar -Dsonar.login=$SONAR_TOKEN
```

Flag

```
sonar:sonar
```

Purpose

Invokes the SonarQube Maven plugin

```
-Dsonar.login=$SONAR_TOKEN
```

Authentication token (from env variable)

Expected output:

```

[INFO] BUILD SUCCESS
[INFO] Analysis total time: 5.697 s
[INFO] More about the report processing at
      http://localhost:9000/api/ce/task?id=AZ3AqabR04yPRjbdvmmww

```

```

pranav@Pranav: ~/sonarqube
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/xbean/xbean-reflect/3.4/xbean-reflect-3.4.jar (134 kB at 485 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-logging/commons-logging-api/1.1/commons-logging-api-1.1.jar (45 kB at 158 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/log4j/log4j/1.2.12/log4j-1.2.12.jar (358 kB at 1.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/3.8.2/junit-3.8.2.jar (121 kB at 388 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/collections/google-collections/1.0/google-collections-1.0.jar (640 kB at 1.8 MB/s)
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/pranav/sonarqube-lab/sample-java-app/target/classes
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.708 s
[INFO] Finished at: 2026-04-24T18:03:37Z
[INFO] -----
pranav@Pranav:~/sonarqube-lab/sample-java-app$ mvn sonar:sonar -Dsonar.login=$SONAR_TOKEN
[INFO] Scanning for projects...
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.pom (12 kB at 39 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom (32 kB at 598 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.jar (50 kB at 887 kB/s)
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar (385 kB at 4.7 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar (45 kB at 530 kB/s)
[INFO] --- sonar-maven-plugin:3.9.1.2184:sonar (default-cli) @ sample-app ---
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.4.pom (3.0 kB at 66 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.2.1/plexus-utils-3.2.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.2.1/plexus-utils-3.2.1.pom (5.3 kB at 109 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.1/plexus-5.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/5.1/plexus-5.1.pom (23 kB at 459 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/api/sonar-scanner-api/2.16.2.588/sonar-scanner-api-2.16.2.588.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/api/sonar-scanner-api/2.16.2.588/sonar-scanner-api-2.16.2.588.pom (5.2 kB at 99 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/api/sonar-scanner-api-parent/2.16.2.588/sonar-scanner-api-parent-2.16.2.588.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/api/sonar-scanner-api-parent/2.16.2.588/sonar-scanner-api-parent-2.16.2.588.pom

```

Phase 7: View Results in Dashboard

Open: <http://localhost:9000/dashboard?id=sample-java-app>

The dashboard shows:

Metric	Value (from our scan)	Rating
Bugs	1	Reliability: D
Vulnerabilities	0	Security: A
Security Hotspots	0	Security Review: A
Code Smells	6	Maintainability: A
Technical Debt	57 min	—
Coverage	0.0%	No unit tests
Duplications	0.0%	—
Quality Gate	✓ Passed	All conditions met

```

pranav@Pranav: ~/sonarqube
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ3AqabR04yPRjbdvmww
[INFO] Analysis total time: 5.697 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.130 s
[INFO] Finished at: 2026-04-24T18:03:57Z
[INFO] -----
pranav@Pranav:~/sonarqube-lab/sample-java-app$ # View all bugs found
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG" \
  | python3 -m json.tool | grep -E "message|severity|component"

# View vulnerabilities
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=VULNERABILITY" \
  | python3 -m json.tool | grep -E "message|severity"

# View code smells
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=CODE_SMELL" \
  | python3 -m json.tool | grep -E "message|severity"

# Check quality gate status
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/qualitygates/project_status?projectKey=sample-java-app" \
  | python3 -m json.tool
Expecting value: line 1 column 1 (char 0)
Expecting value: line 1 column 1 (char 0)
Expecting value: line 1 column 1 (char 0)
Expecting value: line 1 column 1 (char 0)
pranav@Pranav:~/sonarqube-lab/sample-java-app$ cd ~/sonarqube-lab
docker compose down

# To also delete all stored data (full reset)
docker compose down -v
[+] down 3/3
✓Container sonarqube          Removed      1.6s
✓Container sonar-db           Removed      0.5s
✓Network sonarqube-lab_sonarqube-lab Removed      0.3s
[+] down 3/3
✓Volume sonarqube-lab_sonar-db-data Removed      0.1s
✓Volume sonarqube-lab_sonar-data Removed      0.1s
✓Volume sonarqube-lab_sonar-extensions Removed      0.1s
pranav@Pranav:~/sonarqube-lab$

```

Phase 8: Query Results via REST API

```

# View all bugs found
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG" \
  | python3 -m json.tool | grep -E "message|severity|component"

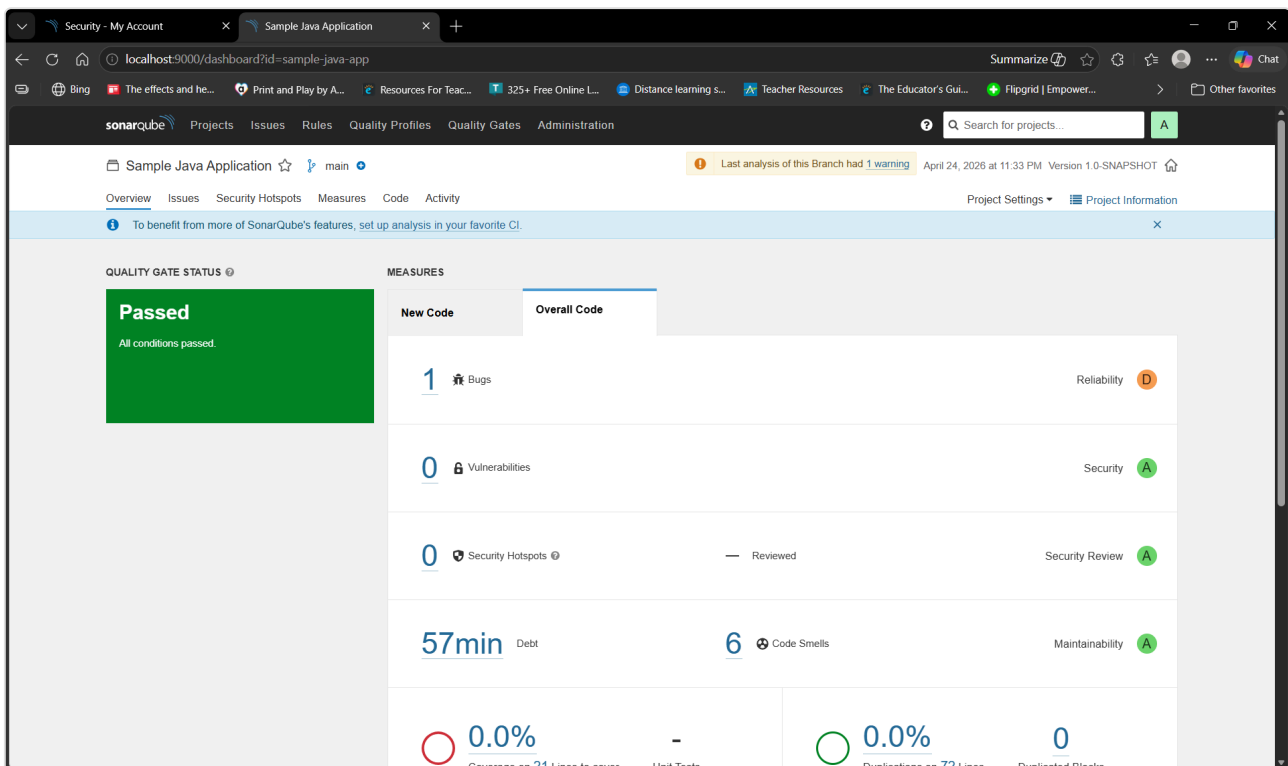
# View vulnerabilities
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=VULNERABILITY" \
  | python3 -m json.tool | grep -E "message|severity"

# View code smells
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=CODE_SMELL" \
  | python3 -m json.tool | grep -E "message|severity"

# Check Quality Gate status
curl -s -u admin:Admin@1234 \
  "http://localhost:9000/api/qualitygates/project_status?projectKey=sample-java-app" \
  | python3 -m json.tool

```

Flag	Purpose
<code>-s</code>	Silent mode — suppresses progress bar noise
<code>-u admin:Admin@1234</code>	HTTP Basic Auth credentials
<code>python3 -m json.tool</code>	Pretty-print JSON output
<code>grep -E "message severity"</code>	Filter to show only relevant fields



Phase 9: Cleanup

```
cd ~/sonarqube-lab

# Stop containers (preserves data in volumes)
docker compose down

# Full reset - remove containers AND all stored data
docker compose down -v
```

Command	Effect
<code>docker compose down</code>	Stops and removes containers + network; volumes are preserved
<code>docker compose down -v</code>	Also removes named volumes — full data reset

Alternative Scanner Method: Docker CLI Scanner

For non-Java projects (or if you don't want Maven), use the SonarQube Scanner Docker image:

Create `sonar-project.properties`:

```
sonar.projectKey=sample-java-app
sonar.projectName=Sample Java Application
sonar.projectVersion=1.0
sonar.sources=src
sonar.java.binaries=target/classes
sonar.sourceEncoding=UTF-8
```

Run the scanner via Docker:

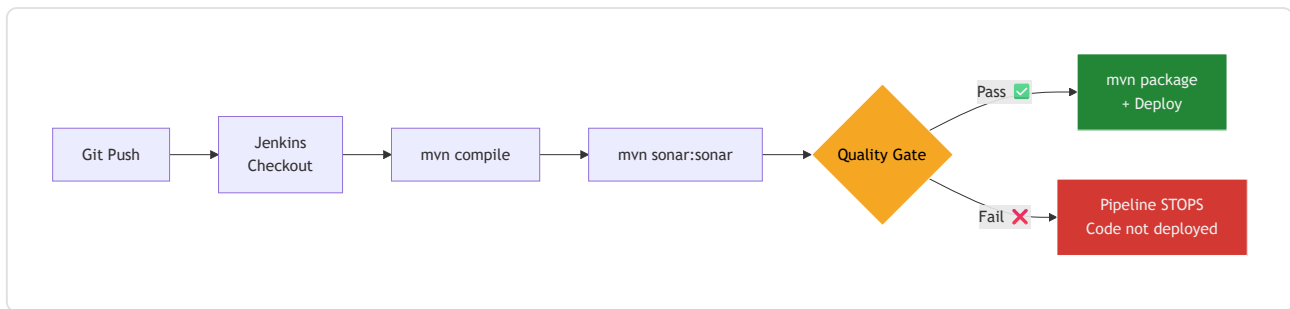
```
docker run --rm \
  --network sonarqube-lab_sonarqube-lab \
  -e SONAR_TOKEN="$SONAR_TOKEN" \
  -v "$(pwd):/usr/src" \
  sonarsource/sonar-scanner-cli \
  -Dsonar.host.url=http://sonarqube:9000 \
  -Dsonar.projectBaseDir=/usr/src \
  -Dsonar.projectKey=sample-java-app
```

Flag	Purpose
<code>--rm</code>	Auto-delete the scanner container after completion
<code>--network</code>	Connect to the same Docker network as SonarQube
<code>-e SONAR_TOKEN</code>	Pass authentication token
<code>-v "\$(pwd):/usr/src"</code>	Mount your project folder into the scanner container
<code>http://sonarqube:9000</code>	Uses container name (not localhost) — because scanner runs inside Docker network

Finding the network name: Run `docker network ls` — it's typically `<folder>-sonarqube-lab`.

Deep Dive: Jenkins CI/CD Integration

In production, SonarQube scans run automatically on every commit via Jenkins:



Example Jenkinsfile:

```

pipeline {
  agent any
  environment {
    SONAR_HOST_URL = 'http://sonarqube:9000'
    SONAR_TOKEN = credentials('sonar-token')
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('SonarQube Analysis') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh 'mvn clean verify sonar:sonar'
        }
      }
    }
    stage('Quality Gate') {
      steps {
        timeout(time: 5, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }
    stage('Build & Deploy') {
      steps {
        sh 'mvn package'
        sh 'docker build -t sample-app .'
        sh 'docker run -d -p 8080:8080 sample-app'
      }
    }
  }
}

```

Comparative Analysis

DevOps Tool Comparison

Feature	SonarQube	Jenkins	Ansible	Chef
Purpose	Code Quality / Static Analysis	CI/CD Automation	Configuration Management	Configuration Management
Architecture	Client-Server	Master-Agent	Agentless (SSH)	Client-Server
Language	Java	Groovy	YAML	Ruby
Learning Curve	Low	Moderate	Low	High
Use Case	Scan code for bugs/vulnerabilities	Build, test, deploy pipelines	Infrastructure as Code	Enterprise config management

How they work together:

- **Jenkins** orchestrates the pipeline (build → test → deploy)
- **SonarQube** scans code quality within the pipeline
- **Ansible/Chef** manages server configuration at deployment

Common Pitfalls & Troubleshooting

Problem	Cause	Fix
SonarQube container starts but web UI never loads	<code>vm.max_map_count</code> too low for Elasticsearch	<code>sudo sysctl -w vm.max_map_count=524288</code>
Invalid value for <code>sonar.java.binaries</code>	Code not compiled — no <code>.class</code> files	Run <code>mvn compile</code> before <code>mvn sonar:sonar</code>
Scan runs but finds 0 issues	Bytecode analysis missing without compilation	Run <code>mvn compile</code> first
<code>401 Unauthorized</code> during scan	Invalid or expired token	Regenerate token in the web UI
<code>version: '3.8'</code> warning in Compose	Obsolete key in Compose v2	Remove the <code>version:</code> line entirely
Java version mismatch error	<code>pom.xml</code> targets Java 11 but JDK 17 is installed	Set <code>maven.compiler.source/target</code> to <code>17</code>
Token committed to Git	Hardcoded in <code>pom.xml</code>	Use <code>\$SONAR_TOKEN</code> env var + <code>-Dsonar.login</code> flag

Lab Fixes Summary

Change	Original (Lab Sheet)	Fixed	Why
Compose version	<code>version: '3.8'</code> included	Removed	Obsolete in Compose v2, produces warnings
Kernel params	Not mentioned	Added <code>sysctl</code> commands	WSL2 defaults too low for Elasticsearch
Java version	Source/target = 11	Changed to 17	Must match installed JDK
Token storage	Hardcoded in <code>pom.xml</code>	Env variable + CLI flag	Security — tokens must never be in source files
Compilation	Straight to <code>mvn sonar:sonar</code>	Added <code>mvn compile</code> first	Scanner needs bytecode for full analysis
API auth	<code>-u admin:YOUR_TOKEN</code>	<code>-u admin:Admin@1234</code>	Use actual credentials for Basic Auth
Health check	Not mentioned	Added <code>curl /api/system/status</code>	Confirms server is truly operational

Best Practices

Security:

- Never hardcode tokens in source files — use environment variables or secrets managers
- Use HTTPS for SonarQube in production
- Give scanner tokens minimal permissions (analyze only)

Code Quality:

- Set Quality Gates to block merges when coverage drops below 80%
- Scan on every pull request, not just nightly
- Fix issues as they appear — don't let technical debt accumulate

CI/CD:

- Cache Maven dependencies between builds
- Run SonarQube analysis in parallel with unit tests
- Configure pipelines to fail fast on Quality Gate failures

Glossary

Term	Definition
SonarQube	Open-source platform for automated static code analysis
Static Analysis	Examining source code for bugs without executing it
Sonar Scanner	CLI tool or Maven plugin that reads code and sends analysis reports to SonarQube
Quality Gate	A set of pass/fail conditions code must meet before deployment
Bug	Code that will likely crash or behave incorrectly at runtime
Vulnerability	A security weakness attackable by malicious users
Code Smell	Code that works but is poorly structured or hard to maintain
Technical Debt	Estimated time needed to fix all detected issues
Coverage	Percentage of code exercised by unit tests
Duplication	Repeated code blocks (copy-paste programming)
Token	Authentication credential generated in the SonarQube UI for scanner access
Maven	Java build tool — compiles code, manages dependencies, runs plugins
pom.xml	Maven project configuration file — defines dependencies and build settings
Bytecode Analysis	Analyzing compiled <code>.class</code> files for deeper bug detection
Quality Profile	Set of rules SonarQube applies during analysis (per language)
Elasticsearch	Search engine embedded in SonarQube for indexing code and issues
JDBC	Java Database Connectivity — how SonarQube connects to PostgreSQL
Idempotent Analysis	Rescanning produces the same results — no duplicate issues created

Optional: Scanner for Other Languages

Node.js / Express

```
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh 'npm install -g sonar-scanner'
            sh '''
            sonar-scanner \
            -Dsonar.projectKey=my-node-app \
            -Dsonar.sources=. \
            -Dsonar.host.url=$SONAR_HOST_URL \
            -Dsonar.login=$SONAR_AUTH_TOKEN
            '''
        }
    }
}
```

Python

```
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
            sonar-scanner \
            -Dsonar.projectKey=my-python-app \
            -Dsonar.sources=. \
            -Dsonar.host.url=$SONAR_HOST_URL \
            -Dsonar.login=$SONAR_AUTH_TOKEN
            '''
        }
    }
}
```

Key insight: Maven is a **Java-specific** build tool. For non-Java projects, use the **universal SonarScanner CLI** directly.

References

- [SonarQube Official Docs](#)
- [SonarQube Docker Hub](#)
- [Maven SonarQube Plugin](#)

Objective

Transition a WordPress + MySQL multi-container application from Docker Compose (single-host, manual management) to Docker Swarm (orchestrated, self-healing, load-balanced) to understand container orchestration concepts including scaling, service discovery, and automatic failure recovery.

Theory

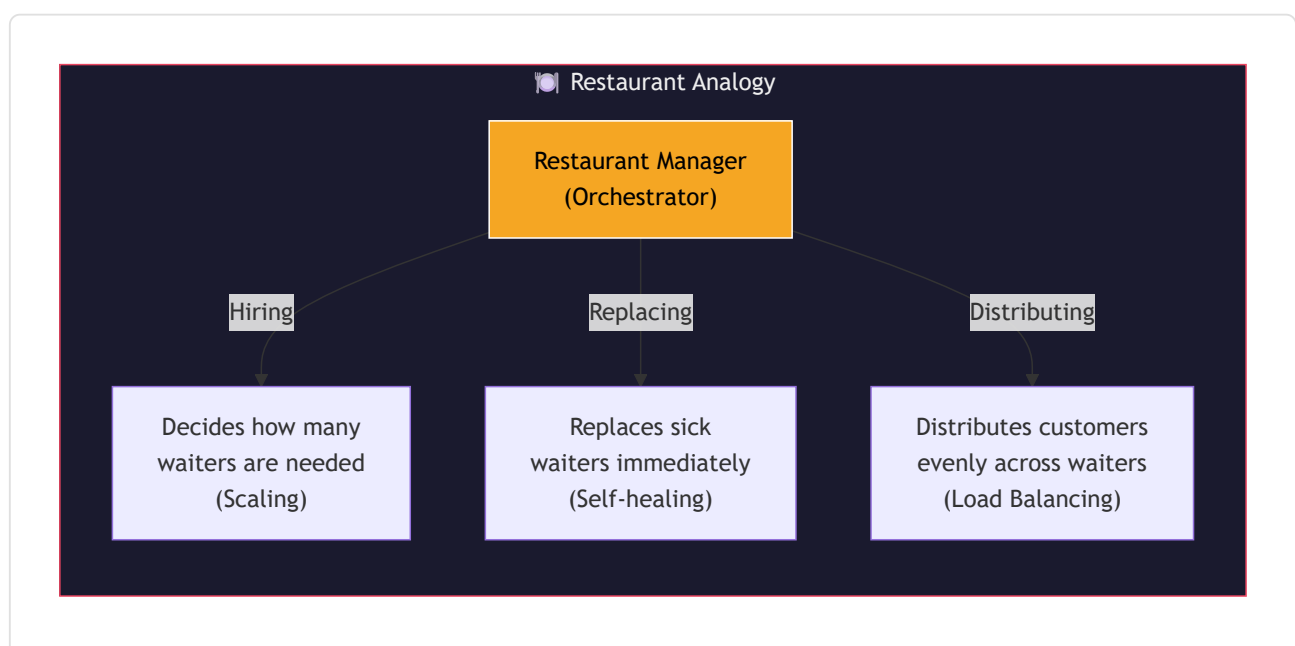
The Problem: Compose Can't Manage Production Workloads

From Experiment 6, you used **Docker Compose** to run WordPress + MySQL together. But Compose has critical limitations:

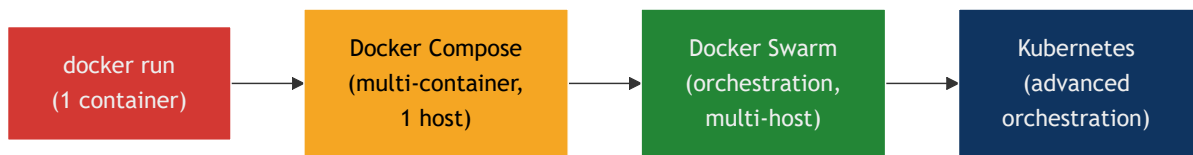
Limitation	What Goes Wrong
No scaling	Scaling WordPress to 3 copies causes port conflicts — all try to bind <code>8080</code>
No self-healing	If a container crashes, it stays dead until you manually restart it
Single host only	All containers must run on one machine — no distributed systems
No load balancing	Traffic hits one container; others sit idle

The Solution: Container Orchestration

Orchestration is the automatic management of containers across a cluster — deciding how many to run, where to place them, and how to recover from failures.



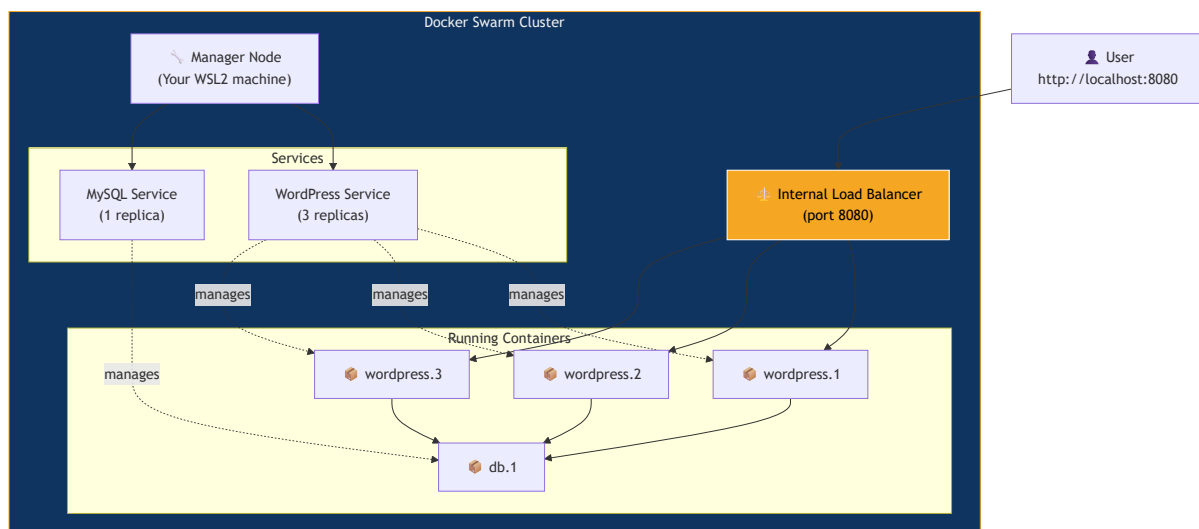
The Container Evolution Path



This experiment focuses on: Moving from Compose → Swarm

What is Docker Swarm?

Docker Swarm is Docker's built-in orchestration tool. It turns a group of Docker hosts into a single virtual system (a "cluster") and manages containers as **services** rather than individual units.



Key Swarm Concepts

Concept	Description
Node	A Docker host (machine) participating in the Swarm
Manager Node	Controls the cluster — schedules tasks, maintains state
Worker Node	Runs containers assigned by the manager
Service	A definition of how to run containers (image, replicas, ports)
Task	A single container instance running as part of a service

Concept	Description
Stack	A group of services deployed together from a Compose file
Replica	A copy of a container — <code>replicas: 3</code> = 3 identical containers
Ingress Network	Swarm's built-in mesh network + load balancer for routing traffic

Compose vs Stack Deploy

Feature	<code>docker compose up</code>	<code>docker stack deploy</code>
Mode	Single host, no orchestration	Swarm-managed orchestration
Scaling	<code>--scale</code> (basic, port conflicts)	<code>docker service scale</code> (built-in LB)
Load Balancing	None	Yes (ingress mesh)
Self-Healing	None	Automatic container replacement
Rolling Updates	None	Zero-downtime updates
Service Discovery	Container names	DNS + Virtual IPs (VIP)
Use Case	Development, testing	Production clusters

Prerequisites

- WSL2 with Ubuntu
- Docker Desktop running (or Docker Engine with Swarm support)
- Familiarity with Experiment 6 (Docker Compose)

Hands-on Lab

Phase 1: Prepare the Environment

Clean up any containers from previous experiments:

```
docker compose down -v 2>/dev/null
docker ps
```

Create the working directory:

```
mkdir -p ~/swarm-lab && cd ~/swarm-lab
```

Phase 2: Create the Docker Compose File

```
# docker-compose.yml (Swarm-compatible version)
services:

  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Note: This is the same WordPress + MySQL stack from Experiment 6, but with `version` and `container_name` removed (see Lab Fixes Summary for why).

```

pranav@Pranav: ~/swarm-lab
pranav@Pranav:~$ mkdir -p ~/swarm-lab
cd ~/swarm-lab
pranav@Pranav:~/swarm-lab$ cat > ~/swarm-lab/docker-compose.yml << 'EOF'
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  EOF
pranav@Pranav:~/swarm-lab$ cat ~/swarm-lab/docker-compose.yml
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:

```

Phase 3: Initialize Docker Swarm

```
docker swarm init --advertise-addr $(hostname -I | awk '{print $1}')
```

Flag	Purpose
<code>docker swarm init</code>	Activates Swarm mode, makes this node a manager
<code>--advertise-addr</code>	Specifies which IP address to use for cluster communication

Why `--advertise-addr`? WSL2 has multiple network interfaces (`lo` + `eth0`). Without specifying, Swarm fails with: *"could not choose an IP address to advertise"*

Verify Swarm is active:

```
docker node ls
```

Expected output:

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
na1mcocekng0to9 ...	Pranav	Ready	Active	Leader

```

pranav@Pranav: ~/swarm-lab
depends_on:
  - db
ports:
  - "8080:80"
restart: always
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wpuuser
  WORDPRESS_DB_PASSWORD: wppass
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
pranav@Pranav:~/swarm-lab$ docker swarm init
Error response from daemon: could not choose an IP address to advertise since this system has multiple addresses on different interfaces (10.255.255.254 on
lo and 192.168.206.154 on eth0) - specify one with --advertise-addr
pranav@Pranav:~/swarm-lab$ docker swarm init --advertise-addr 192.168.206.154
Swarm initialized: current node (na1mcocekng0to9trnfgc7bjk) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1f7q4l9aa06plogzh2pnhzppg19lmo1cccigmeoyq24n07bpqa-9aa75nvpvcjvfffix2z0rin412 192.168.206.154:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
pranav@Pranav:~/swarm-lab$ docker swarm join-token manager
Swarm initialized: current node (xxxxx) is now a manager.
-bash: syntax error near unexpected token `('
pranav@Pranav:~/swarm-lab$ docker node ls
ID                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
na1mcocekng0to9trnfgc7bjk *   Pranav     Ready     Active           Leader             29.2.1
pranav@Pranav:~/swarm-lab$ cd ~/swarm-lab
pranav@Pranav:~/swarm-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
pranav@Pranav:~/swarm-lab$ # Watch until both show 1/1 replicas
watch docker service ls
pranav@Pranav:~/swarm-lab$ docker service ls
ID                NAME                MODE                REPLICAS        IMAGE                PORTS
z4ieajugb2w      wpstack_db          replicated          1/1             mysql:5.7
q5k706zkxsmc     wpstack_wordpress   replicated          1/1             wordpress:latest    *:8080->80/tcp

```

Phase 4: Deploy the Stack to Swarm

```

cd ~/swarm-lab
docker stack deploy -c docker-compose.yml wpstack

```

Argument

Purpose

`docker stack deploy`

Deploy a group of services to Swarm

`-c docker-compose.yml`

Use this Compose file as the blueprint

`wpstack`

Name of the stack — prefixes all service names

Expected output:

```

Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress

```

Note: Swarm ignores the `restart` policy (it uses its own self-healing instead) — you'll see *"Ignoring unsupported options: restart"*.

Verify services are running:

```
docker service ls
```

Expected output:

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
z4ieajiugb2w	wpstack_db	replicated	1/1	mysql:5.7	
q5k706zlkxsmc	wpstack_wordpress	replicated	1/1	wordpress:latest	*:8080→80/tcp

Inspect individual service tasks:

```
docker service ps wpstack_wordpress
docker service ps wpstack_db
```

```
docker ps
```

Container names now follow Swarm's pattern: `wpstack_wordpress.1.xxxxxx` — the stack name + service name + replica number + task ID.

```

pranav@Pranav: ~/swarm-lab
Creating service wpstack_wordpress
pranav@Pranav:~/swarm-lab$ # Watch until both show 1/1 replicas
watch docker service ls
pranav@Pranav:~/swarm-lab$ docker service ls
ID                NAME                MODE                REPLICAS  IMAGE                PORTS
z4ieajugb2w       wpstack_db          replicated          1/1        mysql:5.7
q5k706zlxsmc      wpstack_wordpress   replicated          1/1        wordpress:latest     *:8080->80/tcp
pranav@Pranav:~/swarm-lab$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                MODE                DESIRED STATE  CURRENT STATE          ERROR          PORTS
3pciy4kv4klw      wpstack_wordpress.1 wordpress:latest      Pranav              Running         Running 34 seconds ago
ID                NAME                IMAGE                MODE                DESIRED STATE  CURRENT STATE          ERROR          PORTS
nt2qqqhrzcg9      wpstack_db.1        mysql:5.7            Pranav              Running         Running about a minute ago
pranav@Pranav:~/swarm-lab$ docker ps
CONTAINER ID        IMAGE                COMMAND
1a7e0684a156        wordpress:latest     "docker-entrypoint.s..."
fee78f12b1fd        mysql:5.7            "docker-entrypoint.s..."
7dbb2f1f9b47        jenkins/jenkins:lts "/usr/bin/tini -- /u..."
a97f974ff7ae        myjenkins-blueocean:2.541.3-1 "/usr/bin/tini -- /u..."
pranav@Pranav:~/swarm-lab$ docker stop 7dbb2f1f9b47
7dbb2f1f9b47
pranav@Pranav:~/swarm-lab$ docker stop a97f974ff7ae
a97f974ff7ae
pranav@Pranav:~/swarm-lab$ docker service ps wpstack_db
ID                NAME                IMAGE                MODE                DESIRED STATE  CURRENT STATE          ERROR          PORTS
3pciy4kv4klw      wpstack_wordpress.1 wordpress:latest      Pranav              Running         Running 4 minutes ago
ID                NAME                IMAGE                MODE                DESIRED STATE  CURRENT STATE          ERROR          PORTS
nt2qqqhrzcg9      wpstack_db.1        mysql:5.7            Pranav              Running         Running 3 minutes ago
pranav@Pranav:~/swarm-lab$ docker logs $(docker ps -q --filter name=wpstack_db) --tail 20
2026-04-25T01:49:56.678489Z 0 [Note] InnoDB: 96 redo rollback segment(s) found. 96 redo rollback segment(s) are active.
2026-04-25T01:49:56.678516Z 0 [Note] InnoDB: 32 non-redo rollback segment(s) are active.
2026-04-25T01:49:56.679881Z 0 [Note] InnoDB: 5.7.44 started; log sequence number 12219419
2026-04-25T01:49:56.679223Z 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
2026-04-25T01:49:56.679409Z 0 [Note] Plugin 'FEDERATED' is disabled.
2026-04-25T01:49:56.682695Z 0 [Note] InnoDB: Buffer pool(s) load completed at 260425 1:49:56
2026-04-25T01:49:56.686026Z 0 [Note] Found ca.pem, server-cert.pem and server-key.pem in data directory. Trying to enable SSL support using them.
2026-04-25T01:49:56.686058Z 0 [Note] Skipping generation of SSL certificates as certificate files are present in data directory.
2026-04-25T01:49:56.686061Z 0 [Warning] A deprecated TLS version TLSv1 is enabled. Please use TLSv1.2 or higher.
2026-04-25T01:49:56.686062Z 0 [Warning] A deprecated TLS version TLSv1.1 is enabled. Please use TLSv1.2 or higher.
2026-04-25T01:49:56.686436Z 0 [Warning] CA certificate ca.pem is self signed.
2026-04-25T01:49:56.686476Z 0 [Note] Skipping generation of RSA key pair as key files are present in data directory.

```

Phase 5: Access WordPress

On WSL2, verify the port is listening:

```
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080
# Expected: 302 (WordPress redirect to setup page)
```

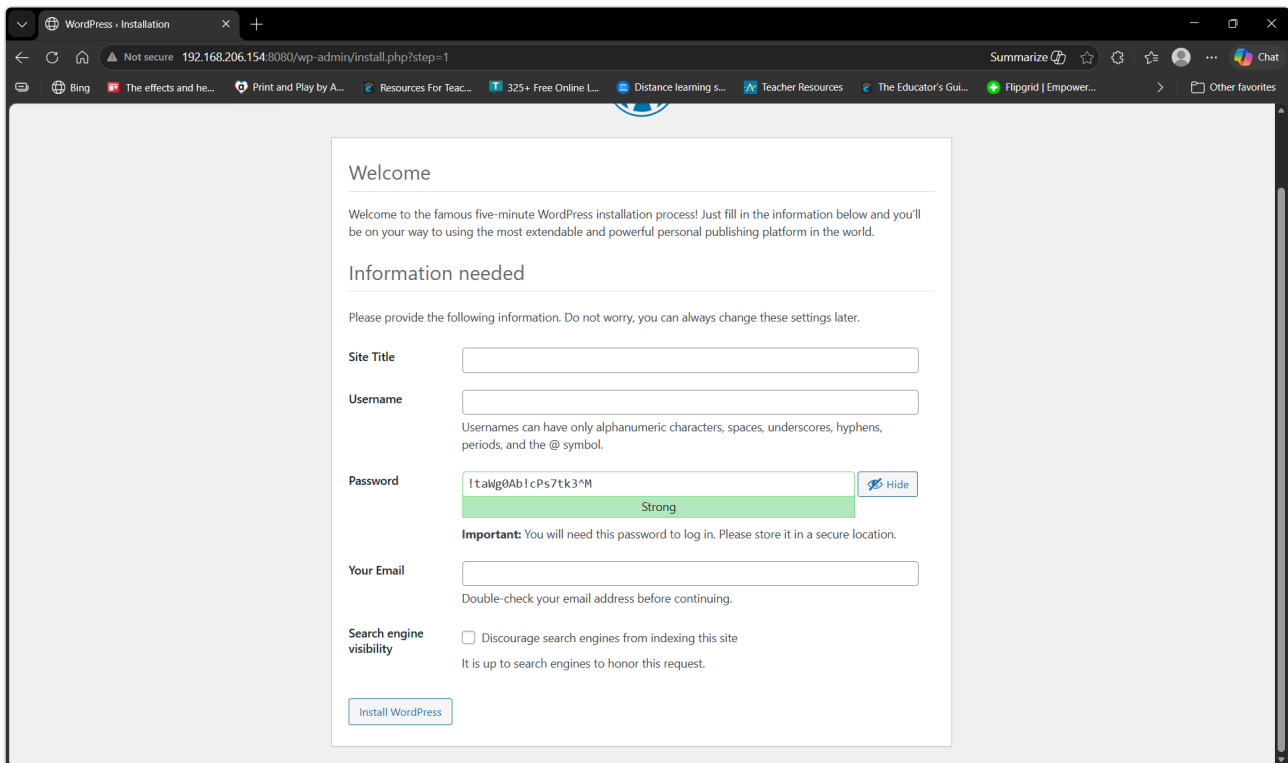
Get the WSL2 IP:

```
hostname -I | awk '{print $1}'
# Example: 192.168.206.154
```

Open in your **Windows browser**:

```
http://192.168.206.154:8080
```

Why not localhost ? WSL2 runs inside a Hyper-V virtual machine. Windows and WSL2 have **different network stacks** — **localhost** in the Windows browser looks for services on Windows itself, not inside WSL2. You must use the WSL2 IP address.



Phase 6: Scale the Application — Swarm's Superpower

This is where Swarm fundamentally differs from Compose.

```
docker service scale wpstack_wordpress=3
```

Expected output:

```
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
```

Verify:

```
docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
z4ieajiugb2w	wpstack_db	replicated	1/1	mysql:5.7	
q5k706zlkxsmc	wpstack_wordpress	replicated	3/3	wordpress:latest	*:8080→80/tcp

```
docker service ps wpstack_wordpress
```

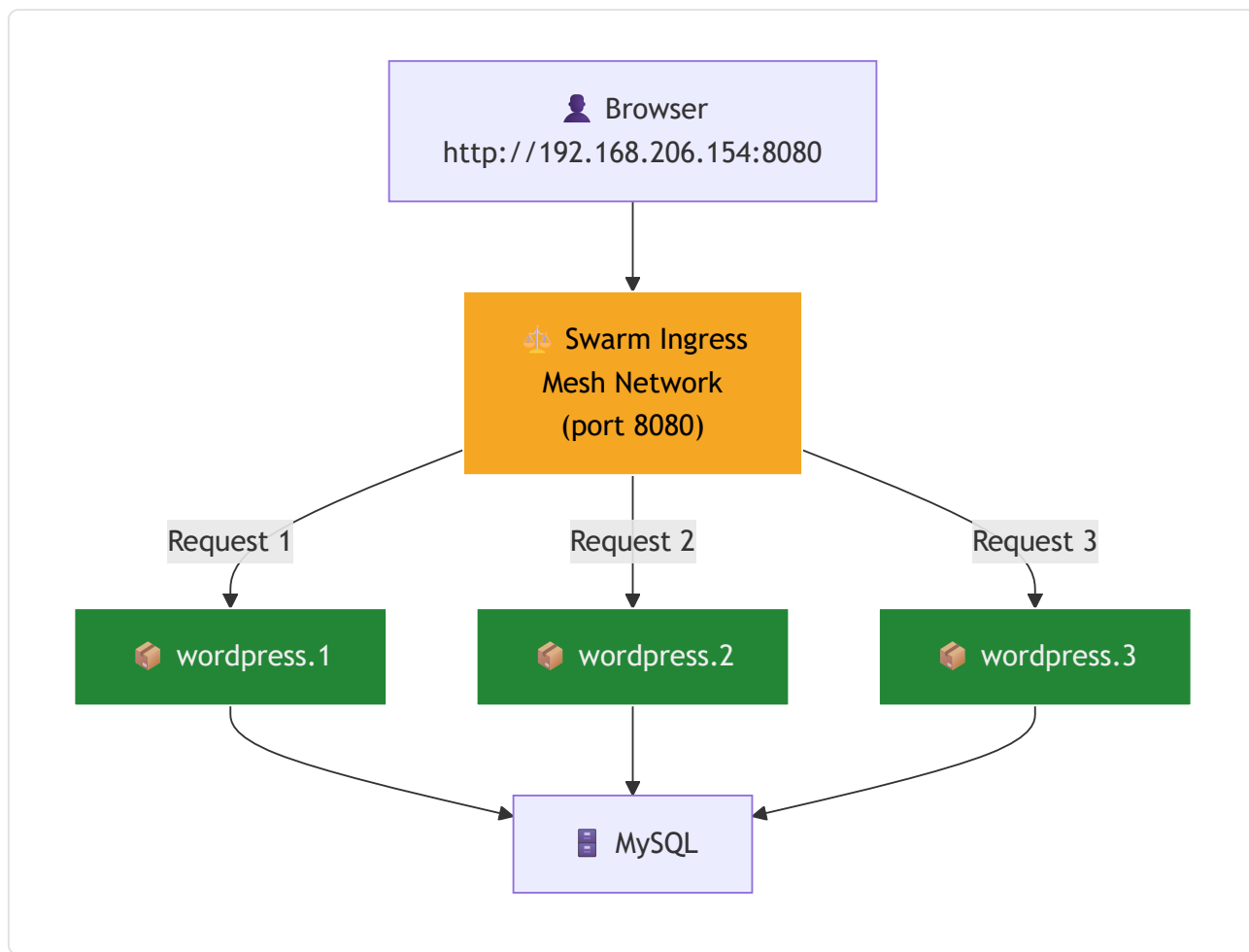
ID	NAME	IMAGE	NODE	STATE	CURRENT STATE	STATE
3pciy4kv4k1w	wpstack_wordpress.1	wordpress:latest	Pranav	Running	Running 6 min ago	
y47niyf3bokh	wpstack_wordpress.2	wordpress:latest	Pranav	Running	Running 25 seconds ago	
b4t0ytboja3a	wpstack_wordpress.3	wordpress:latest	Pranav	Running	Running 25 seconds ago	

```
docker ps | grep wordpress
```

3 WordPress containers running simultaneously!

```
pranav@Pranav: ~/swarm-lab
3pciy4kv4k1w  wpstack_wordpress.1  wordpress:latest  Pranav  Running  Running 3 minutes ago
2026-04-25T01:49:56.678489Z 0 [Note] InnoDB: 96 redo rollback segment(s) found. 96 redo rollback segment(s) are active.
2026-04-25T01:49:56.678516Z 0 [Note] InnoDB: 32 non-redo rollback segment(s) are active.
2026-04-25T01:49:56.679081Z 0 [Note] InnoDB: 5.7.44 started; log sequence number 12219419
2026-04-25T01:49:56.679223Z 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
2026-04-25T01:49:56.679409Z 0 [Note] Plugin 'FEDERATED' is disabled.
2026-04-25T01:49:56.682695Z 0 [Note] InnoDB: Buffer pool(s) load completed at 260425 1:49:56
2026-04-25T01:49:56.686026Z 0 [Note] Found ca.pem, server-cert.pem and server-key.pem in data directory. Trying to enable SSL support using them.
2026-04-25T01:49:56.686058Z 0 [Warning] Skipping generation of SSL certificates as certificate files are present in data directory.
2026-04-25T01:49:56.686061Z 0 [Warning] A deprecated TLS version TLSv1 is enabled. Please use TLSv1.2 or higher.
2026-04-25T01:49:56.686062Z 0 [Warning] A deprecated TLS version TLSv1.1 is enabled. Please use TLSv1.2 or higher.
2026-04-25T01:49:56.686436Z 0 [Warning] CA certificate ca.pem is self signed.
2026-04-25T01:49:56.686476Z 0 [Note] Skipping generation of RSA key pair as key files are present in data directory.
2026-04-25T01:49:56.686647Z 0 [Note] Server hostname (bind-address): '*'; port: 3306
2026-04-25T01:49:56.686689Z 0 [Note] IPv6 is available.
2026-04-25T01:49:56.686696Z 0 [Note] - '::' resolves to '::';
2026-04-25T01:49:56.686706Z 0 [Note] Server socket created on IP: '::'.
2026-04-25T01:49:56.692194Z 0 [Warning] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2026-04-25T01:49:56.697204Z 0 [Note] Event Scheduler: Loaded 0 events
2026-04-25T01:49:56.697520Z 0 [Note] mysqld: ready for connections.
Version: '5.7.44' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server (GPL)
pranav@Pranav:~/swarm-lab$ # 1. Get the actual WSL2 IP
hostname -I | awk '{print $1}'
192.168.206.154
pranav@Pranav:~/swarm-lab$ # 2. Test if the port is actually listening
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080
302pranav@Pranav:~/swarm-lab$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====]
2/3: running [=====]
3/3: running [=====]
verify: Service wpstack_wordpress converged
pranav@Pranav:~/swarm-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
z4ieajugb2w wpstack_db replicated 1/1 mysql:5.7
q5k706zlxsmc wpstack_wordpress replicated 3/3 wordpress:latest *:8080->80/tcp
pranav@Pranav:~/swarm-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
3pciy4kv4k1w wpstack_wordpress.1 wordpress:latest Pranav Running Running 6 minutes ago
y47niyf3bokh wpstack_wordpress.2 wordpress:latest Pranav Running Running 25 seconds ago
b4t0ytboja3a wpstack_wordpress.3 wordpress:latest Pranav Running Running 25 seconds ago
pranav@Pranav:~/swarm-lab$ docker ps | grep wordpress
41152f7ec973 wordpress:latest "docker-entrypoint.s..." 38 seconds ago Up 37 seconds 80/tcp wpstack_wordpress.3.b4t0ytboja3at8c1i6hrb3
sst
115dca638196 wordpress:latest "docker-entrypoint.s..." 38 seconds ago Up 37 seconds 80/tcp wpstack_wordpress.2.y47niyf3bokhb6t3qmk7q
```

Deep Dive: How Do 3 Containers Share Port 8080?



In plain Compose, scaling to 3 would **fail** because all containers try to bind to host port `8080`. In Swarm, the **ingress mesh network** acts as a built-in load balancer:

- It listens on port `8080` **once** on the host
- Routes incoming requests to all healthy replicas using round-robin
- No port conflicts — Swarm manages the internal routing transparently

Phase 7: Test Self-Healing — Automatic Recovery

Self-healing is Swarm's ability to automatically replace failed containers.

Step 1: List WordPress containers and pick one to kill:

```
docker ps | grep wordpress
```

Step 2: Simulate a crash:

```
docker kill <container-id>
```

Step 3: Watch Swarm recreate it instantly:

```
docker service ps wpstack_wordpress
```

Output showing self-healing in action:

ID	NAME	IMAGE	NODE	STATE	CURRENT
3pciy4kv4k1w	wpstack_wordpress.1	wordpress:latest	Pranav	Running	Running
y47niyf3bokh	wpstack_wordpress.2	wordpress:latest	Pranav	Running	Running
r1pnkpnf4h8k	wpstack_wordpress.3	wordpress:latest	Pranav	Running	Running
b4t0ytboja3a	_ wpstack_wordpress.3	wordpress:latest	Pranav	Shutdown	Failed 8

Column

What It Shows

_ wpstack_wordpress.3

The **killed** container — prefixed with `\_` to show it's history

Failed 8 seconds ago

When the failure was detected

exit (137)

Exit code 137 = 128 + 9 = **SIGKILL** — confirms our `docker kill`

wpstack_wordpress.3 (new row)

Swarm spawned a **replacement** within seconds

Verify 3 containers are still running:

```
docker ps | grep wordpress
# Still 3 healthy containers – Swarm fixed it automatically!
```

```
pranav@Pranav: ~/swarm-lab
y47niyf3bokh wpstack_wordpress.2 wordpress:latest Pranav Running Running 25 seconds ago
b4t0ytboja3a wpstack_wordpress.3 wordpress:latest Pranav Running Running 25 seconds ago
pranav@Pranav:~/swarm-lab$ docker ps | grep wordpress
41152f7ec973 wordpress:latest "docker-entrypoint.s..." 38 seconds ago Up 37 seconds 80/tcp wpstack_wordpress.3.b4t0ytboja3at8c1i6hrb3
sst
115dca638196 wordpress:latest "docker-entrypoint.s..." 38 seconds ago Up 37 seconds 80/tcp wpstack_wordpress.2.y47niyf3bokhbwt3qmk7q
gas
1a7e0684a156 wordpress:latest "docker-entrypoint.s..." 6 minutes ago Up 6 minutes 80/tcp wpstack_wordpress.1.3pciy4kv4k1wf8igdb19u0
3yt
pranav@Pranav:~/swarm-lab$ docker ps | grep wordpress
41152f7ec973 "docker-entrypoint.s..." About a minute ago Up About a minute 80/tcp wpstack_wordpress.3.b4t0ytboja3at8
c1i6hrb3sst
115dca638196 wordpress:latest "docker-entrypoint.s..." About a minute ago Up About a minute 80/tcp wpstack_wordpress.2.y47niyf3bokhbwt3qmk7q
6t3qmk7qgas
1a7e0684a156 wordpress:latest "docker-entrypoint.s..." 7 minutes ago Up 7 minutes 80/tcp wpstack_wordpress.1.3pciy4kv4k1wf8
igdb19u03yt
pranav@Pranav:~/swarm-lab$ docker kill 41152f7ec973
41152f7ec973
pranav@Pranav:~/swarm-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE DESIRED STATE CURRENT STATE ERROR PORTS
3pciy4kv4k1w wpstack_wordpress.1 wordpress:latest Pranav Running Running 7 minutes ago
y47niyf3bokh wpstack_wordpress.2 wordpress:latest Pranav Running Running about a minute ago
r1pnkpnf4h8k wpstack_wordpress.3 wordpress:latest Pranav Running Running 2 seconds ago
b4t0ytboja3a \_ wpstack_wordpress.3 wordpress:latest Pranav Shutdown Failed 8 seconds ago "task: non-zero exit (137)"
pranav@Pranav:~/swarm-lab$ docker ps | grep wordpress
5ed7e78cbfff wordpress:latest "docker-entrypoint.s..." About a minute ago Up 58 seconds 80/tcp wpstack_wordpress.3.r1pnkpnf4h8k0h9e6b
mg9va5n
115dca638196 wordpress:latest "docker-entrypoint.s..." 2 minutes ago Up 2 minutes 80/tcp wpstack_wordpress.2.y47niyf3bokhbwt3qmk7q
mk7qgas
1a7e0684a156 wordpress:latest "docker-entrypoint.s..." 8 minutes ago Up 8 minutes 80/tcp wpstack_wordpress.1.3pciy4kv4k1wf8igdb
19u03yt
pranav@Pranav:~/swarm-lab$ docker service scale wpstack_wordpress=1
wpstack_wordpress scaled to 1
overall progress: 1 out of 1 tasks
1/1: running [=====]
verify: Service wpstack_wordpress converged
pranav@Pranav:~/swarm-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
z1ieajugb2w wpstack_db replicated 1/1 mysql:5.7
q5k706zlxsmc wpstack_wordpress replicated 1/1 wordpress:latest *:8080->80/tcp
pranav@Pranav:~/swarm-lab$ # Remove the entire stack
docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
pranav@Pranav:~/swarm-lab$ docker service ls # should be empty
docker ps # no wordpress/mysql containers
```

Phase 8: Scale Down and Cleanup

Scale back to 1 replica:

```
docker service scale wpstack_wordpress=1
```

Remove the entire stack:

```
docker stack rm wpstack
```

Expected output:

```
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Verify everything is gone:

```
docker service ls    # should be empty
docker ps             # no wordpress/mysql containers
```

Clean up volumes and leave Swarm:

```
docker volume prune -f
docker swarm leave --force
```

```

pranav@Pranav: ~/swarm-lab
41152f7ec973
pranav@Pranav:~/swarm-lab$ docker service ps wpstack_wordpress
ID            NAME                IMAGE                NODE    DESIRED STATE    CURRENT STATE                ERROR                PORTS
3pciyl4kv4i1w wpstack_wordpress.1  wordpress:latest    Pranav  Running          Running 7 minutes ago
y47n1yf3bokh  wpstack_wordpress.2  wordpress:latest    Pranav  Running          Running about a minute ago
r1pnkpnf4h8k  wpstack_wordpress.3  wordpress:latest    Pranav  Running          Running 2 seconds ago
bitt3y7boja3a \_ wpstack_wordpress.3  wordpress:latest    Pranav  Shutdown         Failed 8 seconds ago
pranav@Pranav:~/swarm-lab$ docker ps | grep wordpress
5ed7e78cbfff  wordpress:latest    "docker-entrypoint.s..." About a minute ago    Up 58 seconds        80/tcp                wpstack_wordpress.3.r1pnkpnf4h8k0h9e6b
mg9va5n      wordpress:latest    "docker-entrypoint.s..." 2 minutes ago        Up 2 minutes         80/tcp                wpstack_wordpress.2.y47n1yf3bokhbw6t3q
115dca638196 wordpress:latest    "docker-entrypoint.s..." 8 minutes ago        Up 8 minutes         80/tcp                wpstack_wordpress.1.3pciyl4kv4i1wf8igdb
mk7ggas
1a7e0684a156
19u03yt
pranav@Pranav:~/swarm-lab$ docker service scale wpstack_wordpress=1
wpstack_wordpress scaled to 1
overall progress: 1 out of 1 tasks
1/1: running [=====]
verify: Service wpstack_wordpress converged
pranav@Pranav:~/swarm-lab$ docker service ls
ID            NAME                MODE                REPLICAS    IMAGE                PORTS
z4ieajiuqb2w  wpstack_db          replicated          1/1          mysql:5.7
q5k706zkxsmc  wpstack_wordpress   replicated          1/1          wordpress:latest    *:8080->80/tcp
pranav@Pranav:~/swarm-lab$ # Remove the entire stack
docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
pranav@Pranav:~/swarm-lab$ docker service ls # should be empty
ID            NAME                MODE                REPLICAS    IMAGE                PORTS
CONTAINER ID  IMAGE                COMMAND             CREATED      STATUS             PORTS             NAMES
pranav@Pranav:~/swarm-lab$ docker volume prune -f
Deleted Volumes:
200b0eb69313f24c7a81320c7bb924f7d8e474c25a84212aba7fcf20849e5ad2

Total reclaimed space: 262.2kB
pranav@Pranav:~/swarm-lab$ docker swarm leave --force
Node left the swarm.
pranav@Pranav:~/swarm-lab$

```

Deep Dive: Compose File Reusability

The same YAML file works for both development and production:

Command	Mode	Behavior
<code>docker compose up -d</code>	Compose mode	Creates containers — managed by you
<code>docker stack deploy -c file.yml name</code>	Swarm mode	Creates services — managed by Swarm

Key insight: Swarm **extends** Compose, it doesn't replace it. The Compose file is the blueprint; Swarm is the factory that runs it at scale.

Containers vs Services

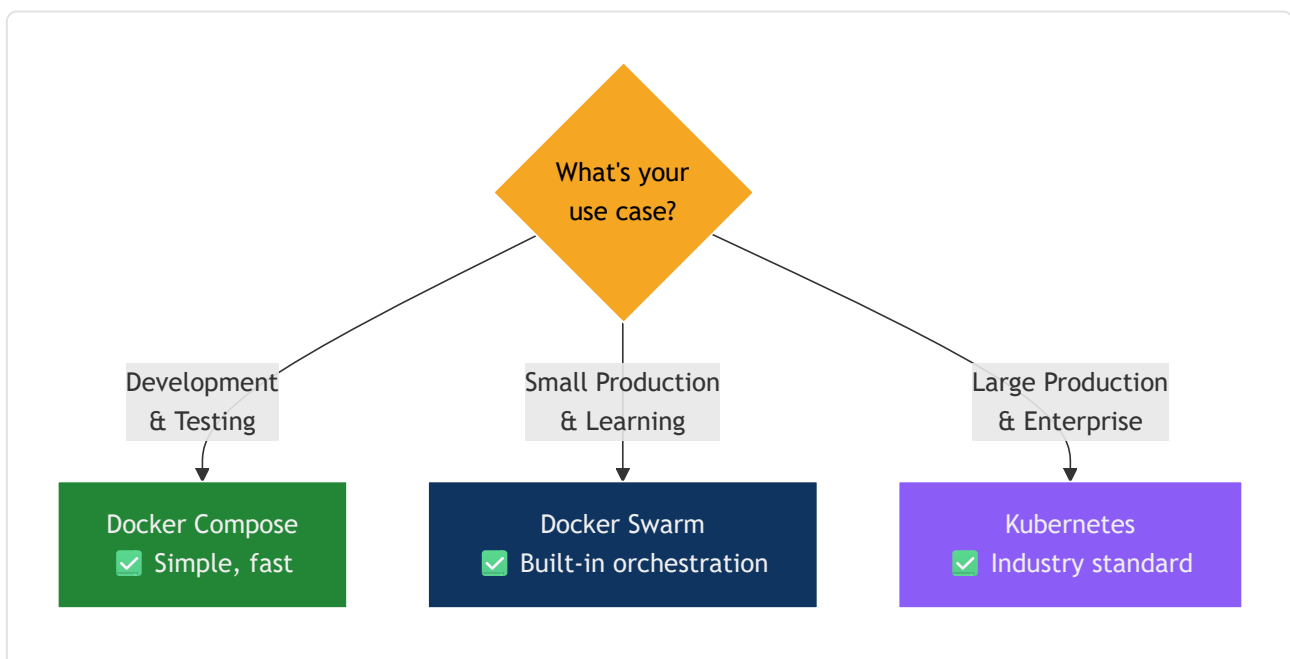
Concept	In Compose	In Swarm
Unit of management	Container	Service
You manage	Individual containers (<code>docker start/stop</code>)	Service definitions (<code>docker service scale</code>)

Concept	In Compose	In Swarm
Naming	You choose (<code>container_name</code>)	Swarm assigns (<code>stack_service.replica.taskid</code>)
Failure recovery	Manual restart	Automatic replacement
Scaling	Port conflicts	Load-balanced, conflict-free

Part C — Compose vs Swarm Analysis

Feature	Docker Compose	Docker Swarm
Scope	Single host only	Multi-node cluster
Scaling	<code>--scale</code> flag (basic, port conflicts)	<code>docker service scale</code> (built-in LB)
Load Balancing	None	Yes (ingress mesh network)
Self-Healing	None (must restart manually)	Yes (automatic container replacement)
Rolling Updates	None	Yes (zero-downtime deployments)
Service Discovery	Via container names	Via DNS + Virtual IPs
Use Case	Development, testing	Simple production clusters
Complexity	Low	Medium

When to Use What



Part D — Multi-Node Swarm (Optional Advanced)

If you have access to multiple VMs or machines:

```
# On manager node - get the join token
docker swarm join-token worker

# On worker node - join the cluster
docker swarm join --token <token> <manager-ip>:2377

# Verify from manager
docker node ls
```

Swarm will then distribute containers across all nodes automatically.

Lab Fixes Summary

Change	Original (Lab Sheet)	Fixed	Why
<code>version</code> key	<code>version: '3.9'</code> included	Removed	Obsolete in Compose v2, causes warnings
<code>container_name</code>	Both services had fixed names	Removed both	Swarm manages names; fixed names block scaling (can't have 3 replicas named <code>wordpress_app</code>)
<code>docker swarm init</code>	No <code>--advertise-addr</code> flag	Added <code>--advertise-addr <eth0-ip></code>	WSL2 has 2 interfaces; Swarm can't auto-pick
Browser access	<code>http://localhost:8080</code>	<code>http://192.168.206.154:8080</code>	WSL2 ≠ Windows localhost (Hyper-V NAT isolation)
Deployment method	<code>docker compose up</code>	<code>docker stack deploy</code>	The entire point — Swarm orchestration vs manual Compose

Exit Code 137 Explained

When you `docker kill` a container:

- Linux sends **SIGKILL** (signal 9) to the process

- Container exits with code **137** (128 + 9)
- Swarm detects the failure and immediately schedules a replacement
- The killed task shows as **Shutdown** / **Failed** in the task history
- A new task is created to maintain the desired replica count

Common Pitfalls & Troubleshooting

Problem	Cause	Fix
<code>could not choose an IP address to advertise</code>	WSL2 has multiple network interfaces	Add <code>--advertise-addr <eth0-ip></code> to <code>docker swarm init</code>
Browser shows "Can't reach this page"	WSL2 and Windows don't share localhost	Access via WSL2 IP (<code>hostname -I</code>) instead of <code>localhost</code>
<code>docker stack deploy</code> fails silently	<code>container_name</code> set in Compose file	Remove all <code>container_name</code> entries — Swarm manages names
Scaling creates containers but they crash	Port conflicts or resource exhaustion	Check <code>docker service ps <service></code> for error messages
<code>Ignoring unsupported options: restart</code>	Swarm has its own restart policy	This is a warning, not an error — safe to ignore
Volumes not cleaned up after <code>stack rm</code>	Swarm preserves volumes by default	Run <code>docker volume prune -f</code> manually

Learning Outcome Questions

1. **Why is Compose not enough for production?** No self-healing, no load balancing, single-host only — a crashed container stays dead until manually restarted.
2. **What does `docker stack deploy` do differently than `docker compose up`?** Stack deploy creates **services** managed by Swarm with automatic scaling, load balancing, and self-healing. Compose creates standalone containers with none of these features.
3. **How does Swarm achieve self-healing?** The manager continuously compares the **desired state** (e.g., 3 replicas) with the **actual state**. When a container dies, the manager schedules a new task to restore the desired count.
4. **What happens if you `docker kill` a Swarm-managed container?** Swarm detects the failure (exit 137), marks the old task as **Shutdown/Failed**, and spawns a new replacement container within seconds.

5. **Can the same Compose file be used for both Compose and Swarm?** Yes — `docker compose up` runs it in development mode, `docker stack deploy` runs it in orchestration mode. Same file, different execution engines.

Glossary

Term	Definition
Orchestration	Automated management of containers — scaling, placement, recovery, and load balancing
Docker Swarm	Docker's built-in orchestration tool for managing clusters of Docker hosts
Manager Node	The node that controls the Swarm cluster — schedules tasks and maintains state
Worker Node	A node that runs containers assigned by the manager
Service	A declarative definition of containers to run (image, replicas, ports, etc.)
Task	A single container instance running as part of a service
Stack	A group of related services deployed from a single Compose file
Replica	One copy of a container in a service — <code>replicas: 3</code> = 3 identical instances
Ingress Mesh	Swarm's built-in network that load-balances traffic across all replicas
Self-Healing	Automatic replacement of failed containers to maintain desired state
Scaling	Increasing or decreasing the number of container replicas
Load Balancing	Distributing incoming requests evenly across container replicas
Rolling Update	Updating containers one at a time with zero downtime
Desired State	The target configuration you declare (e.g., 3 replicas)
Actual State	The current running state — Swarm continuously reconciles this with desired state
VIP (Virtual IP)	A single IP assigned to a service — Swarm routes requests to individual replicas
Exit Code 137	128 + 9 (SIGKILL) — confirms container was forcefully killed
<code>--advertise-addr</code>	Tells Swarm which IP address to use for cluster communication

Quick Reference Card

```
# Initialize Swarm
docker swarm init --advertise-addr $(hostname -I | awk '{print $1}')

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack_service>=<replicas>

# See service tasks (including history)
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Clean up volumes
docker volume prune -f

# Leave Swarm
docker swarm leave --force
```

References

- [Docker Swarm Official Docs](#)
- [Docker Stack Deploy](#)
- [Docker Service Scale](#)

Objective

Deploy, scale, and self-heal a WordPress application on a local Kubernetes cluster using k3d to understand core Kubernetes concepts including Pods, Deployments, Services, and ReplicaSets.

Theory

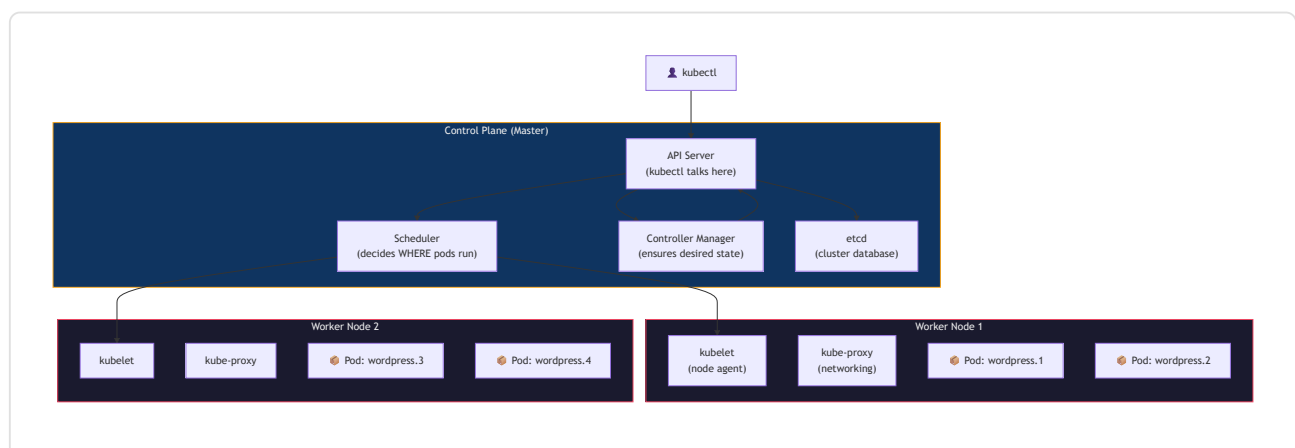
Why Kubernetes After Docker Swarm?

Experiment 11 used Docker Swarm for orchestration. Kubernetes solves the same problems — scaling, self-healing, load balancing — but at a much larger scale with far more features:

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic (<code>service scale</code>)	Advanced (auto-scaling based on CPU/memory)
Ecosystem	Small	Huge (monitoring, logging, service mesh, etc.)
Industry adoption	Rare	Industry standard (AWS EKS, GKE, AKS)
Scheduling	Simple round-robin	Advanced (affinity, taints, tolerations)
Configuration	Compose YAML	Declarative YAML manifests with full API

Verdict: Swarm is great for learning orchestration basics. Kubernetes is what companies actually use in production.

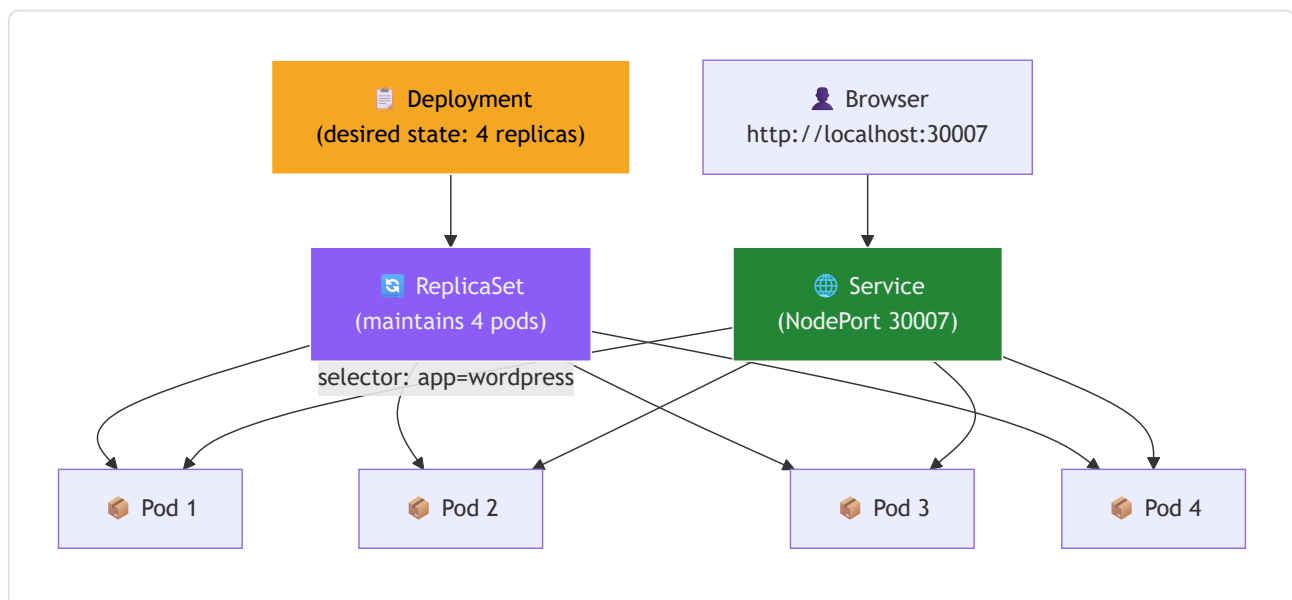
Kubernetes Architecture



Core Concepts — Docker to Kubernetes Translation

Docker Concept	Kubernetes Equivalent	What It Means
Container	Pod	Smallest deployable unit — wraps 1+ containers
Compose service	Deployment	Declares desired state: image, replicas, labels
Load balancing	Service	Stable endpoint that routes traffic to matching pods
Scaling count	ReplicaSet	Ensures N pod copies are always running
Compose file	YAML Manifest	Declarative configuration applied via <code>kubectl apply</code>

How Kubernetes Objects Relate



The chain: You create a **Deployment** → Kubernetes creates a **ReplicaSet** → ReplicaSet creates **Pods** → A **Service** exposes them.

Why k3d Instead of Minikube?

Tool	How It Works	WSL2 Compatibility
Minikube	Runs K8s inside a VM	❌ WSL2 is already a VM — nested VMs fail or need special config
k3d	Runs K8s inside Docker containers	✅ Docker already works on WSL2 — no VM needed
kubeadm	Real multi-VM cluster	⚠️ Requires 2-3 separate VMs with 2+ CPU, 2+ GB RAM each

Prerequisites

- WSL2 with Ubuntu
 - Docker Desktop running
 - Terminal access
-

Hands-on Lab

Phase 1: Install kubectl and k3d

Install kubectl (the Kubernetes CLI):

```
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux-amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
```

Step	Purpose
<code>curl -LO</code>	Downloads the latest stable kubectl binary
<code>chmod +x</code>	Makes it executable
<code>sudo mv</code>	Places it in system PATH

Install k3d (lightweight Kubernetes in Docker):

```
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
k3d version
```

```

pranav@Pranav: ~/k8s-lab
pranav@Pranav:~$ curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
pranav@Pranav:~$ kubectl version --client
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 56.7M  100 56.7M    0     0  632k      0  0:01:31  0:01:31 --:--:-- 609k
[sudo] password for pranav:
Client Version: v1.36.0
Kustomize Version: v5.8.1
pranav@Pranav:~$ curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
k3d version
Preparing to install k3d into /usr/local/bin
k3d installed into /usr/local/bin/k3d
Run 'k3d --help' to see what you can do with it.
k3d version v5.8.3
k3s version v1.31.5-k3s1 (default)
pranav@Pranav:~$ k3d cluster create mylab \
  --port "30007:30007@server:0" \
  --port "8888:80@loadbalancer" \
  --agents 1
INFO[0000] portmapping '8888:80' targets the loadbalancer: defaulting to [servers:*.proxy agents:*.proxy]
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-mylab'
INFO[0000] Created image volume k3d-mylab-images
INFO[0000] Starting new tools node...
INFO[0001] Creating node 'k3d-mylab-server-0'
INFO[0010] Pulling image 'ghcr.io/k3d-io/k3d-tools:5.8.3'
INFO[0011] Pulling image 'docker.io/rancher/k3s:v1.31.5-k3s1'
INFO[0021] Starting node 'k3d-mylab-tools'
INFO[0022] Creating node 'k3d-mylab-agent-0'
INFO[0023] Creating LoadBalancer 'k3d-mylab-serverlb'
INFO[0026] Pulling image 'ghcr.io/k3d-io/k3d-proxy:5.8.3'
INFO[0061] Using the k3d-tools node to gather environment information
INFO[0062] HostIP: using network gateway 172.21.0.1 address
INFO[0062] Starting cluster 'mylab'
INFO[0062] Starting servers...
INFO[0062] Starting node 'k3d-mylab-server-0'
INFO[0066] Starting agents...
INFO[0066] Starting node 'k3d-mylab-agent-0'
INFO[0069] Starting helpers...
INFO[0070] Starting node 'k3d-mylab-serverlb'
INFO[0076] Injecting records for hostAliases (incl. host.k3d.internal) and for 3 network members into CoreDNS configmap...
INFO[0078] Cluster 'mylab' created successfully!
INFO[0078] You can now use it like this:
kubectl cluster-info
pranav@Pranav:~$ kubectl get nodes

```

Phase 2: Create the Kubernetes Cluster

```

k3d cluster create mylab \
  --port "30007:30007@server:0" \
  --port "8888:80@loadbalancer" \
  --agents 1

```

Flag	Purpose
<code>mylab</code>	Cluster name
<code>--port "30007:30007@server:0"</code>	Maps NodePort 30007 from k3d server container to WSL2 host
<code>--port "8888:80@loadbalancer"</code>	Maps LB port 80 to host port 8888
<code>--agents 1</code>	Creates 1 worker node + 1 control plane node

Critical: Port mappings **must** be set at cluster creation time. They cannot be added later without deleting and recreating the cluster.

Verify the cluster:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
k3d-mylab-agent-0	Ready	<none>	102s	v1.31.5+k3s1
k3d-mylab-server-0	Ready	control-plane,master	105s	v1.31.5+k3s1

```
pranav@Pranav: ~/k8s-lab
pranav@Pranav:~$ kubectl get nodes
NAME                STATUS    ROLES                  AGE    VERSION
k3d-mylab-agent-0    Ready     <none>                 102s   v1.31.5+k3s1
k3d-mylab-server-0  Ready     control-plane,master   105s   v1.31.5+k3s1
pranav@Pranav:~$ mkdir -p ~/k8s-lab
cd ~/k8s-lab
pranav@Pranav:~/k8s-lab$ cat > ~/k8s-lab/wordpress-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
        env:
        - name: WORDPRESS_DB_HOST
          value: "localhost"
        - name: WORDPRESS_DB_USER
          value: "root"
        - name: WORDPRESS_DB_PASSWORD
          value: "rootpass"
        - name: WORDPRESS_DB_NAME
          value: "wordpress"
EOF
pranav@Pranav:~/k8s-lab$ cat > ~/k8s-lab/wordpress-service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
  - port: 80
```

Phase 3: Create the WordPress Deployment

```
mkdir -p ~/k8s-lab && cd ~/k8s-lab
```

Create `wordpress-deployment.yaml`:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
        env:
        - name: WORDPRESS_DB_HOST
          value: "mysql-service:3306"
        - name: WORDPRESS_DB_USER
          value: "wpuser"
        - name: WORDPRESS_DB_PASSWORD
          value: "wppass"
        - name: WORDPRESS_DB_NAME
          value: "wordpress"

```

YAML Manifest Breakdown

Field	Purpose
<code>apiVersion: apps/v1</code>	Which Kubernetes API group to use for Deployments
<code>kind: Deployment</code>	Resource type — tells K8s to create a Deployment object
<code>metadata.name</code>	Unique name for this deployment
<code>spec.replicas: 2</code>	Run 2 identical pods
<code>selector.matchLabels</code>	How the Deployment finds its pods — must match <code>template.labels</code>
<code>template</code>	Blueprint for each pod
<code>containers[].image</code>	Docker image to run
<code>containerPort: 80</code>	Port the container listens on
<code>env</code>	Environment variables passed to the container

Why env vars? WordPress requires `WORDPRESS_DB_HOST` etc. to connect to MySQL. Without them, WordPress shows a database connection error.

Phase 4: Create the MySQL Deployment and Services

Create `mysql-deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "rootpass"
            - name: MYSQL_DATABASE
              value: "wordpress"
            - name: MYSQL_USER
              value: "wpuser"
            - name: MYSQL_PASSWORD
              value: "wppass"
          ports:
            - containerPort: 3306
```

Create `mysql-service.yaml` (ClusterIP — internal only):

```

apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306

```

Create `wordpress-service.yaml` (NodePort — external access):

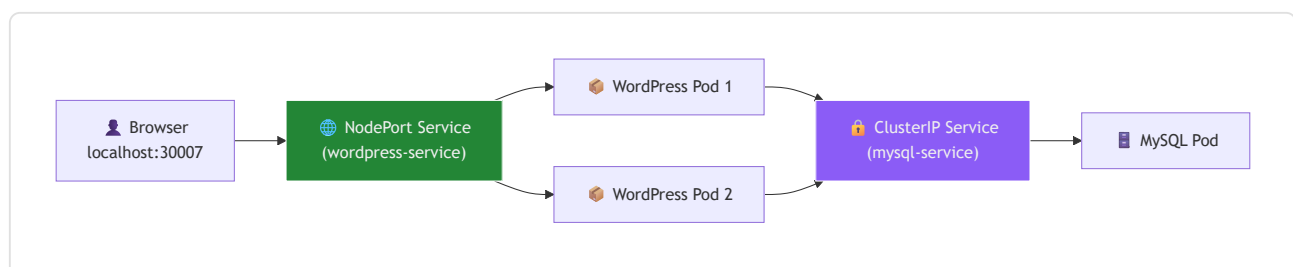
```

apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007

```

Service Types Explained

Type	Visibility	Use Case
ClusterIP (default)	Internal only — pods can reach it, browsers cannot	Databases, internal APIs
NodePort	External — exposed on every node at port 30000–32767	Development, lab access
LoadBalancer	External — provisions a cloud load balancer	Production on AWS/GCP/Azure



```

pranav@Pranav: ~/k8s-lab
pranav@Pranav:~/k8s-lab$ cat > ~/k8s-lab/wordpress-service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
EOF
pranav@Pranav:~/k8s-lab$ cd ~/k8s-lab

# Apply deployment
kubectl apply -f wordpress-deployment.yaml

# Apply service
kubectl apply -f wordpress-service.yaml
deployment.apps/wordpress created
service/wordpress-service created
pranav@Pranav:~/k8s-lab$ watch kubectl get pods
pranav@Pranav:~/k8s-lab$ # Check pods
kubectl get pods

# Check service
kubectl get svc

# Check deployment
kubectl get deployment wordpress
NAME READY STATUS RESTARTS AGE
wordpress-657549f7f5-frtj7 1/1 Running 0 109s
wordpress-657549f7f5-t6tkd 1/1 Running 0 109s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.43.0.1 <none> 443/TCP 4m23s
wordpress-service NodePort 10.43.143.178 <none> 80:30007/TCP 110s
NAME READY UP-TO-DATE AVAILABLE AGE
wordpress 2/2 2 2 110s
pranav@Pranav:~/k8s-lab$ # Test from inside WSL2
curl -s -o /dev/null -w "%{http_code}" http://localhost:30007
500pranav@Pranav:~/k8s-lab$ cat > ~/k8s-lab/mysql-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql

```

Phase 5: Deploy Everything and Verify

```

cd ~/k8s-lab

# Apply all manifests
kubectl apply -f mysql-deployment.yaml
kubectl apply -f mysql-service.yaml
kubectl apply -f wordpress-deployment.yaml
kubectl apply -f wordpress-service.yaml

# Watch pods come up in real-time
watch kubectl get pods

```

Why watch? Pods take 30–60 seconds to pull images. Without live monitoring, students think something is broken and kill the terminal.

Verify everything:

```

kubectl get pods
kubectl get svc
kubectl get deployment wordpress

```

Expected output:

```

NAME                                READY   STATUS    RESTARTS   AGE
mysql-7bdb4b6688-bg7dt             1/1     Running   0           3m20s
wordpress-7c587db787-d8w8n        1/1     Running   0           3m18s
wordpress-7c587db787-nswjl        1/1     Running   0           3m18s

NAME                                TYPE          CLUSTER-IP    PORT(S)      AGE
kubernetes                         ClusterIP     10.43.0.1     443/TCP      4m23s
mysql-service                      ClusterIP     10.43.78.132  3306/TCP     4m21s
wordpress-service                 NodePort      10.43.143.178 80:30007/TCP 110s

NAME      READY   UP-TO-DATE   AVAILABLE   AGE
wordpress 2/2      2             2           110s

```

Test access:

```

curl -s -o /dev/null -w "%{http_code}" http://localhost:30007
# Expected: 302 (WordPress redirect to setup page)

```

Open in your Windows browser: <http://localhost:30007>

Note: Unlike the Swarm experiment (EXP-11), k3d's `--port` flag handles the WSL2→Windows port forwarding automatically — no `netsh` needed.

```

pranav@Pranav: ~/k8s-lab
pranav@Pranav:~/k8s-lab$ # Test from inside WSL2
curl -s -o /dev/null -w "%{http_code}" http://localhost:30007
500pranav@Pranav:~/k8s-lab$ cat > ~/k8s-lab/mysql-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "rootpass"
            - name: MYSQL_DATABASE
              value: "wordpress"
            - name: MYSQL_USER
              value: "wpuser"
            - name: MYSQL_PASSWORD
              value: "wppass"
          ports:
            - containerPort: 3306
EOF
pranav@Pranav:~/k8s-lab$ kubectl delete -f ~/k8s-lab/wordpress-deployment.yaml
cat > ~/k8s-lab/wordpress-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:

```

```

name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
        env:
        - name: WORDPRESS_DB_HOST
          value: "mysql-service:3306"
        - name: WORDPRESS_DB_USER
          value: "wpuser"
        - name: WORDPRESS_DB_PASSWORD
          value: "wppass"
        - name: WORDPRESS_DB_NAME
          value: "wordpress"
EOF
deployment.apps "wordpress" deleted from default namespace
pranav@Pranav: ~/k8s-lab$ kubectl apply -f ~/k8s-lab/mysql-deployment.yaml
kubectl apply -f ~/k8s-lab/mysql-service.yaml
kubectl apply -f ~/k8s-lab/wordpress-deployment.yaml
deployment.apps/mysql created
service/mysql-service created
deployment.apps/wordpress created
pranav@Pranav:~/k8s-lab$ watch kubectl get pods
pranav@Pranav:~/k8s-lab$ curl -s -o /dev/null -w "%{http_code}" http://localhost:30007
302pranav@Pranav:~/k8s-lab$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
pranav@Pranav:~/k8s-lab$ watch kubectl get pods
pranav@Pranav:~/k8s-lab$ kubectl get pods
kubectl get deployment wordpress
NAME                                READY   STATUS    RESTARTS   AGE
mysql-7bdb4b6688-bg7dt             1/1     Running   0           3m20s
wordpress-7c587db787-d2hzj         1/1     Running   0           65s
wordpress-7c587db787-d8w8n         1/1     Running   0           3m18s
wordpress-7c587db787-nswjl         1/1     Running   0           3m18s
wordpress-7c587db787-vpvzw         1/1     Running   0           65s
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
wordpress                           4/4     4             4           3m19s

```

Phase 6: Scale the Application

```
kubectl scale deployment wordpress --replicas=4
```

Argument

Purpose

```
deployment wordpress
```

Target the wordpress Deployment

```
--replicas=4
```

Set desired pod count to 4

Verify:

```

watch kubectl get pods
kubectl get deployment wordpress

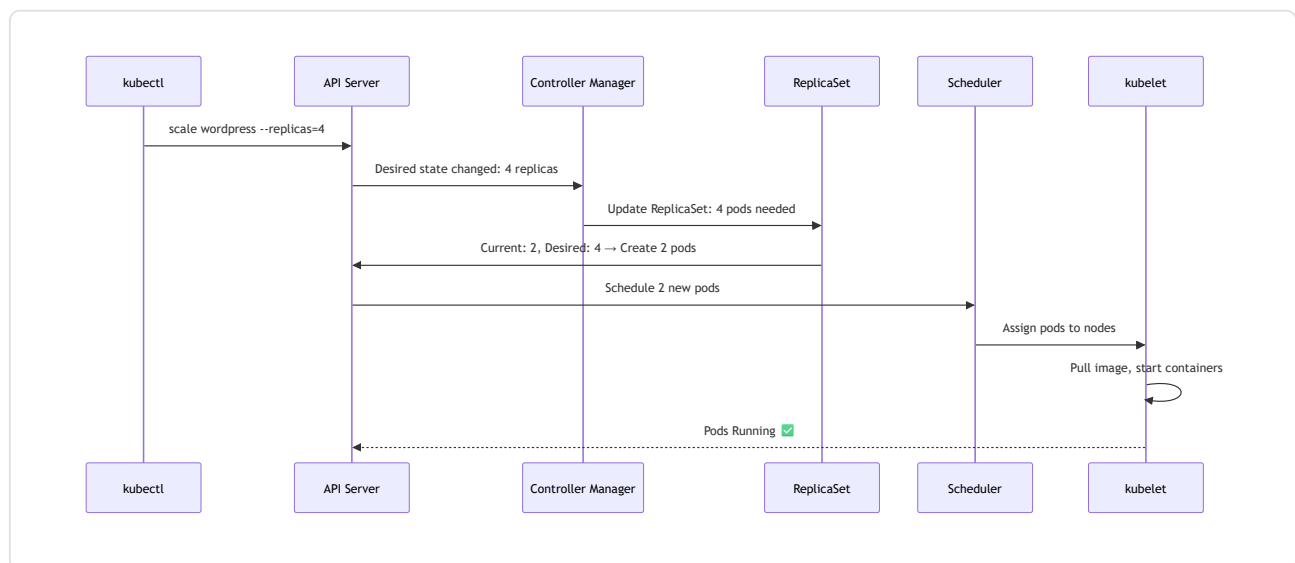
```

Expected:

NAME	READY	STATUS	RESTARTS	AGE	
mysql-7bdb4b6688-bg7dt	1/1	Running	0	3m20s	
wordpress-7c587db787-d2hzj	1/1	Running	0	65s	← new
wordpress-7c587db787-d8w8n	1/1	Running	0	3m18s	
wordpress-7c587db787-nswjl	1/1	Running	0	3m18s	
wordpress-7c587db787-vpvzw	1/1	Running	0	65s	← new

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
wordpress	4/4	4	4	3m19s

Deep Dive: How Scaling Works in Kubernetes



The **Controller Manager** continuously compares desired state (4 replicas) vs actual state. If they differ, it creates or deletes pods until they match.

Phase 7: Self-Healing Demonstration

Kubernetes automatically replaces failed pods — just like Swarm, but with more sophisticated scheduling.

Step 1: List pods and pick one to delete:

```
kubectl get pods
```

Step 2: Delete a pod (simulate a crash):

```
kubectl delete pod <pod-name>
# Example: kubectl delete pod wordpress-7c587db787-d2hzj
```

Step 3: Watch Kubernetes recreate it:

```
watch kubectl get pods
```

The deleted pod disappears and a **new pod** with a different name appears within seconds. The total count stays at 4.

```
pranav@Pranav: ~/k8s-lab
pranav@Pranav:~/k8s-lab$ # Get pod names
kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-7b4db4b6688-bg7dt             1/1     Running   0           3m28s
wordpress-7c587db787-d2hzj          1/1     Running   0           73s
wordpress-7c587db787-d8w8n          1/1     Running   0           3m26s
wordpress-7c587db787-nswjl          1/1     Running   0           3m26s
wordpress-7c587db787-vpvzw          1/1     Running   0           73s
pranav@Pranav:~/k8s-lab$ # Delete it (simulate crash) - replace with actual pod name
kubectl delete pod wordpress-7c587db787-d2hzj
# Immediately watch Kubernetes replace it
watch kubectl get pods
pod "wordpress-7c587db787-d2hzj" deleted from default namespace
pranav@Pranav:~/k8s-lab$ # Delete service and deployment
kubectl delete -f ~/k8s-lab/wordpress-service.yaml
kubectl delete -f ~/k8s-lab/wordpress-deployment.yaml
# Verify everything is gone
kubectl get pods
kubectl get svc
# Delete the k3d cluster entirely
k3d cluster delete mylab
service "wordpress-service" deleted from default namespace
deployment.apps "wordpress" deleted from default namespace
NAME                                READY   STATUS    RESTARTS   AGE
mysql-7b4db4b6688-bg7dt             1/1     Running   0           4m22s
wordpress-7c587db787-d8w8n          1/1     Terminating   0           4m20s
wordpress-7c587db787-frnxf          0/1     Completed      0           25s
wordpress-7c587db787-nswjl          0/1     Completed      0           4m20s
wordpress-7c587db787-vpvzw          0/1     Completed      0           2m7s
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes ClusterIP  10.43.0.1     <none>        443/TCP    13m
mysql-service ClusterIP  10.43.78.132 <none>        3306/TCP   4m21s
INFO[0006] Deleting cluster 'mylab'
INFO[0018] Deleting cluster network 'k3d-mylab'
INFO[0018] Deleting 1 attached volumes...
INFO[0018] Removing cluster details from default kubeconfig...
INFO[0018] Removing standalone kubeconfig file (if there is one)...
INFO[0018] Successfully deleted cluster mylab!
pranav@Pranav:~/k8s-lab$
```

How Self-Healing Works

Step	What Happens
1	You delete pod <code>wordpress-d2hzj</code>
2	ReplicaSet detects: actual = 3, desired = 4
3	ReplicaSet tells API Server: "create 1 new pod"
4	Scheduler assigns it to a node
5	kubelet pulls image and starts the container
6	Actual = 4 again 

Key difference from Swarm: In Swarm, the old task stays in history as `Shutdown/Failed`. In Kubernetes, the old pod is fully removed and a brand-new pod with a new name is created.

Phase 8: Cleanup

```
# Delete Kubernetes resources
kubectl delete -f ~/k8s-lab/wordpress-service.yaml
kubectl delete -f ~/k8s-lab/wordpress-deployment.yaml
kubectl delete -f ~/k8s-lab/mysql-service.yaml
kubectl delete -f ~/k8s-lab/mysql-deployment.yaml

# Verify
kubectl get pods    # pods terminating/gone
kubectl get svc     # only 'kubernetes' ClusterIP remains

# Delete the entire k3d cluster
k3d cluster delete mylab
```

Expected cleanup output:

```
INFO[0006] Deleting cluster 'mylab'
INFO[0018] Deleting cluster network 'k3d-mylab'
INFO[0018] Deleting 1 attached volumes ...
INFO[0018] Removing cluster details from default kubeconfig ...
INFO[0018] Successfully deleted cluster mylab!
```

Part C — Swarm vs Kubernetes Comparison

Feature	Docker Swarm (EXP-11)	Kubernetes (EXP-12)
Deploy command	<code>docker stack deploy</code>	<code>kubectl apply -f</code>
Scale command	<code>docker service scale</code>	<code>kubectl scale deployment</code>
Self-healing	Yes (basic — restart)	Yes (advanced — reschedule to different node)
Config format	docker-compose.yml	YAML manifests (Deployment, Service, etc.)
Load balancing	Ingress mesh (basic)	Services + Ingress controllers (advanced)
Auto-scaling	No	Yes (HPA — Horizontal Pod Autoscaler)
Rolling updates	Basic	Advanced (rollback, canary, blue-green)
Ecosystem	Small	Massive (Helm, Istio, Prometheus, ArgoCD)
Cloud support	Limited	Native (EKS, GKE, AKS)

When to Use What

Tool	Best For
Docker Compose	Development and testing on a single machine
Docker Swarm	Simple production with basic orchestration needs
k3d / Minikube	Learning Kubernetes on your laptop
kubeadm	Real, production-style multi-node clusters
EKS / GKE / AKS	Enterprise production workloads

Part D — Advanced: Real Cluster with kubeadm

For production-style clusters with real VMs (not needed for this lab, but covered for completeness):

Requirements: 2-3 VMs, Ubuntu 22.04+, each with 2+ CPU, 2+ GB RAM.

```
# Step 1: Install on ALL nodes
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
  https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update && sudo apt install -y kubeadm kubelet kubectl
sudo apt-mark hold kubeadm kubelet kubectl

# Step 2: Initialize master node
sudo kubeadm init

# Step 3: Configure kubectl
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Step 4: Install network plugin (Calico)
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml

# Step 5: Join workers (run on each worker node)
kubeadm join <master-ip>:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>

# Step 6: Verify from master
kubectl get nodes
```

Lab Fixes Summary

#	What	Problem	Fix
1	Tool choice	Minikube needs nested VMs on WSL2	Chose k3d (runs inside Docker containers)
2	Installation	kubectl and k3d not installed, lab said "ask instructor"	Added full install commands
3	Cluster creation	No creation command provided	<code>k3d cluster create</code> with explicit <code>--port</code> mappings
4	Deployment YAML	No <code>WORDPRESS_DB_*</code> env vars → WordPress shows DB error	Added env vars pointing to <code>mysql-service:3306</code>
5	Pod monitoring	<code>kubectl get pods</code> is a snapshot — no live feedback	Added <code>watch kubectl get pods</code> throughout
6	Browser access	Lab said <code><node-ip>:30007</code> with no guidance	Used <code>localhost:30007</code> — k3d port mapping handles it
7	Cleanup	No cleanup steps provided	Added <code>kubectl delete</code> + <code>k3d cluster delete mylab</code>

The Two Most Critical Changes

Change 1 + 3 (k3d + port mappings): Minikube would have failed entirely on WSL2 due to nested virtualization. And even with k3d, if `--port` flags aren't set at cluster creation, WordPress is unreachable — ports are trapped inside Docker's internal network with no way to add them later.

Common Pitfalls & Troubleshooting

Problem	Cause	Fix
Pods stuck in <code>ContainerCreating</code>	Image still downloading	Wait 30-60s; check <code>kubectl describe pod <name></code>
Pods in <code>CrashLoopBackOff</code>	App crashes on startup (missing env vars, bad config)	Check logs: <code>kubectl logs <pod-name></code>
WordPress shows "Error establishing DB connection"	No MySQL pod or wrong <code>WORDPRESS_DB_HOST</code>	Deploy MySQL + set host to <code>mysql-service:3306</code>
<code>curl localhost:30007</code> returns nothing	Port not mapped at k3d cluster creation	Delete cluster, recreate with <code>--port "30007:30007@server:0"</code>

Problem	Cause	Fix
<code>kubectl: command not found</code>	kubectl not installed or not in PATH	Install kubectl and move to <code>/usr/local/bin/</code>
Pods show <code>0/1 Running</code>	Container started but readiness probe failing	Check <code>kubectl describe pod <name></code> for probe details

Glossary

Term	Definition
Kubernetes (K8s)	Open-source container orchestration platform — industry standard
kubectl	CLI tool for communicating with the Kubernetes API server
k3d	Lightweight tool that runs K3s (minimal Kubernetes) inside Docker containers
Pod	Smallest deployable unit — wraps one or more containers
Deployment	Declarative definition of desired state: image, replicas, labels
ReplicaSet	Controller that ensures the specified number of pod replicas are running
Service	Stable network endpoint that routes traffic to pods matching a label selector
NodePort	Service type that exposes a port (30000-32767) on every node
ClusterIP	Default Service type — internal only, not accessible from outside the cluster
Label	Key-value metadata attached to resources for selection and grouping
Selector	Query that matches resources by their labels
Namespace	Virtual cluster within a cluster — isolates resources
kubelet	Agent running on each node — manages pods and communicates with API server
kube-proxy	Network proxy on each node — handles Service routing rules
API Server	Central management point — all kubectl commands go through it
Controller Manager	Runs controllers that maintain desired state (ReplicaSet, Deployment, etc.)
Scheduler	Decides which node a new pod should run on based on resources and constraints
etcd	Distributed key-value store — the cluster's "database" for all state
kubeadm	Tool for bootstrapping production-grade Kubernetes clusters
Manifest	YAML file declaring the desired state of Kubernetes resources

Quick Reference Card

```
# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux-amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Install k3d
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash

# Create cluster with port mappings
k3d cluster create mylab --port "30007:30007@server:0" --agents 1

# Apply manifests
kubectl apply -f <file.yaml>

# Watch pods in real-time
watch kubectl get pods

# Scale a deployment
kubectl scale deployment <name> --replicas=N

# Delete a pod (test self-healing)
kubectl delete pod <pod-name>

# Check logs
kubectl logs <pod-name>

# Describe a resource (debugging)
kubectl describe pod <pod-name>

# Delete resources by manifest
kubectl delete -f <file.yaml>

# Delete k3d cluster
k3d cluster delete mylab
```

References

- [Kubernetes Official Docs](#)
- [k3d Documentation](#)
- [kubectl Cheat Sheet](#)
- [Kubernetes Concepts](#)